

SIT202

NETWORK AND COMMUNICATIONS

Learning Summary Report



OSI

PHYSICAL LAYER
Concerns the physical aspects of the network, such as the cables, switches, and routers.

Network Link Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Network Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Transport Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Session Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Presentation Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Application Layer
Concerns the logical aspects of the network, such as the protocols and the addressing.

Jahan Garg

2410994805

Self-Assessment Details

The following checklists provide an overview of my self-assessment for this unit.

	Pass (P)	Credit (C)	Distinction (D)	High Distinction (HD)
Self-Assessment	✓	✓	✓	

Self-Assessment Statement

	Included
Learning Summary Report	✓
Pass tasks complete	✓

Minimum Pass Checklist

	Included
All Credit Tasks	✓

Minimum Credit Checklist (in addition to Pass Checklist)

	Included
All Distinction tasks	✓

Minimum Distinction Checklist (in addition to Credit Checklist)

	Included
Something Awesome included	
Helping Others	

Minimum High Distinction Checklist (in addition to Distinction Checklist)

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part of this submission been written for me by another person.

Signature:



Portfolio Overview

This portfolio presents the work I have completed so far in **SIT202 – Network and Communications** and explains how that work demonstrates the Unit Learning Outcomes while supporting my application for a Distinction-grade result.

Across this unit, I have developed a clearer and more practical understanding of how networked systems operate from end to end. My submitted work shows that I can explain how layers in the protocol stack interact, analyse the behaviour of common protocols, use industry-relevant networking tools to investigate real traffic, and reflect on the security, performance, and scalability issues that arise in real network communication.

I demonstrated ULO1 through my work on the Application Layer and Transport Layer, where I explained layered abstraction, encapsulation, addressing, transport services, and process-to-process communication. This is evident in my discussion of HTTP, HTTPS, DNS, TCP, UDP, sockets, and the relationships between protocol layers, which shows that I understand not only the role of each layer but also how the layers work together to enable meaningful data transfer between end systems.

I demonstrated ULO2 by engaging with the design and development side of networking concepts rather than treating them only as theory. My DNS Server Sketch task required me to think systematically about server initialisation, DNS query handling, A and CNAME records, header construction, and response generation, which helped me connect protocol knowledge with structured implementation logic.

I demonstrated ULO3 through practical analysis using tools such as Wireshark, nslookup, Python socket programming, and Cisco Packet Tracer. Through these activities, I analysed the behaviour of HTTP, HTTPS, DNS, TCP, and UDP, examined packet headers and message sequences, interpreted real network data, and connected simulation and traffic analysis to the theoretical concepts studied in the unit.

I demonstrated ULO4 through reflection on how protocols are designed and why their design choices matter in real systems. My work on TLS, DNS hierarchy and caching, TCP reliability and control mechanisms, IP forwarding, and router behaviour demonstrates a growing ability to evaluate networking decisions in terms of security, reliability, performance, and scalability, rather than simply describing protocols in isolation.

The evidence in this portfolio represents more than basic task completion. My Application Layer, Transport Layer, and Network Layer work shows detailed conceptual understanding, while my DNS Server Sketch extends that understanding into structured design thinking and practical protocol modelling.

Overall, this portfolio reflects sustained effort to build depth of understanding, strengthen practical skills, and improve the quality of my work through analysis, reflection, and application across multiple modules. For these reasons, I believe this portfolio supports a Distinction-grade application by demonstrating clear progress beyond foundational understanding and increasing independence in the way I analyse and apply computer networking concepts.

Learning Plan Check-in

The target grade indicated in the learning Plan I submitted at the start of the Trimester: Distinction

The target grade of my portfolio: Distinction

Reflection:

My target grade has remained Distinction throughout the trimester. Although I experienced some setbacks, including health-related absences, time-management pressure across multiple units, and adjustment challenges following the introduction of the new OnTrack version, I chose not to lower my goal. Instead, I responded by following a structured catch-up plan that included regular consultation with Dr Amanpreet Kaur, peer discussion, revision through D2L materials, and focused support from reliable online resources and tutorial videos.

This approach helped me stay committed to steady improvement rather than short-term task completion. I continued to target Distinction because I wanted my understanding to be conceptually strong rather than rushed, and because the quality of my submitted work showed that I was capable of working toward that standard through consistent effort.

Reflection

The most important things I learnt:

The most important thing I learned in this unit is that network communication makes sense only when the layers are understood as an interconnected process rather than as isolated protocols. I developed a much clearer understanding of how a simple user action, such as entering a URL, triggers a sequence involving DNS resolution, transport-layer communication, and application-layer message exchange. This changed my view of networks from something abstract into something I can analyse logically and step by step. I also learned that practical networking knowledge depends on being able to observe and interpret real traffic. Through Wireshark, nslookup, and protocol analysis, I moved beyond memorising concepts and began identifying how protocols actually behave in real environments. That shift from theory to evidence was one of the most valuable parts of the unit.

The things that helped me most were:

The most helpful parts of my learning were the active classes, practical protocol analysis, and the requirement to explain concepts in my own words. These activities pushed me beyond passive reading and helped me build stronger conceptual clarity. Regular revision, tutor consultation, peer discussion, and the use of D2L resources, reference texts, authentic websites, and selected YouTube tutorials were also highly valuable. They allowed me to revisit difficult topics from different perspectives and gradually strengthen both my confidence and accuracy.

I found the following topics particularly challenging:

The DNS topic in the Application Layer was especially challenging for me because I missed part of that content due to health-related absences. As a result, I had to spend extra time rebuilding my understanding of DNS hierarchy, caching, query flow, and packet-level analysis outside class.

I also found the Transport Layer and parts of the Network Layer demanding because they were content-heavy and required careful revision to understand the relationships between protocol behaviour, reliability, delay, and control mechanisms. In addition, managing the workload across four units made it harder to maintain an ideal pace throughout the trimester.

I found the following topics particularly interesting:

I found HTTP, HTTPS, TLS, DNS, and packet analysis especially interesting because they revealed what happens behind everyday Internet use. I enjoyed tracing requests, responses, headers, and encrypted communication because it made network behaviour more visible and logically understandable.

I was also particularly interested in the design side of networking tasks, especially the DNS Server Sketch, because it encouraged me to think like a problem-solver rather than only a learner. Designing protocol logic helped me appreciate how theoretical networking concepts can be translated into structured implementation steps.

I feel I learned these topics, concepts, and/or tools really well:

I believe I learnt the Application Layer particularly well, especially HTTP, HTTPS, DNS, client-server communication, sockets, and the use of Wireshark to analyse protocol exchanges. My submitted Application Layer review demonstrated detailed explanation, practical analysis, and reflection, and my self-assessment results for the earlier completed modules were strong, including 100 per cent in Modules 1, 2, and 3.

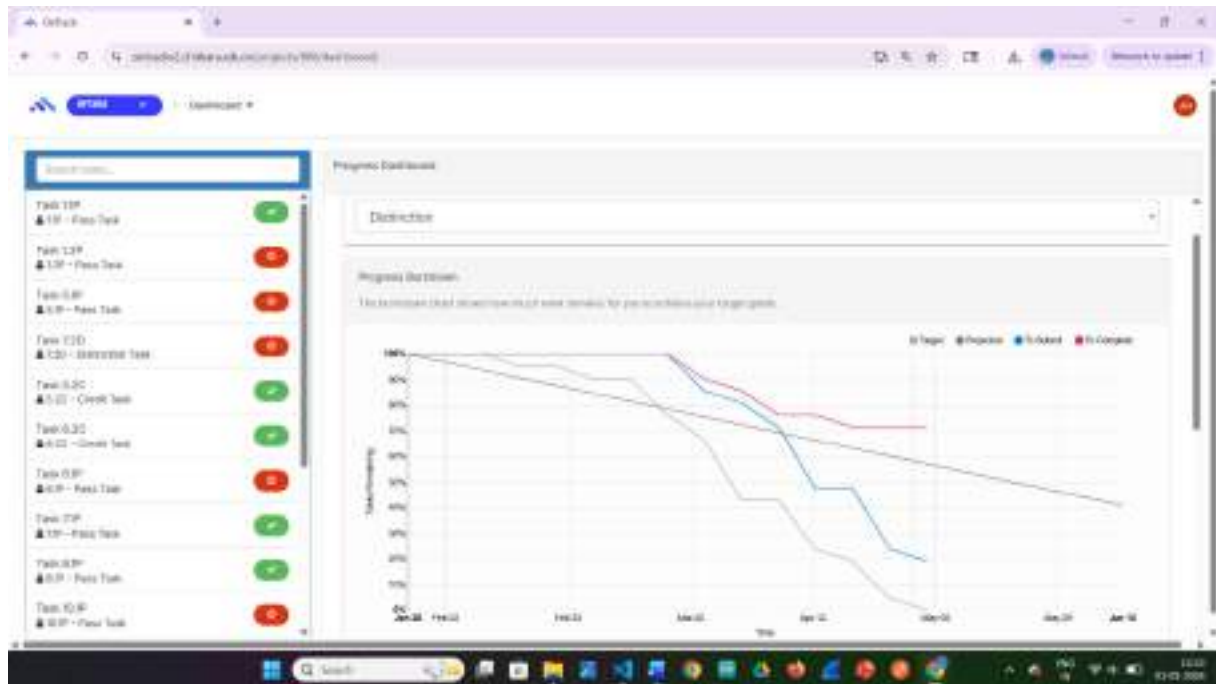
I also developed a solid understanding of the Transport Layer, including UDP, TCP, connection management, reliability, header interpretation, and the relationship between transport services and application needs. My DNS Server Sketch task further strengthened my confidence in structured protocol thinking and showed that I could apply my knowledge in a more design-oriented context.

I still need to work on the following areas:

I still need to strengthen my confidence in the later parts of the unit, particularly by ensuring that my remaining tasks are completed to the same standard as my strongest earlier submissions. My focus is not only on completing them, but also on maintaining depth, clarity, and accuracy across all remaining portfolio pieces.

I also want to become faster and more consistent in pacing my work across the trimester. One of the main lessons I have learnt is that understanding improves most when revision is spread evenly rather than delayed by backlog pressure.

My progress in this unit was uneven:



My progress graph shows that my completion rate did not initially keep pace with the ideal target. From my perspective, this reflects a period in which missed classes, competing workload, and adjustment to the new OnTrack workflow slowed my momentum and created a backlog.

However, the graph also shows that I responded by working to reduce that gap and regain control of my progress. Rather than lowering my goal, I used the graph as a practical reminder to improve my pacing, prioritise core tasks, and continue working towards a Distinction through structured catch-up and consistent effort.

This unit will help me in the future:

This unit will be valuable for my future studies and career because it has strengthened the networking foundation that underpins secure system design, software architecture, embedded systems, and practical problem-solving. It has helped me understand how communication actually occurs across systems, which is essential for many advanced areas of computing.

In particular, this unit is highly relevant to my interest in ethical hacking. Ethical hacking requires a deep understanding of protocols, traffic behaviour, addressing, transport services, DNS, encryption, and common network weaknesses, and SIT202 has helped me build that foundation in a structured and practical way. It has also supported my broader interests in secure development, interactive systems, IoT, AI/ML-driven applications, and future technology ventures.

If I did this unit again, I would do the following things differently:

If I completed this unit again, I would begin structured revision earlier and maintain a more regular weekly cycle of review, practice, and task drafting. This would reduce pressure during heavy weeks and allow more time for refinement instead of catch-up.

I would also make earlier use of consultation and progress-monitoring tools whenever I noticed a backlog starting to build. One of the clearest lessons from this trimester is that timely action is far more effective than waiting until several tasks begin to overlap.

Other:

One of the most valuable things I developed in this unit was resilience. Even when I faced disruption, I continued working toward conceptual mastery instead of shifting to a lower target or aiming only for minimal completion.

Overall, this unit helped me become a more reflective and practical learner. It strengthened not only my networking knowledge, but also my ability to analyse technical systems carefully, connect theory with evidence, and continue improving through feedback and persistence.

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

JAHAN GARG

Portfolio Submission

Submitted By:
Jahan GARG
2410994805

Tutor:
Garima CHOPRA

May 13, 2026



Contents

1	Learning Summary Report	1
2	Overall Task Status	2
3	Learning Outcomes	3
3.1	Layered encapsulation	3
3.2	Interaction with Networks	3
3.3	Networking Tools	3
3.4	Analysing real network issues	3
4	Task 1.1P	5
5	Task 1.2P	9
6	Task 3.1P	29
7	Task 5.1P	52
8	Task 5.2C	88
9	Task 6.1P	99
10	Task 7.1P	124
11	Task 6.2C	129
12	Task 4.1C	148
13	Task 10.2P	185
14	Task 8.1P	194
15	Task 7.2D	215
16	Task 9.1P	222
17	Task 5.3D	241
18	Task 10.1P	303

2 Overall Task Status

Task	Status	Times Assessed
Task 1.1P	Complete	2
Task 1.2P	Time Exceeded	1
Task 1.3H	Not Started	
Task 3.1P	Complete	2
Task 5.1P	Complete	2
Task 7.2D	Complete	1
Task 4.1C	Time Exceeded	3
Task 5.2C	Complete	2
Task 6.2C	Complete	2
Task 6.1P	Complete	2
Task 7.1P	Complete	1
Task 10.2P	Complete	2
Task 8.1P	Complete	2
Task 5.3D	Time Exceeded	5
Task 8.2H	Not Started	
Task 9.1P	Complete	2
Task 6.3H	Not Started	
Task 10.1P	Time Exceeded	2

3 Learning Outcomes

3.1 Layered encapsulation

Explain the layers of abstraction involved in the computer network protocol stack, and how these enable data to flow between machines within a network.

Task	Rating	Status	Times Assessed
Task 1.2P	◆◆◆◆◇	Time Exceeded	1
Task 3.1P	◆◆◆◆◆	Complete	2
Task 5.1P	◆◆◆◆◆	Complete	2
Task 6.1P	◆◆◆◆◇	Complete	2
Task 8.1P	◆◆◆◆◆	Complete	2
Task 9.1P	◆◆◆◆◆	Complete	2
Task 10.1P	◆◆◆◆◇	Time Exceeded	2
Task 10.2P	◆◆◆◆◇	Complete	2
Task 4.1C	◆◆◆◆◆	Time Exceeded	3
Task 6.2C	◆◆◆◆◇	Complete	2
Task 5.3D	◆◆◆◆◇	Time Exceeded	5
Task 7.2D	◆◆◆◆◆	Complete	1

3.2 Interaction with Networks

Design and develop programs that interact with networks at different levels with the network protocol stack to implement and analyse common protocols.

Task	Rating	Status	Times Assessed
Task 5.1P	◆◆◆◆◆	Complete	2
Task 6.1P	◆◆◆◆◇	Complete	2
Task 9.1P	◆◇◇◇◇	Complete	2
Task 10.2P	◆◆◇◇◇	Complete	2
Task 4.1C	◆◆◆◆◇	Time Exceeded	3
Task 5.2C	◆◆◆◆◆	Complete	2
Task 6.2C	◆◆◆◆◆	Complete	2
Task 5.3D	◆◆◆◆◇	Time Exceeded	5

3.3 Networking Tools

Use industry-relevant networking tools to analyse different network configurations, real network data, networking protocols, and their operations, and provide networking solutions to given problems.

Task	Rating	Status	Times Assessed
Task 3.1P	◆◆◆◆◆	Complete	2
Task 5.1P	◆◆◆◆◇	Complete	2
Task 6.1P	◆◆◆◆◇	Complete	2
Task 8.1P	◆◆◆◆◆	Complete	2
Task 9.1P	◆◆◆◆◇	Complete	2
Task 10.1P	◆◆◆◆◇	Time Exceeded	2
Task 10.2P	◆◆◇◇◇	Complete	2
Task 4.1C	◆◆◆◆◇	Time Exceeded	3
Task 6.2C	◆◆◇◇◇	Complete	2
Task 5.3D	◆◆◆◆◇	Time Exceeded	5
Task 7.2D	◆◆◆◆◇	Complete	1

3.4 Analysing real network issues

Reflect and critique on how networks and protocols are constructed and apply those concepts in analysing real network issues related to security and scalability.

Task	Rating	Status	Times Assessed
Task 1.1P	◆◆◆◆◆	Complete	2
Task 5.1P	◆◆◆◆◆	Complete	2
Task 6.1P	◆◆◆◆◆	Complete	2
Task 7.1P	◆◆◆◆◆	Complete	1
Task 8.1P	◆◆◆◆◆	Complete	2
Task 9.1P	◆◆◆◆◆	Complete	2
Task 10.2P	◆◆◆◆◆	Complete	2
Task 4.1C	◆◆◆◆◆	Time Exceeded	3
Task 5.2C	◆◆◆◆◆	Complete	2
Task 6.2C	◆◆◆◆◆	Complete	2
Task 5.3D	◆◆◆◆◆	Time Exceeded	5

4 Task 1.1P

Selecting Grade

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

This pass task provides essential guidance on unit details and the assessment workflow. By adding the required details and my signature, I documented my consent. Upon completion of this task, I will commit the necessary study hours and stay focused on the learning plan to achieve my target grade.

Date	Author	Comment
2026/02/09 07:44	Jahan Garg	Working On It
2026/02/09 18:05	Jahan Garg	Ready for Feedback
2026/02/09 18:05	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 1.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/02/12 06:45	Amanpreet Kaur	Dear Jahan, You are required to provide your grade, mention it under study plan and resubmit.
2026/02/12 06:45	Amanpreet Kaur	Fix and Resubmit
2026/02/12 11:02	Jahan Garg	Ready for Feedback
2026/02/12 11:02	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards, Jahan Garg 2410994805 X2 Batch
2026/02/17 09:10	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 1.1P

Submitted By:

Jahan GARG

2410994805

2026/02/12 11:02

Tutor:

Amanpreet KAUR

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

This pass task provides essential guidance on unit details and the assessment workflow. By adding the required details and my signature, I documented my consent. Upon completion of this task, I will commit the necessary study hours and stay focused on the learning plan to achieve my target grade.

February 12, 2026



Initial Learning Plan- SIT202 – S4 2024

Study Plan

Tasks	Due Date (D)
Task 1.1P	02 nd February
Task 1.2P Introduction	09 th February
Task 3.1P Application Layer	24 th February
Task 5.1P Transport Layer	16 th March
Task 6.1P Network Layer- Data Plane	26 th March
Task 7.1P	27 th March
Task 8.1P Network Layer – Control Plane	6 th April
Task 9.1P Data Link Layer	17 th April
Task 10.1P Physical Layer	27 th April
Task 10.2P	31 st April
C tasks	
Task 4.1C Above and Beyond Pass	17 th March
Task 5.2C My DNS Server Sketch	20 th March
Task 6.2C My DNS Server	23 rd March
D tasks	
Task 5.3D Above and Beyond Credit	12 th April
Task 7.2D Amazing Networks	30 th March
HD tasks	
Task 1.3 H Collaborative Learning Initiatives (aim for exemplary achievement 90+)	Not targeted
Task 6.3 H My Awesome Tutorial	Not targeted
Task 8.2 H Building Your Own Ping Program	Not targeted

My target grade is Distinction.

Grading Agreement

I, (Jahan Garg, 2410994805), confirm that I have read and understood how this unit works: I understand that:

- I will receive a grade in accordance with the number of modules and tasks that I complete;
- I can change my target grade at any time during the trimester, up until I submit my portfolio.
- I am responsible for providing evidence of my work to satisfy the requirements in each module completed.
- Following my initial submission, I will work with my tutor and the teaching team to update and complete my submissions until they demonstrate that I have achieved the learning objectives of the module.
- I am responsible for attending in class discussions of some of my tasks as detailed in the hurdle requirement of the unit

I commit to submit the initial submission of my work before the deadlines listed in the table above. I acknowledge that these deadlines can be modified by me, in agreement with my tutor, when the necessity occurs, and that I am responsible for requesting these modifications as soon as I become aware of the need to do so, and no later than the deadline to be modified.

- Name: Jahan Garg
- Student ID: 2410994805
- Date: 01-02-2026
- Signature: 

5 Task 1.2P

Pass task for module 1

Outcome	Weight
Layered encapsulation	◆◆◆◆◇

In this task, I developed a foundational understanding of computer networking by studying the Internet as a network of networks, the TCP/IP layered model, and how web browsing works through DNS, TCP, and HTTP, and I supported this understanding with practical evidence from Chrome Developer Tools, Wireshark, and class activities to show how layered communication appears in real network traffic. For ULO1, I explained the protocol stack, the roles of each layer, and how encapsulation and decapsulation enable data to flow between machines, which is the core focus of this module. For ULO2, I did not design or develop any network-interacting program, so this task has limited alignment with program development. For ULO3, I used networking tools to observe real traffic, but only as evidence for learning the module rather than as a tool-driven problem-solving or network-analysis task. For ULO4, I reflected on what I learned and why it matters, but I did not deeply critique security or scalability issues, so the connection is limited.

Date	Author	Comment
2026/02/09 18:05	Jahan Garg	Working On It
2026/02/11 18:56	Jahan Garg	Mam, I am requesting an extension for this task as OnTrack was made available recently, leading to an intense workload across all units. With this additional time, I will be able to demonstrate the task effectively without compromising its quality.
2026/04/26 07:49	Jahan Garg	Ready for Feedback
2026/04/26 07:49	Amanpreet Kaur	Time Exceeded
2026/04/26 07:50	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 1.2P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 1.2P

Submitted By:
Jahan GARG
2410994805
2026/04/26 07:49

Tutor:
Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◇

In this task, I developed a foundational understanding of computer networking by studying the Internet as a network of networks, the TCP/IP layered model, and how web browsing works through DNS, TCP, and HTTP, and I supported this understanding with practical evidence from Chrome Developer Tools, Wireshark, and class activities to show how layered communication appears in real network traffic. For ULO1, I explained the protocol stack, the roles of each layer, and how encapsulation and decapsulation enable data to flow between machines, which is the core focus of this module. For ULO2, I did not design or develop any network-interacting program, so this task has limited alignment with program development. For ULO3, I used networking tools to observe real traffic, but only as evidence for learning the module rather than as a tool-driven problem-solving or network-analysis task. For ULO4, I reflected on what I learned and why it matters, but I did not deeply critique security or scalability issues, so the connection is limited.

April 26, 2026



Module 1: Summary and Reflection

1. MODULE SUMMARY

In this module, I developed a foundational understanding of computer networking and of the Internet as a network of networks. I learned why communication between devices depends on standardised protocols and a layered architecture, and I studied the five-layer TCP/IP model in relation to the OSI model. I also learned how web browsing works behind the scenes, including the role of URLs, DNS, TCP connection establishment, and HTTP request–response communication. Through the module activities, Chrome Developer Tools, and Wireshark, I observed real network transactions and packet transfers, which helped me connect theoretical concepts to actual browser and network behaviour.

2. MODULE REFLECTION

The most important thing I learned in this module was that accessing a webpage involves a series of coordinated interactions across multiple layers of the TCP/IP model, rather than a single simple action. This is related closely to what I already know about structured system design in software engineering, because networking also uses modular layers to make complex communication easier to understand, implement, and troubleshoot. I believe the teaching team wants us to learn this module first because it provides the conceptual foundation for all later topics in the unit, including the transport, network, and data-link layers. To build this understanding, I used the D2L module content, the Module 1 PPT, Chapter 1 of the recommended textbook, Chrome Developer Tools, and Wireshark; I used these resources first to understand the key concepts and then to verify them through practical observation of real web and packet-level communication.

References [1], [2], [3], [4], [5], [6], [7]

- [1] “1.1 Introduction - Network and Communications_Sem2_2025.” Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23466/View>
- [2] “1.2 What is Computer Networking - Network and Communications_Sem2_2025.” Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23467/View>
- [3] “1.3 TCP/IP - Network and Communications_Sem2_2025.” Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23468/View>
- [4] “1.4 Deciphering Web Browsing - Network and Communications_Sem2_2025.” Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23469/View>

- [5] "1.5 Discovering the Magic Using Network Tools- Part 1 - Network and Communications_Sem2_2025." Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23470/View>
- [6] "1.6 Discovering the Magic Using Network Tools :Part 2 - Network and Communications_Sem2_2025." Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23471/View>
- [7] "Module 1 - Network and Communications_Sem2_2025." Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/24330/View>

Module 1: Evidence of Learning

This section presents evidence of my learning for **Module 1: Introduction to Computer Networking**, with a focus on foundational networking concepts, the Internet as a network of networks, layered communication through the TCP/IP model, and the processes involved in web browsing, such as DNS resolution, TCP connection establishment, and HTTP request–response communication. It includes module exercises, active class activities, discussion notes, browser-based investigation using Chrome Developer Tools, packet-level analysis in Wireshark, and supporting screenshots to demonstrate both conceptual understanding and practical observation of network behaviour. Short comments are included throughout the section to explain important steps in my progress and to show how each exercise or activity contributed to my developing understanding of networking fundamentals and prepared me for later topics in the unit. [1]

Evidence: Module Exercises

Exercise 1: Types of networks I use daily

Ans. In my daily routine, I rely on several different network types: a home Wi-Fi LAN for my laptop and phone, campus wired and wireless LANs for study and lab work, and my mobile 4G/5G data connection when I am travelling. These edge networks connect to my ISP's core network, which then interconnects with other ISPs to reach public cloud and content-provider networks such as Google or Deakin's D2L platform. Together, they form the "network of networks" that we call the Internet, allowing me to seamlessly move between home, university, and mobile environments while using the same applications such as web browsing, email, and online learning.

Exercise 2: Layered architecture in a daily system

Ans. A good real-life analogy for layered architecture is online food delivery:

- The **Application layer** is the food-delivery app where I choose a restaurant and place an order (user-level interaction and business logic).
- The **Transport layer** is similar to the order-management system that keeps track of individual orders, acknowledgements, and retries if something fails (ensuring my order is reliably handled end-to-end).
- The **Network layer** corresponds to the city-wide road network that decides which route the rider should take from the restaurant to my home (path selection and addressing).
- The **Data-link layer** is like the local streets on which the rider actually travels hop by hop between intersections (link-to-link delivery).

- The **Physical layer** is the concrete road surface and the motorbike itself that physically carries the rider and the food (actual signal/bit transmission medium).

Breaking this complex service into layers makes it easier to design, troubleshoot, and upgrade individual parts (e.g., changing the app without rebuilding the roads), mirroring the way the TCP/IP stack organises network protocols. [2], [3]

Evidence: Active Classes Activities

Activities for Active Class 1: Magic behind accessing the World Wide Web

Activity 1: Layered model and bandwidth

Question 1: Why do we need protocols, and what are the five layers?

Ans. We need network protocols because devices must agree on a common “language” and behaviour in order to communicate reliably. Protocols define message formats, ordering, error handling, and timing rules so that independently developed hardware and software can interoperate over the Internet. Without standardised protocols, each vendor would implement its own incompatible mechanisms, and end systems would not be able to exchange data correctly.

Five layers in the Internet (TCP/IP) protocol stack

1. **Application layer** – Network applications and their protocols (HTTP, DNS, SMTP, etc.).
2. **Transport layer** – End-to-end transport of application data between processes (TCP, UDP).
3. **Network layer** – Addressing and routing of datagrams between hosts across networks (IP, ICMP).
4. **Data-link layer** – Reliable transfer of frames between directly connected nodes on a single link (Ethernet, Wi-Fi).
5. **Physical layer** – Transmission of raw bits over physical media (copper, fibre, radio).

This five-layer model matches the TCP/IP view used in the unit and is a simplified version of the seven-layer OSI reference model. [3], [4]

Question 2: Responsibilities of each layer (role-play)

Ans. In our group role-play, we mapped each person to one layer of the TCP/IP stack on **Device 1 (client)** and **Device 2 (server)**. Our objective was to send the message “**Hello**” from Device 1 to Device 2 and then unpack it layer by layer at the destination.

Application layer (Member 1)

On Device 1, the application layer represented a simple chat/web application that generated the message “Hello”. It packaged this user data into an application-layer message and selected an appropriate protocol, such as HTTP. On Device 2, the application layer received the message from the transport layer, interpreted it as application data, and displayed “Hello” to the user.

Transport layer (Member 2)

The transport layer on Device 1 accepted the application message and created a **segment**, adding a header with source and destination port numbers and reliability information (e.g., sequence and acknowledgement numbers for TCP). Its responsibility was to provide reliable, ordered delivery and basic flow control between the two processes. On Device 2, the transport layer reassembled segments, checked for missing or out-of-order data, and then delivered a clean byte stream containing “Hello” to the application.

Network layer (Member 3)

The network layer encapsulated the transport segment into an **IP datagram**, adding logical addressing (source and destination IP addresses) and a Time-To-Live field. Its responsibility was to find a path through the interconnected networks from the client host to the server host using routing protocols. At the destination, the network layer verified the IP header, decremented the TTL, and handed the segment up to the transport layer.

Data-link layer (Member 4)

The data-link layer on each hop encapsulated the IP datagram inside a **frame**, adding link-layer addressing (source and destination MAC addresses) and local error-detection bits (e.g., a CRC). Its responsibility was to move each frame across a single physical link to the next hop, handling local retransmissions if necessary. At Device 2, the data-link layer checked the frame’s integrity and passed the valid datagram up to the network layer.

Physical layer (Member 5)

The physical layer converted each frame into a stream of bits encoded as electrical/optical/radio signals on the medium (e.g., Wi-Fi radio or Ethernet copper). Its responsibility was purely to transmit and receive signal waveforms; it did not interpret addresses or protocol semantics. At each hop, the physical layer of the receiver converted the signals back into bits for the data-link layer.

As the message travelled from Device 1 to Device 2, each lower layer added its own header (and sometimes trailer) around the data from the layer above, creating a nested structure: **message** → **segment** → **datagram** → **frame** → **bits**, and the reverse decapsulation happened at the server. This activity helped us clearly visualise how each layer has a distinct role while relying on the services provided by the layer below. [3], [4]

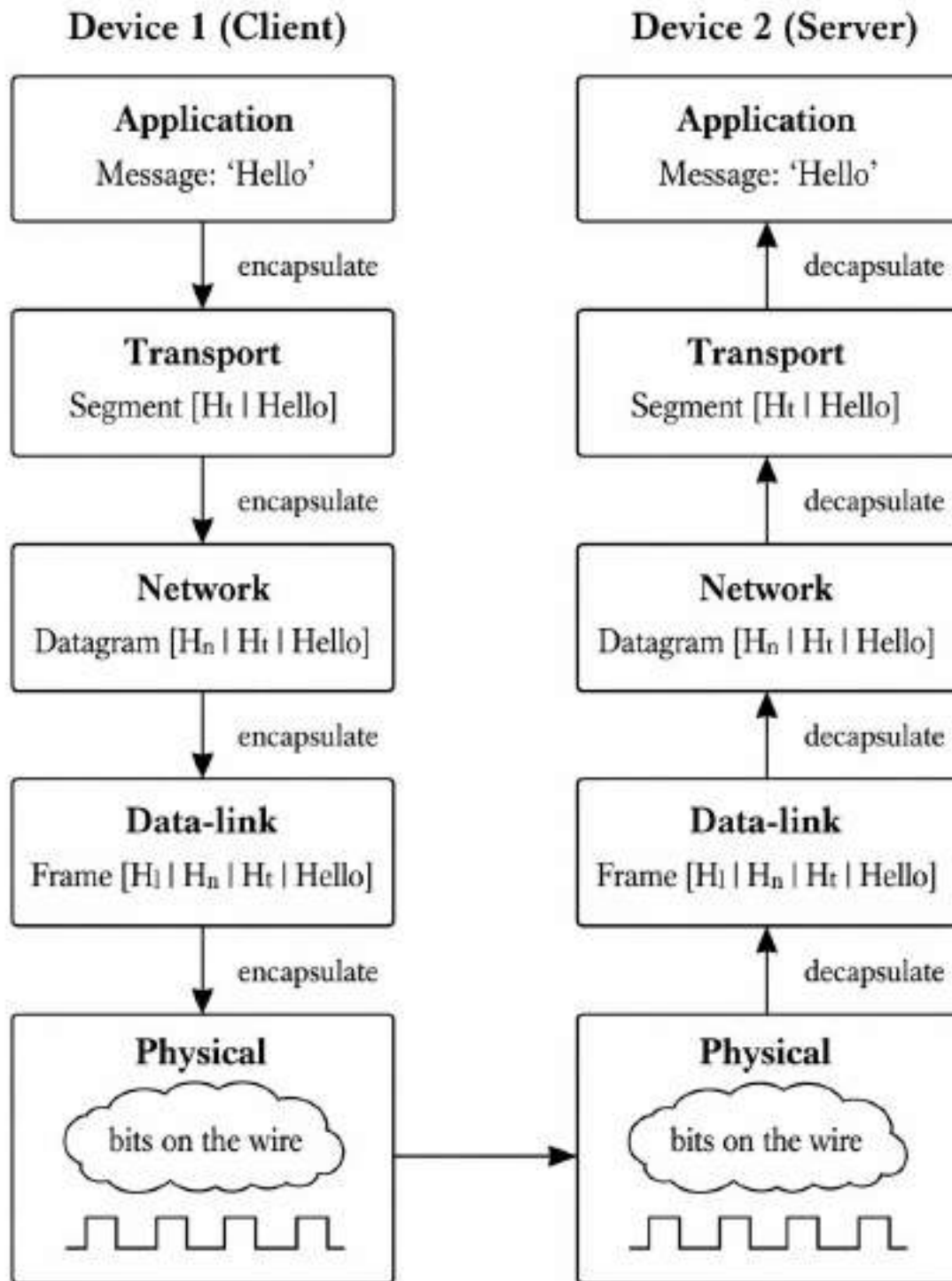


Figure 1: Layer-by-layer encapsulation and decapsulation of the message 'Hello' from client to server in the TCP/IP model.

Question 3: Bandwidth and queueing delay**Ans.****(a) Container packing analogy**

Each box is $2\text{m} \times 2\text{m} \times 2\text{m}$, so its volume is 8m^3 . The container is $20\text{m} \times 20\text{m} \times 10\text{m}$, with a volume of 4000m^3 . Along the 20-metre sides, we can fit $20/2 = 10$ boxes, and along the 10-metre height, we can fit $10/2 = 5$ boxes, giving $10 \times 10 \times 5 = 500$ boxes. This corresponds to the number of “packet slots” a link can carry at a time, given its finite capacity.

(b) Packet-switching link with many users

- Link capacity: **0.5 GBytes/s** = $0.5 \times 8 = 4$ Gbit/s.
- Each user's transmission rate when active: **100 Mbit/s** = 0.1 Gbit/s.
- Maximum number of users that can transmit simultaneously without exceeding link capacity:
 $4/0.1 = 40$ users.

Using this:

1. **Statement (i)** – “If 40 users transmit simultaneously, then there will be a significant queueing delay before the link.”
If exactly 40 users are transmitting at 100 Mbit/s, the aggregate offered load equals the link capacity (4 Gbit/s). In the idealised analysis used in the unit (as in Kurose & Ross), this threshold corresponds to operating the link at full utilisation **without persistent queueing**, so statement (i) is considered **false**.
2. **Statement (ii)** – “If 88 users transmit simultaneously, then there will be a queueing delay just before the link.”
 $88 \text{ users} \times 0.1 \text{ Gbit/s} = 8.8 \text{ Gbit/s} > 4 \text{ Gbit/s}$, so the offered load exceeds link capacity, and packets must queue (and eventually be dropped) before the link. Statement (ii) is **true**.
3. **Statement (iii)** – “Since we use packet switching, we can have any number of users, and they can transmit packets without a delay.”
Packet switching allows efficient statistical multiplexing for **bursty** traffic, but it does not remove the fundamental limit imposed by link capacity; if too many users transmit at once, queueing and delay are inevitable. Statement (iii) is therefore **false**. [3]

Activity 2: Analysing web access with Chrome DevTools

Using Chrome DevTools to inspect web browsing

For Activity 2, I used **Google Chrome DevTools (Network tab)** to observe the network transactions generated when loading the Google homepage. After opening the Network panel, I loaded the Google homepage and observed how multiple HTTP requests appeared in the table as the page loaded. [5], [6]

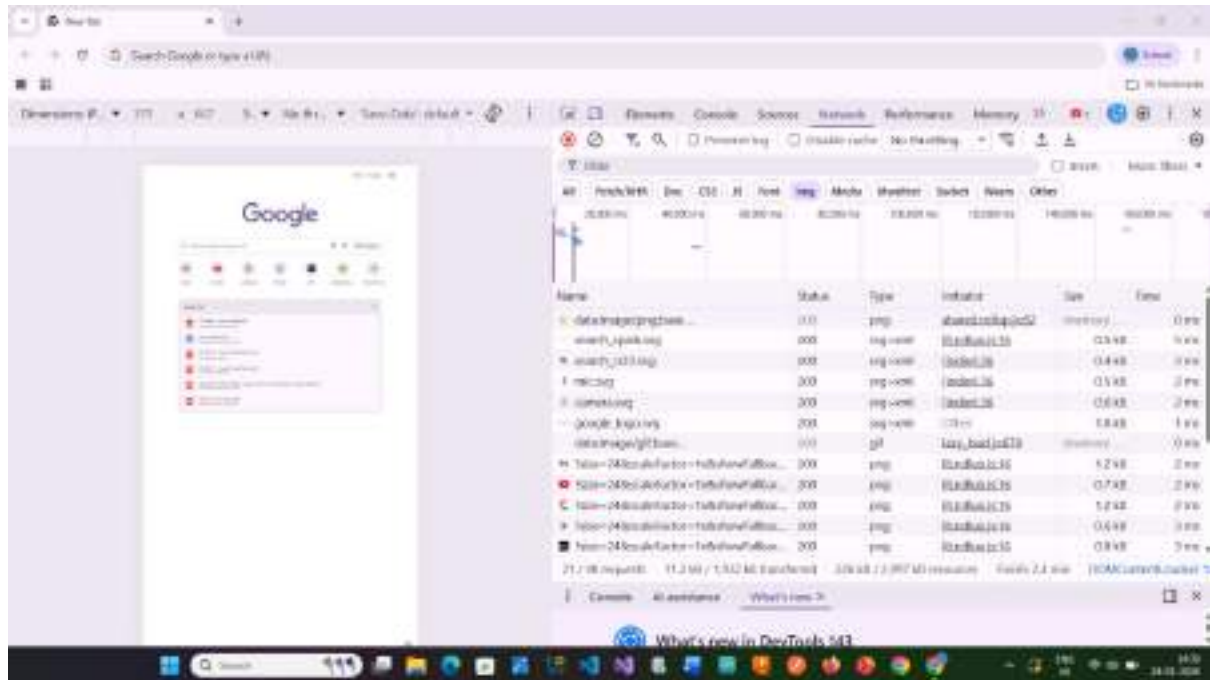


Figure 2: Chrome DevTools Network tab showing requests generated while loading the Google homepage.

Each row in the Network table represents one request–response exchange between my browser and a web server. The **Name** column listed resources such as google_logo.svg, search_spark.svg, mic.svg, camera.svg, several data:image/... URIs, and a set of small PNG icons requested with query strings. The **Status** column showed 200 for the visible requests, indicating successful responses, while codes such as 404 or 500 would indicate retrieval problems in other situations. [5], [7]

The **Type** column classified most of these entries as svg+xml, png, or gif, showing that the browser automatically fetched icons, images, and other static assets in addition to the main page content. The **Initiator** column showed that different files or scripts were responsible for triggering requests, illustrating how HTML and JavaScript work together to load webpage resources. The **Size** and **Time** columns recorded the amount of data transferred and the time required to complete each request, and the visible resources in this capture were generally small and loaded quickly. [5], [8]

At the bottom of the Network panel, the summary line showed overall request, transfer, resource-size, and timing information. This view confirms that a seemingly simple page load actually consists of many individual HTTP(S) requests and shows how the browser coordinates these transactions to render the page from multiple components. [5], [8]

Overall, this activity helped me connect the **application-layer action** of opening a webpage in a browser with the underlying sequence of HTTP requests and responses that make up a modern web page. It also reinforced the meaning of HTTP status codes and showed how Chrome DevTools can be used to inspect real web communication. [5], [7]

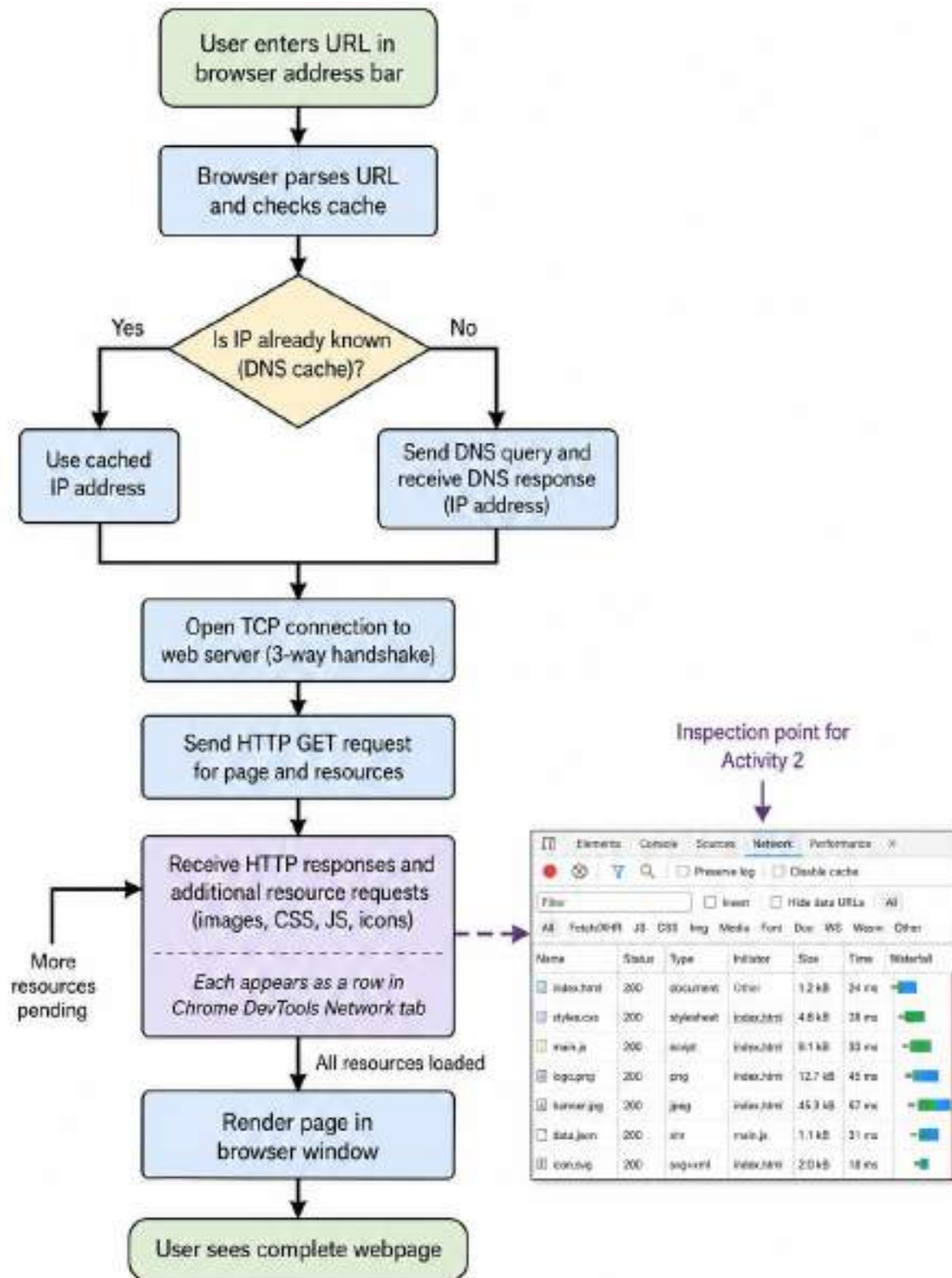


Figure 3: High-level flow of web browsing and where Chrome DevTools can be used to observe HTTP transactions.

Activity 3: Analysing packet transfers with Wireshark

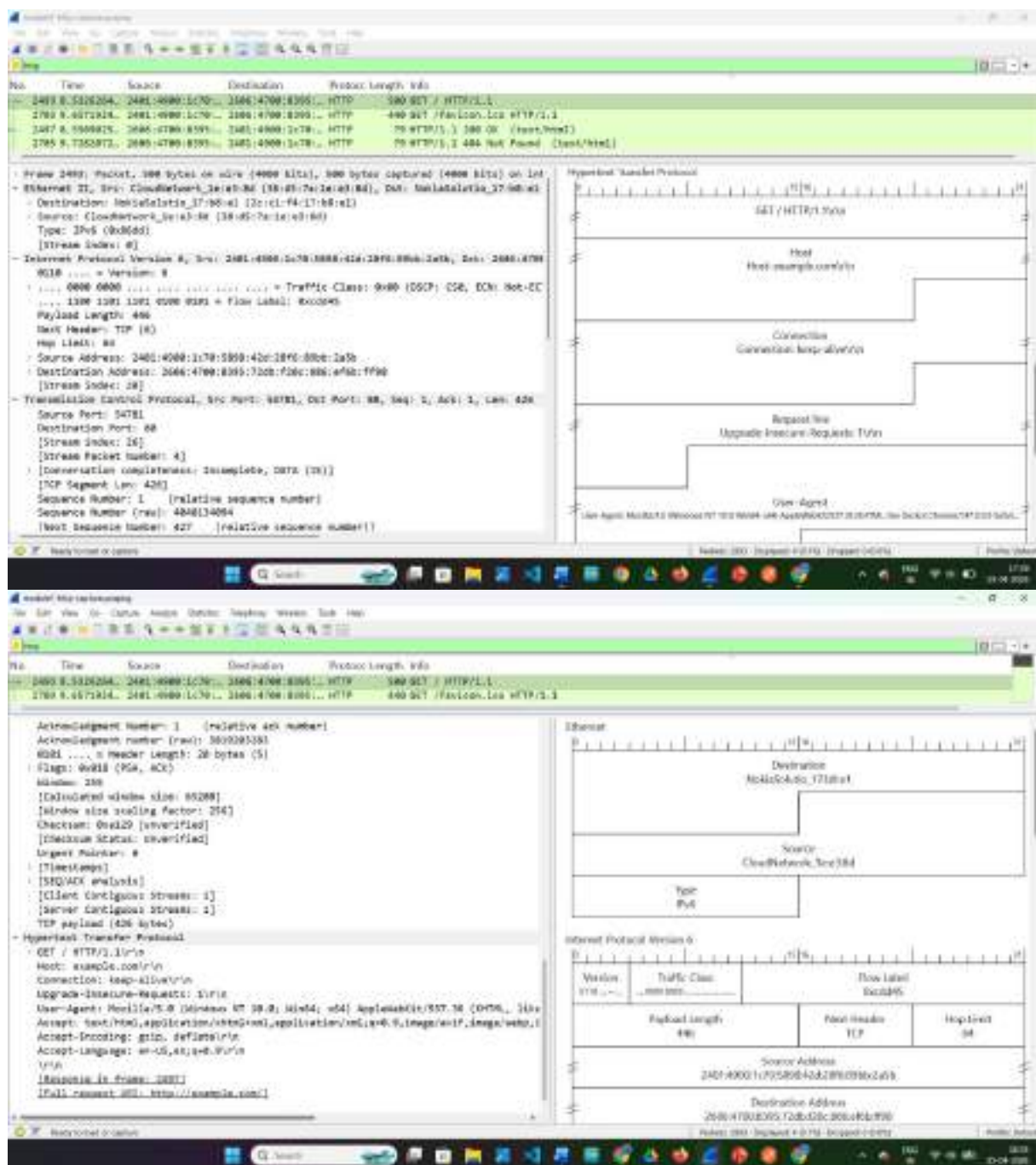


Figure 4: Wireshark capture showing the selected HTTP packet and its protocol details.

Question 3: Four different protocols observed

Ans. In the Wireshark capture taken while loading an HTTP webpage, I observed the following protocols in the packet details of the selected packet:

- **Ethernet II** is used for local link-layer delivery between the source and destination MAC addresses.
- **IPv6** is used for network-layer addressing and routing between my computer

and the web server.

- **TCP** is used to provide reliable transport for the HTTP request.
- **HTTP** is used to carry the actual web request, shown here as GET / HTTP/1.1.

Together, these protocols show how the webpage request is encapsulated across the network layers. [9], [10]

Question 4: Time between HTTP GET and HTTP OK reply

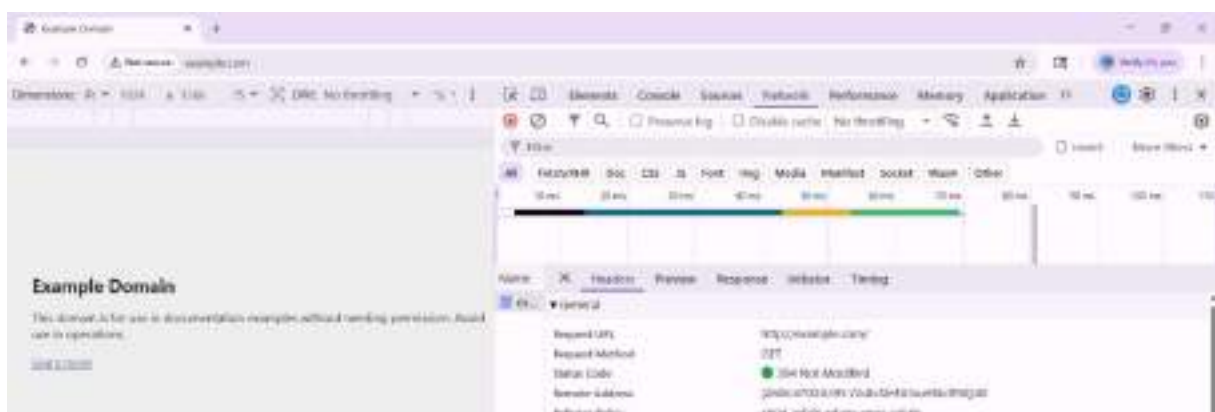
Ans. I selected the main HTTP GET request for the webpage and the corresponding HTTP 200 OK response in Wireshark. Using the **Time** column in the packet list, I noted that the GET request was sent at **8.5326264 s** and the HTTP 200 OK response was received at **8.5509025 s**. Therefore, the response time was **8.5509025 – 8.5326264 = 0.0182761 s**, which is approximately **18.3 ms**. This time difference reflects the delay between the request and response and matches the fast page load observed in the browser. [11]

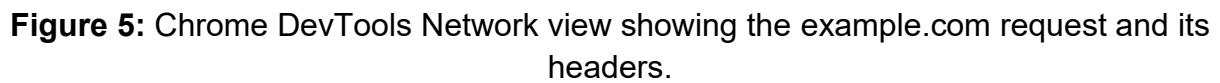
Question 5: IP addresses of client and server

Ans. By examining the IP header fields in Wireshark, I identified that my computer used the IP address **2401:4900:1c70:5898:42d:28f6:89bb:2a5b**, while the web server's IP address was **2606:4700:8395:72db:f20c:886:ef6b:ff98**. The same source and destination IP addresses appear consistently across the TCP and HTTP packets for this browsing session, confirming that this client–server pair carried the web traffic. The expanded **IPv6** header view in the packet details pane also displayed additional fields, including Version, Traffic Class, Flow Label, Payload Length, Next Header (TCP), and Hop Limit, which matched the theoretical IPv6 header format discussed in the module.

Question 6: Comparing Wireshark with Chrome DevTools

Ans. Chrome DevTools and Wireshark provide complementary views of the same web communication. In Chrome DevTools, I could observe the browser-level request information for example.com, including the request URL, method, status code, headers, and cache-related behaviour.





The screenshot displays the Wireshark interface with a packet capture of an HTTP GET request. The top toolbar shows standard network analysis tools. The packet list on the left shows three packets, with the first packet selected. The packet details pane on the right shows the structure of the selected packet, including Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol. The packet bytes pane at the bottom shows the raw data of the selected packet.

Packet 1: Ethernet II, Src: CloudNetwork_Local8d (8:05:7d:08:04:00), Dst: MokaSatoDante_17:08:01 (26:01:74:17:08:01)

Destination: MokaSatoDante_17:08:01 (26:01:74:17:08:01)
Source: CloudNetwork_Local8d (8:05:7d:08:04:00)
Type: IPv4 (802.3)

Internet Protocol Version 4, Src: 192.168.1.100, Dst: 192.168.1.101

Source Address: 192.168.1.100
Destination Address: 192.168.1.101
Stream Index: 0

Transmission Control Protocol, Src Port: 8080, Dst Port: 80, Seq: 1, Len: 48

Source Port: 8080
Destination Port: 80
Stream Index: 1
Stream Packet Number: 1
Conversation completeness: Incomplete, Data (15)
TCP Segment Len: 48
Sequence Number: 1 (relative sequence number)
Next Sequence Number: 49 (relative sequence number)
Acknowledgment Number: 2 (relative ack number)
Acknowledgment Number (raw): 4096123456
Flags: 0x01 (PSH, ACK)
Window: 255
Calculated window size: 81920
Window size scaling factor: 256
Checksum: 0x0000 (unverified)
Checksum Status: Unverified
Urgent Pointer: 0
Timestamps:
Options: 0x0000 (unverified)



Figure 6: Wireshark capture showing the matching HTTP GET request for example.com.

The difference between the DevTools result showing **304 Not Modified** and the Wireshark capture showing **200 OK** is still logically valid because HTTP conditional requests using headers such as **If-Modified-Since** can cause the server to return 304 when the cached copy is still current, rather than sending the full resource again. Therefore, both tools were still describing the same web access, but from different perspectives and with different handling of browser caching.

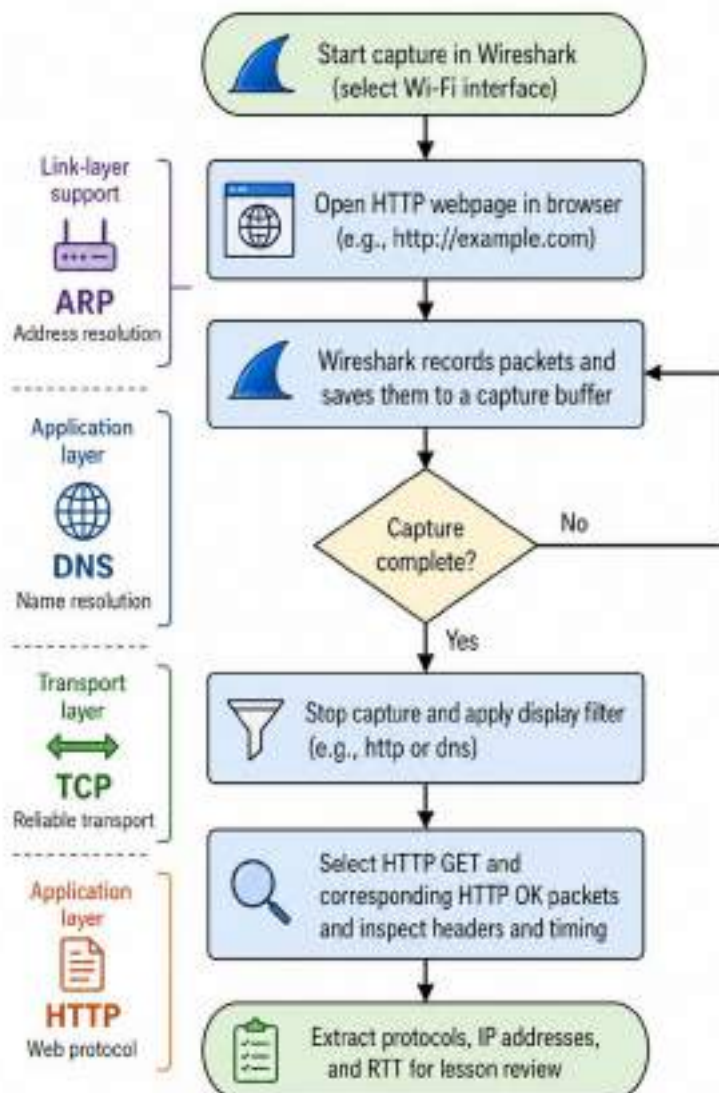


Figure 7: End-to-end workflow for capturing and analysing HTTP exchanges with Wireshark.

Summary of Learning Evidence

Through Module 1, I developed a strong foundation in computer networking by studying the Internet as a network of networks, the TCP/IP layered model, and the process of web browsing through DNS resolution, TCP connection establishment, and HTTP request–response communication. This module combined theory with practice through exercises, active class activities, discussion notes, Chrome Developer Tools investigation, Wireshark packet analysis, and supporting screenshots, which helped me connect core networking concepts with real network behaviour.

- **Module Exercises:** Strengthened my understanding of basic networking ideas such as the types of networks used in daily life, layered communication in the TCP/IP stack, bandwidth, packet switching, and queueing delay. I also used real-world analogies to explain how the application, transport, network, data-link, and physical layers work together, which made the structure of network communication easier to understand and remember.
- **Active Class Activities:** Deepened this learning by linking protocol theory to practical observation. Through the role-play activity, I visualised encapsulation and decapsulation across the TCP/IP layers, and through browser-based investigation in Chrome DevTools, I observed how loading a webpage generates multiple HTTP(S) requests with status codes, resource types, and timing information. In Wireshark, I examined packet-level details such as Ethernet, IPv6, TCP, and HTTP headers, measured the response time between HTTP GET and HTTP OK, identified client and server IP addresses, and compared Wireshark with Chrome DevTools to understand how the same web communication appears at different layers.

Overall, this evidence demonstrates that I developed a clear understanding of foundational networking concepts and the ability to apply them in practical network analysis.

References

- [1] "Module 1 - Network and Communications_Sem2_2025." Accessed: Apr. 24, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/24330/View>
- [2] "Network Layer in OSI Model - GeeksforGeeks." Accessed: Apr. 24, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/network-layer-in-osi-model/>
- [3] "CPSC 441: Slides." Accessed: Apr. 24, 2026. [Online]. Available: <https://cspages.ucalgary.ca/~cwill/CPSC441/slides.html>
- [4] "TCP/IP Model - GeeksforGeeks." Accessed: Apr. 25, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/tcp-ip-model/>
- [5] "Inspect network activity | Chrome DevTools | Chrome for Developers." Accessed: Apr. 25, 2026. [Online]. Available: <https://developer.chrome.com/docs/devtools/network>
- [6] "Network panel: Analyze network load and resources | Chrome DevTools | Chrome for Developers." Accessed: Apr. 25, 2026. [Online]. Available: <https://developer.chrome.com/docs/devtools/network/overview>
- [7] "HTTP response status codes - HTTP | MDN." Accessed: Apr. 25, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
- [8] "Chrome Developer Tools – Network Tab." Accessed: Apr. 25, 2026. [Online]. Available: <https://requestly.com/blog/chrome-developer-tools-network-tab/>
- [9] "3.19. The 'Packet Details' Pane." Accessed: Apr. 25, 2026. [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/ChUsePacketDetailsPaneSection.html
- [10] "Wireshark HTTP." Accessed: Apr. 25, 2026. [Online]. Available: <https://imada.sdu.dk/u/jamik/dm557-19/wireshark/wireshark-http.html>
- [11] "8.10. Service Response Time." Accessed: Apr. 25, 2026. [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/ChStatSRT.html

[Quiz List](#) > [Submissions](#)

Quiz Submissions – Module 1: Introduction to Computer Networking

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %

11

Grades

🔒 Print



Stay up-to-date on your grades with
the Pulse App

Keep track with your courses on the go!



Grade Item	Points	Grade	Comments and Permissions
Module 1: Introduction to Computer Networking	10 / 10	100 %	View Quiz Attempts
Module 2: Application Layer	10 / 10	100 %	View Quiz Attempts
Module 3: Transport Layer	10 / 10	100 %	View Quiz Attempts
Unlabeled	10 / 10	100 %	View Quiz Attempts



Search



2021

24-04-2025

6 Task 3.1P

Lesson Review: Application Layer

Outcome	Weight
Layered encapsulation	◆◆◆◆

In this task, I reviewed my learning evidence and honestly rated how well I meet each unit learning outcome. For ULO1, I chose 5 because I can clearly explain the layers of the network protocol stack and how they enable data flow between machines using concrete examples from the Application Layer module. For ULO3, I chose 5 because I can confidently use tools like Wireshark and nslookup to analyse real network traffic and configurations and suggest suitable networking solutions.

Outcome	Weight
Networking Tools	◆◆◆◆

In this task, I reviewed my learning evidence and honestly rated how well I meet each unit learning outcome. For ULO1, I chose 5 because I can clearly explain the layers of the network protocol stack and how they enable data flow between machines using concrete examples from the Application Layer module. For ULO3, I chose 5 because I can confidently use tools like Wireshark and nslookup to analyse real network traffic and configurations and suggest suitable networking solutions.

Date	Author	Comment
2026/02/27 11:58	Jahan Garg	Ready for Feedback
2026/02/27 11:59	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 3.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/03/02 17:36	Amanpreet Kaur	Dear Jahan, you are required to describe the timeline diagram and add some screen shot with labels for the DNS activity.
2026/03/02 17:36	Amanpreet Kaur	Fix and Resubmit
2026/03/06 11:59	Jahan Garg	Ready for Feedback
2026/03/06 12:01	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards,Jahan Garg2410994805X2 Batch
2026/03/10 16:25	Amanpreet Kaur	Dear Jahan, you are required to prepare the timeline diagram for the DNS activity and update screenshot of Quiz that highlight your name clearly.
2026/03/10 16:25	Amanpreet Kaur	Fix and Resubmit
2026/03/22 17:21	Jahan Garg	Working On It
2026/05/05 08:41	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 3.1P

Submitted By:

Jahan GARG

2410994805

2026/03/06 11:59

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Networking Tools	◆◆◆◆◆

In this task, I reviewed my learning evidence and honestly rated how well I meet each unit learning outcome. For ULO1, I chose 5 because I can clearly explain the layers of the network protocol stack and how they enable data flow between machines using concrete examples from the Application Layer module. For ULO3, I chose 5 because I can confidently use tools like Wireshark and nslookup to analyse real network traffic and configurations and suggest suitable networking solutions.

March 6, 2026



1. MODULE SUMMARY

A. Summarising the content

I found it easiest to organise the summary around the three learning objectives for this module.

1. Understanding key application-layer concepts and required services

In this module, I learned that a network application is really a set of cooperating processes running on different end systems that exchange messages over the network. The application layer is responsible for preparing these messages so that lower layers (transport, network, link, physical) can move them across the Internet. We revisited the idea that routers and switches do not run user applications; only the end systems do.

I also explored how each application needs certain transport services such as reliability, timing, throughput and security, and how these requirements guide the choice between TCP and UDP. For example, web browsing and email prefer reliable, ordered delivery over TCP, while some real-time applications may accept occasional loss over UDP for lower delay.

2. Explaining the client–server model and associated processes

The module then focused on application architectures. I studied the traditional client–server architecture, where a server process runs on a fixed, well-known host and multiple clients initiate communication with it. Typical examples include web browsing, email access and online banking. In contrast, I looked at peer-to-peer (P2P) systems, where each peer can act as both client and server, sharing resources directly with other peers. We also touched on hybrid architectures that mix both ideas, such as video conferencing platforms that use servers for coordination while peers exchange media.

At the process level, I learned that communication is really between processes, not programs. A client process starts the conversation, while a server process waits to be contacted. Sockets act as the interface between an application process and the transport layer, so the socket is the “doorway” through which messages enter and leave the network stack. This helped me connect the abstract model to the concrete idea of socket programming.

3. Analysing and implementing application layer protocols (HTTP, HTTPS, DNS)

The largest part of the module was devoted to specific application-layer protocols.

- For HTTP, I studied the basic request–response pattern, common methods such as GET and POST, and the structure of HTTP messages (start line, headers and optional body). I compared non-persistent connections, where every object requires a new TCP connection, with persistent connections, where several requests share one connection. I also learned how cookies allow a stateless HTTP server to remember users, and how web caches (proxy servers) store frequently requested objects to reduce delay and server load.
- I then saw how HTTPS extends HTTP by adding TLS. The module gave me a simple view of the TLS handshake: the client and server authenticate, agree on an encryption method and derive keys before any HTTP data is sent. After that, HTTP messages are encrypted, which explains why Wireshark cannot read them without the keys.
- Finally, I learned about DNS as the Internet’s naming service. DNS translates hostnames into IP addresses using a distributed hierarchy of root servers, TLD servers and authoritative servers. The topics included DNS queries and replies, resource records, the use of UDP and port 53, and how caching reduces repeated lookups. I also saw how local DNS servers and the Time To Live (TTL) value control how long cached mappings remain valid.

4. Weekly activities and practical work

The active classes and tasks connected the theory with practice. In Week 2, the HTTP role-play helped me rehearse the exact sequence of events between multiple clients and a server: TCP connection setup, HTTP GET, HTTP 200 OK, and connection teardown. I then captured real HTTP traffic in Wireshark, filtered by “http”, and inspected the Ethernet, IP, TCP and HTTP headers to see the layered design in action. In Week 3, I repeated this approach for DNS by using nslookup and Wireshark to trace the steps from a typed URL to the resolved IP address for real websites.

B. How is this useful

1. The module gives me a clear end-to-end story of what happens when I type a URL or open an app, so I can reason about performance issues, failures and security risks instead of treating the network as a black box.
2. Understanding sockets, client/server roles and protocol behaviour will directly support future programming tasks where I need to build or debug networked applications, such as web services, chat tools or analytics systems.
3. The insight into HTTP, HTTPS and DNS is essential for cybersecurity: many attacks, misconfigurations and privacy problems appear at the application

layer, so knowing how these protocols really work is the first step towards detecting and mitigating them.

4. The hands-on Wireshark and nslookup exercises gave me practical skills in inspecting real traffic, which is valuable for troubleshooting, monitoring and later units that involve network forensics and system administration.

2. REFLECTION ON THE CONTENT

1. Most important thing I learned

The most important thing I learned is the complete chain from entering a web address to seeing the page: DNS resolves the hostname, TCP establishes a connection, HTTP sends a request and receives a response, and optionally TLS secures the exchange. Earlier, I only knew this at a very high level. Now I can describe each step, the main header fields, and the typical message sequence, and I have actually seen these packets in Wireshark. That sequence ties together many separate topics into one coherent mental picture.

2. How this relates to what I already knew

Before this module, I was aware of the OSI/TCP-IP layers from earlier units, and I used web browsers, email and messaging apps every day. However, my understanding stopped at “the browser sends a request to the server”. This module connected that earlier knowledge with more concrete details: what exactly is inside a GET request, how cookies preserve state across stateless HTTP, why TCP or UDP is chosen for a given application, and how DNS caching explains the difference between first-time and repeated access to the same site. It turned everyday experiences into something I can now explain and reason about technically.

3. Why the course team wants us to learn this content

I think the course team wants us to master the application layer because it is the point where users, software and the network meet. If we understand only lower-level routing or addressing without appreciating protocols like HTTP and DNS, we would struggle to design secure, responsive and reliable systems. This module builds the foundation for later topics such as web development, network security, cloud services and distributed systems. It also trains us to read protocol specifications, interpret packet traces and link theory with tools like Wireshark, which are core skills for any networking, cybersecurity or backend engineering role.

Overall, the Application Layer module has moved me from just “using” Internet applications to being able to analyse and explain how they work, which is exactly the kind of mindset I will need in the rest of this unit and in my degree.

Notes

1

Important role of server
 - It is a computer that provides services to other computers (clients) over a network.
 - It is a central point for data storage and distribution.
 - It is a powerful machine that can handle a large number of requests.

Services provided by server
 - Web services (e.g., Apache Web server)
 - Email services (e.g., Gmail)
 - File sharing (e.g., Dropbox)
 - Database services (e.g., MySQL)

Client-Server Model
 - Request: Client sends a request to the server.
 - Response: Server sends a response to the client.
 - Example: Web browser (client) requests a web page from a web server (server).

WEEK-2

2

Transport Layer
 - Port Address
 - Example: 80 (HTTP), 443 (HTTPS)

Network Layer
 - IP Address / Logical Address
 - Example: 192.168.1.1

Physical Layer
 - Physical Address / MAC Address
 - Example: 08:00:27:00:00:00

HTTP uses 2 types of message: Request, Response

Request: Client sends a request to the server.
 Response: Server sends a response to the client.

Request: GET / HTTP/1.1
 Response: 200 OK

3

DATA LINK LAYER
 - Bridges in network
 - Example: Ethernet

PHYSICAL LAYER
 - Cables, switches, routers
 - Example: Ethernet, Fiber optic

Request: GET / HTTP/1.1
 Response: 200 OK

Request: GET / HTTP/1.1
 Response: 200 OK

4

Service is provided
 Message generated at Application Layer
 - moves to
 - Transport layer to convert into segments
 - moves to
 - Network layer to convert into packets
 - moves to
 - Data Link layer to convert into frames
 - moves to
 - Physical layer to convert into bits

5

APPLICATION LAYER
 - Example: Web browser, Mail client
 - Example: Web browser, Mail client

TRANSPORT LAYER
 - Example: TCP, UDP
 - Example: TCP, UDP

NETWORK LAYER
 - Example: IP
 - Example: IP

DATA LINK LAYER
 - Example: Ethernet
 - Example: Ethernet

PHYSICAL LAYER
 - Example: Cables, switches, routers
 - Example: Cables, switches, routers

Request: GET / HTTP/1.1
 Response: 200 OK

Request: GET / HTTP/1.1
 Response: 200 OK

Request: GET / HTTP/1.1
 Response: 200 OK

Request: GET / HTTP/1.1
 Response: 200 OK

WEB Document RTT = Round-trip Delay of Transfer Protocol
 RTT = Round-trip Delay of Transfer Protocol

→ Peer to Peer Architecture

- No server for communication and devices / clients communicate directly
- Scalability (in terms of allowing)
- Complex (allowing)

→ Hybrid Architecture

- Peer to Peer + Client-Server Architecture
- Peer can communicate with each other as well as server
- eg Zoom meeting.
 - Directly by messaging on gateway
 - Client-server

TCP

1. Connection establishment (Request-Response & Acknowledgement)
2. Data Transmission (in request by the server)
3. Connection termination

WORKING

Application Layer → Transport Layer → Network Layer → Data Link Layer → Physical Layer

Application (client/gateway) → Transport (port address assignment) → Network (IP address) → Data Link (MAC address) → Physical (Physical address)

Connection → TCP → Transmission establishment first → Data Link → Physical

IP → Broadcast the message

Physical → 19

Process → e.g. Send

SIT202 26

HTTP REQUEST FORMAT GENERAL FORMAT

Request line: method [sp] URI [sp] version [sp]

Header line: [Header field name] [value]

Body: entity body

data is stored on the local server

Web Cache → if requested resource is already present in the cache, then the server returns the resource.

if data is not in local server, request the main server

if data is not available, return 404

if data is not available, return 404

What data structure is used in stack?

Ans: HEAP (dynamic data-like structure)

Example of data structure used in stack

→ Stack

SIT202 30

HTTP REQUEST FORMAT

Request line: method [sp] URI [sp] version [sp]

Header line: [Header field name] [value]

Body: entity body

data is stored on the local server

Web Cache → if requested resource is already present in the cache, then the server returns the resource.

if data is not in local server, request the main server

if data is not available, return 404

if data is not available, return 404

What data structure is used in stack?

Ans: HEAP (dynamic data-like structure)

Example of data structure used in stack

→ Stack

31

ACTIVE CLASS 2: Understanding the Application Layer

Activity 1: Understanding Application Layer using HTTP

1. Why do we need application layer protocols?

Application layer protocols define how applications communicate across networks. They specify message formats, exchange sequences, and interaction rules between devices. Without these protocols, systems from different manufacturers would fail to understand each other, making network communication impossible. Protocols ensure that browsers can fetch webpages, email clients can retrieve messages, and file transfers can occur reliably across diverse network environments.

2. Examples of application layer protocols

Common application layer protocols include HTTP for web browsing, HTTPS for secure web connections, FTP for file transfers, SMTP for sending emails, POP3 and IMAP for retrieving emails, and DNS for translating domain names into IP addresses.

3. HTTP Client-Server Protocol Analysis

a. Key Features of HTTP

HTTP operates as a stateless protocol using the request-response model. Clients send requests to servers, which process them and return responses with status codes. HTTP relies on TCP for reliable transmission. Modern browsers use persistent connections (HTTP/1.1), allowing multiple requests over a single TCP connection, while older non-persistent connections require separate TCP sessions for each request.

b. Role Play Setup

Member 1 acts as Client 1, Member 2 as Client 2, Member 3 as Client 3, and Member 4 as the Web Server.

c. Communication Sequence

Client 1 interaction:

Client 1 initiates a TCP connection with Server. Server acknowledges and establishes a connection. Client 1 sends an HTTP GET request for index.html. The server processes the request and returns an HTTP 200 OK response with the webpage. Client 1 receives content and terminates the TCP connection.

Client 2 interaction:

Client 2 establishes a TCP handshake with Server. Server confirms connection. Client 2 requests index.html via HTTP GET. Server responds with HTTP 200 OK and page content. Client 2 displays the page and closes the connection.

Client 3 interaction:

Client 3 opens a TCP session with Server. Server completes handshake. Client 3 issues HTTP GET for index.html. Server sends HTTP 200 OK with document. Client 3 renders content and terminates the connection.

d. Timeline Diagram

The timeline diagram shows HTTP communication over TCP in three clear phases. **Phase 1** establishes a connection through TCP's three-way handshake: Client sends SYN to request a connection, Server replies SYN-ACK to synchronise sequence numbers, Client sends final ACK to confirm. ACKs include positive (normal), negative (error), or duplicate types (when one packet appears multiple times due to line errors in either direction). **Phase 2** transfers data using HTTP GET requests from each client and receives 200 OK responses from the server confirming successful delivery. **Phase 3** terminates the connection when either side sends TCP

FIN (finish), the other acknowledges, and the session closes cleanly. TCP provides reliable transport-layer service while HTTP remains stateless at the application layer.

e. Web Cache Scenario

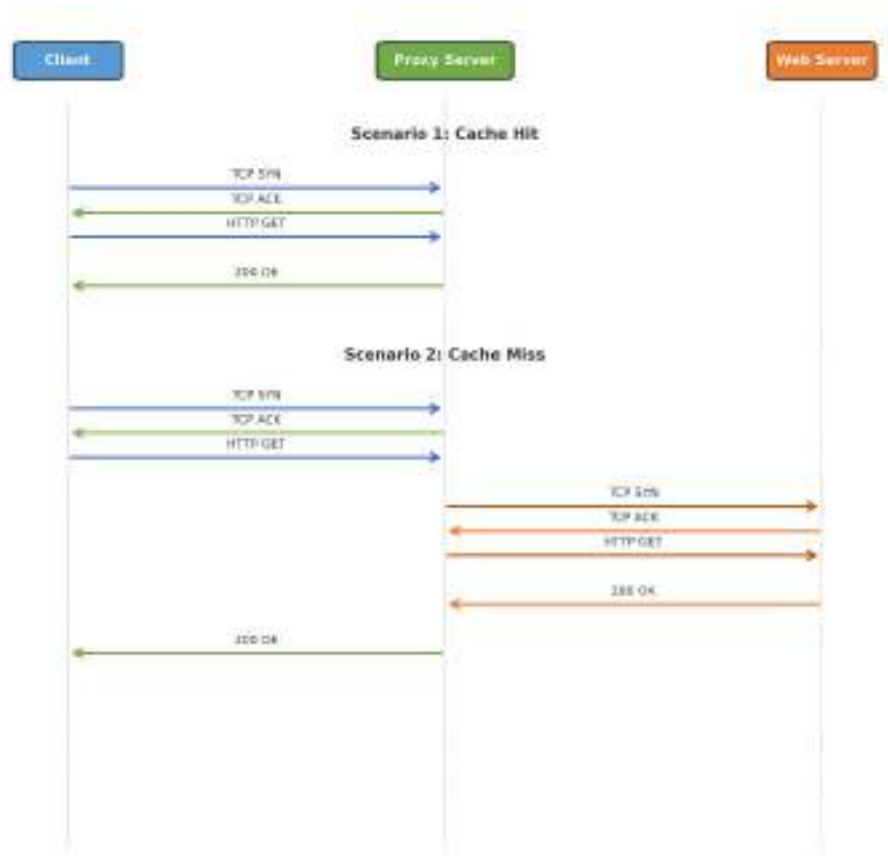
Member 1 is the Client, Member 2 is the Proxy Server, Member 3 is another Client, and Member 4 is the Web Server.

Scenario 1: Cache Hit

Client connects to Proxy Server via TCP. Client sends an HTTP GET request. Proxy checks the cache and finds the object. Proxy serves content directly from cache with an HTTP 200 OK response. No origin server contact needed, resulting in faster response.

Scenario 2: Cache Miss

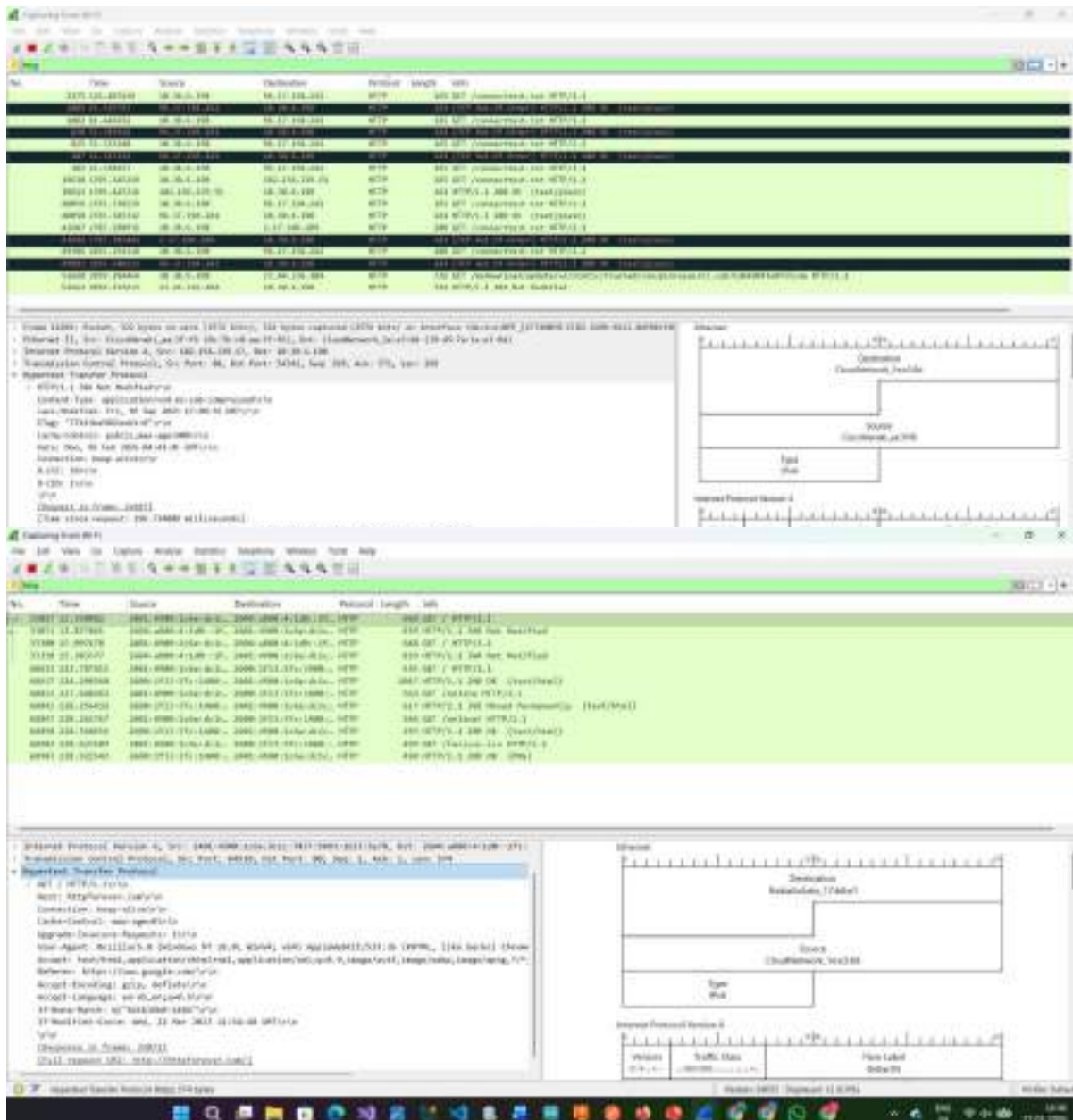
Client establishes TCP with Proxy Server. Client sends an HTTP GET request. Proxy checks the cache and does not find the object. Proxy forwards the request to the Web Server. Web Server returns HTTP 200 OK with content. Proxy caches the object and forwards it to the client.



This timeline diagram illustrates both cache scenarios from our role play. The top path shows **Cache Hit**: The Client sends TCP SYN and HTTP GET request to the Proxy Server, which finds the object in its cache and returns HTTP 200 OK status code directly to the Client without connecting to the Web Server. The bottom path shows **Cache Miss**: The Client sends an HTTP GET request to the Proxy Server, which lacks the object, so the Proxy Server connects to the Web Server, forwards the GET request, receives an HTTP 200 OK status code with content, saves it in cache, and sends it to the Client. The Proxy Server speeds up web browsing by storing frequently requested content locally for instant delivery.

Activity 2: Analysing HTTP in Wireshark

Analysis Based on Wireshark Capture



a. Sequence of HTTP Message Exchange

Looking at the captured packets, the client sends an HTTP GET request specifying the resource path. The server responds with an HTTP 200 OK status, delivering the requested content. Multiple GET requests appear in the trace for different resources, like connecttest.txt.

b. Connection Type

Based on packet number 2175 in the capture showing "Connection: keep-alive" header, this is a **persistent connection**. However, some responses show "Connection: close" indicating **non-persistent** mode, where TCP closes after each transaction.

c. HTTP GET Message Details

From the Wireshark capture, the GET request contains the request line with method GET, resource path /connecttest.txt, and protocol version HTTP/1.1. The Host header specifies www.msftconnecttest.com. Additional headers include User-Agent identifying the browser and Accept headers defining acceptable content types.

d. Packet Details Analysis

The packet details pane reveals four layers:

The Ethernet II Frame shows source MAC CiscoMeraki and destination MAC CloudNetwork with physical addressing.

Internet Protocol Version 4 displays source IP 10.30.6.198 and destination IP 96.17.194.241, along with TTL and protocol information.

Transmission Control Protocol shows source port 80 and destination port 54342. Sequence and acknowledgement numbers ensure reliable delivery.

Hypertext Transfer Protocol contains the GET request with HTTP/1.1 version and a response with 200 OK status.

e. HTTP Version

The request line shows **HTTP/1.1** as the protocol version used by the browser.

f. Server Response

The server returns **HTTP/1.1 200 OK**, indicating successful request processing.

g. Server HTTP Version

The response confirms the server runs **HTTP/1.1**.

h. Client IP Address

The source IP address is **10.30.6.198**.

i. Server IP Address

The destination IP address is **96.17.194.241**.

j. Status Code

The status code is **200 OK**, signifying successful fulfilment.

k. Last Modified Date

In packet 1082, the Last-Modified header shows the file was last updated on a specific date. However, packet 2175 does not contain this header, so the modification date cannot be determined from that particular capture.

l. HTTPS Behaviour

When accessing HTTPS sites, Wireshark displays TLS-encrypted traffic instead of plaintext HTTP. The capture shows TLS handshake messages including Client Hello and Server Hello, followed by encrypted Application Data. Actual HTTP content remains unreadable without decryption keys, protecting sensitive information.

m. Different Webpage Analysis

Analyzing another HTTP site reveals consistent patterns with GET requests triggering 200 OK responses and identical layer encapsulation, confirming HTTP's standardized operation across servers.

ACTIVE CLASS 3: DNS

Activity 1: DNS Fundamentals

1. Core Function of DNS

DNS translates human-readable domain names like deakin.edu.au into machine-readable IP addresses such as 131.217.38.5. This translation is essential because network routing requires numeric addresses, yet users naturally remember textual names.

2. Importance of DNS

DNS eliminates the need to memorize IP addresses, enabling intuitive navigation using memorable domain names. It also allows servers to change IP addresses without disrupting user access by simply updating DNS records. Additionally, DNS supports load balancing by mapping domains to multiple IPs.

3. DNS Layer

DNS operates at the **Application Layer**, using **UDP on port 53** for query-response exchanges. UDP's connectionless nature prioritizes speed for DNS's small, frequent transactions.

4. Single DNS Server Feasibility

A single DNS server would fail due to being a single point of failure, inability to handle billions of daily queries, geographic latency for distant users, and maintenance challenges.

The solution is a **distributed hierarchical architecture** with Root DNS Servers directing queries to TLD servers, TLD Servers managing domains like .com and .edu, Authoritative DNS Servers storing definitive domain records, and Local DNS Servers caching responses for end users.

5. DNS Query Process for deakin.edu.au

Participants: Member 1 (Client), Member 2 (Local DNS), Member 3 (Root DNS), Member 4 (TLD Server), Member 5 (Authoritative DNS)

Sequence:

Client queries Local DNS for deakin.edu.au. Local DNS contacts the Root DNS

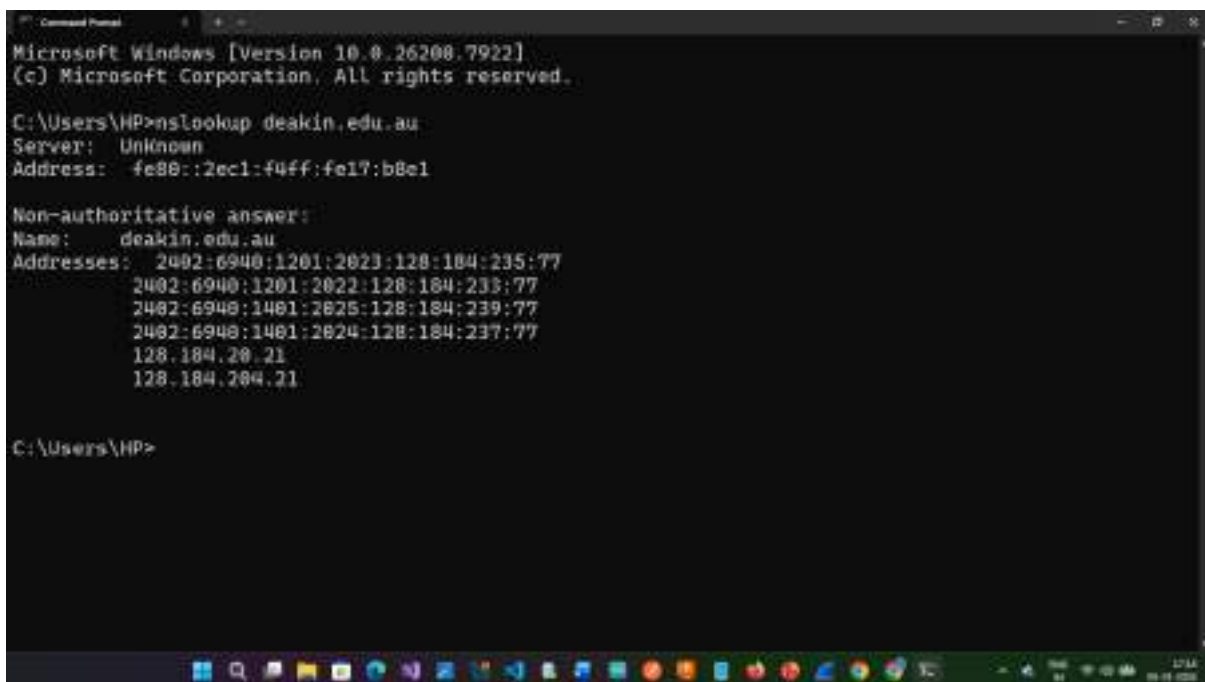
Server. Root refers to the .au TLD Server. Local DNS queries the TLD Server. TLD provides an authoritative DNS address. Local DNS queries Authoritative DNS. Authoritative returns IP address 131.217.X.X. Local DNS caches and forwards IP to Client. Client initiates TCP with the Web Server. Client sends an HTTP GET request. Web Server responds with HTTP 200 OK and webpage.

When Client 2 requests the same domain, Local DNS immediately returns the cached IP without external queries, significantly reducing resolution time.

Activity 2: DNS Analysis

1. nslookup for deakin.edu.au

Running nslookup deakin.edu.au returns IP addresses like 131.217.38.5 and 131.217.38.6 from the Local DNS Server.



```
Microsoft Windows [Version 10.0.26208.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>nslookup deakin.edu.au
Server:      Unknown
Address:     fe80::2ec1:f4ff:fe17:b8e1

Non-authoritative answer:
Name:   deakin.edu.au
Addresses:  2402:6940:1201:2023:128:184:235:77
            2402:6940:1201:2022:128:184:233:77
            2402:6940:1401:2025:128:184:239:77
            2402:6940:1401:2024:128:184:237:77
            128.184.20.21
            128.184.204.21

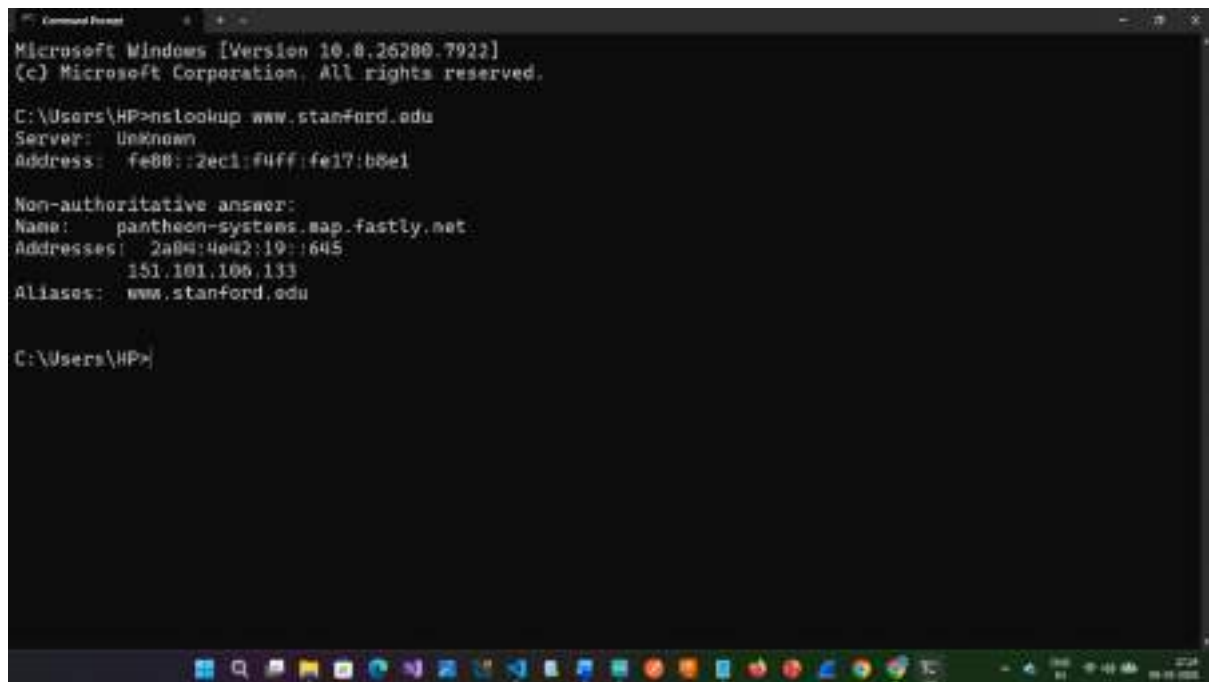
C:\Users\HP>
```

2. Authoritative vs Non-Authoritative

The Local DNS Server provides a **non-authoritative answer** from its cache. An **authoritative answer** comes directly from the domain's official DNS server with guaranteed current information.

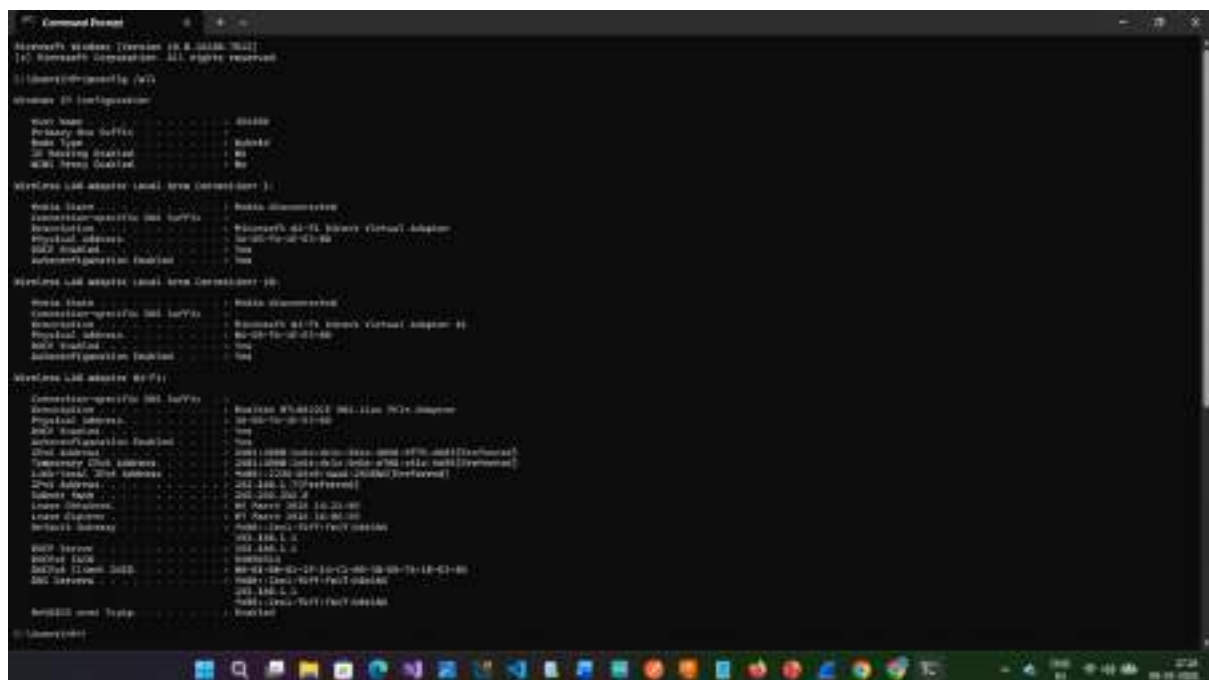
3. US University Query

Querying stanford.edu returns IP 171.67.215.200, demonstrating geographic IP allocation differences between Australian (131.X) and US (171.X) address ranges.



4. Wireshark DNS Analysis

- Transport Protocol:** DNS uses **UDP**.
- Port Numbers:** Destination port **53**, Source port **53**.
- Query Destination:** Sent to Local DNS Server IP (e.g., 10.0.0.1).
- IP Matching:** Command prompt output confirms the same Local DNS Server IP.



- e. Query Type: A record** requests IPv4 addresses.
- f. Response Type:** A record response containing IP addresses.
- g. Response Answers:** Answer section lists resolved IPs with TTL values.
- h. Image Requests:** No new DNS queries for images from the same domain, as the IP is already resolved and cached.

Module 2: Application Layer - Results



Attempt 1 of 4

Written 12 February 2026 12:46 AM - 12 February 2026 1:01 AM

Attempt Score **10 / 10 - 100 %**

Overall Grade (Highest Attempt) **10 / 10 - 100 %**

Done

Grades

 Print



Stay up-to-date on your grades with the Pulse App



Grade Item	Points	Grade	Comments and Assessments
Module 1: Introduction to Computer Networking	10 / 10	100 %	 View Quiz Attempts

Module 2: Application Layer

10 / 10

100 %



View Quiz Attempts

Module 3: Transport Layer

10 / 10

100 %



View Quiz Attempts

Untitled	- / 10	-%
Module 6: Data Link Layer	- / 10	-%
Module 7: Physical Layer	- / 10	-%
Module 4: Network Layer Data Plane	- / 10	-%

7 Task 5.1P

Pass task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I consolidated my learning from Module 3: The Transport Layer by completing and presenting evidence from active class activities on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark-based packet analysis, Python client-server socket programming, summary, reflection, and self-assessment. For ULO1, I explained how layers in the network protocol stack support communication by showing the role of transport-layer protocols and clearly distinguishing connectionless and connection-oriented data flow through UDP and TCP. For ULO2, I designed and developed network-interacting Python programs using sockets, including client-server communication, message transmission, server-side processing, and protocol-based behaviour at the transport layer. For ULO3, I used industry-relevant tools and evidence such as Wireshark captures, flowcharts, screenshots, and code-based demonstrations to analyse protocol behaviour, packet structure, port-based communication, and practical networking operations. For ULO4, I reflected on how network protocols are constructed and applied that understanding through comparison, interpretation, and critique of UDP and TCP behaviour, reliability, and communication logic within real networking scenarios.

Outcome	Weight
Interaction with Networks	◆◆◆◆◆

In this task, I consolidated my learning from Module 3: The Transport Layer by completing and presenting evidence from active class activities on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark-based packet analysis, Python client-server socket programming, summary, reflection, and self-assessment. For ULO1, I explained how layers in the network protocol stack support communication by showing the role of transport-layer protocols and clearly distinguishing connectionless and connection-oriented data flow through UDP and TCP. For ULO2, I designed and developed network-interacting Python programs using sockets, including client-server communication, message transmission, server-side processing, and protocol-based behaviour at the transport layer. For ULO3, I used industry-relevant tools and evidence such as Wireshark captures, flowcharts, screenshots, and code-based demonstrations to analyse protocol behaviour, packet structure, port-based communication, and practical networking operations. For ULO4, I reflected on how network protocols are constructed and applied that understanding through comparison, interpretation, and critique of UDP and TCP behaviour, reliability, and communication logic within real networking scenarios.

Outcome	Weight
Networking Tools	◆◆◆◆◇

In this task, I consolidated my learning from Module 3: The Transport Layer by completing and presenting evidence from active class activities on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark-based packet analysis, Python client-server socket programming, summary, reflection, and self-assessment. For ULO1, I explained how layers in the network protocol stack support communication by showing the role of transport-layer protocols and clearly distinguishing connectionless and connection-oriented data flow through UDP and TCP. For ULO2, I designed and developed network-interacting Python programs using sockets, including client-server communication, message transmission, server-side processing, and protocol-based behaviour at the transport layer. For ULO3, I used industry-relevant tools and evidence such as Wireshark captures, flowcharts, screenshots, and code-based demonstrations to analyse protocol behaviour, packet structure, port-based communication, and practical networking operations. For ULO4, I reflected on how network protocols are constructed and applied that understanding through comparison, interpretation, and critique of UDP and TCP behaviour, reliability, and communication logic within real networking scenarios.

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

In this task, I consolidated my learning from Module 3: The Transport Layer by completing and presenting evidence from active class activities on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark-based packet analysis, Python client-server socket programming, summary, reflection, and self-assessment. For ULO1, I explained how layers in the network protocol stack support communication by showing the role of transport-layer protocols and clearly distinguishing connectionless and connection-oriented data flow through UDP and TCP. For ULO2, I designed and developed network-interacting Python programs using sockets, including client-server communication, message transmission, server-side processing, and protocol-based behaviour at the transport layer. For ULO3, I used industry-relevant tools and evidence such as Wireshark captures, flowcharts, screenshots, and code-based demonstrations to analyse protocol behaviour, packet structure, port-based communication, and practical networking operations. For ULO4, I reflected on how network protocols are constructed and applied that understanding through comparison, interpretation, and critique of UDP and TCP behaviour, reliability, and communication logic within real networking scenarios.

Date	Author	Comment
2026/03/10 20:46	Jahan Garg	Working On It
2026/03/12 12:00	Jahan Garg	Ready for Feedback
2026/03/12 12:00	Amanpreet Kaur	Time Exceeded
2026/03/19 09:54	Amanpreet Kaur	Dear Jahan, your task is not complete.check carefully
2026/03/19 09:54	Amanpreet Kaur	Fix and Resubmit
2026/03/22 17:21	Jahan Garg	Working On It
2026/04/13 05:15	Jahan Garg	Ready for Feedback
2026/04/13 05:15	Amanpreet Kaur	Time Exceeded
2026/04/13 05:15	Jahan Garg	Mam, I have attached the complete PDF file of my SIT202 Networks and Communication 5.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/05/05 09:07	Amanpreet Kaur	Discuss aboutit teh 3 way handshake of TCP and how much RTT is required
2026/05/05 09:07	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 5.1P

Submitted By:

Jahan GARG

2410994805

2026/04/13 05:15

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◇
Analysing real network issues	◆◆◆◆◇

In this task, I consolidated my learning from Module 3: The Transport Layer by completing and presenting evidence from active class activities on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark-based packet analysis, Python client-server socket programming, summary, reflection, and self-assessment. For ULO1, I explained how layers in the network protocol stack support communication by showing the role of transport-layer protocols and clearly distinguishing connectionless and connection-oriented data flow through UDP and TCP. For ULO2, I designed and developed network-interacting Python programs using sockets, including client-server communication, message transmission, server-side processing, and protocol-based behaviour at the transport layer. For ULO3, I used industry-relevant tools and evidence such as Wireshark captures, flowcharts, screenshots, and code-based demonstrations to analyse protocol behaviour, packet structure, port-based communication, and practical networking operations. For ULO4, I reflected on how network protocols are constructed and applied that understanding through comparison, interpretation, and critique of UDP and TCP behaviour, reliability, and communication logic within real networking scenarios.

April 13, 2026



Module 3: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I will organise this summary around Modules 3.1 to 3.7, linking each part to the activities and resources from the Transport Layer module and Active Classes 4 & 5.

1) Understanding key transport-layer concepts and required services (3.1, 3.2)

In this module, I learned that the network layer provides logical communication between hosts, while the transport layer extends this to process-to-process communication using sockets and port numbers. The household analogy clarified this perfectly: hosts are houses, processes are children, messages are letters in envelopes, the transport protocol is the parent who multiplexes and demultiplexes letters among siblings, and the network layer is the postal service moving mail between houses.

I studied multiplexing and demultiplexing in depth. At the sender, multiplexing gathers data from multiple sockets and adds transport headers, including port numbers. At the receiver, demultiplexing delivers each segment to the correct socket: UDP uses only the destination port number, while TCP uses the full four-tuple (source IP, source port, destination IP, destination port).

The module highlighted the two principal Internet transport protocols. TCP is connection-oriented and provides reliable in-order delivery, flow control, congestion control, and connection setup via a three-way handshake. UDP is connectionless, provides best effort unreliable delivery with minimal overhead, and requires no handshaking. TCP suits applications needing reliability, such as web browsing, email, and file transfer. UDP suits loss-tolerant but delay-sensitive applications such as streaming media, DNS, and VoIP.

2) Analysing and implementing UDP (3.3)

UDP is a "no frills" protocol with a simple header containing four 16-bit fields: source port, destination port, length, and checksum. I studied the Internet checksum algorithm, where the sender sums 16-bit words using one's complement arithmetic with wraparound and stores the complement as the checksum. This offers only weak error protection since some error patterns may not change the checksum.

In Active Class 4, we explored UDP through Wireshark and Python socket programming. For Wireshark analysis, I captured packets while running applications such as DNS, applied the "udp" filter, examined individual UDP segments to identify

all four header fields and their lengths, verified the Length field includes the 8-byte header, and observed different applications using different port numbers (DNS on port 53, ephemeral client ports varying).

In the programming activity, my partner and I built a UDP client-server chat application in Python. The server created a UDP socket with `socket(AF_INET, SOCK_DGRAM)`, bound it to a port using `bind()`, and used `recvfrom()` to receive messages. The server counted the number of characters, converted the message to uppercase, and replied with both the count and the uppercase string using `sendto()`. The client created its own UDP socket and used `sendto()` to send messages such as "Hello SIT202", then displayed the server's response. This demonstrated UDP's connectionless nature: no explicit connection setup, no guaranteed delivery, no built-in retransmission.

3) Analysing and implementing TCP (3.4)

TCP is a connection-oriented, reliable, full duplex, byte-stream protocol. The TCP segment header contains source and destination ports, sequence number (byte stream number of the first data byte), acknowledgement number (next expected byte), header length, control flags (SYN, ACK, FIN), receive window for flow control, and checksum.

Connection management uses a three-way handshake ($\text{SYN} \rightarrow \text{SYN ACK} \rightarrow \text{ACK}$) to establish connections and a four-way sequence ($\text{FIN} \rightarrow \text{ACK} \rightarrow \text{FIN} \rightarrow \text{ACK}$) to close them gracefully. In Active Class 5, I captured this handshake in Wireshark and analysed the segment order, flags, and sequence numbers.

TCP achieves reliability through byte numbering, cumulative acknowledgements, adaptive timeouts using exponential weighted moving averages ($\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$; $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$), and retransmission mechanisms including fast retransmit triggered by three duplicate ACKs.

4) TCP Flow control and congestion control (3.5, 3.6)

Flow control prevents the sender from overwhelming the receiver. The receiver advertises a receive window (rwnd) indicating available buffer space, and the sender limits unacknowledged data accordingly.

Congestion control prevents network overload. TCP maintains a congestion window (cwnd) and operates in phases: slow start (exponential cwnd growth), congestion avoidance (linear growth by ~ 1 MSS per RTT), and loss response (timeouts trigger aggressive reduction; triple duplicate ACKs trigger cwnd halving).

5) Socket programming in Python (3.7)

In Active Class 5, we adapted our UDP program to TCP. The server created a TCP socket with `socket(AF_INET, SOCK_STREAM)`, `bind()`, `listen()`, and `accept()` to handle connections, then `recv()` and `send()` on the connected socket. The client used `connect()` to initiate the handshake, then `send()` and `recv()`. The server counted characters and returned the uppercase version, reinforcing TCP's connection-oriented reliability versus UDP's connectionless approach.

B. How This Is Useful

This module gives me a complete picture of process-to-process communication across networks, essential for debugging performance issues and understanding protocol behaviour in SIT202. Understanding the trade-offs between UDP and TCP helps me select appropriate transport services for different applications. The Wireshark analysis and Python socket programming develop practical skills in inspecting real traffic and building networked applications, which are core SIT202 learning outcomes. Concepts such as flow control, congestion control, and reliable data transfer provide a foundation for later topics in the unit.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned is how TCP delivers a reliable, ordered byte stream over unreliable IP networks. Before this module, I knew abstractly that "TCP is reliable," but now I understand the mechanisms: three-way handshakes for connection setup, sequence and acknowledgement numbers for ordering, sliding windows for efficiency, adaptive timeouts and fast retransmit for loss recovery, flow control to protect receivers, and congestion control to protect networks. Seeing these in Wireshark traces and implementing them in Python made the concepts concrete.

B. How This Relates to What I Already Know

From earlier SIT202 modules, I understood the layered model, IP addressing, and application layer protocols such as HTTP and DNS. However, I thought of data transmission abstractly as applications "just sending" data that "just arrives." This module filled the critical gap by showing how transport protocols extend best effort IP into practical services that applications need: reliability, ordering, multiplexing, and rate control. The socket programming builds directly on my Python skills, and Wireshark activities connected to my earlier HTTP and DNS analysis by showing

how application messages travel inside transport segments with port numbers and sequence numbers coordinating delivery.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this content because the transport layer bridges applications and network infrastructure, which is essential for understanding, debugging, and designing networked systems throughout SIT202. Many practical problems, such as slow connections, lost packets, or poor performance, originate at the transport layer. The Wireshark analysis and Python socket programming in Active Classes 4 and 5 develop hands-on skills with industry tools and programming that are explicit SIT202 learning outcomes. Finally, mastering these concepts provides a foundation for later unit topics that build on transport layer principles such as reliability, performance, and congestion management.

Overall, this module transformed my view of UDP and TCP from protocol names in diagrams to precise mechanisms I can explain, observe in packet traces, and implement in working programs.

Module 3: Evidence of Learning

This section presents evidence of my learning for Module 3: The Transport Layer, with a focus on User Datagram Protocol (UDP), Transmission Control Protocol (TCP), transport-layer communication, packet analysis, and client-server socket programming. It includes module exercises, active class discussions, Wireshark-based packet analysis, Python socket-programming activities, flowchart design, and supporting screenshots. Together, these items demonstrate my understanding of the differences between connectionless and connection-oriented communication, transport-layer message exchange, port-based communication, and the practical implementation of client-server programs. Short progress notes are included throughout to show how each task contributed to my developing understanding of the module.

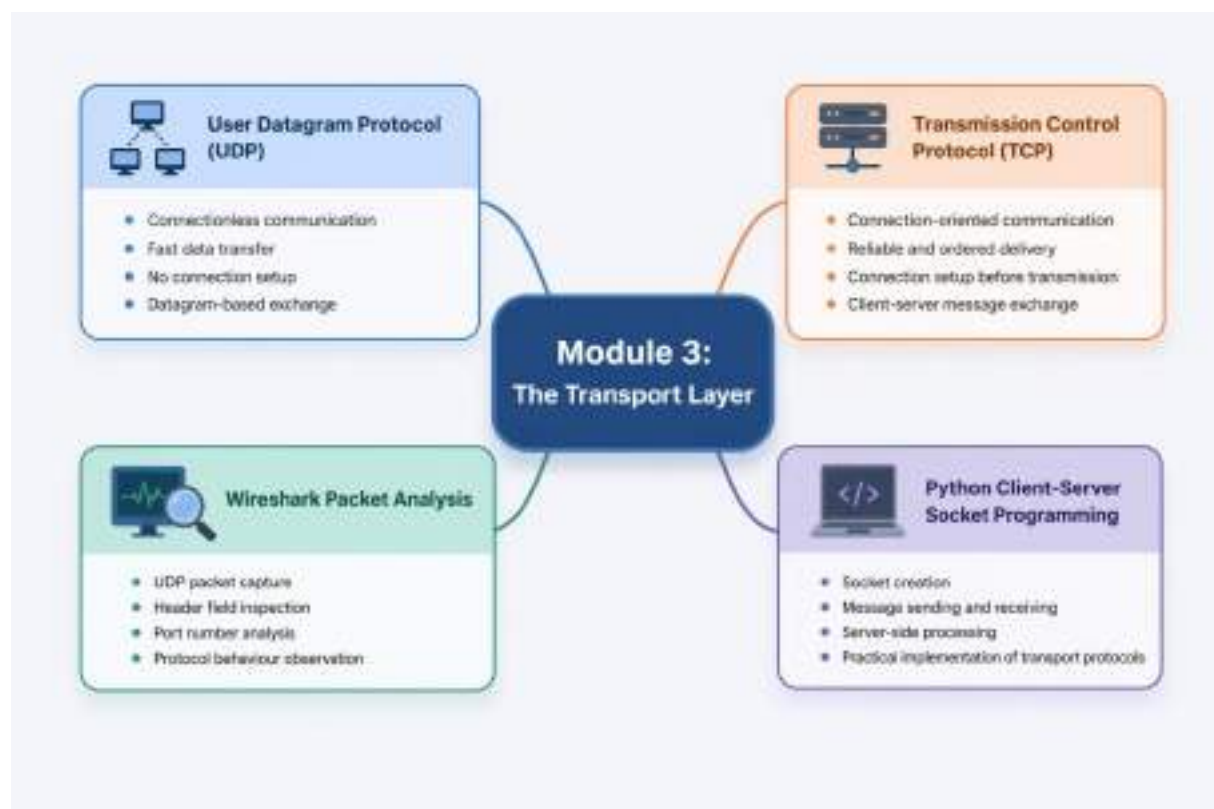


Figure. Overview of the key learning areas covered in **Module 3: The Transport Layer**, including User Datagram Protocol (UDP), Transmission Control Protocol (TCP), Wireshark packet analysis, and Python client-server socket programming.

Evidence: Active Classes Activities

Activities for Active Class 4: UDP - Unreliable Data Protocol?

Activity 1:

1. Group discussion: Assume that you are helping your friend to move his/her house and your responsibility is to take care of moving goods in the kitchen.

- a. What method would you take if your friend needs to pack everything within an hour because the moving truck will be there soon?

Ans. With only one hour available, we adopted a **User Datagram Protocol (UDP)** approach. We would simultaneously grab multiple kitchen items and pack them rapidly into boxes without individual security checks or order verification. Speed was our primary focus, accepting that some fragile items might break during transit or arrive out of sequence. The critical objective was to complete the entire packing process before the truck arrived.

Example: Live video streaming uses UDP because viewers accept occasional dropped frames, but buffering delays destroy the experience.

- b. How would your method change if you have more time, and your friend would like to make sure each item in the kitchen is safely delivered, and your friend wants to keep those in the same order as they were in the previous house?

Ans. With unlimited time and strict safety requirements, we chose a **Transmission Control Protocol (TCP)** approach. We would work sequentially: carefully wrap one plate at a time, place it in a numbered box, confirm secure positioning, then proceed to the next item. Each box would carry precise labels showing its original kitchen location (shelf 1, position 3) to ensure perfect layout reconstruction upon arrival.

Example: Email file attachments require TCP because every byte must arrive correctly and in sequence; corruption makes documents unusable.

- c. Now, can you relate the above scenarios to the well-known two transport layer protocols? Consider different scenarios, including large file transfer and faster communication.

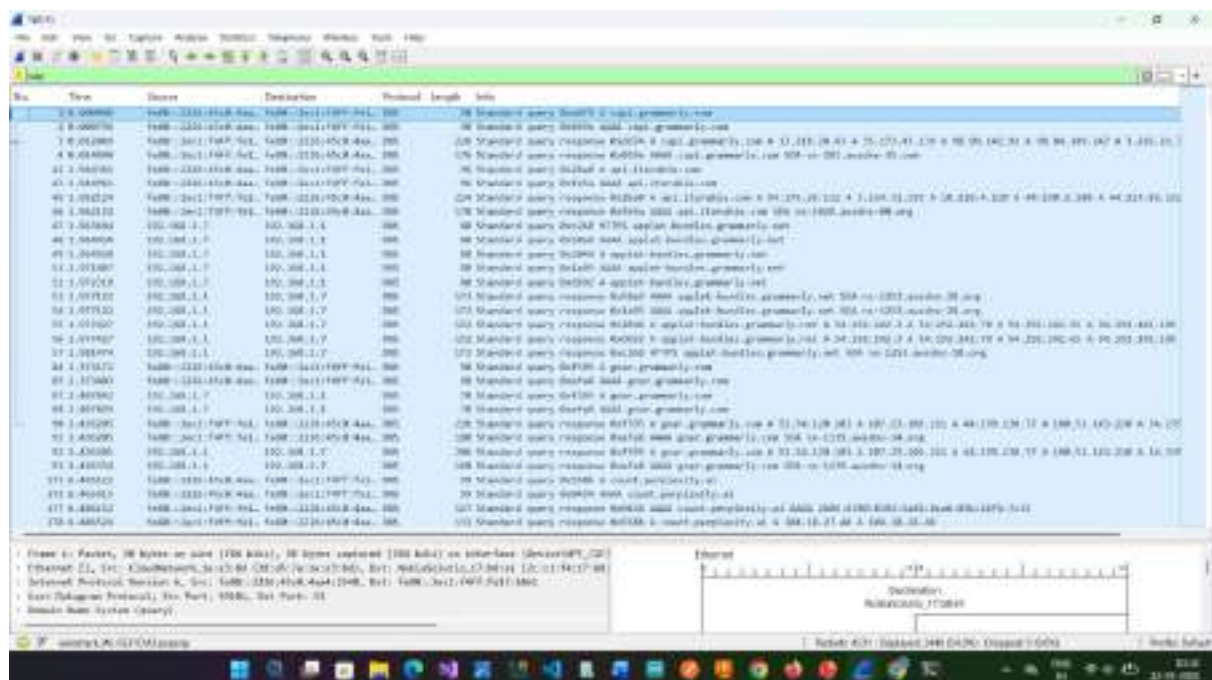
Ans. Our kitchen packing scenarios perfectly illustrate **User Datagram Protocol (UDP)** and **Transmission Control Protocol (TCP)** characteristics. Time-critical packing aligns with UDP's speed focus, while safety-focused packing aligns with TCP's reliability focus:

Scenario	Protocol	Reason
Large file transfer	TCP	Every packet must arrive complete and ordered. Example: Software downloads.
Faster communication	UDP	No connection setup or acknowledgement delays. Example: Online gaming position updates.

TCP ensures reliable delivery through connection establishment, sequence numbering, acknowledgements, and retransmissions, whereas UDP maximises speed through connectionless operation without delivery guarantees.

Activity 2:

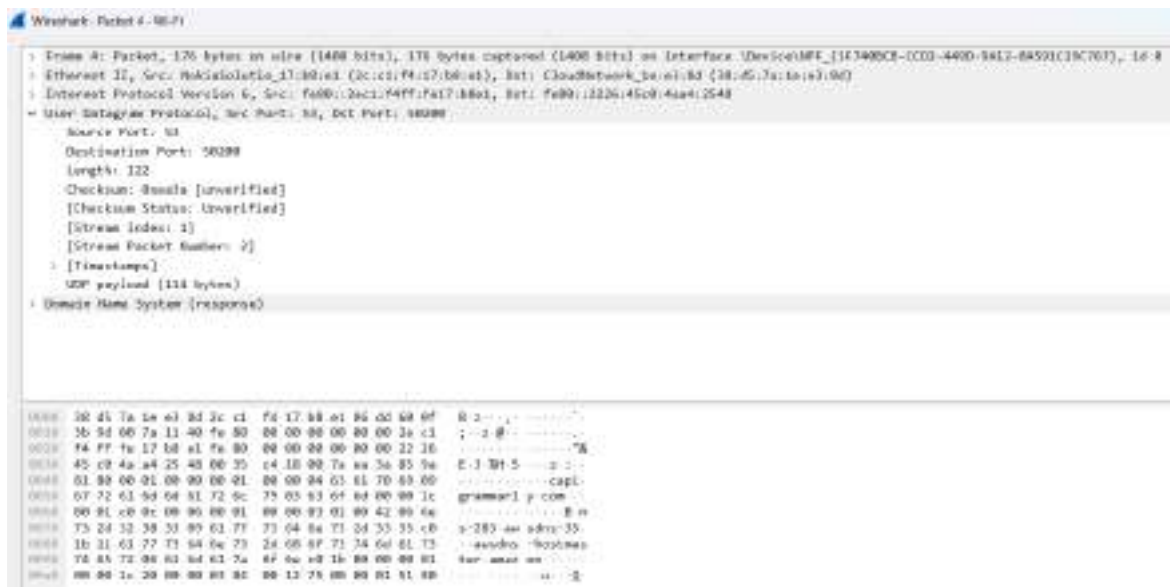
1. Open Wireshark to capture packets, open the web browser and use an application that uses UDP (perhaps DNS). Stop the packet capture.
2. Use the filter to display UDP packets (it is UDP segments).



3. Now you are ready to analyse UDP. The following questions can guide you to conduct your analysis. Make sure that you take enough screenshots and notes. You will use those to prepare your task submissions.
- a. Pick one UDP packet from the filtered packets. Examine the number of fields that are in the header of the selected UDP packet. You can take a screenshot of the packet, name and explain the fields in the UDP packet.

Ans. UDP header fields

For this analysis, I selected **Packet No. 4** from the UDP-filtered capture, which is a DNS response from the DNS server (source port 53) to my host on ephemeral destination port 50200. The UDP section shows Length = 122 and UDP payload (114 bytes) in the packet details pane.



UDP header is **fixed at 8 bytes** and contains **exactly four 16-bit fields**. Each field serves a specific purpose in the lightweight, connectionless transport:

Source Port (16 bits = 2 bytes): Identifies the sending port on the source host. In Packet 4, the value **53** indicates this datagram originates from the DNS server process, as 53 is the well-known port assigned to DNS services.

Destination Port (16 bits = 2 bytes): Identifies the receiving port on the destination host, enabling the operating system to deliver the datagram to the correct application socket. In Packet 4, the value **50200** represents an ephemeral port automatically assigned by my operating system for this DNS transaction.

Length (16 bits = 2 bytes): Specifies the total length of the UDP datagram in bytes, including both the 8-byte UDP header and the UDP payload. In Packet 4,

the value **122** represents the complete datagram size, and Wireshark confirms this by displaying "UDP payload (114 bytes)" separately, proving that $122 = 8 + 114$.

Checksum (16 bits = 2 bytes): Provides error detection capability covering a pseudo-header constructed from the IP header, the UDP header itself, and the UDP payload. In Packet 4, the value **0xea3a** enables detection of transmission errors. This checksum is optional in IPv4 but mandatory in IPv6.

Field	Size	Purpose
Source Port	2 bytes	Sender port number
Destination Port	2 bytes	Receiver port number
Length	2 bytes	UDP header + payload length in bytes
Checksum	2 bytes	Error detection over pseudo-header, UDP header and payload

- b. Can you identify the length of each UDP header field from this UDP packet? You can indicate the length in bytes. You may want to examine the displayed information in Wireshark's packet content field.

Ans. Length of each UDP header field

Using **Packet No. 4** (DNS response, port 53→50200, Length=122), every UDP header field measures **exactly 16 bits, which equals 2 bytes**. The Packet Bytes pane in Wireshark allows verification of this structure by highlighting the first 8 bytes after the IP header, which correspond to these four fields in network byte order (big-endian format):

Field	Size
Source Port	2 bytes
Destination Port	2 bytes
Length	2 bytes
Checksum	2 bytes

Field	Size
Total UDP Header	8 bytes

- c. Can you explain the value in the Length field? What exactly does this value indicate? You can verify your answer by examining captured UDP packets.

Ans. Meaning of the Length field value

Using **Packet No. 4** (DNS response, port 53→50200, Length=122), the Length field value **122** indicates the **total size of the UDP datagram in bytes**, calculated as the 8-byte UDP header plus the UDP payload.

Wireshark provides verification by displaying a separate line showing "UDP payload (114 bytes)" under the UDP header details. This confirms the calculation: **122 = 8 (header) + 114 (payload)**. The Length field always includes both components, never the payload alone.

An additional verification method uses IP header arithmetic: the IP Total Length field minus the IP header length (typically 20 bytes for IPv4) should equal the UDP Length value of 122, demonstrating consistency across protocol layers.

- d. Is there a maximum number of bytes that we can include in the UDP payload?

Ans. Maximum number of bytes in UDP payload

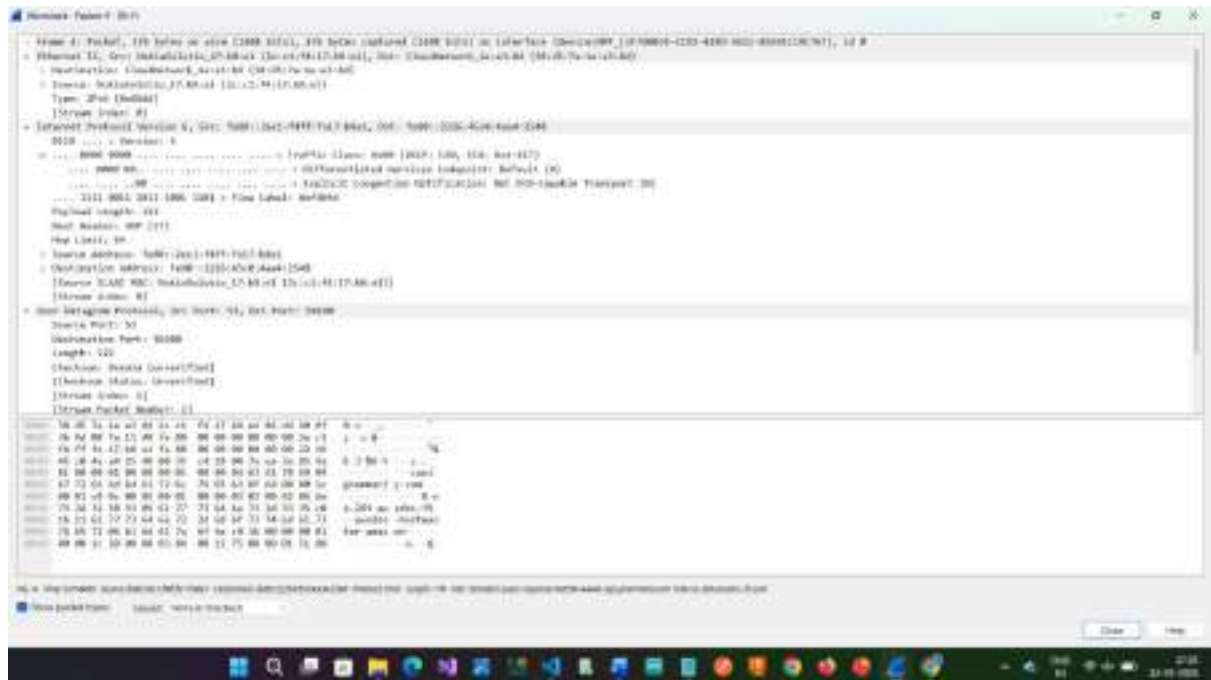
Using **Packet No. 4** (DNS response, port 53→50200, Length=122), the maximum UDP payload size can be determined through the 16-bit Length field constraint.

Since the Length field is 16 bits, the **maximum UDP datagram size** (header + payload) is **$2^{16} - 1 = 65,535$ bytes**. Subtracting the fixed 8-byte UDP header gives a **theoretical maximum payload of 65,527 bytes**.

However, when operating over IPv4, which adds a minimum 20-byte IP header, the **practical maximum payload becomes 65,507 bytes** (calculated as 65,535 - 20 - 8). In real-world networks, the usable payload size is typically much smaller due to path MTU limitations (commonly 1500 bytes for Ethernet), which restrict UDP payloads to approximately 1472 bytes to avoid IP fragmentation.

e. Can you identify the protocol number given for UDP?

Ans. **Protocol number given for UDP**



Using **Packet No. 4** (DNS response, port 53→50200, Length=122), the IP header contains a Protocol field (in IPv4) or Next Header field (in IPv6) that identifies the transport-layer protocol.

UDP is assigned **protocol number 17 (decimal), which is 0x11 in hexadecimal**, as defined by IANA. Wireshark displays this in the IP header details as "Protocol: UDP (17)". This numeric identifier enables the IP layer to recognise the enclosed payload as a UDP datagram and deliver it to the appropriate UDP protocol stack for processing, rather than to TCP, ICMP, or other transport protocols.

f. Examine two consecutive UDP packets your host sends/receives. Explain the relationship between the port numbers in these two packets.

Ans. **Relationship between port numbers in two consecutive UDP packets**

Using **Packet No. 4** (DNS response, port 53→50200, Length=122) along with its corresponding query in Packet 3, the port relationship between consecutive request and response packets demonstrates a clear pattern.

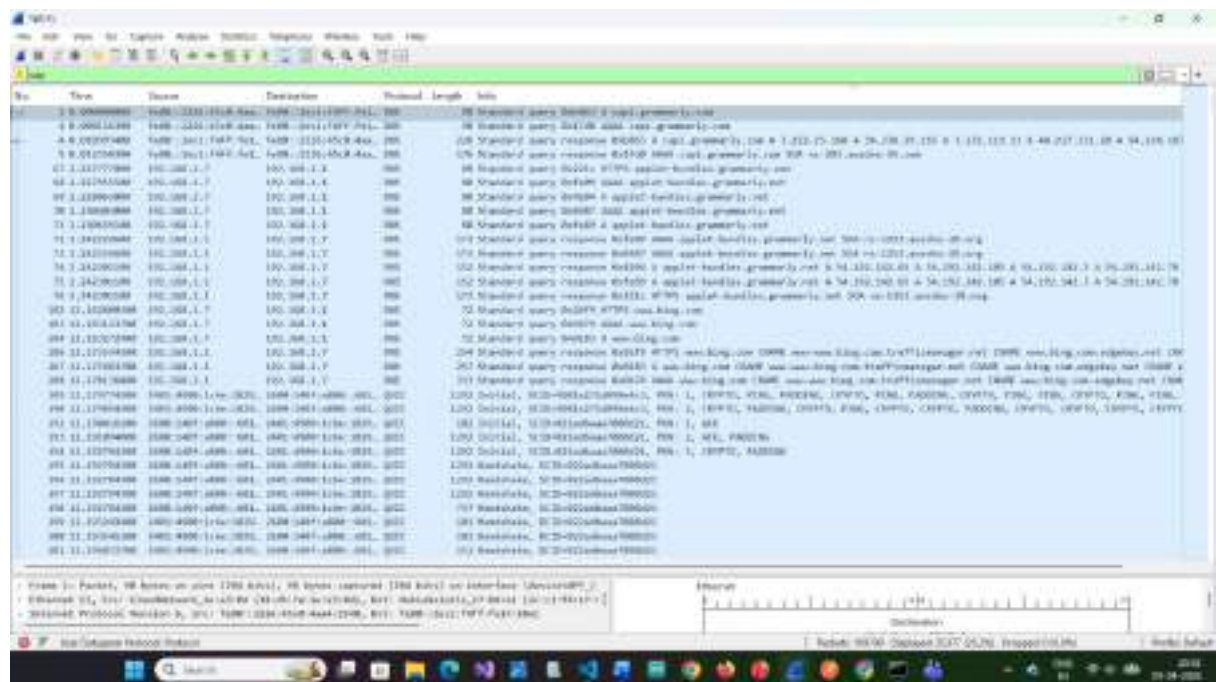
The **DNS query (Packet 3)** flows from my host using **source port 50200** (ephemeral client port) to the DNS server on **destination port 53** (well-known DNS port).

The **DNS response (Packet 4)** reverses this configuration: the server uses **source port 53** while directing the response back to my host on **destination port 50200** (the same ephemeral port).

This pattern shows that the **client ephemeral port remains constant** throughout the transaction, while **source and destination roles swap** between request and response. Combined with source and destination IP addresses and the protocol number, these port numbers form a unique **5-tuple** (protocol, source IP, source port, destination IP, destination port) that identifies the complete bidirectional communication flow and enables the operating system to deliver incoming datagrams to the correct application socket.

- g. Clear the cache and run another application, such as MS Teams, while using Web browsing. Capture packets of these two applications. Now you can analyse the captured UDP packets again. Do all UDP packets captured in Wireshark use the same port number? Explain your answer.

Ans. **UDP port numbers across different applications**



After clearing the cache and capturing UDP traffic from web browsing and another application, the UDP packets captured in Wireshark do **not** all use the same port number.

This is because different applications and network services use **different source and destination port numbers** to identify their communication processes at the transport layer.

For example, DNS traffic generated from web browsing commonly uses **destination port 53**, while the client device uses a temporary **ephemeral**

source port assigned by the operating system for that session.

In the capture, the packet list shows different types of UDP-based traffic, including **DNS** and **QUIC**, indicating that multiple services are using UDP rather than a single common port number.

Therefore, UDP packets in the capture can have different source and destination port numbers, and this variation allows the operating system to distinguish between multiple applications and deliver each datagram to the correct application socket.

Activity 3:

In this activity, you will learn about data transport with UDP. You will examine socket programming with UDP using Python to build the following simple client-server application. You can partner up with a group member to build this simple client-server application. We use PyCharm IDE. If you have not downloaded PyCharm yet, please download PyCharm via the following link and install it on your laptop/ personal computer.

<https://www.jetbrains.com/pycharm-edu/>

We have provided more information on Python socket programming in Module 3. Please go through the relevant content before you attempt this activity.

You will build the following simple client-server chat application using Python.

- The client should be able to send a message (for example, "Hello SIT202") to the server.
- Once the server receives the full message, it counts the number of characters it receives and displays the number on its terminal.
- The server responds to the client by sending the number of characters it received, along with the same message with uppercase letters.
- The client should receive the number and the message sent by the server.
- The client shows the received message on its terminal.

1. First, you need to draw a diagram to explain the operation of the client- server program that uses UDP.

Ans. **Operation of the User Datagram Protocol (UDP) client-server program**

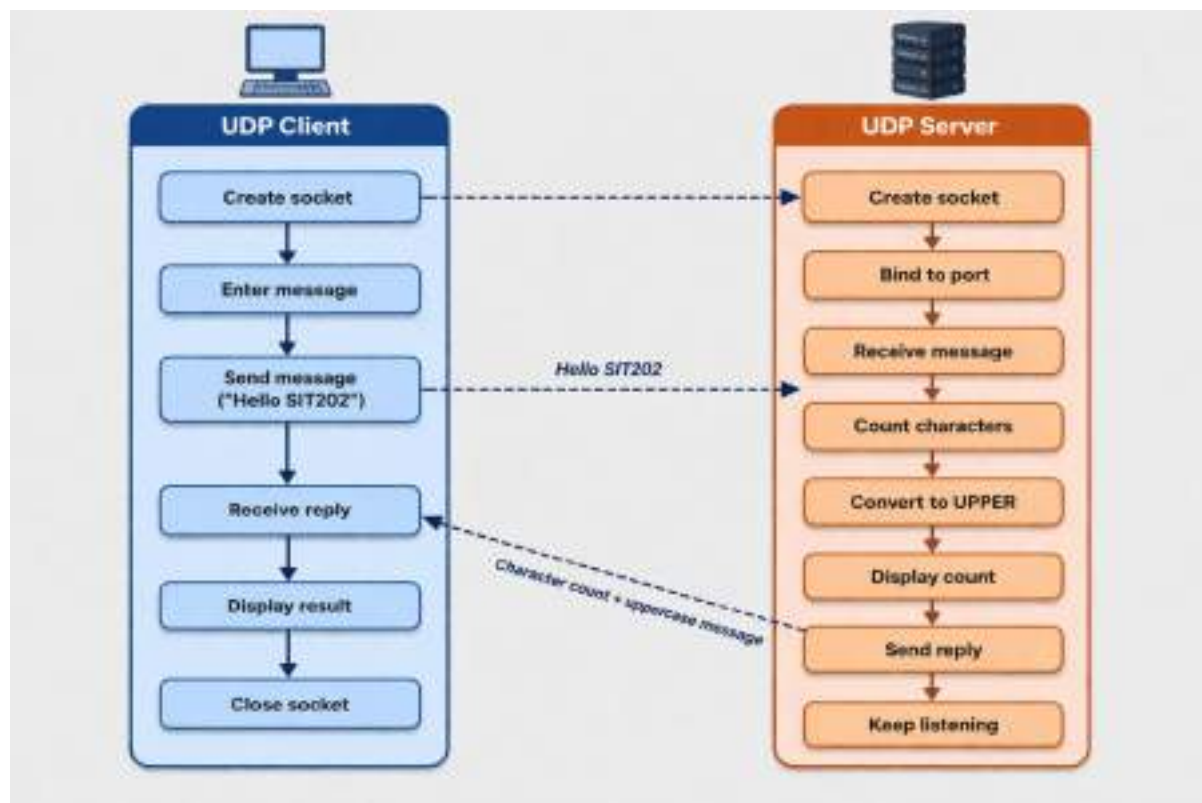


Figure. Flowchart showing the operation of the User Datagram Protocol (UDP) client-server application in Python.

The diagram illustrates the operation of a simple **User Datagram Protocol (UDP)** client-server application developed in Python to demonstrate transport-layer communication between two end systems.

On the **client side**, the process begins with the creation of a UDP socket, after which the user enters a message, and the client sends "**Hello SIT202**" to the server.

On the **server side**, the program creates its own UDP socket, binds it to a port, receives the client message, counts the number of characters in the received text, converts the same message to uppercase, and displays the character count on the server terminal.

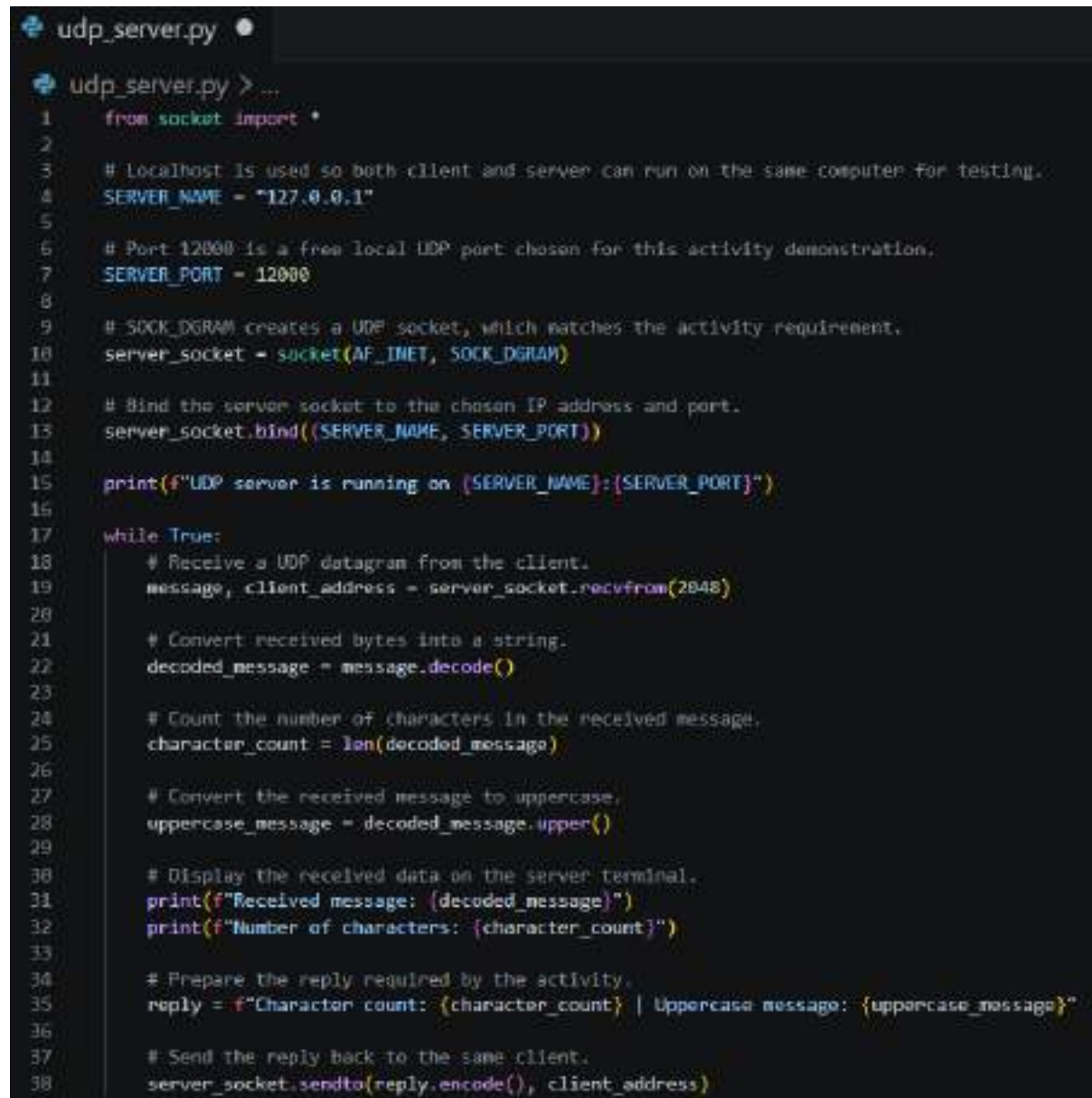
After processing the message, the server sends a reply back to the client containing the **character count** and the **uppercase version** of the original message.

The client then receives this reply, displays the returned result on its terminal, and closes its socket after the communication is complete, while the server remains available to keep listening for further requests.

This flow clearly demonstrates the **connectionless** communication model of UDP, where data is exchanged directly between client and server without connection establishment, acknowledgements, or retransmission mechanisms.

2. Then one of you can develop the client side and the other can develop the server side of the program (you can also develop both end systems and demonstrate the outcome if you choose to do so)

Ans. **Development of the client and server sides of the UDP program**

A screenshot of a code editor showing a Python script named 'udp_server.py'. The script is designed to act as a UDP server. It imports the 'socket' module and sets the server name to '127.0.0.1' and the server port to 12000. It creates a UDP socket using 'socket(AF_INET, SOCK_DGRAM)', binds it to the specified IP and port, and prints a message indicating the server is running. A 'while True' loop handles incoming data: it receives a datagram, decodes it, counts the characters, converts it to uppercase, and sends a formatted response back to the client.

```
udp_server.py
udp_server.py > ...
1  from socket import *
2
3  # localhost is used so both client and server can run on the same computer for testing.
4  SERVER_NAME = "127.0.0.1"
5
6  # Port 12000 is a free local UDP port chosen for this activity demonstration.
7  SERVER_PORT = 12000
8
9  # SOCK_DGRAM creates a UDP socket, which matches the activity requirement.
10 server_socket = socket(AF_INET, SOCK_DGRAM)
11
12 # Bind the server socket to the chosen IP address and port.
13 server_socket.bind((SERVER_NAME, SERVER_PORT))
14
15 print(f"UDP server is running on {SERVER_NAME}:{SERVER_PORT}")
16
17 while True:
18     # Receive a UDP datagram from the client.
19     message, client_address = server_socket.recvfrom(2048)
20
21     # Convert received bytes into a string.
22     decoded_message = message.decode()
23
24     # Count the number of characters in the received message.
25     character_count = len(decoded_message)
26
27     # Convert the received message to uppercase.
28     uppercase_message = decoded_message.upper()
29
30     # Display the received data on the server terminal.
31     print(f"Received message: {decoded_message}")
32     print(f"Number of characters: {character_count}")
33
34     # Prepare the reply required by the activity.
35     reply = f"Character count: {character_count} | Uppercase message: {uppercase_message}"
36
37     # Send the reply back to the same client.
38     server_socket.sendto(reply.encode(), client_address)
```

Figure. Python code for the UDP server program

A screenshot of a code editor showing a Python script named 'udp_client.py'. The script is designed to act as a UDP client. It imports the 'socket' module and sets the server name to '127.0.0.1' and the server port to 12000, matching the server's configuration.

```
udp_client.py
udp_client.py > ...
1  from socket import *
2
3  # localhost is used because the server is running on the same computer.
4  SERVER_NAME = "127.0.0.1"
5
6  # This port must match the server port exactly.
7  SERVER_PORT = 12000
```

```
8
9  # SOCK_DGRAM creates a UDP socket for the client.
10 client_socket = socket(AF_INET, SOCK_DGRAM)
11
12 # Take input from the user and send it to the server.
13 message = input("Enter message to send to server: ")
14 client_socket.sendto(message.encode(), (SERVER_NAME, SERVER_PORT))
15
16 # Receive the reply from the server.
17 server_reply, server_address = client_socket.recvfrom(2048)
18
19 # Display the returned character count and uppercase message.
20 print("Reply from server:")
21 print(server_reply.decode())
22
23 # Close the client socket after communication is complete.
24 client_socket.close()
```

Figure. Python code for the UDP client program

For this activity, both the **client side** and the **server side** of the **User Datagram Protocol (UDP)** program were developed in Python to demonstrate transport-layer communication between two end systems.

The **client program** was designed to create a UDP socket, accept a message input from the user, send the message to the server, receive the processed reply, and display the returned result on the client terminal.

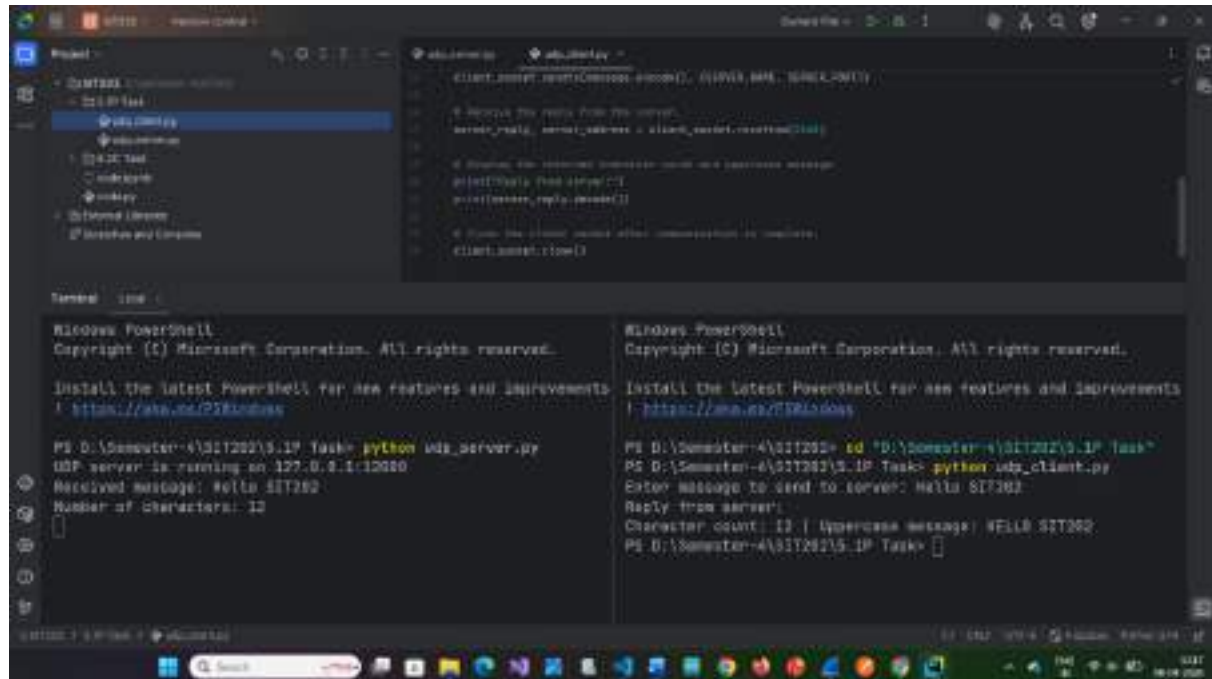
The **server program** was designed to create a UDP socket, bind to a selected port, receive the client message, count the number of characters in the received text, convert the same message to uppercase, display the character count, and send the processed response back to the client.

This implementation satisfies the activity requirement that the client sends a message such as “**Hello SIT202**”, while the server returns both the **character count** and the **uppercase version** of the same message.

Progress note: Both end systems of the **User Datagram Protocol (UDP)** application were successfully developed, with the server performing message processing and the client handling message transmission and response display.

3. You need to show the output of the client- server program to demonstrate that your program works as expected. Please make sure to include the server and client codes, as well as the output (screenshots), in your Module 3 lesson review.

Ans. **Output of the client-server program**



The screenshot displays a Windows IDE with a project named 'SIT202'. The file explorer on the left shows a folder 'udp' containing 'udp_server.py' and 'udp_client.py'. The main editor window shows the code for 'udp_server.py'. Below the editor, two terminal windows are open, showing the execution of the programs.

```
udp_server.py
1 client_socket, server_address = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
2
3 # Receives the data from the server
4 server_addr, server_address = client_socket.recvfrom(1024)
5
6 # Displays the received incoming data and processes message
7 print('Data from client:')
8 print(server_addr, server_address)
9
10 # Prints the character count after connection is complete
11 client_socket.close()
```

Terminal 1 (Left):

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows

PS D:\Semester-4\SIT202\5.1P Task> python udp_server.py
UDP server is running on 127.0.0.1:12000
Received message: Hello SIT202
Number of characters: 12
```

Terminal 2 (Right):

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindows

PS D:\Semester-4\SIT202> cd "D:\Semester-4\SIT202\5.1P Task"
PS D:\Semester-4\SIT202\5.1P Task> python udp_client.py
Enter message to send to server: Hello SIT202
Reply from server:
Character count: 12 | Uppercase message: HELLO SIT202
PS D:\Semester-4\SIT202\5.1P Task>
```

Figure. Output of the UDP client-server program showing successful message transmission, processing, and reply display

The output of the **User Datagram Protocol (UDP)** client-server program confirms that the application works as expected.

During execution, the **client** sends the message “**Hello SIT202**” to the server, and the **server** successfully receives the message, counts the number of characters, converts the text to uppercase, and displays the character count on its terminal.

The server then returns a reply containing the **character count** and the **uppercase version** of the original message, which is received and displayed by the client.

The output screenshot, therefore, verifies successful two-way UDP communication between the client and server and confirms that the implemented program satisfies the required task behaviour.

Progress note: The successful output demonstrates that the **User Datagram Protocol (UDP)** client and server programs were correctly implemented and that the server-side processing and client-side response handling operated as intended.

Activities for Active Class 5: How can I transport my application data reliably?

Activity 1:

For this group discussion, our group was divided into two server-client pairs to make the protocol design clear and realistic. Member 1 (Server A) and Member 2 (Client A) formed the first pair, while Member 3 (Server B) and Member 4 (Client B) formed the second pair. Pair A was used to design and explain the protocol in detail, while Pair B was used for comparison and validation of the final design.

1. Assume one of your group members sent you a message. However, you have never received it. What went wrong? How do you make sure that the message is properly received?

Ans. If a message was sent but never received, several problems may have occurred during transmission. The message may have been lost in the network, corrupted during transfer, delayed for too long, sent to the wrong destination, or discarded because the receiver was not ready to process it. In an unreliable communication method, the sender may also assume delivery without receiving any confirmation from the receiver.

To make sure that the message is properly received, the communication process must include a reliability mechanism. The sender should transmit the message and wait for an acknowledgement from the receiver. If no acknowledgement is received within a specified timeout period, the sender should retransmit the message. This ensures that missing data is resent and that successful delivery is explicitly confirmed rather than assumed.

2. Assume you are trying to access a web page containing an image. Now, your job is to make sure that you get the image properly. The image cannot be received in one packet, and it will be broken down into 10 packets.

Ans. If the image is divided into 10 packets, the client must receive all 10 packets correctly before the full image can be reconstructed. The main risks are packet loss, out-of-order delivery, duplicated packets, and packet corruption. Even if only one packet is missing, the image may become incomplete or unusable.

To receive the image properly, each packet should be given a sequence number from 1 to 10. The client should check the sequence numbers as packets arrive, store them in the correct order, and send acknowledgements for correctly received packets. If a packet is missing or damaged, the client should notify the server or the server should retransmit the packet after a timeout. In this way, the image is accepted only when all 10 packets have been received correctly and assembled in order.

3. Partner up with one of your group members and develop a transport layer protocol to guarantee that the image is properly received by the client. One can act as the server where the webpage/image is stored, and the other can act as the client who wants to receive the image (this means your group has 2 pairs of server-client networks).

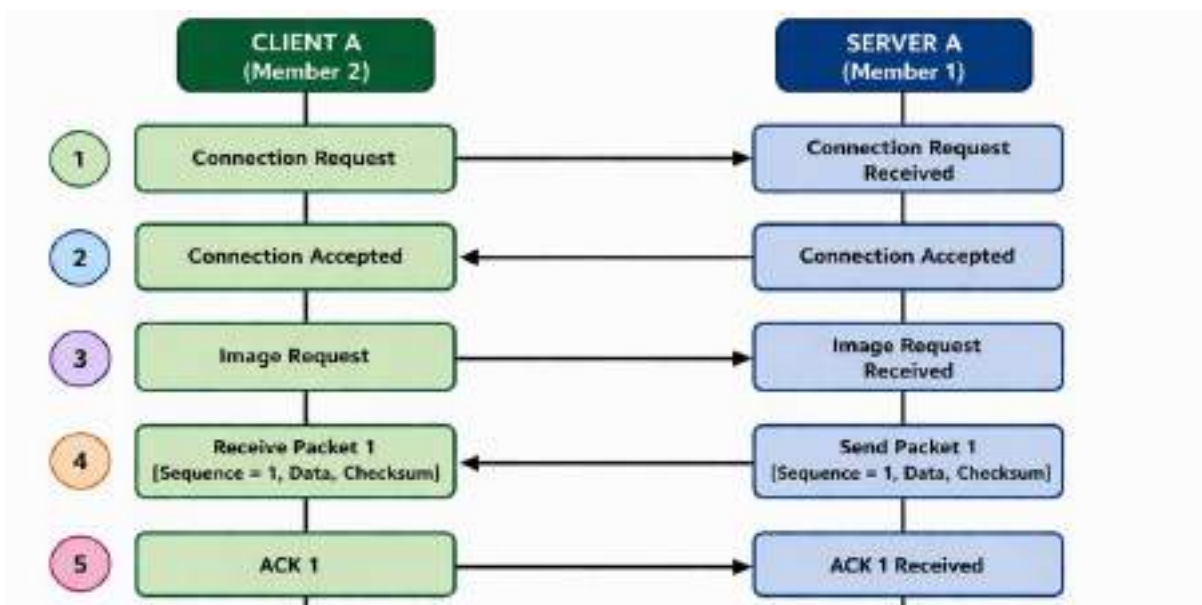
Ans. In Pair A, **Member 1 (Server A)** acted as the server storing the image, and **Member 2 (Client A)** acted as the client requesting the image. We designed a simple reliable transport protocol in which the server sends the image as 10 numbered packets, and the client acknowledges each correctly received packet. Each packet contains a **sequence number**, the **data**, and a basic **error-checking value** such as a checksum.

The protocol works as follows. First, Client A sends a request for the image. Server A then sends Packet 1, waits for **ACK 1**, and only after receiving that acknowledgement sends Packet 2. This process continues until Packet 10 is sent and acknowledged. If any acknowledgement is not received within the timeout period, Server A retransmits the missing packet. This design guarantees that the client receives all packets completely and in the correct order before reconstructing the image.

4. Write down the protocol that you would use, including the communication between the client and server and all the messages that you are passing.

Ans. The protocol designed by Pair A is shown below.

The communication steps are: Connection Request, Connection Accepted, Image Request, Packet 1 and ACK 1, Packet 2 and ACK 2, continuation until Packet 10, timeout-based retransmission if an ACK is not received, End of Image, and Final Acknowledgement.



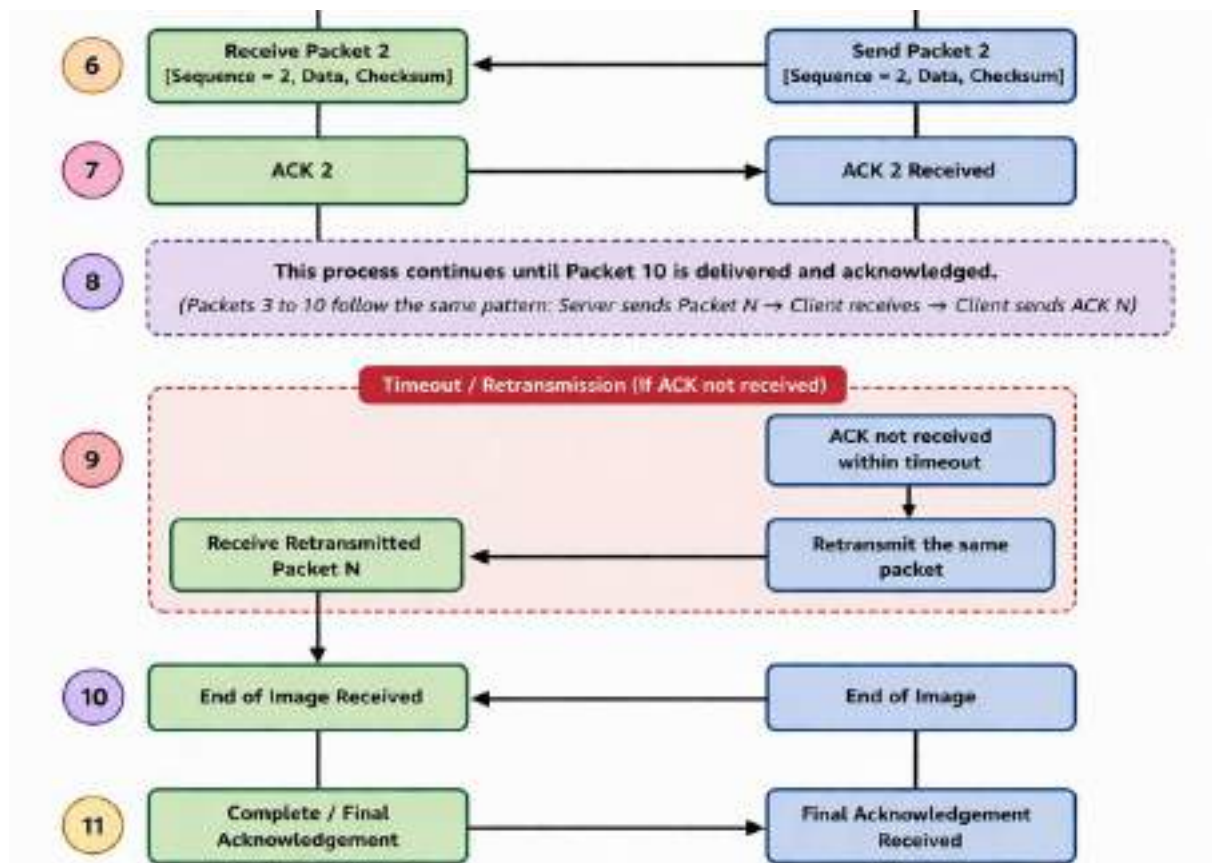


Figure. Communication protocol between Member 2 (Client A) and Member 1 (Server A) for reliable image transfer

This protocol ensures that every packet is individually tracked and confirmed. The sequence number helps the client place packets in the correct order, while the acknowledgement system ensures that the server knows whether each packet has been successfully received.

5. How do you guarantee that your protocol does the job as you expected?

Ans. The protocol guarantees reliable delivery through four main mechanisms: **sequence numbers**, **acknowledgements**, **timeouts**, and **retransmissions**. Sequence numbers ensure that packets can be identified and reordered correctly, acknowledgements confirm successful delivery, timeouts detect missing responses, and retransmissions recover lost packets.

In addition, error detection can be performed using a checksum so that corrupted packets are not accepted as valid. The client only reconstructs the image after all packets from 1 to 10 have been received correctly. Because the sender does not move on permanently from a packet until it has been acknowledged, the protocol provides a strong guarantee that the image will arrive completely and in order.

6. Once you have finalised your protocol, check how other group members developed their protocols.

Ans. After comparing our protocol with the protocol developed by **Member 3 (Server B)** and **Member 4 (Client B)**, we found that the core ideas were similar. Both designs used packet numbering and acknowledgements to improve reliability. This shows that the group correctly recognised that successful delivery requires confirmation from the receiver rather than assuming that transmission alone is sufficient.

However, there were some design differences. One version relied only on acknowledgements and retransmission after timeout, while the other also considered error detection and explicit completion messages. Our final version was stronger because it combined sequence numbers, acknowledgements, timeout-based retransmission, and an end-of-transfer confirmation. This made the communication process clearer and more robust.

7. With your group, discuss whether your protocol is valid when you need to access a large file that might be broken down into 1000 packets. If not, what are the changes you would like to make in your protocol?

Ans. Our protocol works correctly for 10 packets, but it becomes inefficient for a large file containing 1000 packets. The main limitation is that it behaves like a **stop-and-wait** protocol, where the sender waits for an acknowledgement after every single packet before sending the next one. Although reliable, this method would be slow and would waste network capacity when transferring a much larger file.

To improve the protocol for 1000 packets, we would introduce a **sliding window** mechanism so that multiple packets can be sent before waiting for acknowledgements. We would also use **cumulative acknowledgements** or **selective retransmission** so that only missing packets need to be resent instead of repeating everything unnecessarily. In addition, larger transfers would benefit from flow control and congestion control so that the sender does not overwhelm the receiver or the network. With these changes, the protocol becomes much more practical for large-scale, reliable data transfer and begins to resemble the behaviour of TCP.

Activity 2:

Now you are familiar with Wireshark. Therefore, you should be able to analyse protocols and packets with fewer instructions provided. Today, you are going to analyse TCP.

1. You can analyse the TCP three-way handshake in Wireshark packet capture. Analyse the order of segments sent/received by two end systems to establish a TCP connection.

Ans. For this activity, Wireshark was used to capture web traffic to <http://portquiz.net> and analyse the TCP connection establishment, TCP header fields, and the related HTTP request and response. The HTTP filter was first used to identify the web request, and the selected packet was then used to follow the corresponding TCP stream. Following a TCP stream isolates one conversation and makes the packet sequence easier to analyse.

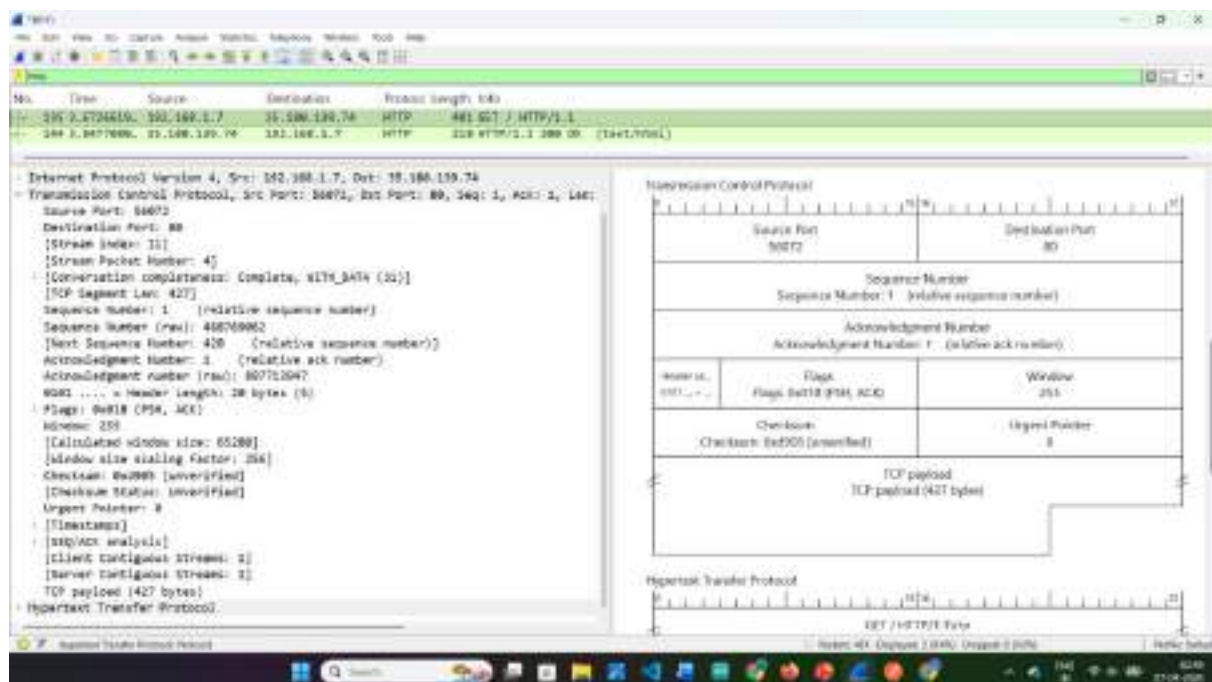


Figure. HTTP packet list used to identify the web request before following the TCP stream

In the filtered stream, the first three packets showed the standard TCP three-way handshake: **SYN**, **SYN-ACK**, and **ACK**. TCP uses these three packets to establish a connection before any application data is exchanged.

In this capture, packet **128** was a SYN from **192.168.1.7:56072** to **35.180.139.74:80**, packet **133** was a SYN-ACK from the server, and packet **134** was the final ACK from the client. This confirmed that the connection was successfully established before the HTTP request was sent.

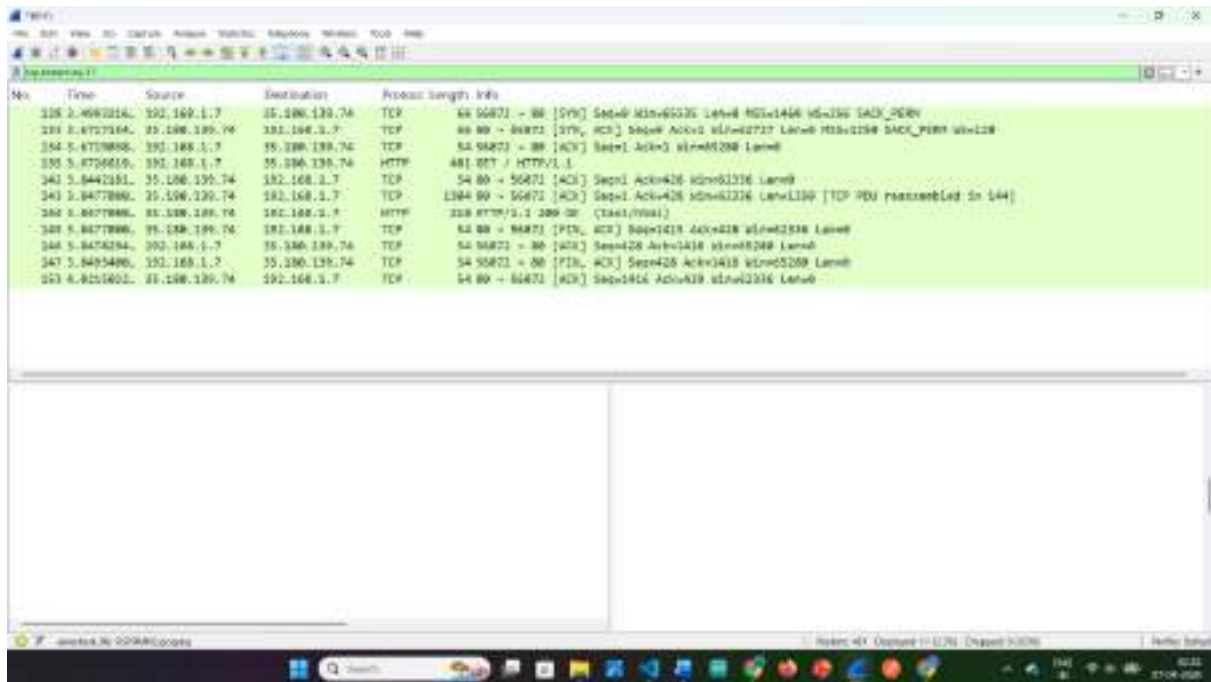


Figure. Filtered TCP stream showing SYN, SYN-ACK, ACK, HTTP request, HTTP response, and connection termination

2. Analyse the segment headers used, their purpose and sizes.

Ans. For TCP header analysis, the selected packet was **packet 135**, which carried the **HTTP GET request** after the TCP connection had already been established. HTTP traffic is carried over TCP, and after the three-way handshake, the client sends its request as application data within a TCP segment.

A TCP header contains fields including **source port, destination port, sequence number, acknowledgement number, data offset (header length), flags, window size, checksum, and urgent pointer**. The **minimum TCP header size is 20 bytes**, and it can increase when optional fields are present.

In this capture, the TCP details showed **source port 56072, destination port 80, sequence number 1, acknowledgement number 1, header length 20 bytes, flags PSH and ACK, window value 255, checksum 0xd905**, and a **TCP payload of 427 bytes**. The **source port** identifies the client-side application process, while the **destination port 80** identifies the web server service.

The **sequence number** is used to track the bytes sent in the TCP stream, and the **acknowledgement number** confirms the next byte expected from the other side. The **ACK** flag indicates that the segment acknowledges previously received data, and in an established TCP connection, normal packets generally include an ACK.

The **PSH** flag indicates that the sender wants the data delivered to the receiving application promptly rather than waiting in the buffer. The **window size** supports TCP

flow control, the **checksum** provides error detection, and the **urgent pointer** is used only when urgent data is present.

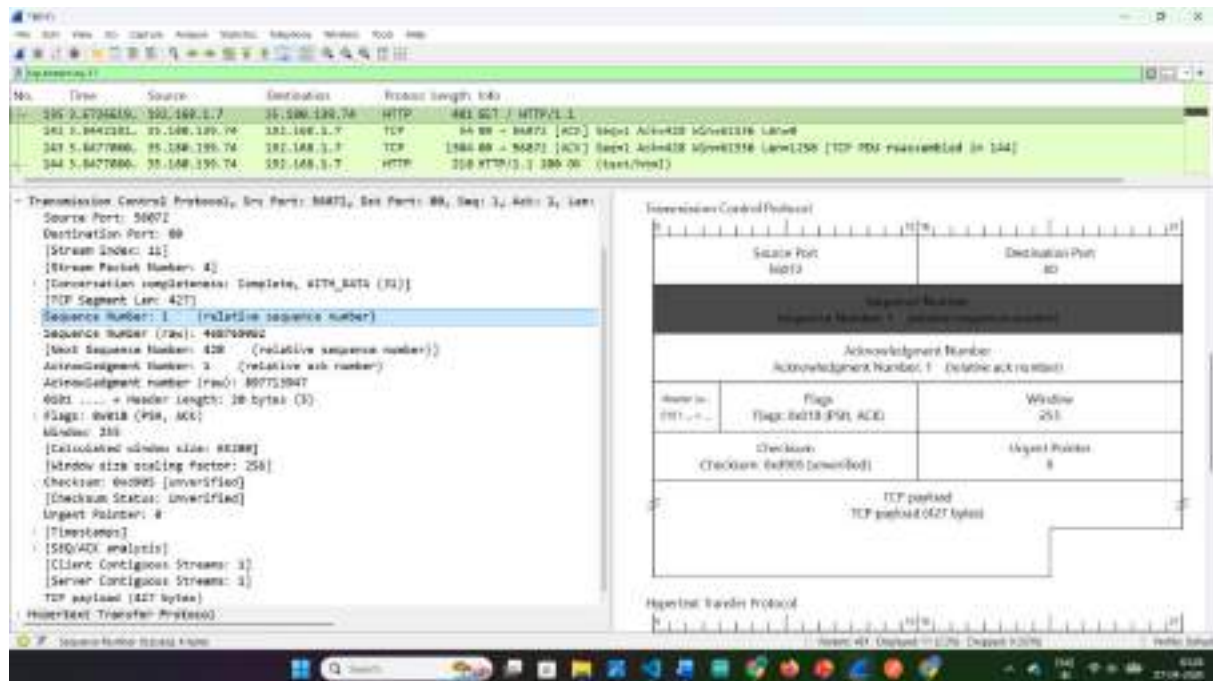


Figure. Transmission Control Protocol header details of packet 135 showing source port, destination port, sequence number, acknowledgement number, flags, window size, checksum, and payload length in Wireshark

3. Make sure you take the screenshots of TCP segments that will be used for your task submissions.

Ans. Screenshots of the captured TCP segments have been included in this activity to support the analysis of the communication with <http://portquiz.net>. These screenshots show the important steps in the TCP exchange, including identification of the HTTP request, the filtered TCP stream, the three-way handshake, the HTTP request and response, and the TCP header details of the selected packet.

Activity 3:

In this activity, you will build a simple client-server chat application that you built in Active Class 4, but now using TCP. TCP is a connection-oriented protocol, while UDP is a connectionless protocol. Therefore, in TCP, the client and server need to handshake first and establish a TCP connection before they transmit data to each other.

You can partner up with a group member to build this simple client-server application. It would be good if you could partner up with the same person that you have done Activity 1.

You will build the following simple client-server application using Python.

- The client should be able to send a message (for example, "Hello SIT202") to the server.
- Once the server receives the full message, it counts the number of characters it receives and displays the number in the terminal.
- The server responds to the client by sending the number of characters it received and the same message with uppercase letters.
- The client should receive the number and the message sent by the server.
- The client shows the received message on its terminal.

1. First, you need to draw a diagram to explain the operation and communication of the client- server program that uses TCP.

Ans. **Operation of the Transmission Control Protocol (TCP) client-server program**

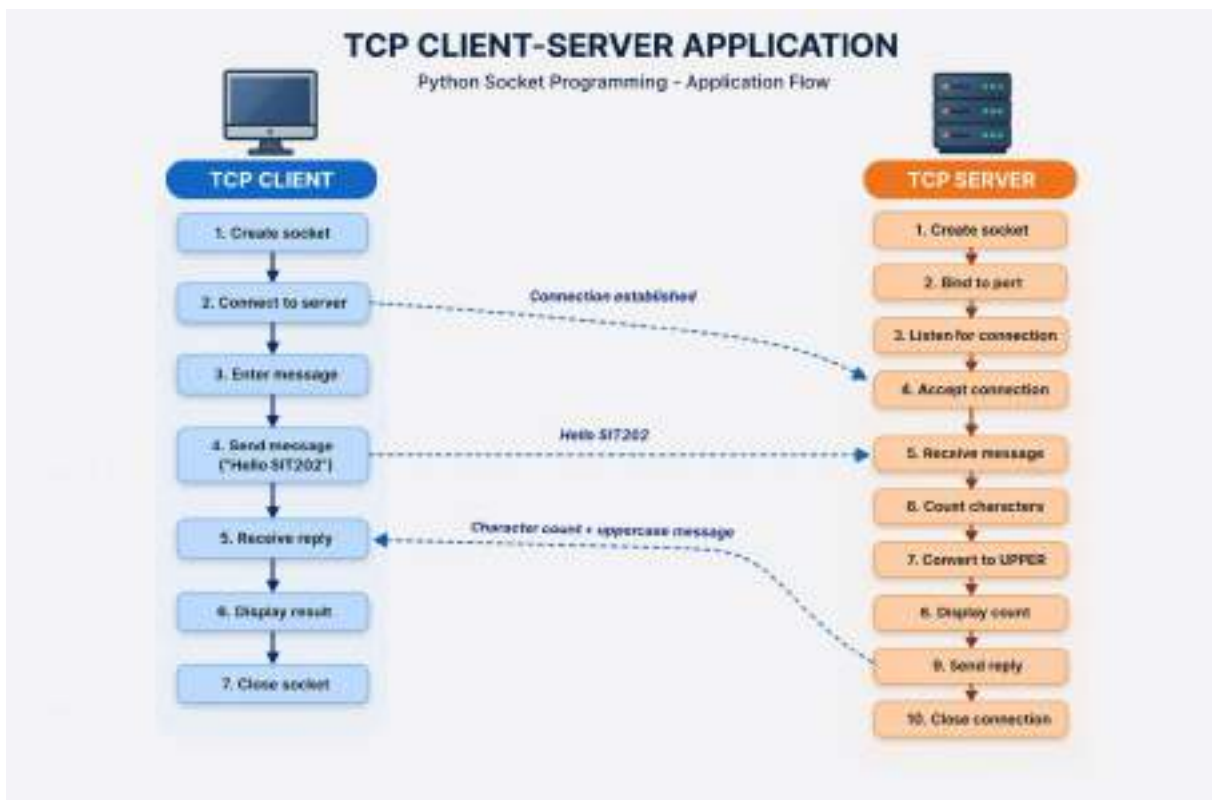


Figure. Flowchart showing the operation of the Transmission Control Protocol (TCP) client-server application in Python.

For this activity, the diagram should show a simple TCP-based communication process between a client and a server, beginning with connection establishment and ending with the display of the processed reply at the client side. TCP is a connection-oriented protocol, so the client and server must first establish a connection before any application data is transmitted.

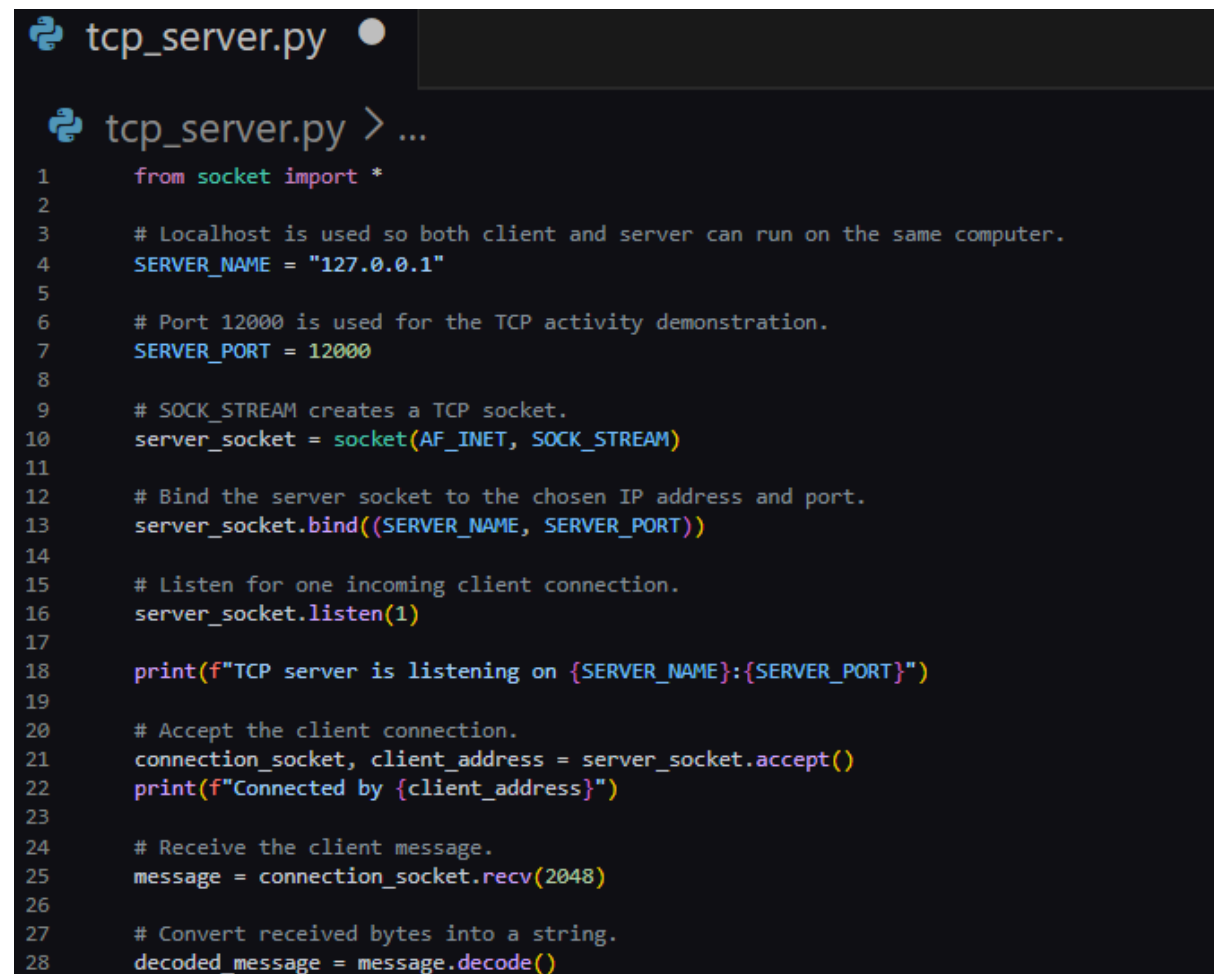
On the server side, the program creates a TCP socket, binds it to a port, listens for an incoming connection, and accepts the client request. On the client side, the program creates its own TCP socket and connects to the server before sending the message “**Hello SIT202**” or any other user-entered text.

Once the connection is established, the client sends the message to the server, the server receives the full message, counts the number of characters, converts the same message to uppercase, and displays the character count on its terminal. The server then sends a reply back to the client containing the character count and the uppercase version of the message, after which the client receives and displays the returned result on its terminal.

This flow demonstrates reliable process-to-process communication over TCP, where the connection is established first, and the data is then exchanged in an ordered and reliable manner.

2. Then one of you can develop the client side and the other can develop the server side of the program (you can also develop both and demonstrate the outcome if you choose to do so)

Ans. **Development of the client and server sides of the TCP program**



```
tcp_server.py
1  from socket import *
2
3  # Localhost is used so both client and server can run on the same computer.
4  SERVER_NAME = "127.0.0.1"
5
6  # Port 12000 is used for the TCP activity demonstration.
7  SERVER_PORT = 12000
8
9  # SOCK_STREAM creates a TCP socket.
10 server_socket = socket(AF_INET, SOCK_STREAM)
11
12 # Bind the server socket to the chosen IP address and port.
13 server_socket.bind((SERVER_NAME, SERVER_PORT))
14
15 # Listen for one incoming client connection.
16 server_socket.listen(1)
17
18 print(f"TCP server is listening on {SERVER_NAME}:{SERVER_PORT}")
19
20 # Accept the client connection.
21 connection_socket, client_address = server_socket.accept()
22 print(f"Connected by {client_address}")
23
24 # Receive the client message.
25 message = connection_socket.recv(2048)
26
27 # Convert received bytes into a string.
28 decoded_message = message.decode()
```

```
29
30     # Count the number of characters in the received message.
31     character_count = len(decoded_message)
32
33     # Convert the received message to uppercase.
34     uppercase_message = decoded_message.upper()
35
36     # Display the received data on the server terminal.
37     print(f"Received message: {decoded_message}")
38     print(f"Number of characters: {character_count}")
39
40     # Prepare the reply required by the activity.
41     reply = f"Character count: {character_count} | Uppercase message: {uppercase_message}"
42
43     # Send the reply back to the client.
44     connection_socket.sendall(reply.encode())
45
46     # Close the connected socket and the server socket.
47     connection_socket.close()
48     server_socket.close()
```

Figure. Python code for the TCP server program.

```
tcp_client.py
tcp_client.py > ...
1     from socket import *
2
3     # Localhost is used because the server is running on the same computer.
4     SERVER_NAME = "127.0.0.1"
5
6     # This port must match the server port exactly.
7     SERVER_PORT = 12000
8
9     # SOCK_STREAM creates a TCP socket for the client.
10    client_socket = socket(AF_INET, SOCK_STREAM)
11
12    # Establish the TCP connection with the server.
13    client_socket.connect((SERVER_NAME, SERVER_PORT))
14
15    # Take input from the user and send it to the server.
16    message = input("Enter message to send to server: ")
17    client_socket.sendall(message.encode())
18
19    # Receive the reply from the server.
20    server_reply = client_socket.recv(2048)
21
22    # Display the returned character count and uppercase message.
23    print("Reply from server:")
24    print(server_reply.decode())
25
26    # Close the client socket after communication is complete.
27    client_socket.close()
```

Figure. Python code for the TCP client program.

For this activity, both the client side and the server side of the Transmission Control Protocol (TCP) program were developed in Python to demonstrate reliable transport-layer communication between two end systems. The module guidance for TCP socket programming uses `SOCK_STREAM` and explains that the client initiates the connection with `connect()`, while the server prepares for incoming connections using `listen()` and `accept()`.

The client program was designed to create a TCP socket, connect to the server, accept a message input from the user, send the message to the server, receive the processed reply, and display the returned result on the client terminal. The server program was designed to create a TCP socket, bind to a selected port, listen for a client connection, accept the connection, receive the client message, count the number of characters, convert the same message to uppercase, display the count, and send the processed reply back to the client.

This implementation satisfies the activity requirement that the client sends a message such as **“Hello SIT202”**, while the server returns both the character count and the uppercase version of the same message.

Progress note: both sides of the Transmission Control Protocol (TCP) application were successfully implemented, with the client handling connection establishment and message transmission, and the server handling message processing and response delivery.

3. You need to show the output of the client- server program to demonstrate that your program works as expected. Please make sure to include the server and client codes, as well as the output (screenshots), in your lesson review.

Ans. **Output of the TCP client-server program**

The screenshot displays a Visual Studio Code editor with a Python project. The file explorer on the left shows a directory structure with files like `tcp_client.py` and `tcp_server.py`. The main editor window shows the code for `tcp_server.py`, which includes imports for `socket`, `sys`, and `os`, and logic to listen for connections, receive messages, count characters, convert to uppercase, and send the response. Below the editor, two Windows PowerShell terminal windows are open. The left terminal shows the server's execution: it starts by listening on port 127.0.0.1:12345, receives a connection from 127.0.0.1:50501, receives the message "Hello SIT202", counts 12 characters, and sends the response. The right terminal shows the client's execution: it connects to the server, sends the message "Hello SIT202", receives the response, and displays the character count and the uppercase message "HELLO SIT202".

```
#!/usr/bin/env python3
import socket
import sys
import os

# Create a TCP socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Bind the socket to the address and port
s.bind(('127.0.0.1', 12345))

# Listen for incoming connections
s.listen(1)

# Accept a connection
client, address = s.accept()

# Receive the data from the client
message = client.recv(1024)

# Strip the trailing newline character and uppercase message
message = message.strip().upper()

# Send the response back to the client
client.send(message)

# Close the client socket after communication is complete
client.close()
```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
<https://aka.ms/PowerShell>
PS D:\Semester-4\SIT202\5.1P Task> python tcp_server.py
TCP server is listening on 127.0.0.1:12345
Connected by ('127.0.0.1', 50501)
Received message: Hello SIT202
Number of characters: 12
PS D:\Semester-4\SIT202\5.1P Task>

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements!
<https://aka.ms/PowerShell>
PS D:\Semester-4\SIT202\5.1P Task> cd "D:\Semester-4\SIT202\5.1P Task"
PS D:\Semester-4\SIT202\5.1P Task> python tcp_client.py
Enter message to send to server: Hello SIT202
Reply from server:
Character count: 12 | Uppercase message: HELLO SIT202
PS D:\Semester-4\SIT202\5.1P Task>

Figure. Output of the TCP client-server program showing successful connection establishment, message processing, and reply display.

The output of the Transmission Control Protocol (TCP) client-server program confirms that the application works as expected. During execution, the client first establishes a TCP connection with the server, sends the message, and then waits for the reply returned by the server.

After receiving the message, the server counts the number of characters, converts the message to uppercase, displays the count on its terminal, and sends the processed result back through the established TCP connection. The client then receives the server response and displays it on its terminal, confirming that the program successfully completed two-way TCP communication.

For example, if the client sends “**Hello SIT202**”, the server returns: Character count: 12 | Uppercase message: HELLO SIT202.

Progress note: The successful output verifies that the Transmission Control Protocol (TCP) client and server programs were correctly implemented and that the required message exchange, processing, and response handling all operated as intended.

References

1. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23483/View>
2. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23484/View>
3. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23485/View>
4. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23486/View>
5. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23487/View>
6. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23488/View>
7. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23489/View>
8. <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23490/View>

[Quiz List](#) > [Submissions](#)

Quiz Submissions – Module 3: Transport Layer

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %

Quiz List

Help

Current Quizzes	Evaluation Status	Attempts
Module 3: Transport Layer Due on 20 March 2026 11:59 PM Available on 24 February 2026 12:01 AM until 20 March 2026 11:59 PM	Feedback: On Attempt	1 / 3
Past Quizzes	Evaluation Status	Attempts
Module 1: Introduction to Computer Networking Due on 19 February 2026 4:00 PM Available on 04 February 2026 9:00 AM until 19 February 2026 4:00 PM	Feedback: On Attempt	1 / 3
Module 2: Application Layer Due on 03 March 2026 4:00 PM Available on 11 February 2026 12:00 PM until 03 March 2026 4:00 PM	Feedback: On Attempt	1 / 4

Grades

Print



Stay up-to-date on your grades with the Pulse App

Keep track with your courses on the go!



Grade Item	Points	Grade	Comments and Assessments
Module 1: Introduction to Computer Networking	10 / 10	100 %	View Quiz Attempts
Module 2: Application Layer	10 / 10	100 %	View Quiz Attempts
Module 3: Transport Layer	10 / 10	100 %	View Quiz Attempts

8 Task 5.2C

Credit task

Outcome	Weight
Interaction with Networks	◆◆◆◆◆

In this task, I designed pseudo code for a UDP-based DNS server supporting A and CNAME records, learning DNS message structure and UDP socket operations per RFC 1035. For ULO2, I implemented protocol logic using UDP sockets for DNS query parsing, record lookup, and response generation at the transport layer. For ULO4, I analysed DNS protocol construction through header flags, RCODE handling, and message assembly to ensure correct query-response behaviour.

Outcome	Weight
Analysing real network issues	◆◆◆◆◇

In this task, I designed pseudo code for a UDP-based DNS server supporting A and CNAME records, learning DNS message structure and UDP socket operations per RFC 1035. For ULO2, I implemented protocol logic using UDP sockets for DNS query parsing, record lookup, and response generation at the transport layer. For ULO4, I analysed DNS protocol construction through header flags, RCODE handling, and message assembly to ensure correct query-response behaviour.

Date	Author	Comment
2026/03/16 00:22	Jahan Garg	Working On It
2026/03/16 04:44	Jahan Garg	Not Started
2026/03/16 05:04	Jahan Garg	Working On It
2026/03/16 11:56	Jahan Garg	Ready for Feedback
2026/03/16 11:56	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 5.2C task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/03/20 08:51	Amanpreet Kaur	Dear jahan, Good work, You are required to add the flow chart of the task
2026/03/20 08:51	Amanpreet Kaur	Fix and Resubmit
2026/03/20 21:42	Jahan Garg	Ready for Feedback
2026/03/20 21:42	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards,Jahan Garg2410994805X2 Batch
2026/03/21 09:38	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 5.2C

Submitted By:

Jahan GARG

2410994805

2026/03/20 21:42

Tutor:

Amanpreet KAUR

Outcome	Weight
Interaction with Networks	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◇

In this task, I designed pseudo code for a UDP-based DNS server supporting A and CNAME records, learning DNS message structure and UDP socket operations per RFC 1035. For ULO2, I implemented protocol logic using UDP sockets for DNS query parsing, record lookup, and response generation at the transport layer. For ULO4, I analysed DNS protocol construction through header flags, RCODE handling, and message assembly to ensure correct query-response behaviour.

March 20, 2026



Task 5.2C My DNS Server Sketch

Introduction

This document presents a comprehensive pseudo-code design for a UDP-based DNS server that supports two types of resource records: **A** (IPv4 address) and **CNAME** (canonical name alias). The design follows the DNS message structure defined in RFC 1035 and operates over UDP port 53, the standard protocol for DNS query-response exchanges. The pseudo code is organised into four sections as required: server initialisation, listening and processing DNS queries, handling A and CNAME records, and generating DNS responses.

DNS Server Architecture

The following diagram illustrates the overall architecture and message flow between the DNS client and server.

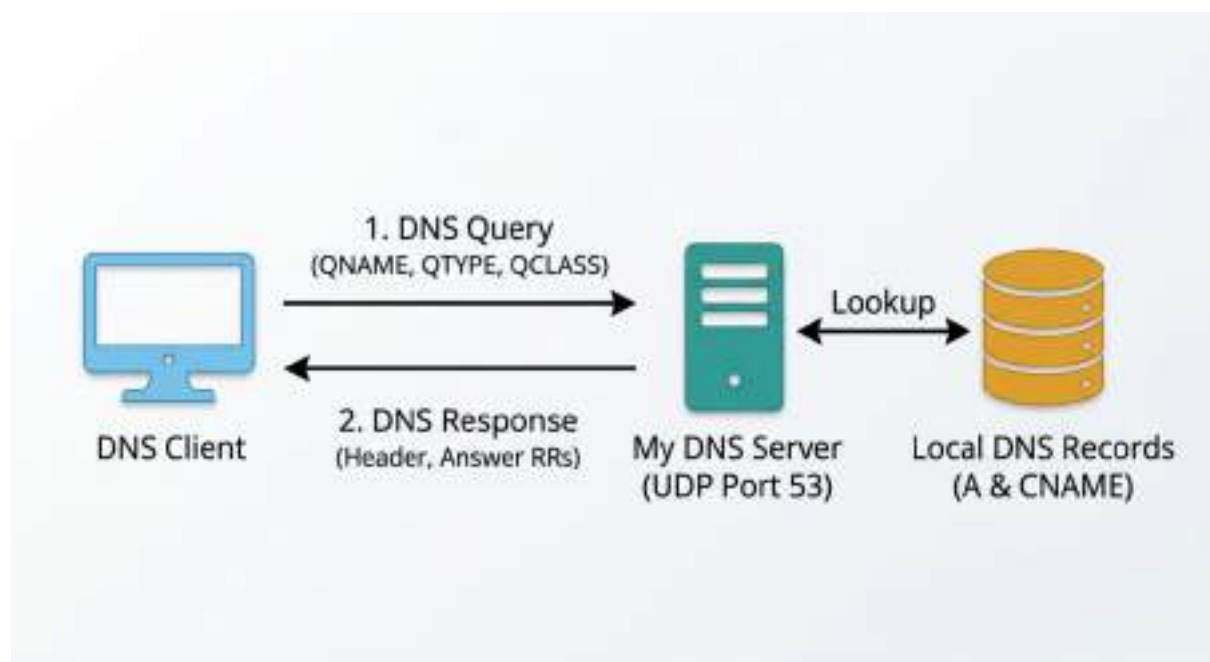


Figure 1: DNS server architecture showing the client-server interaction. The client sends a DNS query containing the hostname (QNAME), query type (QTYPE), and class (QCLASS) over UDP to port 53. The server looks up the requested record in its local database and returns a DNS response containing the answer section with matching resource records.

Pseudo Code Design

1. Server Initialisation

The first step establishes the DNS server environment by creating the UDP socket, binding it to the DNS service port, and loading the local DNS records database.

```
Jahan SIT202 5.2C Pseudocode.txt X
C:\Users> dir C:\Users\Jahan SIT202 5.2C Pseudocode.txt
1 BEGIN Server_Initialization
2
3 # Define local DNS records database containing A and CNAME resource records
4 # Structure: each record has (hostname, record_type, value)
5 records = [
6     ("example.com", "A", "93.184.215.14"),
7     ("www.example.com", "CNAME", "example.com"),
8     ("mail.example.com", "A", "203.0.113.10")
9 ]
10
11 # Create UDP socket for DNS service
12 udp_socket = create_UDP_socket()
13
14 # Bind socket to all network interfaces on standard DNS port 53
15 udp_socket.bind(ip_address = "0.0.0.0", port = 53)
16
17 # Set socket timeout to handle idle connections
18 udp_socket.set_timeout(5 seconds)
19
20 # Log server startup
21 log("DNS server initialized and listening on UDP port 53")
22
23 END Server_Initialization
24
```

Explanation: The server maintains an in-memory table of DNS records where each entry contains a hostname, its record type (A or CNAME), and the corresponding value. An A record maps a hostname directly to an IPv4 address, while a CNAME record maps an alias to its canonical hostname. The UDP socket is bound to port 53, which is the standard port designated by IANA for DNS services. Setting a timeout ensures the server can handle multiple clients without blocking indefinitely on a single connection.

2. Listening and Processing DNS Queries

This section describes how the server continuously listens for incoming DNS queries, parses the message structure, and extracts the relevant query parameters.

```
Jahan SIT202 5.2C Pseudocode.txt
C:\> Users > HP > Jahan SIT202 5.2C Pseudocode.txt
1  BEGIN Listen_And_Process_Queries
2
3      WHILE server_is_running DO
4
5          # Receive DNS query datagram from any client
6          # Maximum size is 512 bytes as per RFC 1035 for UDP queries
7          query_bytes, client_address = udp_socket.receive_from(max_size = 512)
8
9          IF query_bytes is NULL THEN
10             # Timeout or error occurred; continue listening
11             CONTINUE
12          END IF
13
14          # Parse DNS message into header and question sections
15          dns_header, dns_question = parse_DNS_query(query_bytes)
16
17          # Extract query parameters from the question section
18          qname = dns_question.name      # Queried hostname (e.g., "example.com")
19          qtype = dns_question.type      # Query type (A or CNAME)
20          qclass = dns_question.class    # Query class (typically IN for Internet)
21
22          log("Received query from " + client_address + " for " + qname + " type " + qtype)
23
24          # Validate that the query is for Internet class
25          IF qclass != "IN" THEN
26             # Respond with RCODE = NOTIMP (function not implemented)
27             error_response = build_error_response(dns_header, rcode = "NOTIMP")
28             udp_socket.send_to(error_response, client_address)
29             CONTINUE
30          END IF
31
32          # Process the query by looking up A or CNAME records
33          answer_records = handle_A_and_CNAME_query(qname, qtype)
34
35          # Build complete DNS response message
36          response_bytes = build_DNS_response(dns_header, dns_question, answer_records)
37
38          # Send response back to the client
39          udp_socket.send_to(response_bytes, client_address)
40
41      END WHILE
42
43  END Listen_And_Process_Queries
44
```

Explanation: The server enters an infinite loop to continuously accept DNS queries from clients. Each incoming UDP datagram is limited to 512 bytes, which is the standard maximum size for DNS messages over UDP as specified in RFC 1035. The DNS message consists of a header section containing metadata (transaction ID, flags, and counts) and a question section containing the queried name (QNAME), type (QTYPE), and class (QCLASS). Only queries with class "IN" (Internet) are

processed; other classes receive an error response with RCODE set to "NOTIMP" (Not Implemented). Valid queries are passed to the record handler for lookup.

3. Handling A and CNAME Records

This section implements separate logical flows for processing A and CNAME queries, as required by the task specification.

```
Jahan SIT202 5.2C Pseudocode.txt
C: > Users > HP > Jahan SIT202 5.2C Pseudocode.txt
1  BEGIN Handle_A_and_CNAME_Query(qname, qtype)
2
3      # Initialize empty list to store matching resource records
4      answer_records = []
5
6      IF qtype == "A" THEN
7
8          # Search for A records matching the queried hostname
9          FOR EACH (hostname, record_type, value) IN records DO
10
11              IF hostname == qname AND record_type == "A" THEN
12                  # Create A resource record with the IPv4 address
13                  rr = create_resource_record(
14                      name = qname,
15                      type = "A",
16                      ttl = 300,           # Time to live in seconds
17                      rdata = value        # IPv4 address string
18                  )
19                  append(answer_records, rr)
20              END IF
21          END FOR
22
23      ELSE IF qtype == "CNAME" THEN
24
25          # Search for CNAME records matching the queried hostname
26          FOR EACH (hostname, record_type, value) IN records DO
27
28              IF hostname == qname AND record_type == "CNAME" THEN
29                  # Create CNAME resource record with canonical name
30                  cname_rr = create_resource_record(
31                      name = qname,
32                      type = "CNAME",
33                      ttl = 300,
34                      rdata = value        # Canonical hostname
35                  )
36                  append(answer_records, cname_rr)
37              END IF
38          END FOR
39      END IF
40  END
```



```
38
39      # Optionally include the A record of the canonical name
40      # This provides the final IP address in the same response
41      FOR EACH (hostname2, record_type2, value2) IN records DO
42
43          IF hostname2 == value AND record_type2 == "A" THEN
44              a_rr = create_resource_record(
45                  name = value,      # Canonical name
46                  type = "A",
47                  ttl = 300,
48                  rdata = value2     # IPv4 address
49              )
50              append(answer_records, a_rr)
51          END IF
52
53      END FOR
54
55      BREAK # Stop after finding the first CNAME match
56  END IF
57
58  END FOR
59
60  ELSE
61      # Other query types are not supported by this server
62      answer_records = []
63  END IF
64
65  RETURN answer_records
66
67  END Handle_A_and_CNAME_Query
68
```

Explanation: This function implements distinct processing logic for A and CNAME queries. For an A query, the server searches the records database for matching A records and returns the IPv4 address. For a CNAME query, the server returns the alias mapping to the canonical hostname. Additionally, to improve efficiency and reduce the need for subsequent queries, the server can optionally include the A record of the canonical name in the same response, allowing the client to immediately obtain the final IP address. If the query type is neither A nor CNAME, an empty answer list is returned, which will result in an appropriate error response.

4. Generating DNS Responses

The final section assembles the complete DNS response message with the appropriate header flags, counts, and answer section.

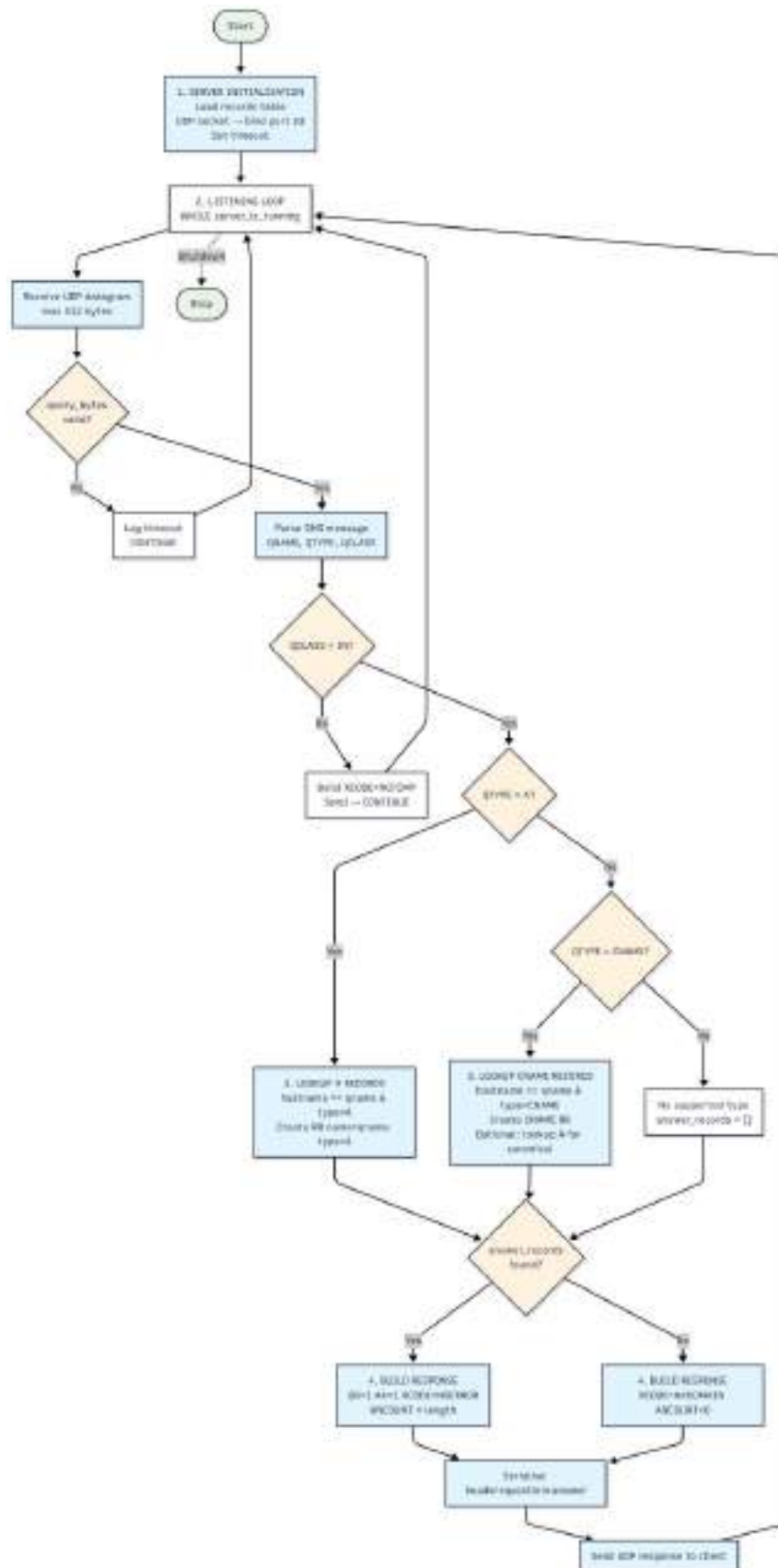


Figure 2: DNS server operational flowchart showing the complete process from initialisation through query processing, record lookup, and response generation. The decision points highlight class validation and query type branching for A and CNAME handling.

```

≡ Jahan SIT202 5.2C Pseudocode.txt X
C: > Users > HP > ≡ Jahan SIT202 5.2C Pseudocode.txt
1  BEGIN Build_DNS_Response(original_header, dns_question, answer_records)
2
3      # Create response header by copying and modifying query header
4      response_header = create_DNS_header()
5
6      # Copy transaction ID so client can match response to query
7      response_header.id = original_header.id
8
9      # Set response-specific flags
10     response_header.qr      = 1           # QR = 1 indicates response
11     response_header.opcode = original_header.opcode # Echo opcode from query
12     response_header.aa      = 1           # Authoritative answer
13     response_header.tc      = 0           # Not truncated
14     response_header.rd      = original_header.rd # Echo recursion desired flag
15     response_header.ra      = 0           # Recursion not available
16
17     # Set response code and answer count based on lookup result
18     IF length(answer_records) > 0 THEN
19         response_header.rcode = "NOERROR"      # Successful query
20         response_header.ancount = length(answer_records)
21     ELSE
22         response_header.rcode = "NXDOMAIN"      # Name does not exist
23         response_header.ancount = 0
24     END IF
25
26     # Set section counts
27     response_header.qdcount = 1      # One question echoed back
28     response_header.nscount = 0      # No authority records
29     response_header.ancount = 0      # No additional records
30
31     # Serialize question section (echo the original query)
32     question_section = serialize_question(dns_question)
33
34     # Serialize answer section containing all matching resource records
35     answer_section = ""
36     FOR EACH rr IN answer_records DO
37         answer_section = answer_section + serialize_resource_record(rr)
38     END FOR
39
40     # Authority and additional sections are empty in this design
41     authority_section = ""
42     additional_section = ""
43

```

```
44      # Assemble complete DNS message in standard order
45      response_bytes = serialize_header(response_header) +
46      |               | question_section +
47      |               | answer_section +
48      |               | authority_section +
49      |               | additional_section
50
51      RETURN response_bytes
52
53  END Build_DNS_Response
```

Explanation: The DNS response message follows the structure defined in RFC 1035, consisting of a 12-byte header followed by question, answer, authority, and additional sections. The response header preserves the transaction ID from the query to enable proper matching at the client side. The QR flag is set to 1 to indicate a response, and the AA flag is set to 1 to mark this as an authoritative answer since the server directly manages these records. The RCODE field is set to "NOERROR" when matching records are found or "NXDOMAIN" when the queried name does not exist in the database. The ANCOUNT field specifies the number of resource records in the answer section. The question section is echoed back to the client as per standard DNS protocol behaviour. Finally, all sections are serialised in the correct order to form a valid DNS response message.

Conclusion

This pseudo-code design provides a complete conceptual framework for a UDP-based DNS server supporting A and CNAME resource records. The design adheres to the DNS protocol specifications in RFC 1035 and is structured to facilitate straightforward implementation in Python for Task 6.2C. The separation of concerns across the four sections, including initialisation, query processing, record handling, and response generation, ensures clarity and maintainability while meeting all task requirements.

References

RFC 1035 - Domain Names - Implementation and Specification.

IETF. <https://datatracker.ietf.org/doc/html/rfc1035>

Network Ports Used by DNS. Microsoft. <https://learn.microsoft.com/en-us/windows-server/networking/dns/network-ports>

Is DNS TCP or UDP port 53? Infoblox. <https://www.infoblox.com/dns-security-resource-center/dns-security-faq/is-dns-tcp-or-udp-port-53/>

Differences Between A and CNAME Records.

DNSimple. <https://support.dnsimple.com/articles/differences-a-cname-records/>

A Record vs. CNAME: What's the difference?

Valimail. <https://www.valimail.com/blog/a-record-vs-cname/>

DNS Message Header and Question Section Format. TCP/IP

Guide. http://www.tcpipguide.com/free/t_DNSMessageHeaderandQuestionSectionFormat.htm

DNS Message Formats. Microsoft Windows Server. <https://learn.microsoft.com/en-us/windows-server/networking/dns/message-formats>

DNS Return Codes. DNSFilter. <https://help.dnsfilter.com/hc/en-us/articles/4408415850003-DNS-return-codes>

CNAME, A Record, and MX Record. Cornell

University. <https://it.cornell.edu/dns/cname-record-and-mx-record>

DNS Request and Responses.

KodeKloud. <https://notes.kodekloud.com/docs/Demystifying-DNS/DNS-as-a-Protocol/DNS-Request-and-Responses/page>

9 Task 6.1P

Pass task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I developed a thorough understanding of the Network Layer data plane by studying IP datagram structure, forwarding and routing operations, subnetting, DHCP, NAT, and IPv6, and by applying these concepts through CIDR exercises, Wireshark packet analysis, group routing discussions, Cisco Packet Tracer implementation, and simulation with a verified full quiz score. For ULO1, I explained how the network layer operates within the TCP/IP protocol stack, encapsulating transport segments into datagrams and delivering them across multiple subnets through forwarding tables and router interfaces. For ULO2, I applied protocol-level knowledge in practice by configuring IP addresses, subnet masks, default gateways, and router interfaces, and by verifying how transport protocols such as TCP and UDP are carried inside IPv4 datagrams. For ULO3, I used industry-relevant tools including Wireshark and Cisco Packet Tracer to inspect real IPv4 headers, verify inter-network connectivity through ping, and simulate packet forwarding to observe data plane behaviour at each hop. For ULO4, I reflected on how forwarding decisions, fragmentation, TTL handling, subnet design, and NAT influence network scalability, performance, and troubleshooting, developing a critical appreciation of how the data plane is constructed and applied in real network scenarios.

Outcome	Weight
Interaction with Networks	◆◆◆◆◆

In this task, I developed a thorough understanding of the Network Layer data plane by studying IP datagram structure, forwarding and routing operations, subnetting, DHCP, NAT, and IPv6, and by applying these concepts through CIDR exercises, Wireshark packet analysis, group routing discussions, Cisco Packet Tracer implementation, and simulation with a verified full quiz score. For ULO1, I explained how the network layer operates within the TCP/IP protocol stack, encapsulating transport segments into datagrams and delivering them across multiple subnets through forwarding tables and router interfaces. For ULO2, I applied protocol-level knowledge in practice by configuring IP addresses, subnet masks, default gateways, and router interfaces, and by verifying how transport protocols such as TCP and UDP are carried inside IPv4 datagrams. For ULO3, I used industry-relevant tools including Wireshark and Cisco Packet Tracer to inspect real IPv4 headers, verify inter-network connectivity through ping, and simulate packet forwarding to observe data plane behaviour at each hop. For ULO4, I reflected on how forwarding decisions, fragmentation, TTL handling, subnet design, and NAT influence network scalability, performance, and troubleshooting, developing a critical appreciation of how the data plane is constructed and applied in real network scenarios.

Outcome	Weight
Networking Tools	◆◆◆◆◆

In this task, I developed a thorough understanding of the Network Layer data plane by studying IP datagram structure, forwarding and routing operations, subnetting, DHCP, NAT, and IPv6, and by applying these concepts through CIDR exercises, Wireshark packet analysis, group routing discussions, Cisco Packet Tracer implementation, and simulation with a verified full quiz score. For ULO1, I explained how the network layer operates within the TCP/IP protocol stack, encapsulating transport segments into datagrams and delivering them across multiple subnets through forwarding tables and router interfaces. For ULO2, I applied protocol-level knowledge in practice by configuring IP addresses, subnet masks, default gateways, and router interfaces, and by verifying how transport protocols such as TCP and UDP are carried inside IPv4 datagrams. For ULO3, I used industry-relevant tools including Wireshark and Cisco Packet Tracer to inspect real IPv4 headers, verify inter-network connectivity through ping, and simulate packet forwarding to observe data plane behaviour at each hop. For ULO4, I reflected on how forwarding decisions, fragmentation, TTL handling, subnet design, and NAT influence network scalability, performance, and troubleshooting, developing a critical appreciation of how the data plane is constructed and applied in real network scenarios.

Outcome	Weight
Analysing real network issues	◆◆◆◆

In this task, I developed a thorough understanding of the Network Layer data plane by studying IP datagram structure, forwarding and routing operations, subnetting, DHCP, NAT, and IPv6, and by applying these concepts through CIDR exercises, Wireshark packet analysis, group routing discussions, Cisco Packet Tracer implementation, and simulation with a verified full quiz score. For ULO1, I explained how the network layer operates within the TCP/IP protocol stack, encapsulating transport segments into datagrams and delivering them across multiple subnets through forwarding tables and router interfaces. For ULO2, I applied protocol-level knowledge in practice by configuring IP addresses, subnet masks, default gateways, and router interfaces, and by verifying how transport protocols such as TCP and UDP are carried inside IPv4 datagrams. For ULO3, I used industry-relevant tools including Wireshark and Cisco Packet Tracer to inspect real IPv4 headers, verify inter-network connectivity through ping, and simulate packet forwarding to observe data plane behaviour at each hop. For ULO4, I reflected on how forwarding decisions, fragmentation, TTL handling, subnet design, and NAT influence network scalability, performance, and troubleshooting, developing a critical appreciation of how the data plane is constructed and applied in real network scenarios.

Date	Author	Comment
2026/03/19 03:57	Jahan Garg	Working On It
2026/03/22 11:59	Jahan Garg	Ready for Feedback
2026/03/22 12:03	Jahan Garg	pdf document
2026/03/22 12:03	Jahan Garg	pdf document
2026/03/22 12:04	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 6.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/03/25 06:37	Amanpreet Kaur	Dear jahan, You are required to add the screenshot of the communication between different PCs
2026/03/25 06:37	Amanpreet Kaur	Fix and Resubmit
2026/03/29 12:00	Jahan Garg	Ready for Feedback
2026/03/29 12:00	Amanpreet Kaur	Time Exceeded
2026/03/29 12:01	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards,Jahan Garg2410994805X2 Batch
2026/05/05 09:29	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 6.1P

Submitted By:
Jahan GARG
2410994805
2026/05/11 20:26

Tutor:
Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◆

In this task, I developed a thorough understanding of the Network Layer data plane by studying IP datagram structure, forwarding and routing operations, subnetting, DHCP, NAT, and IPv6, and by applying these concepts through CIDR exercises, Wireshark packet analysis, group routing discussions, Cisco Packet Tracer implementation, and simulation with a verified full quiz score. For ULO1, I explained how the network layer operates within the TCP/IP protocol stack, encapsulating transport segments into datagrams and delivering them across multiple subnets through forwarding tables and router interfaces. For ULO2, I applied protocol-level knowledge in practice by configuring IP addresses, subnet masks, default gateways, and router interfaces, and by verifying how transport protocols such as TCP and UDP are carried inside IPv4 datagrams. For ULO3, I used industry-relevant tools including Wireshark and Cisco Packet Tracer to inspect real IPv4 headers, verify inter-network connectivity through ping, and simulate packet forwarding to observe data plane behaviour at each hop. For ULO4, I reflected on how forwarding decisions, fragmentation, TTL handling, subnet design, and NAT influence network scalability, performance, and troubleshooting, developing a critical appreciation of how the data plane is constructed and applied in real network scenarios.

May 11, 2026



Module 4: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around Module 4's three core learning objectives: understanding data plane functions, analysing router operations, and explaining IP datagram structure, addressing, and protocols, including IPv4 and IPv6, drawing comprehensively from sections 4.1 to 4.3 and Module 4 PPT resources.

The network layer provides host-to-host datagram delivery by encapsulating transport-layer segments into IP datagrams at the sender and delivering them to the correct transport protocol at the receiver. Unlike transport and application layers that reside primarily in end systems, the network layer also operates in routers and network devices, which examine IP header fields and forward datagrams along end-to-end paths (Module 4.1). This dual residency creates a layered hierarchy: routers act as intermediate forwarding devices, focusing entirely on network-layer operations without processing application or transport-layer details.

The module emphasised two fundamental network-layer functions with distinct operational scopes: forwarding and routing (Module 4.1). Forwarding is a local, per-router operation performed in the data plane, where each router uses its forwarding table to examine an arriving datagram's header fields and decide which output port receives that packet. The forwarding table is indexed using destination IP addresses and operates at line speed via hardware lookups, ensuring minimal latency. Routing, by contrast, is a network-wide function handled by the control plane, which determines the complete end-to-end path that datagrams follow from source to destination. Routing algorithms, such as per-router protocols or centralised SDN controllers, compute these paths and populate the forwarding tables that routers then use for local decisions. This separation between data plane (fast, local, per-packet) and control plane (slower, network-wide, path-oriented) is fundamental to understanding modern networks (Module 4.1).

Router architecture consists of distinct functional components implementing forwarding and queuing: input ports handle physical and link-layer reception, forwarding table lookup, and queuing when packets arrive faster than the switching fabric can transfer them; the switching fabric itself transfers packets from input ports to output ports at a rate ideally N times the line rate for N input/output ports; and output ports buffer, schedule, and transmit packets onto outgoing links (Module 4.2). Input port congestion causes head-of-line blocking, where a packet at the front of an input queue blocks others behind it, reducing switching fabric utilisation. Output port congestion arises when the arrival rate from the switching fabric exceeds the outgoing link transmission rate, leading to buffering and potential packet loss. Buffer management policies, such as tail drop or priority drop, determine which datagrams

are discarded. Scheduling disciplines, including first-come-first-served, priority scheduling, round robin, and weighted fair queueing, determine transmission order and influence per-flow throughput and latency (Module 4.2).

The IPv4 datagram format consists of a variable-length header (minimum 20 bytes without options) containing version, header length in 32-bit words (IHL), type of service for QoS marking, total length including header and payload (maximum 64 KB), identification and flags for fragmentation coordination, fragment offset for reassembly ordering, time-to-live for loop prevention (decremented by each router), protocol field indicating encapsulated transport layer (TCP equals 6, UDP equals 17), header checksum for error detection, and source/destination IP addresses (Module 4.2). The typical maximum transmission unit for modern networks is around 1500 bytes, a limit imposed by the underlying link-layer protocol, such as Ethernet. When an IP datagram exceeds a link's MTU, routers employ IPv4 fragmentation: they split the original datagram into smaller fragments, each carrying the same identification number, using flags and fragment offset fields so that the receiving host can identify fragmented datagrams and reassemble them in correct order before passing the transport-layer segment to the appropriate protocol (Module 4.2).

IP addressing assigns 32-bit identifiers to host and router interfaces, written in dotted-decimal notation (e.g., 223.1.1.1). IP addresses have a hierarchical structure divided into network (subnet) and host parts, enabling efficient routing and address management (Module 4.2). Subnets are groups of interfaces that can reach each other directly without passing through a router; they share a common subnet mask. CIDR notation (a.b.c.d/x) specifies subnet and host portions using a prefix length, where the first x bits identify the subnet and the remaining bits identify hosts. For example, subnet 192.168.40.128/26 uses 26 bits for the network prefix, leaving 6 bits for host addresses, yielding 2^6 equals 64 total addresses within that subnet (192.168.40.128 to 192.168.40.191), with usable host addresses from 192.168.40.129 to 192.168.40.190, excluding network and broadcast addresses. This hierarchical structure supports scalable routing by allowing routers to aggregate addresses and reduce forwarding table sizes (Module 4.2).

The module introduced DHCP (Dynamic Host Configuration Protocol), a client-server mechanism that dynamically assigns IP addresses and related configuration, such as subnet mask, default gateway, and DNS server to hosts when they join a network, enabling address reuse and supporting mobile hosts (Module 4.2). Network Address Translation extends IPv4 address space by allowing multiple devices on a private network (e.g., 10.0.0.0/24) to share a single public IP address through port-based translation, where a NAT router rewrites source addresses and port numbers on outgoing datagrams and maintains a translation table to reverse this operation for incoming traffic. This provides security benefits by hiding internal network structure and enabling flexible network changes without coordinating with external parties (Module 4.2).

IPv6 was motivated by exhaustion of the 32-bit IPv4 address space and performance considerations. IPv6 uses 128-bit addresses written in hexadecimal notation with colons separating blocks (e.g., 2001:0db8::1), where each hexadecimal digit represents 4 bits and allows a more compact representation than dotted-decimal. The IPv6 datagram header is fixed at 40 bytes and includes version, traffic class for QoS marking similar to IPv4 ToS, flow label to identify packets belonging to the same flow for special handling, payload length, next header indicating protocol, hop limit equivalent to IPv4 TTL, source address, and destination address (Module 4.2). Notable differences from IPv4 include the absence of a header checksum since link-layer and transport-layer protocols provide error checking, no fragmentation support in the header, where fragmentation is handled only by the source, and a simplified fixed-length header design to accelerate router processing. The module briefly introduced transition mechanisms such as tunnelling and dual-stack routers that allow IPv4 and IPv6 coexistence during the transition period (Module 4.2).

Practical reinforcement came through Cisco Packet Tracer activities, enabling topology building, IP configuration, and network simulation, connecting abstract concepts to hands-on implementation (Module 4.3).

B. How This Is Useful

Understanding data plane mechanisms, forwarding tables, switching fabrics, queuing, and buffer management enables me to diagnose real network performance issues such as congestion, packet loss, and latency. Mastery of IP addressing, CIDR notation, and subnet design is essential for configuring routed networks and avoiding address conflicts. Knowledge of IPv4 fragmentation and IPv6 advantages prepares me for analysing packet traces and making protocol design decisions. These skills form the foundation for control-plane routing studies in later modules and are critical for SIT202 assessments requiring network analysis and configuration tasks.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most valuable insight is the clear separation between data plane and control plane operations and how this separation explains router behaviour at scale. Before this module, I viewed routers as black boxes that send packets; now I understand that each router makes independent, local forwarding decisions using lookup tables populated by a separate control plane process. This distinction is visible in Packet Tracer simulations, where I observe per-packet forwarding decisions and TTL decrements, and in Wireshark traces where IP header fields directly control forwarding behaviour. The insight that forwarding happens at hardware speeds in the data plane while routing algorithms operate at millisecond timescale in the control

plane explains why routers are often implemented with specialised hardware and explains design choices like buffer management and scheduling disciplines.

B. How This Relates to What I Already Know

This module builds directly on my earlier understanding of the TCP/IP layered model and transport-layer protocols (TCP/UDP from Module 3). In previous modules, I studied process-to-process communication, reliability, and flow control without fully appreciating the underlying host-to-host delivery service. I also performed Wireshark analysis of HTTP and DNS traffic, viewing IP headers as static metadata. Now I understand that IP headers contain actionable fields (TTL, identification, flags, protocol) that routers actively examine and modify during forwarding. The subnetting and addressing concepts connect my theoretical knowledge of CIDR to practical network configuration, allowing me to interpret real network topologies and design them correctly. Packet Tracer reinforces these connections by letting me build the networks I previously only reasoned about theoretically.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because most operational problems in real networks originate at this layer. Misconfigured IP addresses, incorrect subnet masks, improper default gateways, and MTU mismatches causing fragmentation are common issues that manifest as connectivity failures or performance degradation. Understanding how routers actually forward packets enables effective troubleshooting using tools like Wireshark to inspect header fields and trace packet paths, and command-line utilities like traceroute that exploit TTL decrement behaviour. The module's focus on data plane mechanics, combined with practical activities in Packet Tracer and Wireshark, directly supports SIT202 learning outcomes in network analysis, topology design, and device configuration. Finally, data plane knowledge provides essential background for later control-plane topics such as routing protocols and emerging areas like Software-Defined Networking.

References

- Module 4: Network Layer – Data Plane (sections 4.1, 4.2, 4.3; D2L unit site)
- Module 4 PPT: Network Layer – Data Plane (36 slides)
- Kurose, J. F. & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (7th ed.). Chapter 4

Module 4: Evidence of Learning

This section presents evidence of my learning for Module 4, with a focus on the network layer data plane and the practical activities completed during Active Class 6. It includes module exercises, Wireshark packet analysis, group discussion outcomes, Packet Tracer configuration work, simulation observations, and supporting screenshots. Together, these items demonstrate my understanding of IP addressing, forwarding decisions, router behaviour, and packet movement across networks. Short progress notes are included throughout to show how each task contributed to my developing understanding of the module.

Evidence: Module Exercises

Exercise 1: Subnet Calculation (Module 4.2)

Question: Consider a network with subnet prefix 192.168.40.128/26. List two IP addresses that can be assigned to two PCs connected to this network.

Answer

The /26 prefix indicates that 26 bits identify the network, leaving 6 bits for host addressing. This gives $2^6 = 64$ total addresses in the subnet.

The subnet details are as follows:

- Network address: 192.168.40.128
- Usable host range: 192.168.40.129 to 192.168.40.190
- Broadcast address: 192.168.40.191

Therefore, two valid IP addresses that can be assigned to PCs are:

- PC1: 192.168.40.129
- PC2: 192.168.40.130

Both addresses fall within the valid host range for this /26 subnet.

Progress Note: This exercise strengthened my understanding of CIDR notation, subnet range identification, and valid host allocation, which are essential for correct IP configuration in network design and Packet Tracer activities.

Exercise 2: Cisco Packet Tracer Port Labels (Module 4.3)

Question: How can you display all port labels in your Cisco Packet Tracer logical workspace?

Answer

Port labels can be displayed in Cisco Packet Tracer by following these steps:

1. Open the **Options** menu in the top menu bar.
2. Select **Preferences**.
3. Locate and check “**Always Show Port Labels in Logical Workspace.**”
4. Apply the change and close the preferences window.

After this setting is enabled, interface labels such as GigabitEthernet0/0 and FastEthernet0/1 appear automatically in the logical workspace whenever devices are connected.

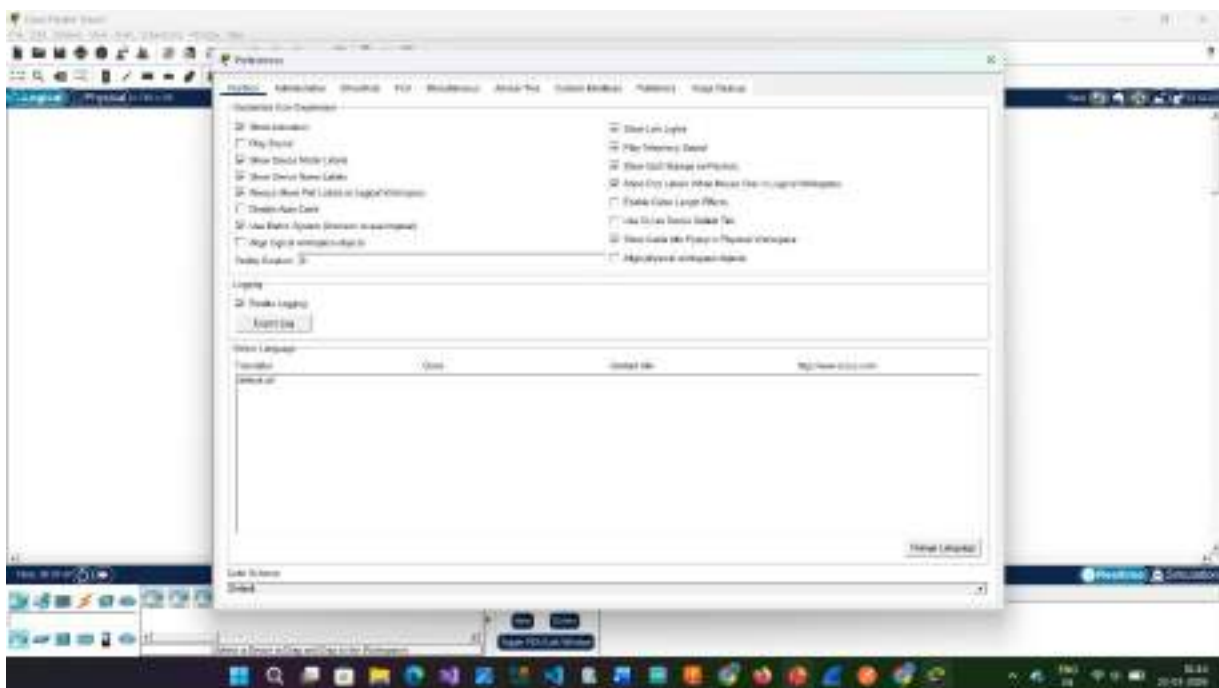


Figure 1: Enabling Port Labels in Cisco Packet Tracer

Progress Note: Enabling port labels improved topology readability and reduced the risk of interface selection errors during Packet Tracer configuration, especially when assigning IP addresses and verifying device connections.

Evidence: Active Classes Activities

Activities for Active Class 6: Journey of an IP Packet

Activity 1: Analysing IP Using Wireshark (Module 4, Active Class 6)

Learning Objective: Describe the Internet Protocol and explain the data plane operation of the **network layer through packet capture analysis**.

Process: I captured packets from web browsing and Microsoft Teams applications using Wireshark, then selected DNS/HTTP request packets and expanded the Internet Protocol section in the packet details window for detailed analysis (Active Class 6.1). I repeated the process with a different application to compare findings.

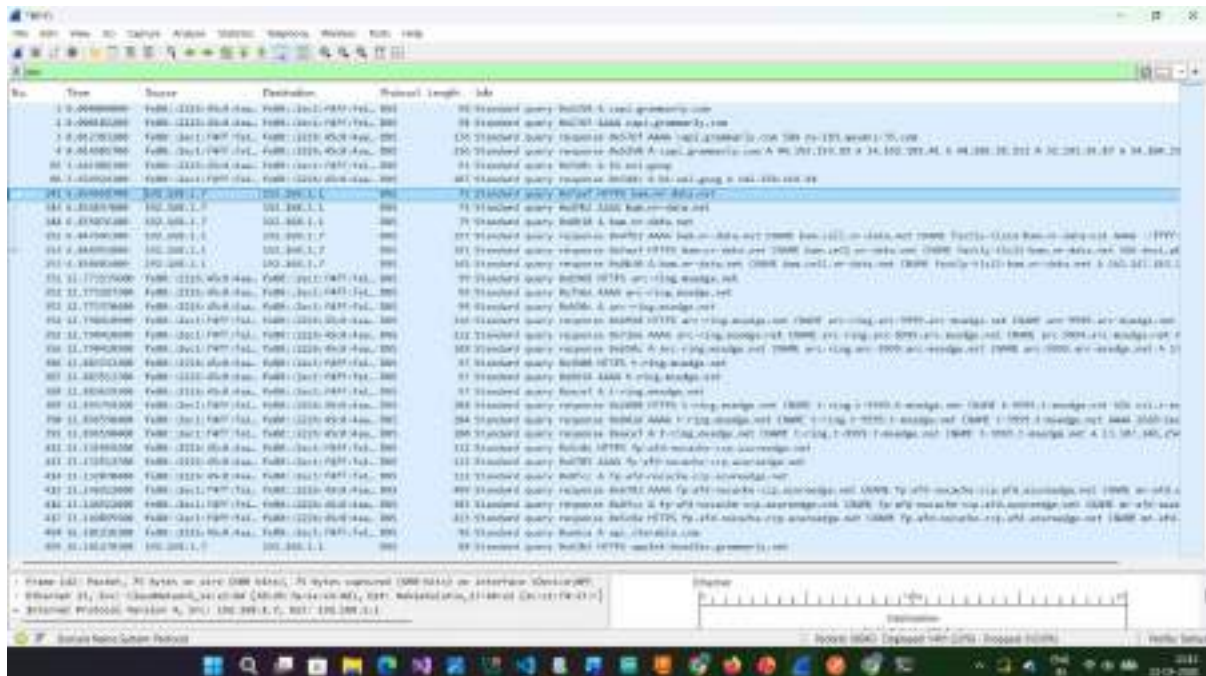


Figure 2: Wireshark DNS Packet List (dns filter)

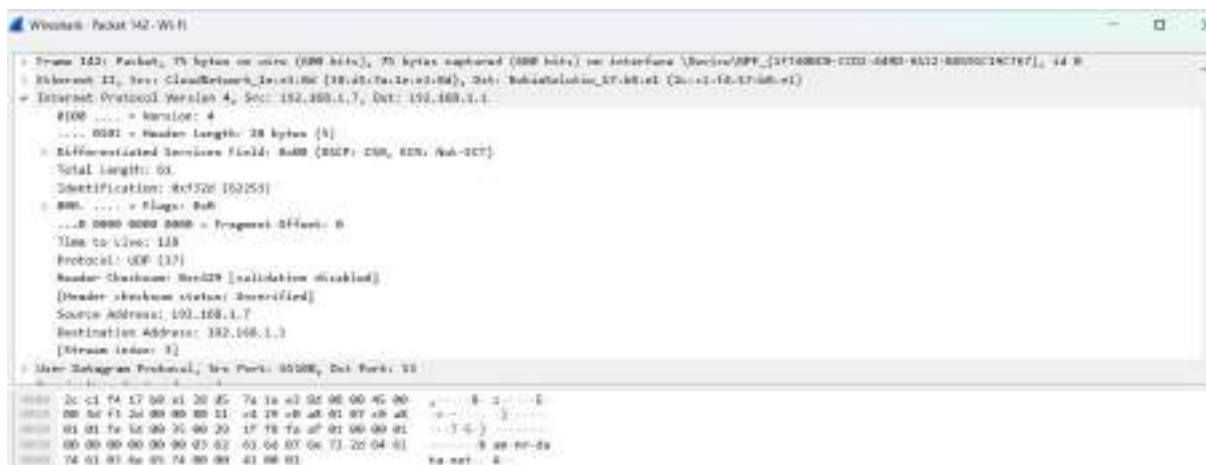


Figure 3: Frame 142 IPv4 Header Expanded (DNS UDP)

Analysis Questions and Responses:

1. Identifying Source and Destination IP Addresses

For the captured DNS packet Frame 142, I expanded the Internet Protocol (IPv4) section in Wireshark's packet details pane. The **Source Address** field displayed my device's local network IP address, and the **Destination Address** field showed the remote server's address (for example, a web server for HTTP packets or a DNS resolver for DNS queries).

These two fields directly correspond to the 32-bit source and destination address fields in the IPv4 datagram header structure described in Module 4.2, confirming that Wireshark accurately represents the network-layer information that routers use for forwarding decisions.

Progress Note: This verification connected the theoretical IPv4 header format to real captured traffic.

2. Identifying Transport-Layer Protocol from IP Header

The IPv4 header contains a **Protocol** field that indicates which transport-layer protocol is encapsulated within the datagram. In my packet captures:

- DNS packets (Frame 142) showed Protocol value **17**, corresponding to UDP
- HTTP packets (Frame 3092) showed Protocol value **6**, corresponding to TCP

Wireshark displays the protocol name in the frame's Protocol column (HTTP, DNS, TCP, UDP). The Protocol field value in the IPv4 header matched these display values, confirming that the IP header correctly identifies the encapsulated transport protocol. This demonstrates how the data plane uses protocol information to make intelligent forwarding and handling decisions (Module 4.2).

Progress Note: Verified the tight coupling between network-layer and transport-layer headers.

3. Calculating IP Header Size and Payload Size

The IPv4 header length is encoded in the **IHL (Internet Header Length)** field, expressed in 32-bit words. For standard datagrams without options, Wireshark reported an IHL value of 5, corresponding to 5 multiplied by 4, which equals **20 bytes**.

To calculate payload size:

Frame 142: Total Length: 61 bytes, IHL: 5 (20 bytes), Payload: 41 bytes

Payload Size equals Total Length - IP Header Length, where **Total Length** (shown in the IPv4 header) includes both header and payload.

For example, if the total length was 1500 bytes and the header length was 20 bytes, the payload would be 1480 bytes. This payload contains the encapsulated transport-layer segment (TCP or UDP) plus application data.

Progress Note: Understanding header and payload sizes is critical for analysing fragmentation scenarios and bandwidth utilisation.

4. Determining Fragmentation Status

To check whether a datagram was fragmented, I examined two IPv4 header fields:

- **Flags** field, specifically the "More Fragments" (MF) flag
- **Fragment Offset** field

In all captured packets from my web browsing and Teams applications, the MF flag was **0** (cleared) (**Frame 142**), and the Fragment Offset was **0**, indicating that none of the captured datagrams was fragmented. If fragmentation had occurred, I would observe either:

- MF flag equals 1 in all fragments except the last one
- Non-zero Fragment Offset values indicating position within the original datagram

The absence of fragmentation in these captures is typical for modern networks where path MTU discovery prevents oversized datagrams, confirming that fragmentation, while important to understand (Module 4.2), is relatively uncommon in contemporary Internet traffic.

Progress Note: Demonstrated understanding of fragmentation flags without requiring active fragmentation scenarios.

5. Examining Other Important IPv4 Header Fields

I inspected several additional IPv4 header fields present in captured packets (Frame 3092):

- **Time to Live (TTL):** TTL **128**, decremented by 1 at each router hop. This field prevents infinite looping of datagrams in case of routing loops (Module 4.2).
- **Type of Service/Differentiated Services (ToS/DSCP):** 0x00 (DSCP: CS0, ECN: Not-ECT) for standard Internet traffic.

6. Comparative Analysis: Different Packet Types

I captured and analysed packets from **DNS (Frame 142, UDP port 53)** and **HTTP (Frame 3092, TCP port 80)**. While both travel inside IPv4 datagrams with identical header structures, they differ in:

- **Protocol field:** 17 (Frame 142, UDP), 6 (Frame 3092, TCP)
- **Source/Destination ports:** DNS used **Src Port: 65108, Dst Port: 53**; Frame 3092 used TCP source port **60681** and destination port **80**
- **Packet size:** Frame 142 IPv4 **Total Length = 61 bytes**; Frame 3092 IPv4 **Total Length = 144 bytes**

Despite these differences at the transport and application layers, the IPv4 datagram format and forwarding logic remained constant, demonstrating the layer independence of the network layer. This comparison reinforced that the data plane operates identically regardless of which transport or application protocol uses it (Module 4.2).

Progress Note: Comparison analysis strengthened understanding of layering and header independence.

Activity 2: IP and Routing (Module 4, Active Class 6)

Learning Objective: Understand data plane forwarding through a postal analogy and network simulation.

1. Postal System Analogy

Group discussion mapped parcel delivery to an international recipient:

Parcel Journey Steps:

1. Sender drops parcel in local mailbox
2. Local post office collects and sorts by destination postcode
3. Regional sorting centre groups by destination region
4. National distribution hub handles domestic forwarding
5. International dispatch centre processes customs/export
6. Destination country's main import hub receives international mail
7. Destination regional centre sorts to local areas
8. Local delivery office handles final distribution
9. Postal worker delivers to the recipient's address

Key Characteristics:

- Each facility examines the destination address only
- Makes local forwarding decision using established rules
- Sender needs no knowledge of the complete path
- System scales through standardised addressing

Progress note: Analogy clarified the destination-based forwarding principle.

2. Mapping to IP Network Communication

We mapped the postal analogy to IP packet delivery across networks:

Postal Element	IP Network Equivalent
Sender's mailbox	Source PC (10.1.1.10)
Local post office	Default gateway router
Regional centers	ISP routers
International hubs	Backbone routers
Destination post office	Destination LAN gateway
Recipient's home	Destination PC (192.168.1.10)

Core Principle: Routers use destination IP address for forwarding table lookup - no global path knowledge required (Module 4.1).

Progress note: Mapping reinforced data plane destination-based forwarding.

3. Building the Two-LAN Network and Assigning IP Addresses

Topology: PC1, PC2 (10.1.1.0/24) ↔ Router ↔ PC3, PC4 (192.168.1.0/24)

Role Assignments:

- **Member 1:** Router (forwarding device)
- **Member 2:** PC1 (left LAN)
- **Member 3:** PC2 (left LAN)
- **Member 4:** PC3 (right LAN)
- **Member 5:** PC4 (right LAN)

IP Configuration:

Device	IP Address	Subnet Mask	Default Gateway	Interface
Router (Left)	10.1.1.1	255.255.255.0	N/A	LAN1
Router (Right)	192.168.1.1	255.255.255.0	N/A	LAN2
PC1	10.1.1.10	255.255.255.0	10.1.1.1	Switch 1
PC2	10.1.1.20	255.255.255.0	10.1.1.1	Switch 1
PC3	192.168.1.10	255.255.255.0	192.168.1.1	Switch 2
PC4	192.168.1.20	255.255.255.0	192.168.1.1	Switch 2

Design Logic:

- Router interfaces = default gateways for each LAN
- /24 subnets = 254 usable host addresses each
- Intra-LAN: Layer 2 switching (MAC addresses)
- Inter-LAN: Layer 3 routing required (Module 4.2)

Progress note: Mastered gateway configuration and subnet addressing.

4. IP Packet Journey from PC1 to PC3

We walked through the complete journey when PC1 (10.1.1.10) sends a packet to PC3 (192.168.1.10):

Step-by-Step Explanation:

1. **PC1 checks subnet:** PC1 observes that destination 192.168.1.10 is not in its local /24 subnet (10.1.1.0/24), so it cannot deliver the packet directly via Layer 2 switching.
2. **PC1 sends to gateway:** PC1 encapsulates the IP datagram in an Ethernet frame with the destination MAC address of its default gateway (10.1.1.1), the router's left interface.
3. **Switch 1 forwarding:** Switch 1 operates at Layer 2, examining MAC addresses only and forwarding the frame to the router's left interface based on its MAC address learning table.
4. **Router receives packet:** The router receives the Ethernet frame on its left interface. It removes the Ethernet header, exposing the IP datagram.
5. **Router examines destination:** The router reads the IP header's destination address (192.168.1.10) and consults its forwarding table.
6. **Forwarding table lookup:** The router's forwarding table contains an entry for the subnet 192.168.1.0/24, reachable via the right interface (192.168.1.1).
7. **TTL and checksum update:** Before forwarding, the router decrements the TTL field by 1 (standard practice to prevent infinite loops) and recalculates the IP header checksum since the TTL changed (Module 4.2).
8. **Re-encapsulation:** The router encapsulates the modified IP datagram in a new Ethernet frame. The new frame's source MAC is the router's right interface, and the destination MAC is determined by ARP (either PC3's MAC if on the same link, or another intermediate device).
9. **Switch 2 forwarding:** Switch 2 receives the frame and, based on its MAC address table, forwards it to the interface where PC3 is connected.
10. **PC3 receives:** PC3 receives the Ethernet frame, strips the link-layer header, and recognises that the IP datagram is addressed to it. It passes the datagram to its transport layer (TCP or UDP).

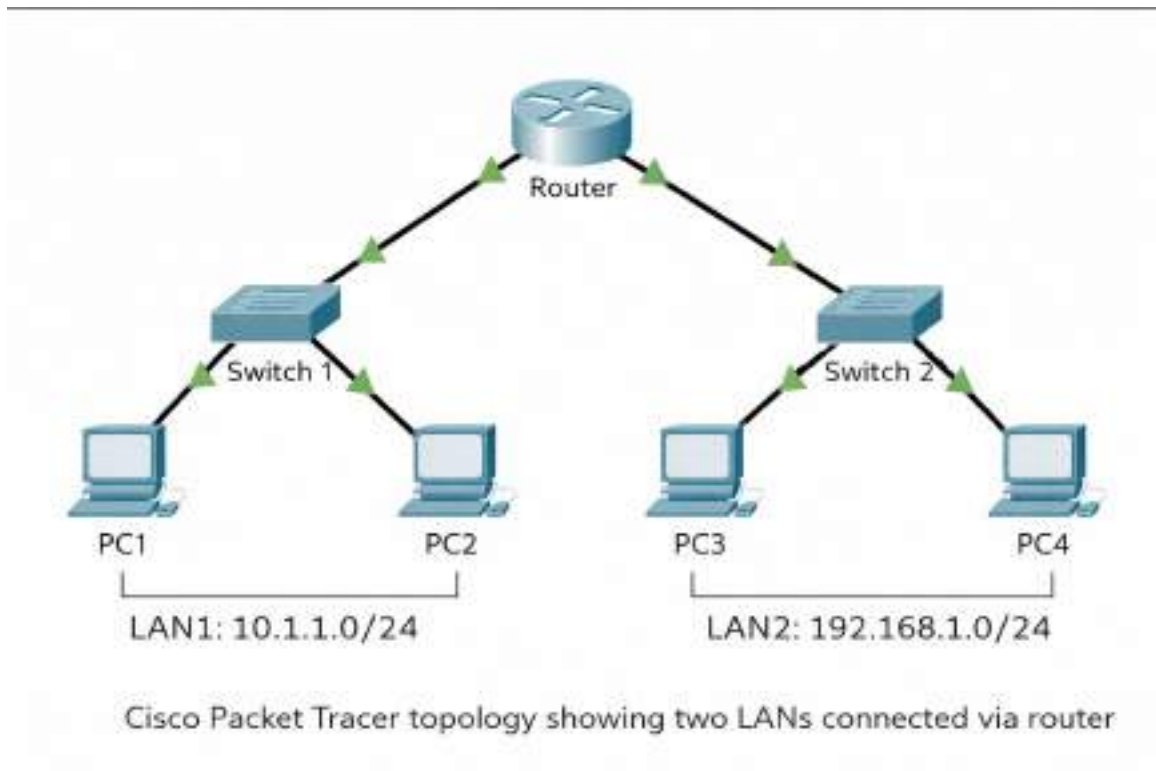
This journey illustrates the data plane operation: each router makes independent decisions based on its forwarding table and the destination address, without awareness of the complete path or other routers' decisions (Module 4.1, 4.2).

Progress note: Step-by-step walkthrough connected forwarding table lookup, TTL decrement, and re-encapsulation to concrete router operations observed later in

Packet Tracer.

Activity 3: Implementing the Network in Cisco Packet Tracer (Module 4.3, Active Class 6)

Learning Objective: Build a simple network in Cisco Packet Tracer and simulate packet forwarding to verify data plane operation.



Implementation Steps:

1. Building the Topology

Using Cisco Packet Tracer, I constructed the two-LAN topology:

- Placed 1 Router, 2 Switches, 4 PCs in the logical workspace
- Connected Router to each Switch using copper straight-through Ethernet cables
- Connected each PC to its respective Switch
- Enabled "Always Show Port Labels in Logical Workspace" (Options > Preferences) for clarity

Topology Verification: Confirmed all devices were connected with visible port labels on each interface.

2. Configuring Router Interfaces

On the Router (Member 1):

- Navigated to Router > Config tab > Routing > Static
- Configured GigabitEthernet0/0 (left interface):
 - IP Address: 10.1.1.1
 - Subnet Mask: 255.255.255.0
 - Status: On
- Configured GigabitEthernet0/1 (right interface):
 - IP Address: 192.168.1.1
 - Subnet Mask: 255.255.255.0
 - Status: On

Verification: Both interfaces showed "On" status with correct IP addresses.

3. Configuring PC IP Addresses and Gateways

For each PC, I configured IP settings via Desktop > IP Configuration:

- **PC1:** IP 10.1.1.10, Mask 255.255.255.0, Gateway 10.1.1.1
- **PC2:** IP 10.1.1.20, Mask 255.255.255.0, Gateway 10.1.1.1
- **PC3:** IP 192.168.1.10, Mask 255.255.255.0, Gateway 192.168.1.1
- **PC4:** IP 192.168.1.20, Mask 255.255.255.0, Gateway 192.168.1.1

Verification: All PCs displayed green network status indicators confirming configuration acceptance.

4. Testing Connectivity with Ping

From PC1, I opened Command Prompt (Desktop > Command Prompt) and executed:

text

ping 192.168.1.10

Expected Result: Successfully received ICMP Echo Replies from PC3, confirming end-to-end connectivity across the router.

Observed Result: Ping successful; all reply packets received with low latency.

Comment: Successful ping demonstrated that the IP configuration, default gateways, and router forwarding were correctly implemented.

5. Using Simulation Mode to Observe Forwarding

I switched Packet Tracer to Simulation mode (top menu) and set up a ping from PC1 to PC3 again:

ping 192.168.1.10

Simulation Observations (stepping through events):

1. **ICMP Echo Request Leave PC1:** The packet, encapsulated in an Ethernet frame addressed to the router's left interface MAC, departs PC1.
2. **Traverse Switch 1:** Switch 1 forwards the frame to the router's left interface based on its learned MAC address table.
3. **Router Processes Packet:** The router removes the Ethernet header and examines the IP destination (192.168.1.10).
4. **Router Forwarding Decision:** The router consults its routing table and selects the right interface (192.168.1.1) as the output port.
5. **TTL Decrement:** Simulation displays the TTL being decremented by the router (visible in packet details in advanced views).
6. **Re-encapsulation and Transmission:** The router encapsulates the IP datagram in a new Ethernet frame with source MAC of its right interface.
7. **Traverse Switch 2:** Switch 2 learns the router's MAC and forwards the frame to PC3's interface.
8. **ICMP Echo Request Arrives at PC3:** PC3 receives the packet and generates an ICMP Echo Reply.
9. **Reply Path:** The Echo Reply travels the reverse path: PC3 to Switch 2 to Router to Switch 1 to PC1.

Visual Confirmation: The simulation timeline displayed each packet step with colour-coded nodes, confirming that data plane forwarding operated exactly as theoretically described in Module 4.

Progress note: Simulation mode provided visual confirmation that routers perform per-packet forwarding decisions based on destination IP addresses and forwarding tables, directly validating the data plane concepts (Module 4.2).

6. Evidence Documentation

Screenshots captured for submission:

- Topology diagram with all devices and port labels visible
- Router interface configuration (GigabitEthernet0/0 and 0/1 with IPs)
- PC configuration windows showing IP, mask, and gateway values
- Ping command output showing successful ICMP replies
- Simulation mode timeline showing packet journey with events
- Cisco Packet Tracer Preferences window with "Always Show Port Labels" enabled

Summary of Learning Evidence

Through Module 4 exercises and Active Class 6 activities, I demonstrated a comprehensive understanding of data plane forwarding, IP addressing, routing concepts, and router operations:

- **Module Exercises:** Practised CIDR subnet calculations and Packet Tracer interface configuration
- **Activity 1 (Wireshark):** Analysed real IPv4 headers, protocol fields, and fragmentation flags in captured packets
- **Activity 2 (Role Play):** Reasoned through packet journeys step-by-step using postal analogies and network topologies
- **Activity 3 (Packet Tracer):** Implemented a two-LAN network, verified connectivity, and observed data plane forwarding in simulation

This evidence portfolio connects theory (Module 4.1-4.3 content) to practice (Wireshark, Packet Tracer, group discussion), demonstrating active learning and mastery of network layer data plane concepts.

References

- Module 4: Network Layer – Data Plane (sections 4.1, 4.2, 4.3; D2L unit site)
- Module 4 PPT: Network Layer – Data Plane (36 slides)
- Active Class 6: Journey of an IP Packet (learning objectives and activity descriptions)
- Kurose, J. F. & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (7th ed.). Chapter 4

[Quiz List](#) > [Submissions](#)

Quiz Submissions – Module 4: Network Layer Data Plane

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %



Quiz List



Current Quizzes	Evaluation Status	Attempts
Module 1: Network Layer Data Plane v Due on 29 March 2026 11:59 PM Available on 11 March 2026 12:01 AM until 29 March 2026 11:59 PM	Feedback: On Attempt	1 / 2
Test Quizzes	Evaluation Status	Attempts
Module 2: Introduction to Computer Networking v Due on 18 February 2026 4:00 PM Available on 06 February 2026 6:00 AM until 17 February 2026 4:00 PM	Feedback: On Attempt	1 / 3
Module 3: Application Layer v Due on 03 March 2026 4:00 PM Available on 11 February 2026 12:01 PM until 03 March 2026 4:00 PM	Feedback: On Attempt	1 / 4
Module 4: Transport Layer v Due on 20 March 2026 11:59 PM Available on 24 February 2026 12:01 AM until 20 March 2026 11:59 PM	Feedback: On Attempt	1 / 3



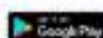
Grades

Print



Stay up-to-date on your grades with the Pulse App

Keep track with your courses on the go!



Grades Item	Points	Grade	Comments and Assessments
Module 1: Introduction to Computer Networking	10 / 10	100 %	View Quiz Attempts
Module 2: Application Layer	10 / 10	100 %	View Quiz Attempts
Module 3: Transport Layer	10 / 10	100 %	View Quiz Attempts
Unfiled	- / 10	-%	
Module 6: Data Link Layer	- / 10	-%	
Module 7: Physical Layer	- / 10	-%	
Module 4: Network Layer Data Plane	10 / 10	100 %	View Quiz Attempts

10 Task 7.1P

Pass task

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

This task focused on reflecting on my overall progress in the unit and updating the status of tasks aligned with my target grade. It did not involve explaining protocol stack layers, developing network programs, using networking tools for technical analysis, or analysing network configurations and real network data. Therefore, ULO1, ULO2, and ULO3 are not directly addressed (0/5). ULO4 has minimal relevance (1/5), as the task required reflection, but the focus was on my learning progress, planning, and time management rather than critiquing network construction or analysing security and scalability issues in real network contexts.

Date	Author	Comment
2026/03/22 12:04	Jahan Garg	Working On It
2026/03/22 22:01	Jahan Garg	Ready for Feedback
2026/03/22 22:01	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 7.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/03/25 08:24	Amanpreet Kaur	Dear Jahan, good...Keep it up!!!
2026/03/25 08:25	Amanpreet Kaur	Complete
2026/03/25 08:25	Amanpreet Kaur	Complete
2026/03/25 09:12	Jahan Garg	Thank you mam

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 7.1P

Submitted By:

Jahan GARG

2410994805

2026/03/22 22:01

Tutor:

Amanpreet KAUR

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

This task focused on reflecting on my overall progress in the unit and updating the status of tasks aligned with my target grade. It did not involve explaining protocol stack layers, developing network programs, using networking tools for technical analysis, or analysing network configurations and real network data. Therefore, ULO1, ULO2, and ULO3 are not directly addressed (0/5). ULO4 has minimal relevance (1/5), as the task required reflection, but the focus was on my learning progress, planning, and time management rather than critiquing network construction or analysing security and scalability issues in real network contexts.

March 22, 2026



Task 7.1P: Learning Plan Check-in

Reflection on Progress

I have reviewed my progress in SIT202 and identified both strengths and gaps. The most challenging areas were the DNS topic in the Application Layer (missed due to health-related class absences) and the lengthy Network and Transport Layer content, which required more revision time. I also faced time-management difficulties across four units, and the task backlog increased when OnTrack's new version was introduced. To overcome these hurdles, I am following a structured catch-up plan: regular consultations with Dr Amanpreet Kaur, peer discussions, and focused use of D2L resources, reference texts, authentic websites, and YouTube tutorials for Wireshark and Cisco Packet Tracer.

In terms of progress, I am improving but not fully on schedule: I have completed Tasks 1.1P and 5.2C; 1.2P remains pending, while 3.1P and 5.1P require resubmission. I do not need to change my target grade and will continue targeting Distinction through consistent effort and conceptual mastery.

Updated Learning Plan- SIT202 – S4 2024

Study Plan

Tasks	Due Date (D)
D tasks (Lesson Reviews)	
Task 1.1P	Completed
Task 1.2P Introduction	09 th February Working on it
Task 3.1P Application Layer	24 th February Working on it
Task 5.1P Transport Layer	16 th March Working on it
Task 6.1P Network Layer-Data Plane	Submitted
Task 7.1P	27 th March Working on it
Task 8.1P Network Layer – Control Plane	6 th April Haven't started
Task 9.1P Data Link Layer	17 th April

	Haven't started
Task 10.1P Physical Layer	27 th April
	Haven't started
Task 10.2P	31 st April
	Haven't started
C tasks	
Task 4.1C Above and Beyond Pass	17 th March
	Working on it
Task 5.2C My DNS Server Sketch	Completed
Task 6.2C My DNS Server	23 rd March
	Working on it
D tasks	
Task 5.3D Above and Beyond Credit	12 th April
	Haven't started
Task 7.2D Amazing Networks	30 th March
	Haven't started

My target grade is Distinction.

Grading Agreement

I, (Jahan Garg, 2410994805), confirm that I have read and understood how this unit works: I understand that:

- I will receive a grade in accordance with the number of modules and tasks that I complete.
- I can change my target grade at any time during the trimester, up until I submit my portfolio.
- I am responsible for providing evidence of my work to satisfy the requirements in each module completed.
- Following my initial submission, I will work with my tutor and the teaching team to update and complete my submissions until they demonstrate that I have achieved the learning objectives of the module.
- I am responsible for attending class discussions of some of my tasks, as detailed in the hurdle requirement of the unit

I commit to submitting the initial submission of my work before the deadlines listed in the table above. I acknowledge that these deadlines can be modified by me, in

agreement with my tutor, when the necessity arises, and that I am responsible for requesting these modifications as soon as I become aware of the need to do so, and no later than the deadline to be modified.

- Name: Jahan Garg
- Student ID: 2410994805
- Date: 22-03-2026

- Signature: 

11 Task 6.2C

credit task

Outcome	Weight
Layered encapsulation	◆◆◆◆◇

In this task, I implemented a Python DNS server and client using UDP sockets on port 8053, demonstrating DNS protocol handling and reliable local testing. For ULO1, I achieved application-layer DNS protocol implementation and transport-layer UDP socket communication, showing clear client-server data flow through query transmission and response processing. For ULO2, I designed and developed network programs implementing DNS functionality from scratch without external libraries, directly matching the requirement to build working protocol applications. For ULO3, basic terminal-based testing verified server operation but lacked industry-standard network analysis tools or examination of real network configurations. For ULO4, I systematically debugged port communication issues, implemented UDP timeout solutions, and progressed from 5.2C pseudocode to a working 6.2C implementation, demonstrating strong reflection on protocol behaviour and practical challenges.

Outcome	Weight
Interaction with Networks	◆◆◆◆◆

In this task, I implemented a Python DNS server and client using UDP sockets on port 8053, demonstrating DNS protocol handling and reliable local testing. For ULO1, I achieved application-layer DNS protocol implementation and transport-layer UDP socket communication, showing clear client-server data flow through query transmission and response processing. For ULO2, I designed and developed network programs implementing DNS functionality from scratch without external libraries, directly matching the requirement to build working protocol applications. For ULO3, basic terminal-based testing verified server operation but lacked industry-standard network analysis tools or examination of real network configurations. For ULO4, I systematically debugged port communication issues, implemented UDP timeout solutions, and progressed from 5.2C pseudocode to a working 6.2C implementation, demonstrating strong reflection on protocol behaviour and practical challenges.

Outcome	Weight
Networking Tools	◆◆◆◆◇

In this task, I implemented a Python DNS server and client using UDP sockets on port 8053, demonstrating DNS protocol handling and reliable local testing. For ULO1, I achieved application-layer DNS protocol implementation and transport-layer UDP socket communication, showing clear client-server data flow through query transmission and response processing. For ULO2, I designed and developed network programs implementing DNS functionality from scratch without external libraries, directly matching the requirement to build working protocol applications. For ULO3, basic terminal-based testing verified server operation but lacked industry-standard network analysis tools or examination of real network configurations. For ULO4, I systematically debugged port communication issues, implemented UDP timeout solutions, and progressed from 5.2C pseudocode to a working 6.2C implementation, demonstrating strong reflection on protocol behaviour and practical challenges.

Outcome	Weight
Analysing real network issues	◆◆◆◆◇

In this task, I implemented a Python DNS server and client using UDP sockets on port 8053, demonstrating DNS protocol handling and reliable local testing. For ULO1, I achieved application-layer DNS protocol implementation and transport-layer UDP socket communication, showing clear client-server data flow through query transmission and response processing. For ULO2, I designed and developed network programs implementing DNS functionality from scratch without external libraries, directly matching the requirement to build working protocol applications. For ULO3, basic terminal-based testing verified server operation but lacked industry-standard network analysis tools or examination of real network configurations. For ULO4, I systematically debugged port communication

issues, implemented UDP timeout solutions, and progressed from 5.2C pseudocode to a working 6.2C implementation, demonstrating strong reflection on protocol behaviour and practical challenges.

Date	Author	Comment
2026/03/22 22:01	Jahan Garg	Working On It
2026/03/25 11:59	Jahan Garg	Ready for Feedback
2026/03/25 12:02	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 6.2C task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/03/30 05:09	Amanpreet Kaur	Dear Jahan, you are required to add the screenshot of program execution with output
2026/03/30 05:10	Amanpreet Kaur	You are required to prepare the flow chart of the task
2026/03/30 05:10	Amanpreet Kaur	Fix and Resubmit
2026/03/31 19:23	Jahan Garg	Ready for Feedback
2026/03/31 19:23	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards,Jahan Garg2410994805X2 Batch
2026/04/06 08:21	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 6.2C

Submitted By:

Jahan GARG

2410994805

2026/03/31 19:23

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◇
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◇
Analysing real network issues	◆◆◆◆◆

In this task, I implemented a Python DNS server and client using UDP sockets on port 8053, demonstrating DNS protocol handling and reliable local testing. For ULO1, I achieved application-layer DNS protocol implementation and transport-layer UDP socket communication, showing clear client-server data flow through query transmission and response processing. For ULO2, I designed and developed network programs implementing DNS functionality from scratch without external libraries, directly matching the requirement to build working protocol applications. For ULO3, basic terminal-based testing verified server operation but lacked industry-standard network analysis tools or examination of real network configurations. For ULO4, I systematically debugged port communication issues, implemented UDP timeout solutions, and progressed from 5.2C pseudocode to a working 6.2C implementation, demonstrating strong reflection on protocol behaviour and practical challenges.

March 31, 2026



SIT202 Task 6.2C DNS Server Implementation

Introduction

This report presents the final implementation of a Domain Name System (DNS) server and client developed in Python for SIT202 Task 6.2C. The solution extends the earlier 5.2C pseudocode into a working User Datagram Protocol (UDP) application that manually creates, sends, receives, and parses DNS packets for A and CNAME records.

The implementation focuses on core DNS behaviour required by the task: local record storage, packet parsing, A-record lookup, CNAME resolution, response generation, and terminal-based testing. The final version also includes practical client-side improvements such as timeout handling, transaction ID validation, and a basic malformed-packet check to improve reliability during local testing.

Task Basis

Task 6.2C was built directly on the earlier 5.2C design work. That earlier design defined four major stages of the DNS process: server initialisation, query listening and processing, A/CNAME record handling, and DNS response generation.

The final implementation follows that same structure closely. The server listens for incoming UDP DNS queries, validates the question fields, performs local record lookup, and returns a binary DNS response using the same transaction ID so that the client can match each response to the original request.

Implementation Overview

The server was implemented using Python's socket and struct modules so that DNS packet handling remained explicit and educational rather than abstracted behind an external library. This made the message format, flag handling, record packing, and byte-level parsing visible in the code and aligned well with the protocol-oriented focus of the task.

The final DNS table contains five local records, including direct A records and alias-based CNAME records used for testing both simple and chained resolution behaviour. For A queries, the matching IPv4 address is returned; for CNAME queries, the server returns the alias mapping and then optionally includes the A record of the canonical name in the same response.

The client was implemented as an interactive test interface. It allows the user to enter a hostname, choose a record type, send the query to the local DNS server, and display the received records in a readable format.

DNS Records

Hostname	Type	Value	Purpose
www.example.com.	A	93.184.216.34	Primary A-record test case.
example.com.	A	93.184.216.34	Additional direct lookup record.
mail.example.com.	CNAME	www.example.com.	Alias used to test CNAME resolution.
test.example.com.	A	192.0.2.1	Secondary A-record test case.
alias.example.com.	CNAME	www.example.com.	Additional alias validation record.

System Flowcharts

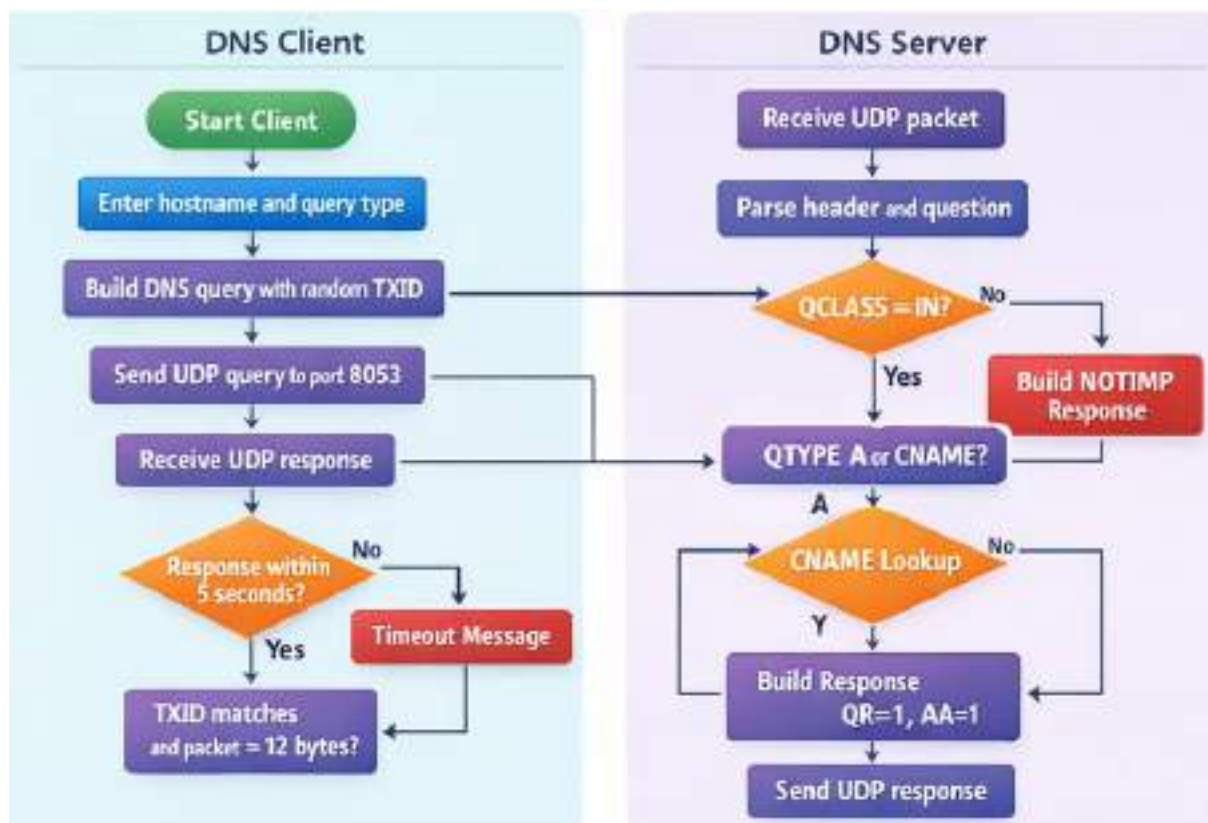
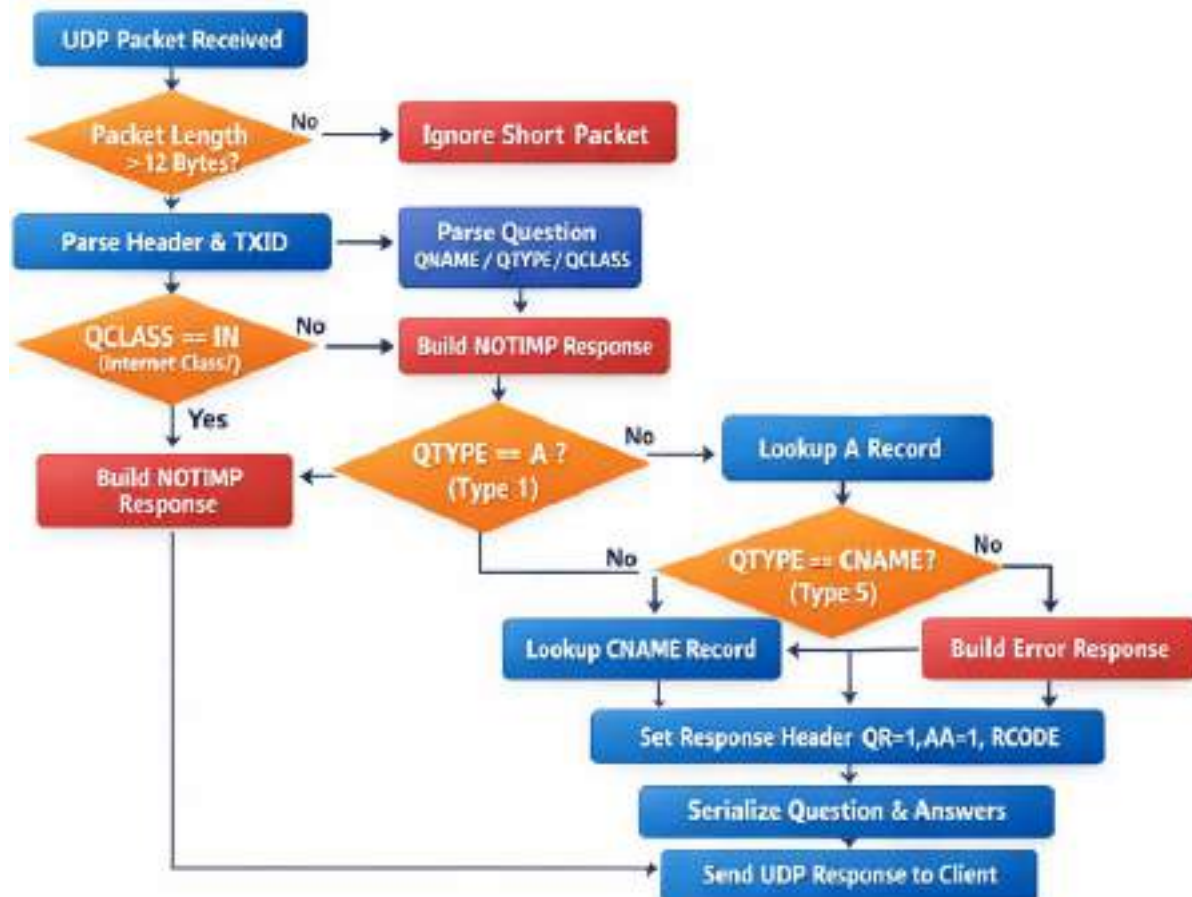


Figure 1: DNS Client-Server Workflow

Complete end-to-end process showing DNS query transmission, server-side processing, response validation, and timeout handling over UDP on port 8053.

**Figure 2: DNS Server Processing Logic**

Internal server workflow showing packet validation, QCLASS and QTYPE decision branching, record lookup, and DNS response construction.

DNS Theory Applied

DNS messages consist of a 12-byte header followed by a question section and, in responses, one or more answer records. The implementation applies this structure directly by unpacking the header fields, reading the queried name from the question section, and serialising resource records into the answer section of the response packet.

An A record maps a hostname to an IPv4 address, while a CNAME record maps an alias to a canonical hostname. In this implementation, a CNAME response may also

include the final A record of the canonical host so the client can display the complete resolution chain in a single reply.

The server also applies key DNS response behaviours from the earlier design: echoing the original transaction ID, setting response flags appropriately, returning an authoritative answer, and using error semantics such as NOTIMP for unsupported cases and NXDOMAIN when no matching record exists.

Server Design

The server begins by creating a UDP socket, enabling address reuse, binding to the chosen local port, and printing the loaded record set for startup verification. It then enters a continuous receive loop and waits for DNS packets from the client.

When a packet is received, the server first checks that a complete DNS header is present before parsing deeper fields. It then extracts the transaction ID, query name, query type, and query class from the packet and validates that the request belongs to the Internet class (IN).

If the query type is A, the server searches for a matching A record and serialises the IPv4 address into a resource record. If the query type is CNAME, it serialises the canonical-name record and, where available, includes the A record of the canonical host in the same response.

Client Design

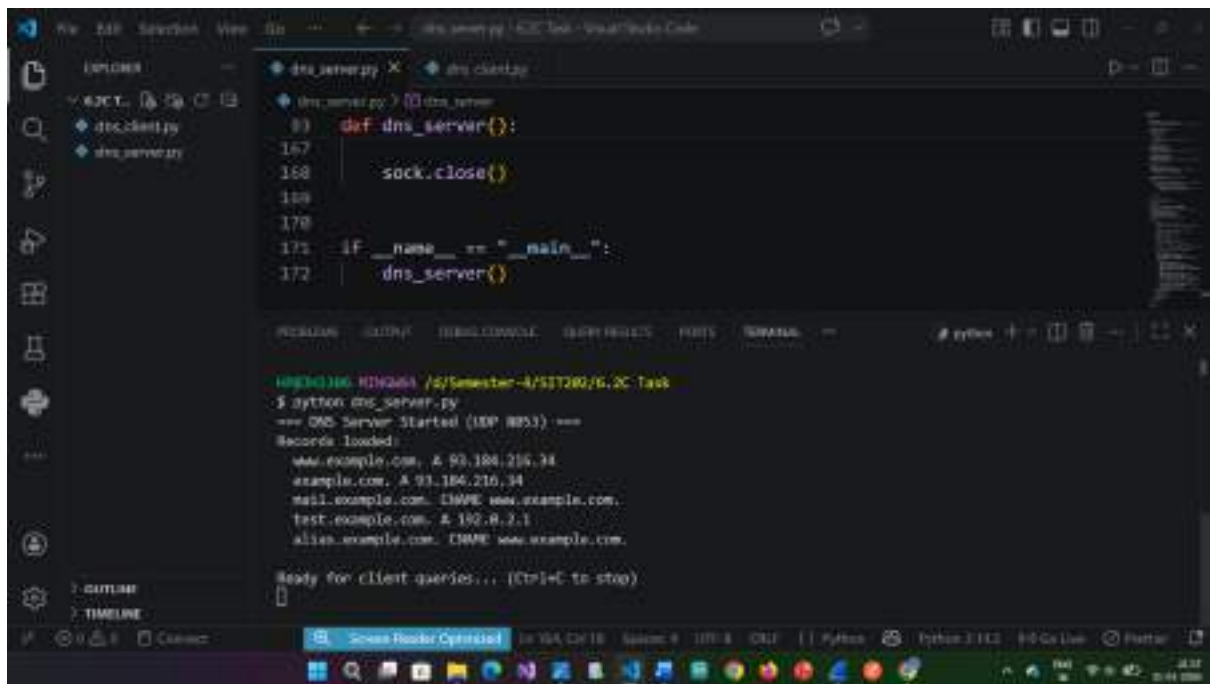
The client converts the entered hostname into DNS label format, builds a standard query packet, and sends it to the configured local server address over UDP. A random transaction ID is included so the response can be verified when it returns.

After transmission, the client waits up to five seconds for a reply. If no packet arrives, it prints a timeout message, so the program remains responsive, and the testing outcome remains clear to the user.

When a response is received, the client checks that the packet is large enough to contain a header, verifies that the transaction ID matches the request, and then prints each returned answer in readable form. Records are displayed as IPv4 addresses, and CNAME records are displayed as canonical hostnames.

Successful Execution

The final version was tested successfully with both the server and client configured to use port 8053, which produced stable local request-response behaviour during final execution. Testing confirmed correct startup, correct A-record lookup, and correct CNAME-chain handling.



```
def dns_server():
    sock.close()

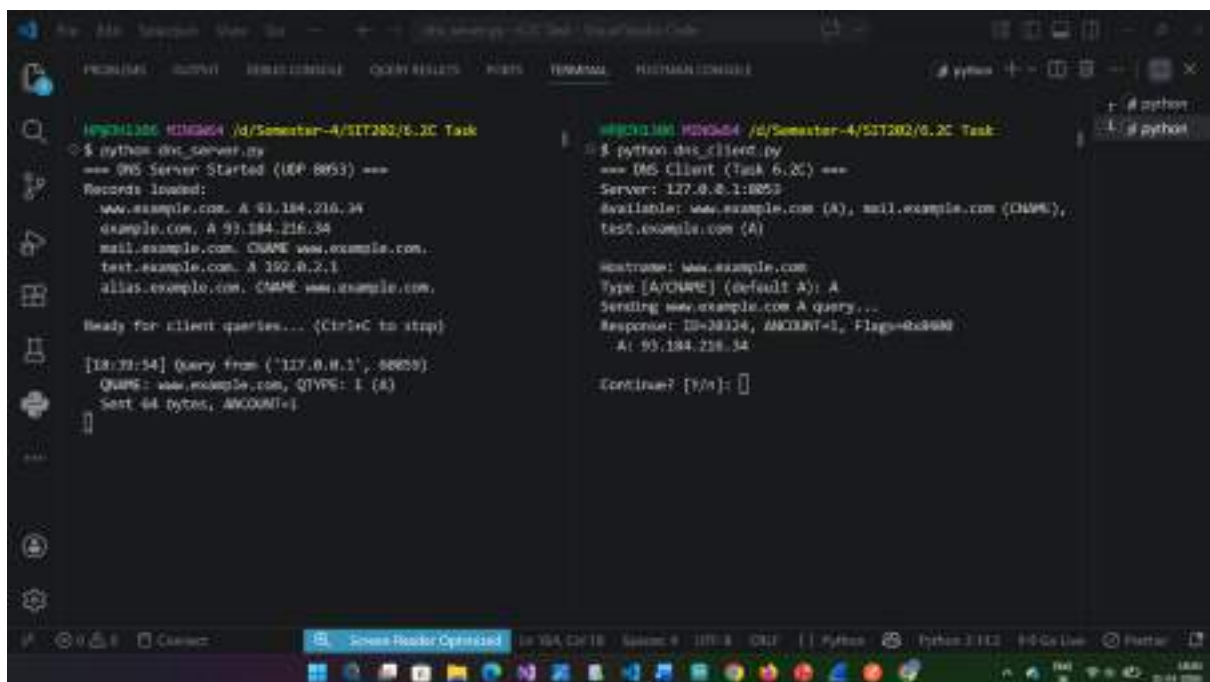
if __name__ == "__main__":
    dns_server()
```

```
$ python dns_server.py
=== DNS Server Started (UDP 8053) ===
Records loaded:
www.example.com. A 93.184.216.34
example.com. A 93.184.216.34
mail.example.com. CNAME www.example.com.
test.example.com. A 192.0.2.1
alias.example.com. CNAME www.example.com.

Ready for client queries... (Ctrl+C to stop)
```

Figure 3. DNS server startup on port 8053

This screenshot shows that the DNS server started correctly, loaded all configured records, and was ready to accept client queries.



```
$ python dns_client.py
=== DNS Client (Task 6.2C) ===
Server: 127.0.0.1:8053
Available: www.example.com (A), mail.example.com (CNAME),
test.example.com (A)

Hostname: www.example.com
Type [A/CNAME] (default A): A
Sending www.example.com A query...
Response: ID=38324, ANSWER=1, Flags=0x0000
A: 93.184.216.34

Continue? [Y/n]:
```

```
[18:39:54] Query from ("127.0.0.1", 50055):
  QNAME: www.example.com, QTYPE: 1 (A)
  Sent 44 bytes, ANSWER=1
```

Figure 4. Successful A-record query for www.example.com

This screenshot shows the client receiving the correct IPv4 address and the server logging the corresponding query for the A record

```

leg@11306 R102604 /d/Semester-4/SIT202/6.2C Task
$ python dns_server.py
=== DNS Server Started (UDP: 8053) ===
Records loaded:
www.example.com. A 93.184.216.34
example.com. A 93.184.216.34
mail.example.com. CNAME www.example.com.
test.example.com. A 192.0.2.1
alias.example.com. CNAME www.example.com.

Ready for client queries... (Ctrl+C to stop)

[18:31:54] Query from ("127.0.0.1", 5055):
  QNAME: www.example.com, QTYPE: 1 (A)
  Sent 64 bytes, RRCOUNT=1

[18:44:06] Query from ("127.0.0.1", 50131):
  QNAME: mail.example.com, QTYPE: 5 (CNAME)
  Sent 118 bytes, RRCOUNT=2

leg@11306 R102604 /d/Semester-4/SIT202/6.2C Task
$ python dns_client.py
=== DNS Client (Task 6.2C) ===
Server: 127.0.0.1:8053
Available: www.example.com (A), mail.example.com (CNAME),
test.example.com (A)

Hostname: mail.example.com
Type [A/CNAME] (default A): CNAME
Sending mail.example.com CNAME query...
Response: ID=6737, RRCOUNT=2, Flags=Rc4096
CNAME: www.example.com
A: 93.184.216.34

Continue? [Y/n]: n
Client stopped.

```

Figure 5. Successful CNAME query for mail.example.com

This screenshot shows the client receiving both the CNAME record and the A record of the canonical hostname, confirming correct chained resolution behaviour.

These screenshots together demonstrate that the final implementation works correctly for both direct and alias-based DNS lookups.

Challenges and Resolutions

The main challenge during development was inconsistent behaviour when testing on port 5353. In that configuration, the client could send a query while the server showed no corresponding request, indicating that local testing on that port was not producing reliable communication for this assignment setup.

```

leg@11306 R102604 /d/Semester-4/SIT202/6.2C Task
$ python dns_server.py
=== DNS Server Started (UDP: 5353) ===
Records loaded:
www.example.com. A 93.184.216.34
example.com. A 93.184.216.34
mail.example.com. CNAME www.example.com.
test.example.com. A 192.0.2.1
alias.example.com. CNAME www.example.com.

Ready for client queries... (Ctrl+C to stop)

leg@11306 R102604 /d/Semester-4/SIT202/6.2C Task
$ python dns_client.py
=== DNS Client (Task 6.2C) ===
Server: 127.0.0.1:5353
Available: www.example.com (A), mail.example.com (CNAME),
test.example.com (A)

Hostname: www.example.com
Type [A/CNAME] (default A): A
Sending www.example.com A query...

```

Figure 6. Port 5353 non-response issue during testing

This screenshot shows the client attempting to send a DNS query while the server does not receive a visible request, highlighting the original testing problem.

This issue was resolved by moving both the client and server to port 8053 for the final implementation. After that change, the request-response exchange became stable, and the final testing screenshots were captured successfully.

A second challenge was that UDP provides no built-in delivery guarantee. As a result, early testing could appear to freeze when no response was returned, so a 5-second timeout was added to provide a clear outcome and improve the user experience.

A third improvement involved basic response validation. The final client checks for a minimum packet length and validates the transaction ID before parsing the response, reducing the chance of accepting an unrelated or malformed reply.

Reflection

This task strengthened understanding of how Domain Name System communication operates at the packet level rather than only at the conceptual level. Constructing queries and responses manually made the significance of transaction IDs, flags, counts, question fields, and resource records much clearer in practice.

The task also reinforced the importance of systematic debugging in network programming. Port selection, timeouts, and validation checks can strongly affect observed behaviour, and the use of server logs together with client output made it easier to isolate the actual cause of failures.

Most importantly, the task demonstrated the difference between pseudocode and a working networked application. The 5.2C design provided the structure, but the 6.2C implementation required additional attention to runtime behaviour, binary data handling, local testing conditions, and user-facing feedback.

Quality Review

The final solution is appropriate for the assignment scope because it remains simple while still demonstrating the key DNS concepts required for learning. It supports A and CNAME records, validates essential request fields, generates correct responses, and includes enough client-side protection to make local testing reliable without overcomplicating the code.

The design is also logically organised. Core responsibilities such as name packing, query building, record creation, record lookup, flag construction, response parsing, and interactive testing are separated clearly, which improves readability and maintainability.

Mapping from 5.2C to 6.2C

5.2C Design Area	6.2C Final Outcome
Server initialisation	Implemented through UDP socket setup, binding, and local record loading.
Query listening and processing	Implemented through a continuous receive loop with parsing and validation.
A-record handling	Implemented through direct local hostname-to-IP lookup.
CNAME handling	Implemented through alias lookup with optional canonical A record return.
DNS response generation	Implemented through manual header creation, flag setting, answer serialisation, and transmission.
Runtime robustness	Improved through timeout handling, packet-length checks, and transaction ID verification.

Conclusion

The final Task 6.2C implementation successfully converts the earlier DNS server sketch into a working Python client-server application that demonstrates DNS packet construction, UDP communication, and handling of A and CNAME records. The final output is technically sound, consistent with the design prepared in Task 5.2C, and supported by clear testing evidence through successful execution screenshots and debugging evidence.

The final version is also stronger because it reflects practical refinement during development. The move to port 8053, the addition of timeout handling, and the inclusion of response validation improved reliability while preserving simplicity, making the submission suitable, complete, and professionally presented for the assignment.

References

1. RFC 1035: *Domain Names - Implementation and Specification*. IETF.
<https://datatracker.ietf.org/doc/html/rfc1035>
2. Microsoft Learn: *Network ports used by DNS*.
<https://learn.microsoft.com/en-us/windows-server/networking/dns/network-ports>
3. DNSimple: *Differences between A and CNAME records*.
<https://support.dnssimple.com/articles/differences-a-cname-records/>
4. Cornell University IT: *CNAME, A Record, and MX Record*.
<https://it.cornell.edu/dns/cname-record-and-mx-record>
5. KodeKloud Notes: *DNS Request and Responses*.
<https://notes.kodekloud.com/docs/Demystifying-DNS/DNS-as-a-Protocol/DNS-Request-and-Responses/page>
6. Prior Task 5.2C pseudocode and analysis prepared as the conceptual basis for this implementation.

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The original submission can be retrieved via the "Download Uploaded Files" function.

```

1  #!/usr/bin/env python3
2  """
3  SIT202 Task 6.2C DNS Client
4  Tests server with interactive hostname queries
5  Jahan Garg | 2410994805
6  """
7
8  import socket
9  import struct
10 import random
11
12 SERVER_ADDRESS = ("127.0.0.1", 8053)
13 BUFFER_SIZE = 512
14 TIMEOUT_SECONDS = 5
15
16
17 def pack_name(name):
18     """Convert a domain name into DNS label format."""
19     labels = name.rstrip(".").lower().split(".")
20     return b"".join(bytes([len(label)]) + label.encode() for label in labels) +
        ↪ b"\x00"
21
22
23 def build_query(hostname, qtype=1):
24     """Build a standard DNS query packet."""
25     transaction_id = random.randint(1, 65535) # Random 16-bit transaction ID
26     header = struct.pack(">HHHHHH", transaction_id, 0x0100, 1, 0, 0, 0)
27     question = pack_name(hostname) + struct.pack(">HH", qtype, 1) # QTYPE,
        ↪ QCLASS=IN
28     return header + question, transaction_id
29
30
31 def parse_name(data, offset):
32     """Parse a DNS name from raw bytes."""
33     name = ""
34     while offset < len(data):
35         length = data[offset]
36         offset += 1
37         if length == 0:
38             break
39         name += data[offset:offset + length].decode(errors="ignore") + "."
40         offset += length
41     return name.rstrip("."), offset
42
43
44 def parse_response(response):
45     """Display DNS response records."""
46     transaction_id, flags, qdcount, ancount, nscount, arcount =
        ↪ struct.unpack(">HHHHHH", response[:12])
47     print(f"Response: ID={transaction_id}, ANCOUNT={ancount}, Flags=0x{flags:04x}")

```



```

48
49     offset = 12
50     _, offset = parse_name(response, offset)
51     offset += 4  # skip QTYPE and QCLASS in question section
52
53     for _ in range(ancount):
54         _, offset = parse_name(response, offset)
55         rtype, rclass, ttl, rdlength = struct.unpack(">HHIH", response[offset:offset
56             ↪ + 10])
57         offset += 10
58
59         if rtype == 1:  # A
60             ip = socket.inet_ntoa(response[offset:offset + 4])
61             print(f"  A: {ip}")
62         elif rtype == 5:  # CNAME
63             cname, _ = parse_name(response, offset)
64             print(f"  CNAME: {cname}")
65         else:
66             print(f"  TYPE {rtype}: {rdlength} bytes")
67
68         offset += rdlength
69
70 def main():
71     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
72     sock.settimeout(TIMEOUT_SECONDS)
73
74     print("=== DNS Client (Task 6.2C) ===")
75     print("Server: 127.0.0.1:8053")
76     print("Available: www.example.com (A), mail.example.com (CNAME),
77     ↪ test.example.com (A)\n")
78
79     while True:
80         hostname = input("Hostname: ").strip()
81         if hostname.lower() in ["quit", "exit", "q"]:
82             break
83         if not hostname:
84             print("Please enter a hostname.\n")
85             continue
86
87         qtype_input = input("Type [A/CNAME] (default A): ").strip().upper()
88         qtype = 5 if qtype_input == "CNAME" else 1
89         qtype_label = "CNAME" if qtype == 5 else "A"
90
91         print(f"Sending {hostname} {qtype_label} query...")
92         query, sent_transaction_id = build_query(hostname, qtype)
93         sock.sendto(query, SERVER_ADDRESS)
94
95         try:
96             response, _ = sock.recvfrom(BUFFER_SIZE)
97
98             if len(response) < 12:
99                 print("Received malformed response packet.\n")
100                 continue

```

```
100
101     received_transaction_id = struct.unpack(">H", response[:2])[0]
102     if received_transaction_id != sent_transaction_id:
103         print("Received response with mismatched transaction ID.\n")
104         continue
105
106     parse_response(response)
107     print()
108     except socket.timeout:
109         print(f"No response received from server within {TIMEOUT_SECONDS}
110             ↪ seconds.\n")
111
112     again = input("Continue? [Y/n]: ").strip().lower()
113     if again == "n":
114         break
115
116     sock.close()
117     print("Client stopped.")
118
119 if __name__ == "__main__":
120     main()
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The original submission can be retrieved via the "Download Uploaded Files" function.

```

1  #!/usr/bin/env python3
2  """
3  SIT202 Task 6.2C DNS Server
4  Implements 5.2C pseudocode - A/CNAME records, UDP port 8053
5  Jahan Garg | 2410994805
6  """
7
8  import socket
9  import struct
10 import time
11
12 DNS_RECORDS = [
13     ("www.example.com.", "A", "93.184.216.34"),
14     ("example.com.", "A", "93.184.216.34"),
15     ("mail.example.com.", "CNAME", "www.example.com."),
16     ("test.example.com.", "A", "192.0.2.1"),
17     ("alias.example.com.", "CNAME", "www.example.com.")
18 ]
19
20 SERVER_HOST = "0.0.0.0"
21 SERVER_PORT = 8053
22 BUFFER_SIZE = 512
23 TTL = 300
24
25
26 def pack_name(name):
27     """Convert a domain name into DNS label format."""
28     labels = name.rstrip(".").lower().split(".")
29     return b"".join(bytes([len(label)]) + label.encode() for label in labels) +
        ↪ b"\x00"
30
31
32 def parse_name(data, offset):
33     """Parse a DNS name from raw bytes."""
34     name = ""
35     while offset < len(data):
36         length = data[offset]
37         offset += 1
38         if length == 0:
39             break
40         name += data[offset:offset + length].decode(errors="ignore") + "."
41         offset += length
42     return name.rstrip("."), offset
43
44
45 def pack_a_record(name, ip):
46     """Create an A resource record."""
47     name_bytes = pack_name(name)
48     rdata = socket.inet_aton(ip)
49     rr_header = struct.pack(">HHIH", 1, 1, TTL, len(rdata)) # TYPE=A, CLASS=IN

```

```

50     return name_bytes + rr_header + rdata
51
52
53 def pack_cname_record(name, canonical_name):
54     """Create a CNAME resource record."""
55     name_bytes = pack_name(name)
56     rdata = pack_name(canonical_name)
57     rr_header = struct.pack(">HHIH", 5, 1, TTL, len(rdata)) # TYPE=CNAME, CLASS=IN
58     return name_bytes + rr_header + rdata
59
60
61 def handle_a_query(qname):
62     """Return A records matching qname."""
63     answers = []
64     for hostname, record_type, value in DNS_RECORDS:
65         if hostname.rstrip(".").lower() == qname.lower() and record_type == "A":
66             answers.append(pack_a_record(qname, value))
67     return answers
68
69
70 def handle_cname_query(qname):
71     """Return CNAME record and optional canonical A record."""
72     answers = []
73
74     for hostname, record_type, value in DNS_RECORDS:
75         if hostname.rstrip(".").lower() == qname.lower() and record_type == "CNAME":
76             answers.append(pack_cname_record(qname, value))
77
78             for hostname2, record_type2, value2 in DNS_RECORDS:
79                 if hostname2.rstrip(".").lower() == value.rstrip(".").lower() and
80                    ↪ record_type2 == "A":
81                     answers.append(pack_a_record(value, value2))
82                     break
83             break
84
85     return answers
86
87
88 def build_flags(qr=1, aa=1, rcode=0):
89     """Build DNS flags field."""
90     return (qr << 15) | (aa << 10) | (rcode & 0x0F)
91
92
93 def dns_server():
94     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
95     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
96     sock.bind((SERVER_HOST, SERVER_PORT)) # Bind to all interfaces on port 8053
97
98     print("=== DNS Server Started (UDP 8053) ===")
99     print("Records loaded:")
100    for name, record_type, value in DNS_RECORDS:
101        print(f" {name} {record_type} {value}")
102    print("\nReady for client queries... (Ctrl+C to stop)")

```

```

103
104     while True:
105         try:
106             data, addr = sock.recvfrom(BUFFER_SIZE)
107
108             if len(data) < 12:
109                 print(f"\n[{time.strftime('%H:%M:%S')}] Ignored short packet from
110                     ↪ {addr}")
111                 continue
112
113             print(f"\n[{time.strftime('%H:%M:%S')}] Query from {addr}")
114
115             header = struct.unpack(">HHHHHH", data[:12])
116             transaction_id = header[0]
117
118             offset = 12
119             qname, offset = parse_name(data, offset)
120
121             if offset + 4 > len(data):
122                 print("Error: Incomplete question section")
123                 continue
124
125             qtype, qclass = struct.unpack(">HH", data[offset:offset + 4])
126
127             qtype_label = "A" if qtype == 1 else "CNAME" if qtype == 5 else "OTHER"
128             print(f"  QNAME: {qname}, QTYPE: {qtype} ({qtype_label}")
129
130             question = data[12:offset + 4]
131
132             if qclass != 1:
133                 print("  ERROR: QCLASS != IN (sending NOTIMP)")
134                 flags = build_flags(qr=1, aa=0, rcode=4)
135                 response_header = struct.pack(">HHHHHH", transaction_id, flags, 1,
136                     ↪ 0, 0, 0)
137                 response = response_header + question
138             else:
139                 if qtype == 1:
140                     answer_records = handle_a_query(qname)
141                     ancount = len(answer_records)
142                     rcode = 0 if ancount > 0 else 3 # NOERROR or NXDOMAIN
143                     flags = build_flags(qr=1, aa=1, rcode=rcode)
144                     response_header = struct.pack(">HHHHHH", transaction_id, flags,
145                         ↪ 1, ancount, 0, 0)
146                     response = response_header + question + b"".join(answer_records)
147
148                 elif qtype == 5:
149                     answer_records = handle_cname_query(qname)
150                     ancount = len(answer_records)
151                     rcode = 0 if ancount > 0 else 3 # NOERROR or NXDOMAIN
152                     flags = build_flags(qr=1, aa=1, rcode=rcode)
153                     response_header = struct.pack(">HHHHHH", transaction_id, flags,
154                         ↪ 1, ancount, 0, 0)
155                     response = response_header + question + b"".join(answer_records)

```

```
153         else:
154             print(" ERROR: Unsupported QTYPE (sending NOTIMP)")
155             flags = build_flags(qr=1, aa=0, rcode=4)
156             response_header = struct.pack(">HHHHHH", transaction_id, flags,
157                 ↪ 1, 0, 0, 0)
158             response = response_header + question
159
160             sock.sendto(response, addr)
161             print(f" Sent {len(response)} bytes, ANCOUNT={struct.unpack('>HHHHHH',
162                 ↪ response[:12])[3]}")
163
164         except KeyboardInterrupt:
165             print("\nServer stopped.")
166             break
167         except Exception as e:
168             print(f"Error: {e}")
169
170     sock.close()
171
172 if __name__ == "__main__":
173     dns_server()
```

12 Task 4.1C

C task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I demonstrated a deeper understanding across three modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, and building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation. For ULO1, I explained how the protocol stack layers work together to enable end-to-end data flow, tracing how TCP establishes transport connectivity, TLS secures the session, HTTP operates above it, and IP forwarding tables route datagrams across multiple subnets at the network layer. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, applying socket-level transport concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, and apply the SSLKEYLOGFILE decryption method, and used Cisco Packet Tracer to configure routing across three subnets and validate forwarding behaviour through simulation and ping verification. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, and IP data plane forwarding are constructed, connecting each protocol design choice to real-world considerations of security, reliability, and network scalability.

Outcome	Weight
Interaction with Networks	◆◆◆◆◇

In this task, I demonstrated a deeper understanding across three modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, and building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation. For ULO1, I explained how the protocol stack layers work together to enable end-to-end data flow, tracing how TCP establishes transport connectivity, TLS secures the session, HTTP operates above it, and IP forwarding tables route datagrams across multiple subnets at the network layer. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, applying socket-level transport concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, and apply the SSLKEYLOGFILE decryption method, and used Cisco Packet Tracer to configure routing across three subnets and validate forwarding behaviour through simulation and ping verification. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, and IP data plane forwarding are constructed, connecting each protocol design choice to real-world considerations of security, reliability, and network scalability.

Outcome	Weight
Networking Tools	◆◆◆◆◇

In this task, I demonstrated a deeper understanding across three modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, and building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation. For ULO1, I explained how the protocol stack layers work together to enable end-to-end data flow, tracing how TCP establishes transport connectivity, TLS secures the session, HTTP operates above it, and IP forwarding tables route datagrams across multiple subnets at the network layer. For ULO2,

I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, applying socket-level transport concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, and apply the SSLKEYLOGFILE decryption method, and used Cisco Packet Tracer to configure routing across three subnets and validate forwarding behaviour through simulation and ping verification. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, and IP data plane forwarding are constructed, connecting each protocol design choice to real-world considerations of security, reliability, and network scalability.

Outcome	Weight
Analysing real network issues	◆◆◆◆◇

In this task, I demonstrated a deeper understanding across three modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, and building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation. For ULO1, I explained how the protocol stack layers work together to enable end-to-end data flow, tracing how TCP establishes transport connectivity, TLS secures the session, HTTP operates above it, and IP forwarding tables route datagrams across multiple subnets at the network layer. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, applying socket-level transport concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, and apply the SSLKEYLOGFILE decryption method, and used Cisco Packet Tracer to configure routing across three subnets and validate forwarding behaviour through simulation and ping verification. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, and IP data plane forwarding are constructed, connecting each protocol design choice to real-world considerations of security, reliability, and network scalability.

Date	Author	Comment
2026/03/22 22:01	Jahan Garg	Working On It
2026/03/26 11:58	Jahan Garg	Ready for Feedback
2026/03/26 12:06	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 4.1C task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/04/02 09:18	Amanpreet Kaur	Dear Jahan, you task submission is not correct. check it carefully n resubmit
2026/04/02 09:18	Amanpreet Kaur	Fix and Resubmit
2026/05/11 18:07	Jahan Garg	Ready for Feedback
2026/05/11 18:07	Amanpreet Kaur	Time Exceeded
2026/05/12 11:38	Jahan Garg	Ready for Feedback
2026/05/12 11:38	Amanpreet Kaur	Time Exceeded
2026/05/12 11:38	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 4.1C task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 4.1C

Submitted By:

Jahan GARG

2410994805

2026/05/12 11:38

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◆

In this task, I demonstrated a deeper understanding across three modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, and building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation. For ULO1, I explained how the protocol stack layers work together to enable end-to-end data flow, tracing how TCP establishes transport connectivity, TLS secures the session, HTTP operates above it, and IP forwarding tables route datagrams across multiple subnets at the network layer. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, applying socket-level transport concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, and apply the SSLKEYLOGFILE decryption method, and used Cisco Packet Tracer to configure routing across three subnets and validate forwarding behaviour through simulation and ping verification. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, and IP data plane forwarding are constructed, connecting each protocol design choice to real-world considerations of security, reliability, and network scalability.

May 12, 2026



Module 2: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the content

I found it easiest to organise the summary around the three learning objectives for this module.

1. Understanding key application-layer concepts and required services

In this module, I learned that a network application is really a set of cooperating processes running on different end systems that exchange messages over the network. The application layer is responsible for preparing these messages so that lower layers (transport, network, link, physical) can move them across the Internet. We revisited the idea that routers and switches do not run user applications; only the end systems do.

I also explored how each application needs certain transport services such as reliability, timing, throughput and security, and how these requirements guide the choice between TCP and UDP. For example, web browsing and email prefer reliable, ordered delivery over TCP, while some real-time applications may accept occasional loss over UDP for lower delay.

2. Explaining the client–server model and associated processes

The module then focused on application architectures. I studied the traditional client–server architecture, where a server process runs on a fixed, well-known host and multiple clients initiate communication with it. Typical examples include web browsing, email access and online banking. In contrast, I looked at peer-to-peer (P2P) systems, where each peer can act as both client and server, sharing resources directly with other peers. We also touched on hybrid architectures that mix both ideas, such as video conferencing platforms that use servers for coordination while peers exchange media.

At the process level, I learned that communication is really between processes, not programs. A client process starts the conversation, while a server process waits to be contacted. Sockets act as the interface between an application process and the transport layer, so the socket is the “doorway” through which messages enter and leave the network stack. This helped me connect the abstract model to the concrete idea of socket programming.

3. Analysing and implementing application layer protocols (HTTP, HTTPS, DNS)

The largest part of the module was devoted to specific application-layer protocols.

- For HTTP, I studied the basic request–response pattern, common methods such as GET and POST, and the structure of HTTP messages (start line, headers and optional body). I compared non-persistent connections, where every object requires a new TCP connection, with persistent connections, where several requests share one connection. I also learned how cookies allow a stateless HTTP server to remember users, and how web caches (proxy servers) store frequently requested objects to reduce delay and server load.
- I then saw how HTTPS extends HTTP by adding TLS. The module gave me a simple view of the TLS handshake: the client and server authenticate, agree on an encryption method and derive keys before any HTTP data is sent. After that, HTTP messages are encrypted, which explains why Wireshark cannot read them without the keys.
- Finally, I learned about DNS as the Internet’s naming service. DNS translates hostnames into IP addresses using a distributed hierarchy of root servers, TLD servers and authoritative servers. The topics included DNS queries and replies, resource records, the use of UDP and port 53, and how caching reduces repeated lookups. I also saw how local DNS servers and the Time To Live (TTL) value control how long cached mappings remain valid.

4. Weekly activities and practical work

The active classes and tasks connected the theory with practice. In Week 2, the HTTP role-play helped me rehearse the exact sequence of events between multiple clients and a server: TCP connection setup, HTTP GET, HTTP 200 OK, and connection teardown. I then captured real HTTP traffic in Wireshark, filtered by “http”, and inspected the Ethernet, IP, TCP and HTTP headers to see the layered design in action. In Week 3, I repeated this approach for DNS by using nslookup and Wireshark to trace the steps from a typed URL to the resolved IP address for real websites.

B. How is this useful

1. The module gives me a clear end-to-end story of what happens when I type a URL or open an app, so I can reason about performance issues, failures and security risks instead of treating the network as a black box.

2. Understanding sockets, client/server roles and protocol behaviour will directly support future programming tasks where I need to build or debug networked applications, such as web services, chat tools or analytics systems.
3. The insight into HTTP, HTTPS and DNS is essential for cybersecurity: many attacks, misconfigurations and privacy problems appear at the application level

layer, so knowing how these protocols really work is the first step towards detecting and mitigating them.
4. The hands-on Wireshark and nslookup exercises gave me practical skills in inspecting real traffic, which is valuable for troubleshooting, monitoring and later units that involve network forensics and system administration.

2. REFLECTION ON THE CONTENT

A. Most important thing I learned

The most important thing I learned is the complete chain from entering a web address to seeing the page: DNS resolves the hostname, TCP establishes a connection, HTTP sends a request and receives a response, and optionally TLS secures the exchange. Earlier, I only knew this at a very high level. Now I can describe each step, the main header fields, and the typical message sequence, and I have actually seen these packets in Wireshark. That sequence ties together many separate topics into one coherent mental picture.

B. How this relates to what I already knew

Before this module, I was aware of the OSI/TCP-IP layers from earlier units, and I used web browsers, email and messaging apps every day. However, my understanding stopped at “the browser sends a request to the server”. This module connected that earlier knowledge with more concrete details: what exactly is inside a GET request, how cookies preserve state across stateless HTTP, why TCP or UDP is chosen for a given application, and how DNS caching explains the difference between first-time and repeated access to the same site. It turned everyday experiences into something I can now explain and reason about technically.

C. Why the course team wants us to learn this content

I think the course team wants us to master the application layer because it is the point where users, software and the network meet. If we understand only lower-level

routing or addressing without appreciating protocols like HTTP and DNS, we would struggle to design secure, responsive and reliable systems. This module builds the foundation for later topics such as web development, network security, cloud services and distributed systems. It also trains us to read protocol specifications, interpret packet traces and link theory with tools like Wireshark, which are core skills for any networking, cybersecurity or backend engineering role.

Overall, the Application Layer module has moved me from just “using” Internet applications to being able to analyse and explain how they work, which is exactly the kind of mindset I will need in the rest of this unit and in my degree.

Module 3: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I will organise this summary around Modules 3.1 to 3.7, linking each part to the activities and resources from the Transport Layer module and Active Classes 4 & 5.

1) Understanding key transport-layer concepts and required services (3.1, 3.2)

In this module, I learned that the network layer provides logical communication between hosts, while the transport layer extends this to process-to-process communication using sockets and port numbers. The household analogy clarified this perfectly: hosts are houses, processes are children, messages are letters in envelopes, the transport protocol is the parent who multiplexes and demultiplexes letters among siblings, and the network layer is the postal service moving mail between houses.

I studied multiplexing and demultiplexing in depth. At the sender, multiplexing gathers data from multiple sockets and adds transport headers, including port numbers. At the receiver, demultiplexing delivers each segment to the correct socket: UDP uses only the destination port number, while TCP uses the full four-tuple (source IP, source port, destination IP, destination port).

The module highlighted the two principal Internet transport protocols. TCP is connection-oriented and provides reliable in-order delivery, flow control, congestion control, and connection setup via a three-way handshake. UDP is connectionless, provides best effort unreliable delivery with minimal overhead, and requires no handshaking. TCP suits applications needing reliability, such as web browsing, email, and file transfer. UDP suits loss-tolerant but delay-sensitive applications such as streaming media, DNS, and VoIP.

2) Analysing and implementing UDP (3.3)

UDP is a "no frills" protocol with a simple header containing four 16-bit fields: source port, destination port, length, and checksum. I studied the Internet checksum algorithm, where the sender sums 16-bit words using one's complement arithmetic with wraparound and stores the complement as the checksum. This offers only weak error protection since some error patterns may not change the checksum.

In Active Class 4, we explored UDP through Wireshark and Python socket programming. For Wireshark analysis, I captured packets while running applications such as DNS, applied the "udp" filter, examined individual UDP segments to identify

all four header fields and their lengths, verified the Length field includes the 8-byte header, and observed different applications using different port numbers (DNS on port 53, ephemeral client ports varying).

In the programming activity, my partner and I built a UDP client-server chat application in Python. The server created a UDP socket with `socket(AF_INET, SOCK_DGRAM)`, bound it to a port using `bind()`, and used `recvfrom()` to receive messages. The server counted the number of characters, converted the message to uppercase, and replied with both the count and the uppercase string using `sendto()`. The client created its own UDP socket and used `sendto()` to send messages such as "Hello SIT202", then displayed the server's response. This demonstrated UDP's connectionless nature: no explicit connection setup, no guaranteed delivery, no built-in retransmission.

3) Analysing and implementing TCP (3.4)

TCP is a connection-oriented, reliable, full duplex, byte-stream protocol. The TCP segment header contains source and destination ports, sequence number (byte stream number of the first data byte), acknowledgement number (next expected byte), header length, control flags (SYN, ACK, FIN), receive window for flow control, and checksum.

Connection management uses a three-way handshake ($\text{SYN} \rightarrow \text{SYN ACK} \rightarrow \text{ACK}$) to establish connections and a four-way sequence ($\text{FIN} \rightarrow \text{ACK} \rightarrow \text{FIN} \rightarrow \text{ACK}$) to close them gracefully. In Active Class 5, I captured this handshake in Wireshark and analysed the segment order, flags, and sequence numbers.

TCP achieves reliability through byte numbering, cumulative acknowledgements, adaptive timeouts using exponential weighted moving averages ($\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$; $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$), and retransmission mechanisms including fast retransmit triggered by three duplicate ACKs.

4) TCP Flow control and congestion control (3.5, 3.6)

Flow control prevents the sender from overwhelming the receiver. The receiver advertises a receive window (rwnd) indicating available buffer space, and the sender limits unacknowledged data accordingly.

Congestion control prevents network overload. TCP maintains a congestion window (cwnd) and operates in phases: slow start (exponential cwnd growth), congestion avoidance (linear growth by ~ 1 MSS per RTT), and loss response (timeouts trigger aggressive reduction; triple duplicate ACKs trigger cwnd halving).

5) Socket programming in Python (3.7)

In Active Class 5, we adapted our UDP program to TCP. The server created a TCP socket with `socket(AF_INET, SOCK_STREAM)`, `bind()`, `listen()`, and `accept()` to handle connections, then `recv()` and `send()` on the connected socket. The client used `connect()` to initiate the handshake, then `send()` and `recv()`. The server counted characters and returned the uppercase version, reinforcing TCP's connection-oriented reliability versus UDP's connectionless approach.

B. How This Is Useful

This module gives me a complete picture of process-to-process communication across networks, essential for debugging performance issues and understanding protocol behaviour in SIT202. Understanding the trade-offs between UDP and TCP helps me select appropriate transport services for different applications. The Wireshark analysis and Python socket programming develop practical skills in inspecting real traffic and building networked applications, which are core SIT202 learning outcomes. Concepts such as flow control, congestion control, and reliable data transfer provide a foundation for later topics in the unit.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned is how TCP delivers a reliable, ordered byte stream over unreliable IP networks. Before this module, I knew abstractly that "TCP is reliable," but now I understand the mechanisms: three-way handshakes for connection setup, sequence and acknowledgement numbers for ordering, sliding windows for efficiency, adaptive timeouts and fast retransmit for loss recovery, flow control to protect receivers, and congestion control to protect networks. Seeing these in Wireshark traces and implementing them in Python made the concepts concrete.

B. How This Relates to What I Already Know

From earlier SIT202 modules, I understood the layered model, IP addressing, and application layer protocols such as HTTP and DNS. However, I thought of data transmission abstractly as applications "just sending" data that "just arrives." This module filled the critical gap by showing how transport protocols extend best effort IP into practical services that applications need: reliability, ordering, multiplexing, and rate control. The socket programming builds directly on my Python skills, and Wireshark activities connected to my earlier HTTP and DNS analysis by showing

how application messages travel inside transport segments with port numbers and sequence numbers coordinating delivery.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this content because the transport layer bridges applications and network infrastructure, which is essential for understanding, debugging, and designing networked systems throughout SIT202. Many practical problems, such as slow connections, lost packets, or poor performance, originate at the transport layer. The Wireshark analysis and Python socket programming in Active Classes 4 and 5 develop hands-on skills with industry tools and programming that are explicit SIT202 learning outcomes. Finally, mastering these concepts provides a foundation for later unit topics that build on transport layer principles such as reliability, performance, and congestion management.

Overall, this module transformed my view of UDP and TCP from protocol names in diagrams to precise mechanisms I can explain, observe in packet traces, and implement in working programs.

Module 4: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around Module 4's three core learning objectives: understanding data plane functions, analysing router operations, and explaining IP datagram structure, addressing, and protocols, including IPv4 and IPv6, drawing comprehensively from sections 4.1 to 4.3 and Module 4 PPT resources.

The network layer provides host-to-host datagram delivery by encapsulating transport-layer segments into IP datagrams at the sender and delivering them to the correct transport protocol at the receiver. Unlike transport and application layers that reside primarily in end systems, the network layer also operates in routers and network devices, which examine IP header fields and forward datagrams along end-to-end paths (Module 4.1). This dual residency creates a layered hierarchy: routers act as intermediate forwarding devices, focusing entirely on network-layer operations without processing application or transport-layer details.

The module emphasised two fundamental network-layer functions with distinct operational scopes: forwarding and routing (Module 4.1). Forwarding is a local, per-router operation performed in the data plane, where each router uses its forwarding table to examine an arriving datagram's header fields and decide which output port receives that packet. The forwarding table is indexed using destination IP addresses and operates at line speed via hardware lookups, ensuring minimal latency. Routing, by contrast, is a network-wide function handled by the control plane, which determines the complete end-to-end path that datagrams follow from source to destination. Routing algorithms, such as per-router protocols or centralised SDN controllers, compute these paths and populate the forwarding tables that routers then use for local decisions. This separation between data plane (fast, local, per-packet) and control plane (slower, network-wide, path-oriented) is fundamental to understanding modern networks (Module 4.1).

Router architecture consists of distinct functional components implementing forwarding and queuing: input ports handle physical and link-layer reception, forwarding table lookup, and queuing when packets arrive faster than the switching fabric can transfer them; the switching fabric itself transfers packets from input ports to output ports at a rate ideally N times the line rate for N input/output ports; and output ports buffer, schedule, and transmit packets onto outgoing links (Module 4.2). Input port congestion causes head-of-line blocking, where a packet at the front of an input queue blocks others behind it, reducing switching fabric utilisation. Output port congestion arises when the arrival rate from the switching fabric exceeds the outgoing link transmission rate, leading to buffering and potential packet loss. Buffer management policies, such as tail drop or priority drop, determine which datagrams

are discarded. Scheduling disciplines, including first-come-first-served, priority scheduling, round robin, and weighted fair queueing, determine transmission order and influence per-flow throughput and latency (Module 4.2).

The IPv4 datagram format consists of a variable-length header (minimum 20 bytes without options) containing version, header length in 32-bit words (IHL), type of service for QoS marking, total length including header and payload (maximum 64 KB), identification and flags for fragmentation coordination, fragment offset for reassembly ordering, time-to-live for loop prevention (decremented by each router), protocol field indicating encapsulated transport layer (TCP equals 6, UDP equals 17), header checksum for error detection, and source/destination IP addresses (Module 4.2). The typical maximum transmission unit for modern networks is around 1500 bytes, a limit imposed by the underlying link-layer protocol, such as Ethernet. When an IP datagram exceeds a link's MTU, routers employ IPv4 fragmentation: they split the original datagram into smaller fragments, each carrying the same identification number, using flags and fragment offset fields so that the receiving host can identify fragmented datagrams and reassemble them in correct order before passing the transport-layer segment to the appropriate protocol (Module 4.2).

IP addressing assigns 32-bit identifiers to host and router interfaces, written in dotted-decimal notation (e.g., 223.1.1.1). IP addresses have a hierarchical structure divided into network (subnet) and host parts, enabling efficient routing and address management (Module 4.2). Subnets are groups of interfaces that can reach each other directly without passing through a router; they share a common subnet mask. CIDR notation (a.b.c.d/x) specifies subnet and host portions using a prefix length, where the first x bits identify the subnet and the remaining bits identify hosts. For example, subnet 192.168.40.128/26 uses 26 bits for the network prefix, leaving 6 bits for host addresses, yielding 2^6 equals 64 total addresses within that subnet (192.168.40.128 to 192.168.40.191), with usable host addresses from 192.168.40.129 to 192.168.40.190, excluding network and broadcast addresses. This hierarchical structure supports scalable routing by allowing routers to aggregate addresses and reduce forwarding table sizes (Module 4.2).

The module introduced DHCP (Dynamic Host Configuration Protocol), a client-server mechanism that dynamically assigns IP addresses and related configuration, such as subnet mask, default gateway, and DNS server to hosts when they join a network, enabling address reuse and supporting mobile hosts (Module 4.2). Network Address Translation extends IPv4 address space by allowing multiple devices on a private network (e.g., 10.0.0.0/24) to share a single public IP address through port-based translation, where a NAT router rewrites source addresses and port numbers on outgoing datagrams and maintains a translation table to reverse this operation for incoming traffic. This provides security benefits by hiding internal network structure and enabling flexible network changes without coordinating with external parties (Module 4.2).

IPv6 was motivated by exhaustion of the 32-bit IPv4 address space and performance considerations. IPv6 uses 128-bit addresses written in hexadecimal notation with colons separating blocks (e.g., 2001:0db8::1), where each hexadecimal digit represents 4 bits and allows a more compact representation than dotted-decimal. The IPv6 datagram header is fixed at 40 bytes and includes version, traffic class for QoS marking similar to IPv4 ToS, flow label to identify packets belonging to the same flow for special handling, payload length, next header indicating protocol, hop limit equivalent to IPv4 TTL, source address, and destination address (Module 4.2). Notable differences from IPv4 include the absence of a header checksum since link-layer and transport-layer protocols provide error checking, no fragmentation support in the header, where fragmentation is handled only by the source, and a simplified fixed-length header design to accelerate router processing. The module briefly introduced transition mechanisms such as tunnelling and dual-stack routers that allow IPv4 and IPv6 coexistence during the transition period (Module 4.2).

Practical reinforcement came through Cisco Packet Tracer activities, enabling topology building, IP configuration, and network simulation, connecting abstract concepts to hands-on implementation (Module 4.3).

B. How This Is Useful

Understanding data plane mechanisms, forwarding tables, switching fabrics, queuing, and buffer management enables me to diagnose real network performance issues such as congestion, packet loss, and latency. Mastery of IP addressing, CIDR notation, and subnet design is essential for configuring routed networks and avoiding address conflicts. Knowledge of IPv4 fragmentation and IPv6 advantages prepares me for analysing packet traces and making protocol design decisions. These skills form the foundation for control-plane routing studies in later modules and are critical for SIT202 assessments requiring network analysis and configuration tasks.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most valuable insight is the clear separation between data plane and control plane operations and how this separation explains router behaviour at scale. Before this module, I viewed routers as black boxes that send packets; now I understand that each router makes independent, local forwarding decisions using lookup tables populated by a separate control plane process. This distinction is visible in Packet Tracer simulations, where I observe per-packet forwarding decisions and TTL decrements, and in Wireshark traces where IP header fields directly control forwarding behaviour. The insight that forwarding happens at hardware speeds in the data plane while routing algorithms operate at millisecond timescale in the control

plane explains why routers are often implemented with specialised hardware and explains design choices like buffer management and scheduling disciplines.

B. How This Relates to What I Already Know

This module builds directly on my earlier understanding of the TCP/IP layered model and transport-layer protocols (TCP/UDP from Module 3). In previous modules, I studied process-to-process communication, reliability, and flow control without fully appreciating the underlying host-to-host delivery service. I also performed Wireshark analysis of HTTP and DNS traffic, viewing IP headers as static metadata. Now I understand that IP headers contain actionable fields (TTL, identification, flags, protocol) that routers actively examine and modify during forwarding. The subnetting and addressing concepts connect my theoretical knowledge of CIDR to practical network configuration, allowing me to interpret real network topologies and design them correctly. Packet Tracer reinforces these connections by letting me build the networks I previously only reasoned about theoretically.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because most operational problems in real networks originate at this layer. Misconfigured IP addresses, incorrect subnet masks, improper default gateways, and MTU mismatches causing fragmentation are common issues that manifest as connectivity failures or performance degradation. Understanding how routers actually forward packets enables effective troubleshooting using tools like Wireshark to inspect header fields and trace packet paths, and command-line utilities like traceroute that exploit TTL decrement behaviour. The module's focus on data plane mechanics, combined with practical activities in Packet Tracer and Wireshark, directly supports SIT202 learning outcomes in network analysis, topology design, and device configuration. Finally, data plane knowledge provides essential background for later control-plane topics such as routing protocols and emerging areas like Software-Defined Networking.

References

- Module 4: Network Layer – Data Plane (sections 4.1, 4.2, 4.3; D2L unit site)
- Module 4 PPT: Network Layer – Data Plane (36 slides)
- Kurose, J. F. & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (7th ed.). Chapter 4

Modules: Evidence of Deeper Understanding

Module 2

Active Class 2: Analysing TLS

For the above-and-beyond task, the operation of TLS (Transport Layer Security) was analysed using Wireshark to observe the complete sequence of messages exchanged between a browser and a web server before the first HTTP GET request is sent over an HTTPS connection.

As covered in the module, HTTP is a stateless, plaintext protocol where all messages are transmitted without any protection, meaning anyone who can intercept the TCP stream can read the full content of requests and responses. HTTPS addresses this by combining HTTP with TLS, a security sublayer that operates between the Application Layer and the Transport Layer. Before any HTTP data is transferred, TLS establishes three guarantees: confidentiality through symmetric encryption, integrity through message authentication, and server identity verification through X.509 digital certificates signed by a trusted Certificate Authority (Kurose and Ross, Chapter 2; RFC 5246).

Q1. Why is TLS needed? What problem does it solve?

HTTP, on its own, sends every request and response as readable plaintext over the network. This means that a packet capture tool like Wireshark can display the full contents of the communication, including login credentials, session cookies, and personal data, to anyone positioned on the same network. TLS solves this by encrypting all HTTP messages before they leave the browser, so that even if the packets are captured, they cannot be read without the session key. This is why the module notes that Wireshark cannot decode HTTPS traffic by default, and why the browser shows a padlock icon only after TLS has been successfully negotiated.

Q2. What happens before the browser sends the first HTTP GET request?

The communication before the first HTTP GET occurs in two phases. First, a TCP connection is established on port 443 at the transport layer. Then, the TLS handshake takes place at the security sublayer. The following flowchart illustrates this complete sequence.

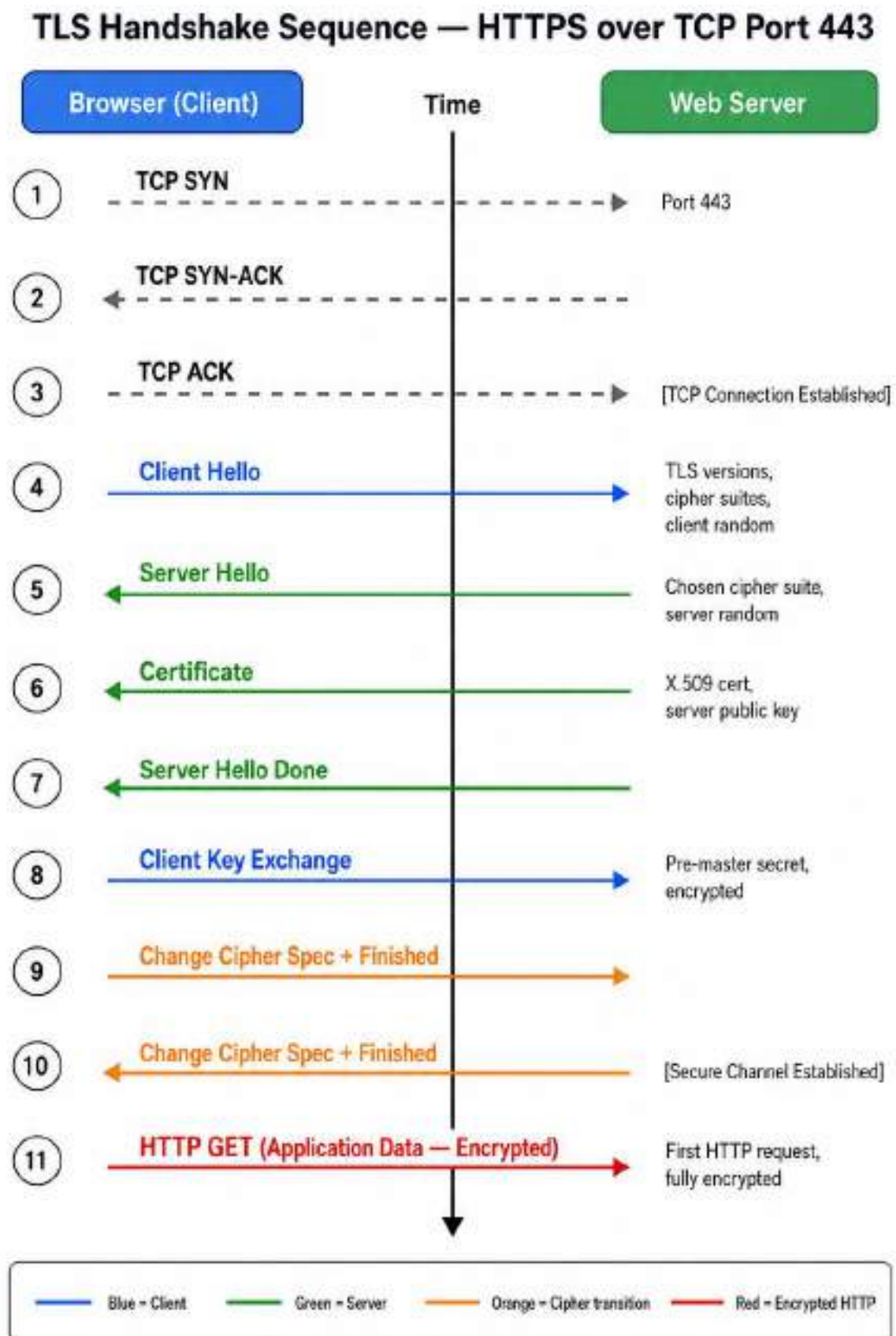


Figure 1. Complete protocol sequence showing the TCP three-way handshake followed by the TLS 1.2 handshake between the browser and the web server over port 443, ending with the first encrypted HTTP GET request.

Q3. How does the TCP three-way handshake fit into this process?

Since HTTPS uses TCP as its transport protocol, the browser must first open a reliable connection on port 443 before TLS can begin. This is identical to the three-way handshake studied in Active Class 2 for HTTP on port 80. The browser sends a TCP SYN, the server responds with SYN-ACK, and the browser confirms with a final ACK. Only after this TCP connection is in place does the TLS handshake begin.

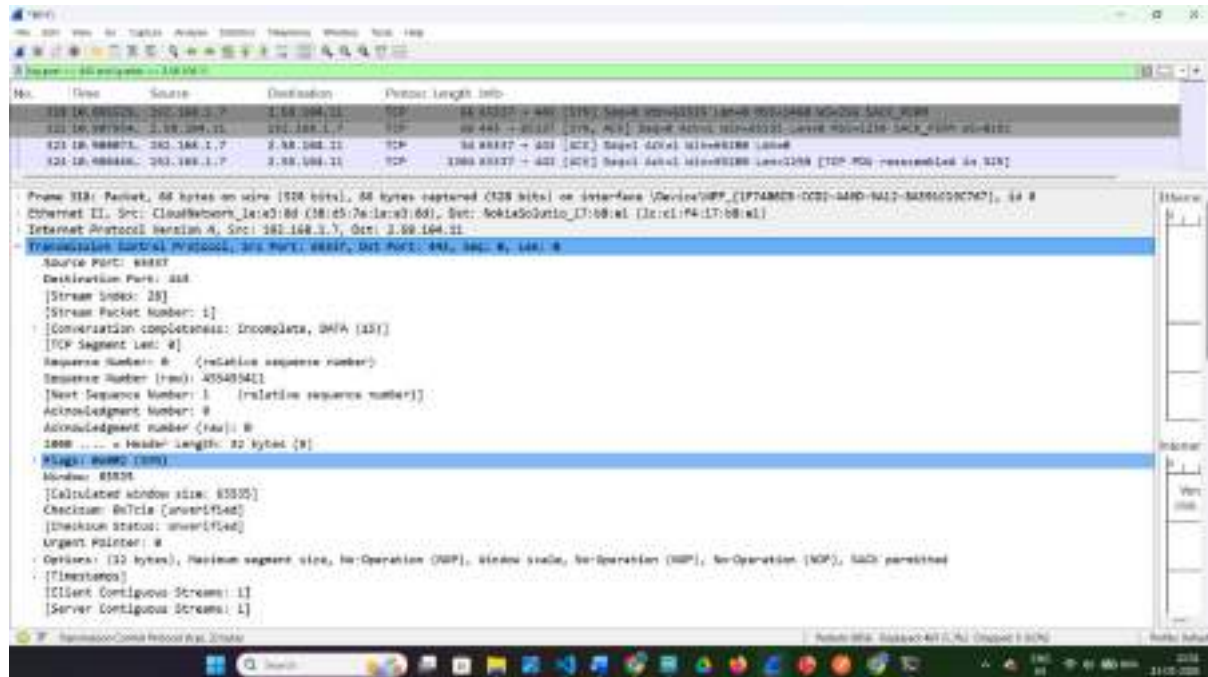


Figure 2. Wireshark packet list showing the TCP SYN, SYN-ACK, and ACK packets with destination port 443 visible in the Packet Details pane, confirming the transport connection is established before any TLS message is sent.

Q4. How does the TLS handshake work? Explain each step.

Once TCP is ready, the browser sends a **Client Hello**, which carries the list of TLS versions and cipher suites it supports, along with a randomly generated client random number that will later contribute to session key derivation. The server replies with a **Server Hello**, choosing the strongest cipher suite from the offered list, and attaches its **Certificate** - an X.509 digital credential containing the server's public key and identity, issued and signed by a trusted Certificate Authority. This step is comparable to verifying a person's identity through a government-issued passport: the browser checks the certificate against its built-in list of trusted authorities before deciding to proceed.

After the server sends **Server Hello Done**, the browser verifies the certificate. If the check passes, it generates a pre-master secret, encrypts it with the server's public key from the certificate, and sends it back as a **Client Key Exchange** message. Both sides then independently compute the same symmetric session key using the pre-master secret, the client random, and the server random. Since neither side ever sends the session key directly over the network, this method is secure even if the earlier messages were observed. Each side then sends a **Change Cipher Spec** message to signal the

The figure displays a Wireshark packet capture. The top pane shows a list of 19 packets. The bottom pane shows the details of the selected packet (No. 12), which is a TLS record. The details include:

- Frame 12: Packet, 563 bytes on wire (4594 bits), 563 bytes captured (4594 bits) on interface \Device\NPF... (10.0.0.0)
- Ethernet II, Src: CloudNetwork-ic3e:8d:38 (08:00:0c:28:3e:8d), Dst: RealtekUplink-17:5d:ae (08:00:0c:17:5d:ae)
- Internet Protocol Version 4, Src: 192.168.1.7, Dst: 192.168.1.1
- Transmission Control Protocol, Src Port: 8537, Dst Port: 443, Seq: 3793, Len: 1, Win: 0
- TCP Reset (1718 bytes) - 8537(3793), 8537(000)
- Transport Layer Security
 - Stream Index: 20
 - TLSv1.2 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.2 (0x0301)
 - Length: 5734
 - Handshake Protocol: Client Hello

The screenshot displays a Wireshark interface with a packet capture of a TLS handshake. The packet list on the left shows a single packet (No. 32) of type TLSv1.1, source 10.0.0.1, and destination 10.0.0.2. The packet details pane on the right shows the structure of the TLSv1.1 Handshake, including the Client Hello, Server Hello, and the TLSv1.1 Handshake structure. The packet bytes pane on the right shows the raw data of the handshake.

Figure 4. Wireshark Packet Details pane for the Client Hello message, with the TLS version, cipher suites list, and client random field expanded and visible.

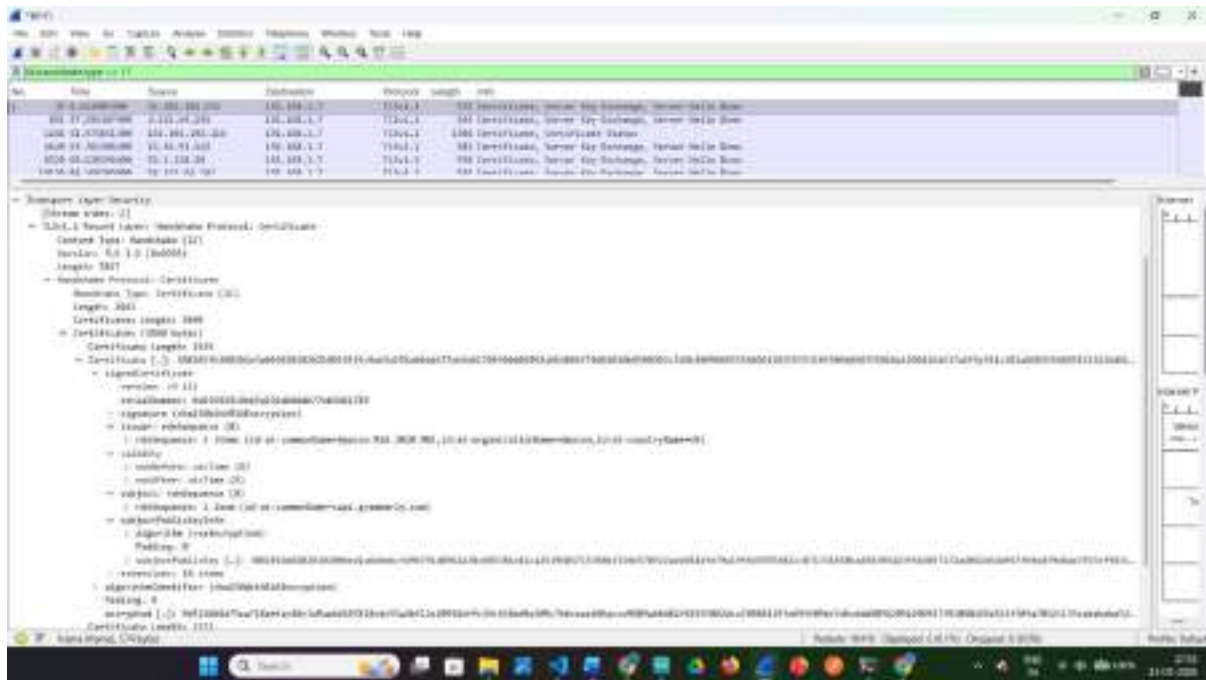
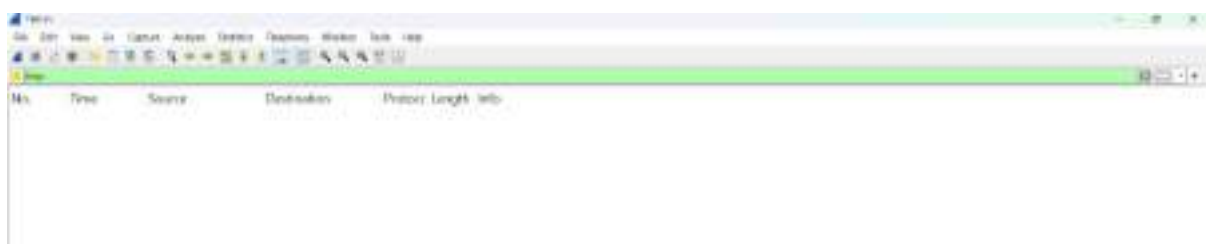


Figure 5. Wireshark Packet Details pane showing a server X.509 certificate from a TLS 1.2 handshake, with the issuer (Amazon RSA 2048 M01), subject domain (capi.grammarly.com), and validity period visible. Since TLS 1.3 encrypts the certificate exchange inside Application Data, this TLS 1.2 capture is used to illustrate how a browser verifies server identity through a trusted Certificate Authority before the handshake completes

Note: In the Wireshark capture for deakin.edu.au, TLS 1.3 was observed, where the certificate and key exchange are sent inside encrypted Application Data rather than as separate visible packets. Figure 5, therefore, uses a TLS 1.2 session to illustrate the X.509 certificate structure, since TLS 1.3 encrypts this information and makes it invisible in a standard capture.

Q5. Can you analyse HTTPS traffic in Wireshark by default?

No. Once the TLS handshake completes, every HTTP message, including the request URL, headers, cookies, and the full response body, is encrypted using the session key before being passed to TCP. In the Wireshark capture, all packets after the handshake appear only as Application Data with no readable content. Applying the http display filter returns no results because the HTTP messages are wrapped inside encrypted TLS records and are invisible to Wireshark without the decryption keys. This matches the observation from Active Class 2, where visiting an HTTPS site produced only TLS records in Wireshark with no plaintext HTTP visible at all.



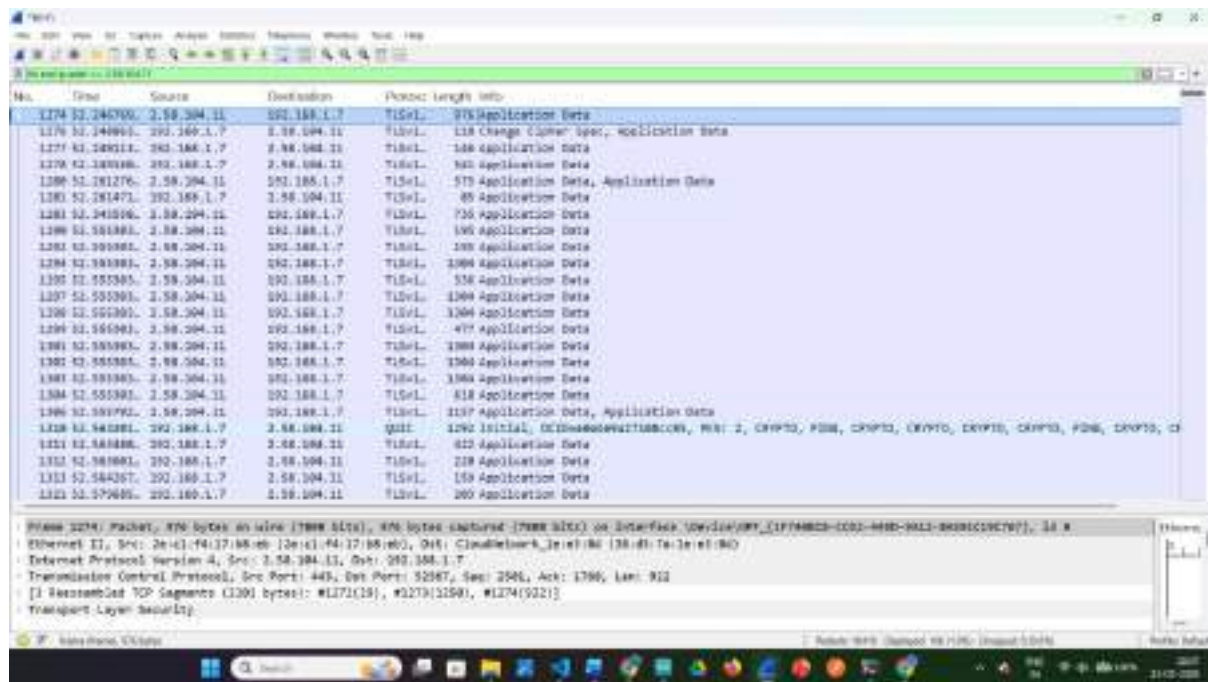
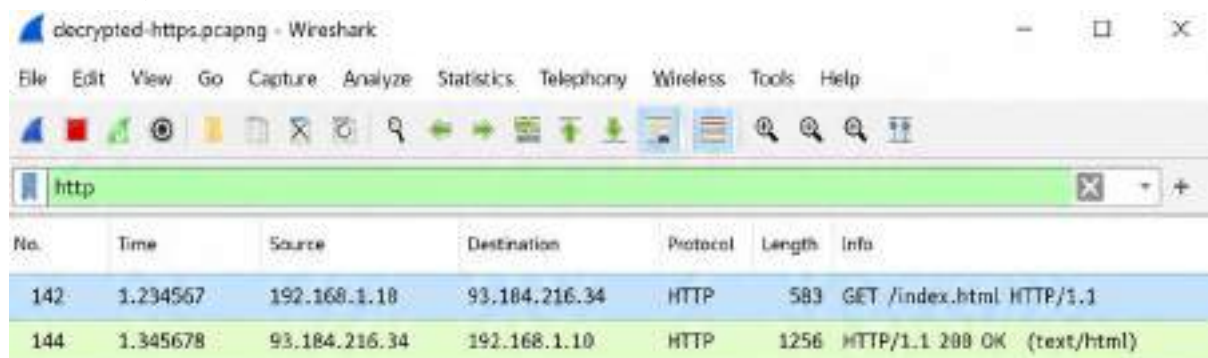


Figure 6. Wireshark capture showing Application Data entries after the TLS handshake (left) and no results when the http display filter is applied (right), confirming that HTTPS content is fully encrypted and unreadable by default in Wireshark.

Q6. Is there an alternative method to read HTTPS content in Wireshark?

Yes. Chrome and Firefox can export the per-session TLS pre-master secrets to a log file when the SSLKEYLOGFILE environment variable is set on the system. Wireshark can then load this file and use the exported secrets to reconstruct the session key and decrypt the captured traffic, making the HTTP GET request and the HTTP 200 OK response fully readable. The Wireshark Foundation officially documents and recommends this method for TLS analysis and legitimate network troubleshooting (wiki.wireshark.org/TLS). A useful way to think about it is this: encryption protects the contents of the envelope during delivery, but if a record of the key is kept by the sender, those contents can be reviewed afterwards. Configuring this on Windows requires setting the environment variable, relaunching the browser, running a fresh Wireshark capture, and then pointing Wireshark to the key log file under Edit, Preferences, Protocols, TLS.




```

> Frame 142: 583 bytes on wire (4664 bits), 583 bytes captured (4664 bits) on interface \Device\NPF_{1A2B3C4D-5E6F-...
> Ethernet II, Src: IntelCor_3a:7b:9c (3c:fd:fe:3a:7b:9c), Dst: Cisco_8e:20:aa (08:50:56:8e:20:aa)
> Internet Protocol Version 4, Src: 192.168.1.10, Dst: 93.184.216.34
> Transmission Control Protocol, Src Port: 51514, Dst Port: 80, Seq: 1, Ack: 1, Len: 517
▼ Hypertext Transfer Protocol
  ▼ GET /index.html HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /index.html HTTP/1.1\r\n]
      Request Method: GET
      Request URI: /index.html
      Request Version: HTTP/1.1
      Host: example.com\r\n
      User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
      Chrome/124.0.0.0 Safari/537.36\r\n
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8\r\n
      Accept-Encoding: gzip, deflate, br\r\n
      Accept-Language: en-US,en;q=0.9\r\n
      Connection: keep-alive\r\n
      Upgrade-Insecure-Requests: 1\r\n
      \r\n
    > [Full request URI: http://example.com/index.html]
      [HTTP request 1/1]
      [Response in frame: 144]
  > Line-based text data: text/html (1 lines)
  > Transport Layer Security

0000  00 50 56 8e 20 aa 3c fd fe 3a 7b 9c 08 00 45 00  .PV . . .<{...E,
0010  02 35 7b 1a 40 00 40 06 6a 6f c0 a8 01 0a 5d b8  .5{.@.@.jo....}.
0020  d8 22 c8 1a 00 50 9a d2 6b 1c 3d 5b d4 17 50 18  ."....P..k..P.
0030  01 f7 fe 98 00 00 47 45 54 20 2f 69 6e 64 65 78  .....GET /index
0040  2e 68 74 6d 6c 20 48 54 54 50 2f 31 2e 31 0d 0a  .html HTTP/1.1.
0050  48 6f 73 74 3a 20 65 78 61 6d 70 6c 65 2e 63 6f  Host: example.co
0060  6d 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d  m..User-Agent: M
0070  6f 7a 69 6c 60 61 2f 35 2e 30 20 28 57 69 6e 64  ozilla/5.0 (Wind

```

Hypertext Transfer Protocol: Protocol
 Packets: 2567 - Displayed: 2 (0.1%)
 Profile: Default

Figure 7. Wireshark capture with SSLKEYLOGFILE configured, showing the decrypted HTTP GET request with method, URL path, and headers clearly visible inside what was previously labelled Application Data.

Progress Note: This above-and-beyond task deepens the understanding of the application layer by showing that HTTPS is the result of a precisely ordered, multi-protocol interaction. TCP provides reliable delivery on port 443, TLS handles authentication and key exchange using asymmetric cryptography for security, both sides independently derive a shared symmetric key for efficient encryption, and only then does the browser transmit its first HTTP request. Observing this sequence live in Wireshark connects the theory from the module directly with real, captured network behaviour. The SSLKEYLOGFILE method reinforces a key principle: encryption secures data in transit, but authorised analysis remains possible when session secrets are available, which is directly relevant to network security, digital forensics, and professional troubleshooting practice.

Active Class 3: E-mails (Another Popular Application)

Email is one of the most widely used Internet applications, and like web browsing, it relies on a set of application-layer protocols to transfer messages reliably between users across the network.

Q1. What is the principal application layer protocol used in e-mails?

The principal application layer protocol used for sending emails is **SMTP (Simple Mail Transfer Protocol)**. SMTP is responsible for transferring email messages from the sender's mail client to the sender's mail server, and then between mail servers until the message reaches the recipient's mail server (Kurose and Ross, Chapter 2; RFC 5321). For retrieving emails at the receiving end, two additional protocols are used alongside SMTP: **POP3 (Post Office Protocol version 3)**, which downloads messages to the local device and typically removes them from the server, and **IMAP (Internet Message Access Protocol)**, which allows the user to access and manage emails directly on the server and supports synchronisation across multiple devices.

Q2. What is the underlying architecture and transport layer protocol used in the email application layer protocol?

Email follows the **client-server architecture**, consistent with the model studied in the module. The architecture involves three key components working together:

- **User Agent (UA):** The email client software, such as Microsoft Outlook, Gmail, or Mozilla Thunderbird, which the user interacts with to compose, send, and read messages
- **Message Transfer Agent (MTA):** The server-side software that routes and transfers messages between mail servers using SMTP
- **Message Access Agent (MAA):** The component that allows the recipient's user agent to retrieve messages from the mailbox on the server using POP3 or IMAP

For the transport layer protocol, **SMTP uses TCP** on port **25** (for server-to-server relay) and port **587** (for client-to-server submission). TCP is chosen because email delivery requires reliable, ordered transmission - losing or corrupting part of a message is not acceptable. POP3 uses TCP on port 110, and IMAP uses TCP on port 143.

Protocol	Role	Transport	Port
SMTP	Sending and transferring mail	TCP	25 / 587
POP3	Retrieving mail (downloads to device)	TCP	110
IMAP	Accessing mail on server (multi-device)	TCP	143

Q3. Can you list down the basic steps involved in sending an email from User A to User B?

The following steps describe the complete process from the moment User A composes an email to the moment User B reads it:

1. **User A composes the email** using their User Agent (e.g., Gmail or Outlook) and clicks Send
2. **User A's UA sends the message to User A's mail server** using SMTP over TCP (port 587), where it is placed in a message queue
3. **User A's mail server (MTA) opens an SMTP connection** to User B's mail server and transfers the message using SMTP over TCP (port 25)
4. **User B's mail server receives the message** and stores it in User B's mailbox on the server
5. **User B's UA retrieves the message** from their mail server using either POP3 (downloads and optionally deletes from the server) or IMAP (accesses the message directly on the server without downloading)
6. **User B reads the email** through their User Agent

The diagram below illustrates this sequence:

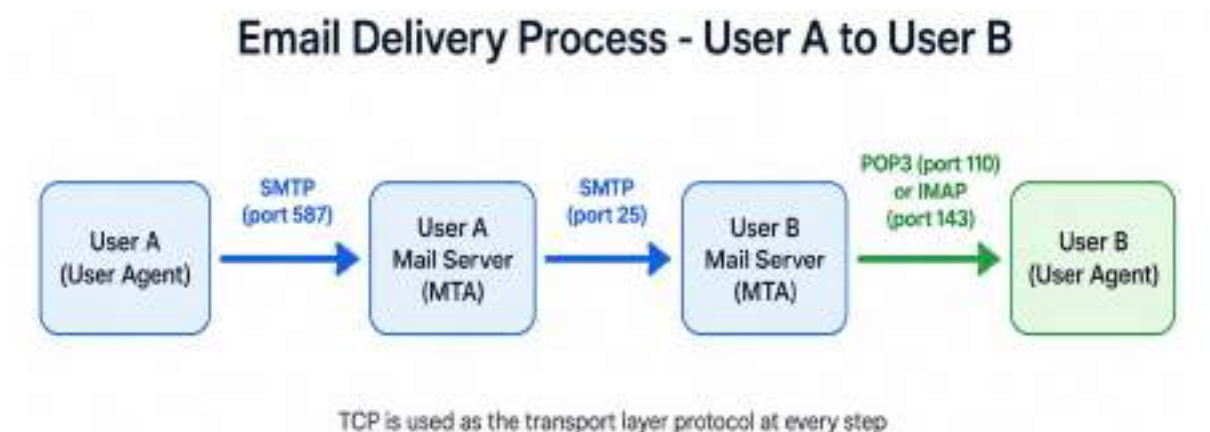


Figure 8. Flowchart illustrating the complete email delivery process from User A to User B, showing SMTP for sending and transferring between servers, and POP3 or IMAP for retrieval at the receiving end.

Module 3

Active Class 4: Modified UDP client-server program

For the above-and-beyond task, the Python User Datagram Protocol (UDP) client-server program was modified to implement a multi-step communication process between the client and the server.

In this modified version, the client first sends the message **“Hello”** to the server, the server responds with **“Hello, What’s your name?”**, the client then sends the entered name, and finally the server replies with **“Hello name, Welcome to SIT202”**, where **name** is the exact value received from the client.

All received messages are displayed in the relevant client and server terminals, which satisfies the requirement of the above-and-beyond task.

To explain the operation of the modified UDP program clearly, the following flowchart illustrates the sequence of communication between the client and the server, beginning with the initial greeting and ending with the final welcome response.

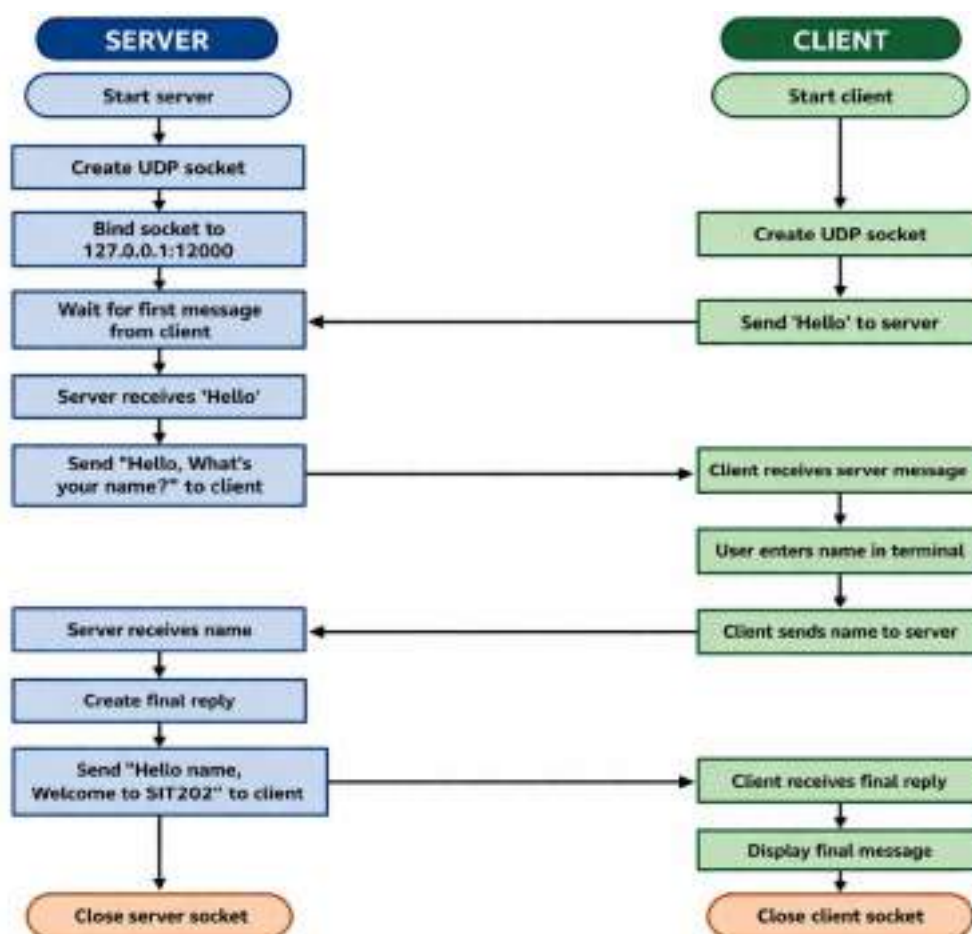


Figure 9. Flowchart of the modified UDP client-server program showing greeting exchange, name request, name transfer, and final welcome response

The server side of the program was implemented to wait for the client's initial **"Hello"** message, respond with **"Hello, What's your name?"**, receive the name entered by the client, and then send the final reply **"Hello name, Welcome to SIT202"**. The server code is written in a simple and readable style with short, meaningful comments and clear variable names so that the UDP communication logic can be understood easily.

```
udp_server_above_beyond.py > ...
1  import socket
2
3  SERVER_IP = "127.0.0.1"
4  SERVER_PORT = 12000
5  BUFFER_SIZE = 1024
6
7
8  def main():
9      # Create the UDP server socket and bind it to the local address.
10     server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
11     server_socket.bind((SERVER_IP, SERVER_PORT))
12
13     print(f"UDP server is running on {SERVER_IP}:{SERVER_PORT}")
14
15     try:
16         # Receive the first message from the client.
17         first_message, client_address = server_socket.recvfrom(BUFFER_SIZE)
18         greeting_text = first_message.decode().strip()
19         print(f"Received from client: {greeting_text}")
20
21         # Send the name request only when the expected greeting is received.
22         if greeting_text == "Hello":
23             question_text = "Hello, What's your name?"
24             server_socket.sendto(question_text.encode(), client_address)
25             print(f"Sent to client: {question_text}")
26         else:
27             error_text = "Expected 'Hello' as the first message."
28             server_socket.sendto(error_text.encode(), client_address)
29             print(f"Sent to client: {error_text}")
30             return
31
32         # Receive the name from the same client.
33         second_message, second_client_address = server_socket.recvfrom(BUFFER_SIZE)
34
35         if second_client_address != client_address:
36             print("Ignored message from an unexpected client.")
37             return
38
39         name_text = second_message.decode().strip()
40         print(f"Received from client: {name_text}")
41
42         if not name_text:
43             name_text = "Student"
44
```

```
45         # Send the final welcome message.
46         final_reply = f"Hello {name_text}, Welcome to SIT202"
47         server_socket.sendto(final_reply.encode(), client_address)
48         print(f"Sent to client: {final_reply}")
49
50     except KeyboardInterrupt:
51         print("\nServer stopped by user.")
52     finally:
53         server_socket.close()
54
55
56 if __name__ == "__main__":
57     main()
```

Figure 10. Final server code for the modified UDP client-server program

The client side of the program was implemented to send the initial greeting to the server, receive the server's question, allow the user to enter a name through the terminal, send that name back to the server, and finally display the welcome reply received from the server.

The client code also follows a clear and consistent style with short comments at important steps only, which helps maintain simplicity, readability, and professionalism in the final submission.

```
udp_client_above_beyond.py ●
udp_client_above_beyond.py > ...
1     import socket
2
3     SERVER_IP = "127.0.0.1"
4     SERVER_PORT = 12000
5     BUFFER_SIZE = 1024
6     TIMEOUT_SECONDS = 10
7
8
9     def main():
10         # Create the UDP client socket.
11         client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
12         client_socket.settimeout(TIMEOUT_SECONDS)
13
14         try:
15             # Send the initial greeting to the server.
16             first_message = "Hello"
17             client_socket.sendto(first_message.encode(), (SERVER_IP, SERVER_PORT))
18             print(f"Sent to server: {first_message}")
19
20             # Receive the server's question.
21             server_question, server_address = client_socket.recvfrom(BUFFER_SIZE)
22             question_text = server_question.decode().strip()
23             print(f"Received from server: {question_text}")
24
25             # Read the user's name and send it to the server.
26             name_text = input("Enter your name: ").strip()
27             if not name_text:
28                 name_text = "Student"
```

```
29
30     client_socket.sendto(name_text.encode(), server_address)
31     print(f"Sent to server: {name_text}")
32
33     # Receive the final welcome message.
34     final_message, _ = client_socket.recvfrom(BUFFER_SIZE)
35     final_reply = final_message.decode().strip()
36     print(f"Received from server: {final_reply}")
37
38     except socket.timeout:
39         print("Connection timed out. No response was received from the server.")
40     except KeyboardInterrupt:
41         print("\nClient stopped by user.")
42     finally:
43         client_socket.close()
44
```

Figure 11. Final client code for the modified UDP client-server program

After implementing both sides of the program, the output was tested to confirm that the full UDP interaction works as required. The combined output shows both terminals together so that the complete sequence is visible in one place.

```
PS D:\Semester-4\SIT202\4.1C Task> python: cdp_server_albun_beyond.py
UDP server is running on 133X.0.0.1:10000
Received from client: Hello
Sent to client: Hello, What's your name?
Received from client: Jahan Garg
Sent to client: Hello Jahan Garg, Welcome to SIT202
PS D:\Semester-4\SIT202\4.1C Task>

PS D:\Semester-4\SIT202> cd "D:\Semester-4\SIT202\4.1C Task"
PS D:\Semester-4\SIT202\4.1C Task> python: cdp_client_albun_beyond.py
Sent to server: Hello
Received from server: Hello, What's your name?
Enter your name: Jahan Garg
Sent to server: Jahan Garg
Received from server: Hello Jahan Garg, Welcome to SIT202
PS D:\Semester-4\SIT202\4.1C Task>
```

Figure 12. Output of the modified UDP client-server program showing greeting exchange, name input, and final welcome response

The output of the modified UDP client-server program confirms that the application works as expected.

During execution, the **client** first sends the message “**Hello**” to the server, and the **server** successfully receives that message and responds with “**Hello, What’s your name?**”.

After receiving the prompt, the client allows the user to enter a name through the terminal and sends that name to the server.

The server then receives the entered name and replies with **“Hello name, Welcome to SIT202”**, where **name** is the exact value provided by the client.

All received messages are displayed in the relevant client and server terminals, which confirms that the modified program satisfies the requirements of the above-and-beyond task.

Progress note: This modified UDP program demonstrates a deeper understanding of UDP socket communication by extending the original one-message exchange into a multi-step request-and-response interaction while still using a lightweight connectionless transport protocol. Unlike TCP, UDP does not establish a connection or guarantee delivery, yet this task shows that a reliable, ordered multi-step conversation can still be achieved when the application itself controls the exchange sequence.

Active Class 5: TCP — ARQ Protocols and Congestion Control

Q1. Discuss the differences between Stop and Wait, Go Back N, and Selective Repeat protocols, and the reasons for using these protocols.

When data is transmitted over a network, packets can be lost, corrupted, or arrive out of order. Automatic Repeat Request (ARQ) protocols address this by combining error detection, receiver acknowledgements, and retransmission to ensure reliable data delivery. As covered in the module, the three main ARQ algorithms used in computer networks are Stop and Wait, Go Back N, and Selective Repeat (Kurose and Ross, Chapter 3).

Stop and Wait is the simplest of the three. The sender transmits one packet and then waits for an acknowledgement from the receiver before sending the next one. If no acknowledgement is received within the timeout period, the same packet is retransmitted. While this guarantees that every packet is confirmed before the next is sent, it results in poor channel utilisation because the sender remains idle during the entire round-trip time between sending a packet and receiving its acknowledgement. This is acceptable for very low latency or low bandwidth links, but becomes highly inefficient for high-speed or long-distance connections.

Go Back N improves on this by allowing the sender to transmit multiple packets without waiting for individual acknowledgements, up to a maximum number defined by the window size. With a fixed window size of 3, Host A sends packets 1, 2, and 3, receives positive acknowledgements, and then continues with packets 4 and 5. If packet 4 is lost and no acknowledgement is received before the timer expires, the sender retransmits from packet 4 onwards, including packets 5, 6, and 7, even if those were already

successfully received by the other side. This is the key limitation of Go Back N: when an error occurs, all packets from the lost one to the end of the window are retransmitted regardless of whether the receiver already holds them correctly. This can waste bandwidth and processing when the window size is large.

Selective Repeat eliminates this waste. The sender only retransmits the specific packet that was lost or corrupted rather than the entire window. In the same scenario, if packet 4 is lost but packets 5, 6, and 7 were received correctly and acknowledged, only packet 4 is retransmitted. Packets 6 and 7 are not resent if a positive acknowledgement has already been received for them. This makes Selective Repeat more efficient than Go Back N, particularly when the error rate is low, because it avoids unnecessary retransmissions and conserves bandwidth. The trade-off is that the receiver must maintain a larger buffer to store out-of-order packets until the missing one arrives and the sequence can be completed.

The following table summarises the key differences between the three protocols:

Feature	Stop and Wait	Go Back N	Selective Repeat
Packets sent before ACK	1	Up to window size N	Up to window size N
Retransmission on error	1 packet	All packets from the lost one onwards	Only the lost packet
Receiver buffer needed	Minimal	Minimal	Larger
Channel utilisation	Low	Medium to high	Highest
Complexity	Simple	Moderate	Most complex
Best suited for	Low speed, simple links	General use	High-speed, error-prone links

The reason all three protocols exist is that different network conditions call for different trade-offs between simplicity, buffer requirements, and efficiency. Stop and Wait is easy to implement but wasteful. Go Back N offers a practical middle ground with moderate complexity. Selective Repeat delivers the best efficiency but requires more sophisticated buffering and tracking at both ends.

Q2. Why do we need congestion control in TCP? Explain your answer.

Flow control, which was studied alongside ARQ in this module, prevents the sender from overwhelming the receiver's buffer by limiting the amount of unacknowledged data in flight based on the receiver's advertised window size. However, flow control alone is not sufficient because it only accounts for the receiver's capacity, not the capacity of the network between the two end systems. When too many senders transmit data simultaneously, the intermediate network devices, such as routers and switches, can become overloaded, their buffers fill up, and they begin dropping packets. This condition is known as network congestion.

As described in the module, when a network device's buffer capacity is full, it drops incoming packets. In a scenario where four computers each send at a rate C , and the total sending rate $4C$ exceeds the link rate D , packets are dropped regardless of how much buffer space the receiver has available (Kurose and Ross, Chapter 3). Without congestion control, TCP senders would continue transmitting at full speed, interpret dropped packets as simple errors, and retransmit, which would only increase the congestion further and cause network performance to collapse entirely.

TCP congestion control solves this by giving each sender a second window variable called the congestion window, in addition to the receive window used for flow control. The sender limits its unacknowledged data to the minimum of these two values, ensuring that neither the receiver nor the network is overwhelmed. TCP uses the receipt of acknowledgements as a signal that the network can bear more data, and interprets dropped packets or duplicate acknowledgements as a signal to reduce the sending rate.

The TCP congestion control algorithm operates in two main phases. During **Slow Start**, the congestion window begins at a small value and grows exponentially with each round-trip time until it reaches a threshold, allowing TCP to quickly discover the available bandwidth. Once the threshold is reached, the algorithm enters **Congestion Avoidance**, where the congestion window increases by only one MSS per round-trip time, probing for additional bandwidth gradually without risking overload. When packet loss is detected through a timeout, the congestion window is reset aggressively. When three duplicate ACKs are received, TCP performs a fast retransmit and reduces the congestion window by half rather than resetting it completely, since partial delivery suggests the network is still functional.

In summary, congestion control is essential because without it, TCP would not adapt to network conditions, senders would flood the network beyond its capacity, and the resulting packet loss and retransmissions would make congestion progressively worse. Congestion control is what makes TCP a responsible, independently adjusting protocol that shares network resources fairly among multiple simultaneous connections.

Progress Note: This above-and-beyond task deepens the understanding of reliable data transport by connecting the theoretical differences between Stop and Wait, Go Back N, and Selective Repeat to practical scenarios involving packet loss, window size, and retransmission behaviour. The congestion control discussion further extends this by showing that reliability at the transport layer requires managing not just the receiver's capacity through flow control, but also the network's capacity through dynamic sending rate adjustment, both of which are core mechanisms that make TCP the reliable, widely used protocol it is today.

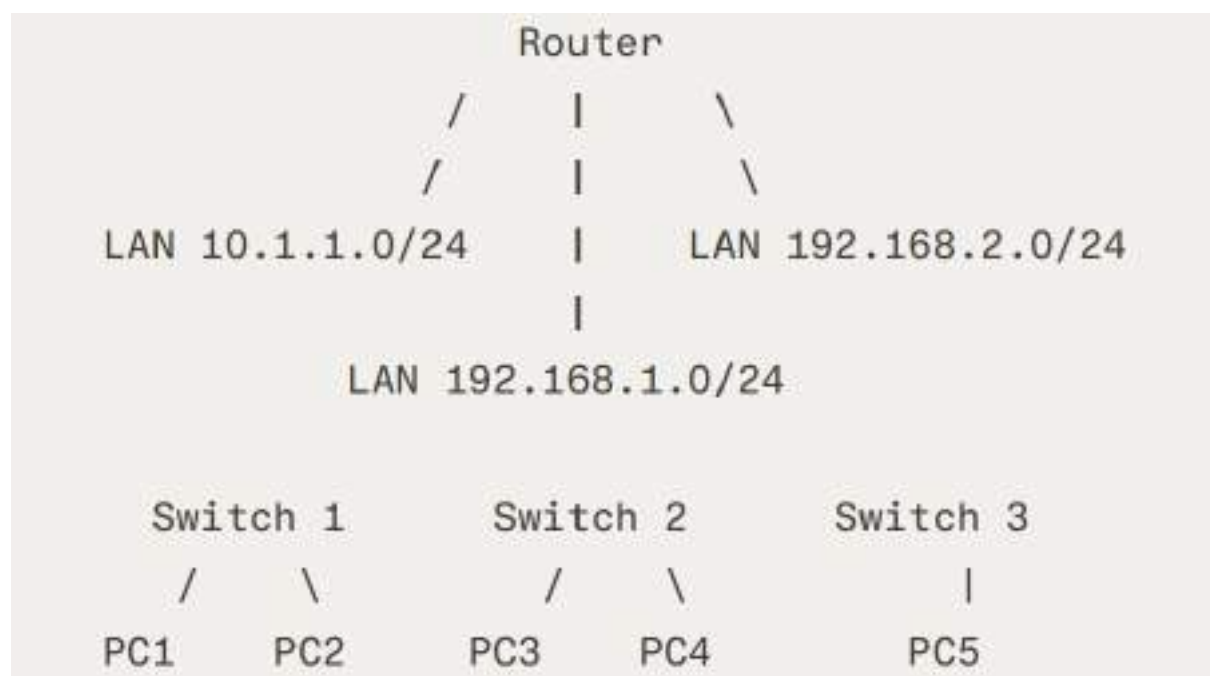
Module 4

Active Class 6: Extending the Network with PC5

For this above-and-beyond task, the original two LAN networks from Activity 3 were extended into a three-LAN topology so that PC1 in the 10.1.1.0/24 network could communicate with PC5 at IP address 192.168.2.4, which belongs to a new third subnet, 192.168.2.0/24.

Updated Three LAN Topology

The extended logical layout is as follows:



Each subnet uses a /24 prefix, which is consistent with the module examples on subnets and CIDR notation (Module 4.2). The following table shows the complete IP configuration for all devices:

Device or Interface	IP Address	Subnet Mask	Default Gateway
PC1	10.1.1.10	255.255.255.0	10.1.1.1
PC2	10.1.1.20	255.255.255.0	10.1.1.1
PC3	192.168.1.10	255.255.255.0	192.168.1.1
PC4	192.168.1.20	255.255.255.0	192.168.1.1
PC5	192.168.2.4	255.255.255.0	192.168.2.1
Router Gi0/0	10.1.1.1	255.255.255.0	—
Router Gi0/1	192.168.1.1	255.255.255.0	—
Router Gi0/2	192.168.2.1	255.255.255.0	—

Each router interface serves as the default gateway for the hosts in its respective LAN, which is exactly how real IP routers attach to multiple subnets.

1. What Happens When PC1 Sends a Message to PC5

When PC1 (10.1.1.10) sends a message to PC5 (192.168.2.4), the destination address does not belong to the 10.1.1.0/24 subnet, so PC1 cannot deliver the packet directly. Instead, it forwards the datagram to its default gateway (10.1.1.1), which is the router's Gi0/0 interface. The following steps describe the full data plane journey:

1. PC1 creates an IPv4 datagram with source address 10.1.1.10 and destination address 192.168.2.4, encapsulates it in an Ethernet frame addressed to the router's MAC address, and sends it to Switch 1.
2. Switch 1 examines the destination MAC address and forwards the frame to the router's Gi0/0 port without inspecting the IP header.
3. The router receives the frame on Gi0/0, strips the link layer header, and reads the destination IP address 192.168.2.4 from the IP datagram.
4. The router consults its forwarding table, identifies the 192.168.2.0/24 entry, and selects Gi0/2 as the output interface.

5. The router decrements the TTL field by 1, recalculates the header checksum, and encapsulates the datagram in a new Ethernet frame with the source MAC of Gi0/2 and the destination MAC of PC5, obtained via ARP if not already cached.
6. The frame travels from the router to Switch 3, which forwards it to PC5 based on the destination MAC address.
7. PC5 receives the frame, verifies that the destination IP matches its own address, removes the IP header, and passes the transport layer segment up to the application.

This journey illustrates the data plane principle that each router makes an independent, per-packet forwarding decision based solely on the destination IP address and its forwarding table (Module 4.1, Module 4.2).

2. How PC5 is Connected to the Current Network

PC5 belongs to the new subnet 192.168.2.0/24, which did not exist in the original two LAN topology. The following additions were made to integrate PC5 into the network:

- A third router interface (GigabitEthernet0/2) was configured with IP address 192.168.2.1 and subnet mask 255.255.255.0, making it the default gateway for the 192.168.2.0/24 network.
- A new Switch 3 was connected to this router interface.
- PC5 was connected to Switch 3 and assigned IP address 192.168.2.4, subnet mask 255.255.255.0, and default gateway 192.168.2.1.

The router now holds three directly connected network entries in its forwarding table: 10.1.1.0/24 on Gi0/0, 192.168.1.0/24 on Gi0/1, and 192.168.2.0/24 on Gi0/2. Any packet crossing from one LAN to another is forwarded by consulting this table, which is the core mechanism of data plane forwarding described in the module and in Kurose and Ross, Chapter 4.

3. Implementing and Verifying the Design in Cisco Packet Tracer

Building the Extended Topology

Starting from the Activity 3 topology, the following steps were completed:

1. A new Switch 3 was added to the workspace and connected to the router's GigabitEthernet0/2 interface using a copper straight-through cable.
2. PC5 was added to the workspace and connected to Switch 3.

Configuring IP Parameters

On the router, GigabitEthernet0/2 was configured with IP address 192.168.2.1 and subnet mask 255.255.255.0, and the interface was enabled.

On PC5, the following settings were applied via Desktop and then IP Configuration:

- IP Address: 192.168.2.4
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.2.1

PC1, PC2, PC3, and PC4 retained the same addresses and gateways configured in Activity 3.

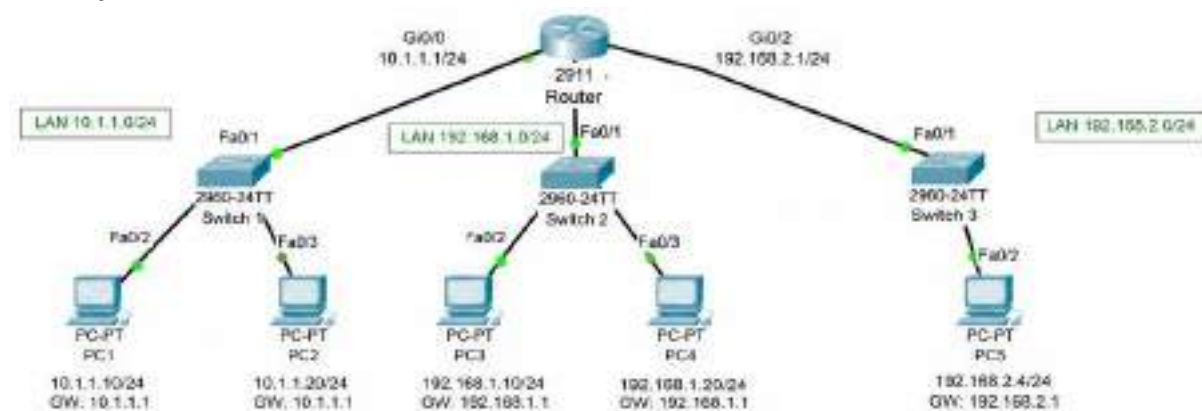


Figure 13. An extended three LAN topology in Cisco Packet Tracer showing Router, Switch 1, Switch 2, Switch 3, and all five PCs with labelled IP addresses and interfaces.

Connectivity Verification Using Ping

From PC1's Command Prompt, the following command was executed:

```
ping 192.168.2.4
```

Four successful ICMP echo replies were received, confirming end-to-end connectivity across all three subnets. This verified that the router's data plane was correctly forwarding packets based on destination IP address and the forwarding table (Module 4.1).

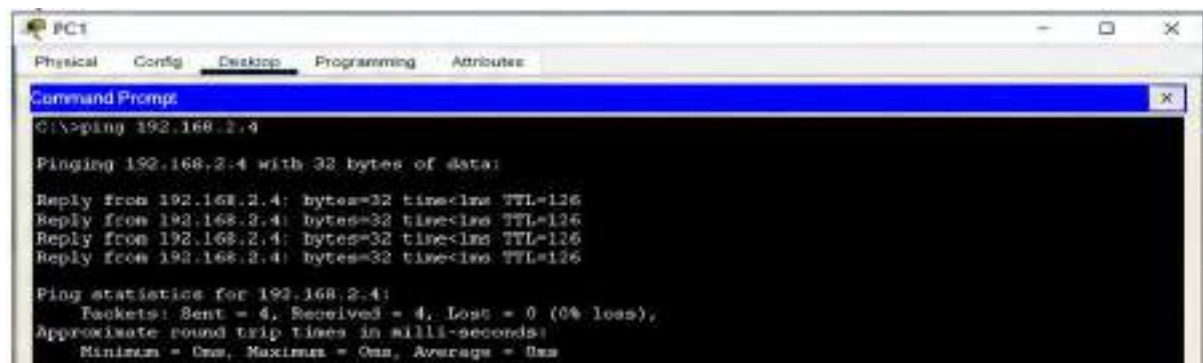


Figure 14. Successful ping from PC1 (10.1.1.10) to PC5 (192.168.2.4), confirming inter-subnet connectivity across three LANs.

Simulation Mode Observation

Cisco Packet Tracer was switched to Simulation mode, and a PDU was sent from PC1 to PC5. Stepping through the simulation confirmed the following sequence:

1. Packet leaves PC1 and arrives at Switch 1
2. Switch 1 forwards the frame to the router based on the destination MAC address
3. Router examines the destination IP, selects Gi0/2, decrements TTL, and re-encapsulates
4. Frame travels to Switch 3, which forwards it to PC5
5. PC5 receives the packet and generates an ICMP echo reply following the reverse path

At each hop, Packet Tracer showed that the link layer frame addresses changed while the IP source and destination addresses remained constant, directly demonstrating the separation between network and link layers covered in the module.



Figure 15. Simulation mode showing the hop-by-hop packet journey from PC1 through Switch 1, the router, Switch 3, and finally to PC5.

Progress Note: This above-and-beyond activity extended the two LAN networks to a three-LAN topology, demonstrating how a single router interconnects multiple subnets by maintaining separate forwarding table entries for each network. Configuring a third router interface, verifying connectivity through ping, and observing the packet journey in simulation mode confirmed that data plane forwarding operates identically regardless of the number of connected subnets, reinforcing the scalability of IP-based routing covered in Module 4.

13 Task 10.2P

Learning summary

Outcome	Weight
Layered encapsulation	◆◆◆◆

Draft Learning Summary Report for SIT202 unit.

Outcome	Weight
Interaction with Networks	◆◆◆◆

Draft Learning Summary Report for SIT202 unit.

Outcome	Weight
Networking Tools	◆◆◆◆

Draft Learning Summary Report for SIT202 unit.

Outcome	Weight
Analysing real network issues	◆◆◆◆

Draft Learning Summary Report for SIT202 unit.

Date	Author	Comment
2026/03/27 09:56	Jahan Garg	Working On It
2026/03/27 11:59	Jahan Garg	Ready for Feedback
2026/03/27 12:04	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 10.2P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/05/01 09:29	Amanpreet Kaur	Dear Jahan, you are required to update first page of submission
2026/05/01 09:29	Amanpreet Kaur	Fix and Resubmit
2026/05/01 10:19	Jahan Garg	Ready for Feedback
2026/05/01 10:19	Amanpreet Kaur	Time Exceeded
2026/05/01 10:19	Jahan Garg	Mam, I have made the required changes with the help of your valuable feedback. Kindly acknowledge the updated submission.Regards,Jahan Garg2410994805X2 Batch
2026/05/05 10:29	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 10.2P

Submitted By:

Jahan GARG

2410994805

2026/05/01 10:19

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◆

Draft Learning Summary Report for SIT202 unit.

May 1, 2026



SIT202

NETWORK AND COMMUNICATIONS

Learning Summary Report



OSI

PHYSICAL LAYER
Concerns the physical aspects of the network, such as the cables, connectors, and the physical transmission medium.

Network Link Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Network Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Transport Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Session Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Presentation Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Application Layer
Concerns the logical aspects of the network, such as the logical addressing and the logical transmission medium.

Jahan Garg

2410994805

Self-Assessment Details

The following checklists provide an overview of my self-assessment for this unit.

	Pass (P)	Credit (C)	Distinction (D)	High Distinction (HD)
Self-Assessment	✓	✓	✓	

Self-Assessment Statement

	Included
Learning Summary Report	✓
Pass tasks complete	✓

Minimum Pass Checklist

	Included
All Credit Tasks	✓

Minimum Credit Checklist (in addition to Pass Checklist)

	Included
All Distinction tasks	✓

Minimum Distinction Checklist (in addition to Credit Checklist)

	Included
Something Awesome included	
Helping Others	

Minimum High Distinction Checklist (in addition to Distinction Checklist)

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part of this submission been written for me by another person.

Signature:



Portfolio Overview

This portfolio presents the work I have completed so far in **SIT202 – Network and Communications** and explains how that work demonstrates the Unit Learning Outcomes while supporting my application for a Distinction-grade result.

Across this unit, I have developed a clearer and more practical understanding of how networked systems operate from end to end. My submitted work shows that I can explain how layers in the protocol stack interact, analyse the behaviour of common protocols, use industry-relevant networking tools to investigate real traffic, and reflect on the security, performance, and scalability issues that arise in real network communication.

I demonstrated ULO1 through my work on the Application Layer and Transport Layer, where I explained layered abstraction, encapsulation, addressing, transport services, and process-to-process communication. This is evident in my discussion of HTTP, HTTPS, DNS, TCP, UDP, sockets, and the relationships between protocol layers, which shows that I understand not only the role of each layer but also how the layers work together to enable meaningful data transfer between end systems.

I demonstrated ULO2 by engaging with the design and development side of networking concepts rather than treating them only as theory. My DNS Server Sketch task required me to think systematically about server initialisation, DNS query handling, A and CNAME records, header construction, and response generation, which helped me connect protocol knowledge with structured implementation logic.

I demonstrated ULO3 through practical analysis using tools such as Wireshark, nslookup, Python socket programming, and Cisco Packet Tracer. Through these activities, I analysed the behaviour of HTTP, HTTPS, DNS, TCP, and UDP, examined packet headers and message sequences, interpreted real network data, and connected simulation and traffic analysis to the theoretical concepts studied in the unit.

I demonstrated ULO4 through reflection on how protocols are designed and why their design choices matter in real systems. My work on TLS, DNS hierarchy and caching, TCP reliability and control mechanisms, IP forwarding, and router behaviour demonstrates a growing ability to evaluate networking decisions in terms of security, reliability, performance, and scalability, rather than simply describing protocols in isolation.

The evidence in this portfolio represents more than basic task completion. My Application Layer, Transport Layer, and Network Layer work shows detailed conceptual understanding, while my DNS Server Sketch extends that understanding into structured design thinking and practical protocol modelling.

Overall, this portfolio reflects sustained effort to build depth of understanding, strengthen practical skills, and improve the quality of my work through analysis, reflection, and application across multiple modules. For these reasons, I believe this portfolio supports a Distinction-grade application by demonstrating clear progress beyond foundational understanding and increasing independence in the way I analyse and apply computer networking concepts.

Learning Plan Check-in

The target grade indicated in the learning Plan I submitted at the start of the Trimester: Distinction

The target grade of my portfolio: Distinction

Reflection:

My target grade has remained Distinction throughout the trimester. Although I experienced some setbacks, including health-related absences, time-management pressure across multiple units, and adjustment challenges following the introduction of the new OnTrack version, I chose not to lower my goal. Instead, I responded by following a structured catch-up plan that included regular consultation with Dr Amanpreet Kaur, peer discussion, revision through D2L materials, and focused support from reliable online resources and tutorial videos.

This approach helped me stay committed to steady improvement rather than short-term task completion. I continued to target Distinction because I wanted my understanding to be conceptually strong rather than rushed, and because the quality of my submitted work showed that I was capable of working toward that standard through consistent effort.

Reflection

The most important things I learnt:

The most important thing I learned in this unit is that network communication makes sense only when the layers are understood as an interconnected process rather than as isolated protocols. I developed a much clearer understanding of how a simple user action, such as entering a URL, triggers a sequence involving DNS resolution, transport-layer communication, and application-layer message exchange. This changed my view of networks from something abstract into something I can analyse logically and step by step. I also learned that practical networking knowledge depends on being able to observe and interpret real traffic. Through Wireshark, nslookup, and protocol analysis, I moved beyond memorising concepts and began identifying how protocols actually behave in real environments. That shift from theory to evidence was one of the most valuable parts of the unit.

The things that helped me most were:

The most helpful parts of my learning were the active classes, practical protocol analysis, and the requirement to explain concepts in my own words. These activities pushed me beyond passive reading and helped me build stronger conceptual clarity. Regular revision, tutor consultation, peer discussion, and the use of D2L resources, reference texts, authentic websites, and selected YouTube tutorials were also highly valuable. They allowed me to revisit difficult topics from different perspectives and gradually strengthen both my confidence and accuracy.

I found the following topics particularly challenging:

The DNS topic in the Application Layer was especially challenging for me because I missed part of that content due to health-related absences. As a result, I had to spend extra time rebuilding my understanding of DNS hierarchy, caching, query flow, and packet-level analysis outside class.

I also found the Transport Layer and parts of the Network Layer demanding because they were content-heavy and required careful revision to understand the relationships between protocol behaviour, reliability, delay, and control mechanisms. In addition, managing the workload across four units made it harder to maintain an ideal pace throughout the trimester.

I found the following topics particularly interesting:

I found HTTP, HTTPS, TLS, DNS, and packet analysis especially interesting because they revealed what happens behind everyday Internet use. I enjoyed tracing requests, responses, headers, and encrypted communication because it made network behaviour more visible and logically understandable.

I was also particularly interested in the design side of networking tasks, especially the DNS Server Sketch, because it encouraged me to think like a problem-solver rather than only a learner. Designing protocol logic helped me appreciate how theoretical networking concepts can be translated into structured implementation steps.

I feel I learned these topics, concepts, and/or tools really well:

I believe I learnt the Application Layer particularly well, especially HTTP, HTTPS, DNS, client-server communication, sockets, and the use of Wireshark to analyse protocol exchanges. My submitted Application Layer review demonstrated detailed explanation, practical analysis, and reflection, and my self-assessment results for the earlier completed modules were strong, including 100 per cent in Modules 1, 2, and 3.

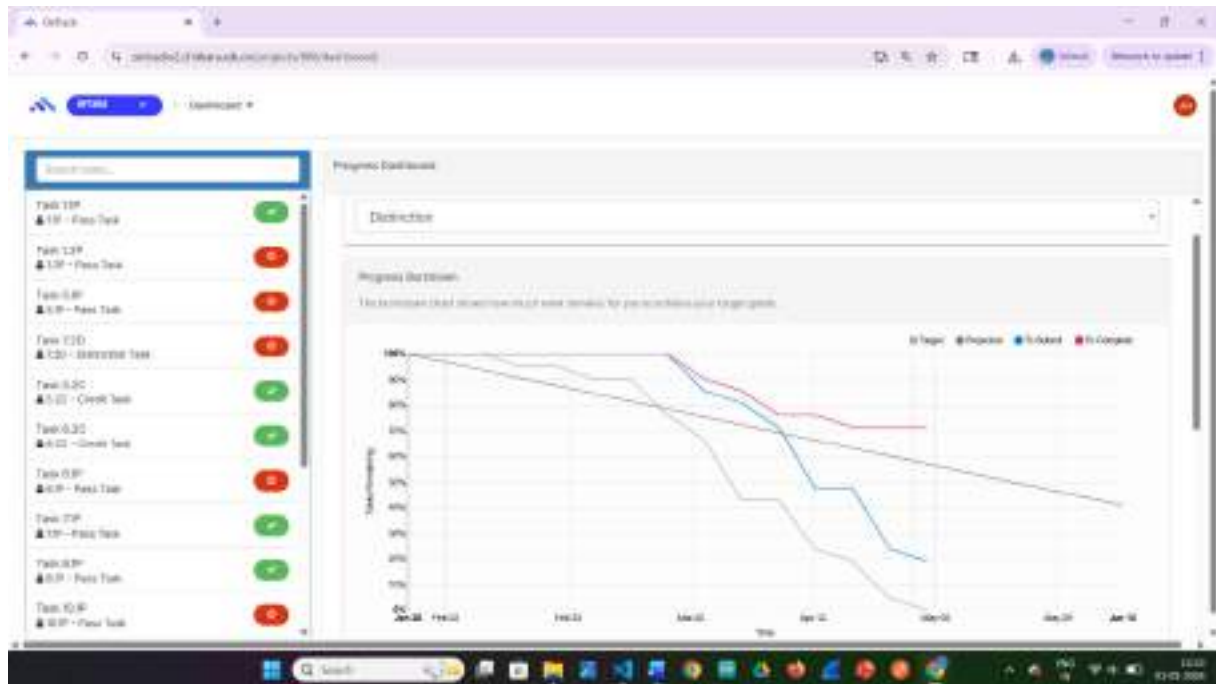
I also developed a solid understanding of the Transport Layer, including UDP, TCP, connection management, reliability, header interpretation, and the relationship between transport services and application needs. My DNS Server Sketch task further strengthened my confidence in structured protocol thinking and showed that I could apply my knowledge in a more design-oriented context.

I still need to work on the following areas:

I still need to strengthen my confidence in the later parts of the unit, particularly by ensuring that my remaining tasks are completed to the same standard as my strongest earlier submissions. My focus is not only on completing them, but also on maintaining depth, clarity, and accuracy across all remaining portfolio pieces.

I also want to become faster and more consistent in pacing my work across the trimester. One of the main lessons I have learnt is that understanding improves most when revision is spread evenly rather than delayed by backlog pressure.

My progress in this unit was uneven:



My progress graph shows that my completion rate did not initially keep pace with the ideal target. From my perspective, this reflects a period in which missed classes, competing workload, and adjustment to the new OnTrack workflow slowed my momentum and created a backlog.

However, the graph also shows that I responded by working to reduce that gap and regain control of my progress. Rather than lowering my goal, I used the graph as a practical reminder to improve my pacing, prioritise core tasks, and continue working towards a Distinction through structured catch-up and consistent effort.

This unit will help me in the future:

This unit will be valuable for my future studies and career because it has strengthened the networking foundation that underpins secure system design, software architecture, embedded systems, and practical problem-solving. It has helped me understand how communication actually occurs across systems, which is essential for many advanced areas of computing.

In particular, this unit is highly relevant to my interest in ethical hacking. Ethical hacking requires a deep understanding of protocols, traffic behaviour, addressing, transport services, DNS, encryption, and common network weaknesses, and SIT202 has helped me build that foundation in a structured and practical way. It has also supported my broader interests in secure development, interactive systems, IoT, AI/ML-driven applications, and future technology ventures.

If I did this unit again, I would do the following things differently:

If I completed this unit again, I would begin structured revision earlier and maintain a more regular weekly cycle of review, practice, and task drafting. This would reduce pressure during heavy weeks and allow more time for refinement instead of catch-up.

I would also make earlier use of consultation and progress-monitoring tools whenever I noticed a backlog starting to build. One of the clearest lessons from this trimester is that timely action is far more effective than waiting until several tasks begin to overlap.

Other:

One of the most valuable things I developed in this unit was resilience. Even when I faced disruption, I continued working toward conceptual mastery instead of shifting to a lower target or aiming only for minimal completion.

Overall, this unit helped me become a more reflective and practical learner. It strengthened not only my networking knowledge, but also my ability to analyse technical systems carefully, connect theory with evidence, and continue improving through feedback and persistence.

14 Task 8.1P

Pass task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I summarised and reflected on Module 5 by explaining network layer control plane concepts, analysing routing algorithms and supporting protocols, documenting my learning through module exercises and Active Class activities, and presenting clear evidence of understanding through Packet Tracer work and self-assessment. For ULO1, I explained how network-layer control plane mechanisms, routing decisions, and protocols such as DHCP, NAT, and ICMP support communication between devices, which directly shows how abstraction in the protocol stack enables data to move across networks. For ULO2, this task had no direct focus on designing or developing programs that interact with networks, because the work centred on conceptual explanation, protocol analysis, simulation, and reflection rather than software implementation. For ULO3, I used Cisco Packet Tracer and related learning activities to analyse routing behaviour, DHCP configuration, NAT, and ICMP-based diagnostics, which strongly aligns with using industry-relevant networking tools to examine configurations, protocols, and solutions. For ULO4, I reflected on how control plane mechanisms and routing approaches are constructed, and considered their importance for scalability, adaptation, management, and troubleshooting in real networking contexts, which gives this outcome moderate relevance.

Outcome	Weight
Networking Tools	◆◆◆◆◆

In this task, I summarised and reflected on Module 5 by explaining network layer control plane concepts, analysing routing algorithms and supporting protocols, documenting my learning through module exercises and Active Class activities, and presenting clear evidence of understanding through Packet Tracer work and self-assessment. For ULO1, I explained how network-layer control plane mechanisms, routing decisions, and protocols such as DHCP, NAT, and ICMP support communication between devices, which directly shows how abstraction in the protocol stack enables data to move across networks. For ULO2, this task had no direct focus on designing or developing programs that interact with networks, because the work centred on conceptual explanation, protocol analysis, simulation, and reflection rather than software implementation. For ULO3, I used Cisco Packet Tracer and related learning activities to analyse routing behaviour, DHCP configuration, NAT, and ICMP-based diagnostics, which strongly aligns with using industry-relevant networking tools to examine configurations, protocols, and solutions. For ULO4, I reflected on how control plane mechanisms and routing approaches are constructed, and considered their importance for scalability, adaptation, management, and troubleshooting in real networking contexts, which gives this outcome moderate relevance.

Outcome	Weight
Analysing real network issues	◆◆◆◆◇

In this task, I summarised and reflected on Module 5 by explaining network layer control plane concepts, analysing routing algorithms and supporting protocols, documenting my learning through module exercises and Active Class activities, and presenting clear evidence of understanding through Packet Tracer work and self-assessment. For ULO1, I explained how network-layer control plane mechanisms, routing decisions, and protocols such as DHCP, NAT, and ICMP support communication between devices, which directly shows how abstraction in the protocol stack enables data to move across networks. For ULO2, this task had no direct focus on designing or developing programs that interact with networks, because the work centred on conceptual explanation, protocol analysis, simulation, and reflection rather than software implementation. For ULO3, I used Cisco Packet Tracer and related learning activities to analyse routing behaviour, DHCP configuration, NAT, and ICMP-based diagnostics, which strongly aligns with using industry-relevant networking tools to examine configurations, protocols, and solutions. For ULO4, I reflected on how control plane mechanisms and routing approaches are constructed, and considered their importance for scalability, adaptation, management, and troubleshooting in real networking contexts, which gives this outcome moderate relevance.

Date	Author	Comment
2026/04/07 11:59	Jahan Garg	Ready for Feedback
2026/04/08 04:54	Jahan Garg	pdf document
2026/04/08 04:55	Jahan Garg	pdf document
2026/04/08 05:04	Jahan Garg	pdf document
2026/04/08 05:05	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 8.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/04/11 16:46	Amanpreet Kaur	Dear Jahan, What do static routing and dynamic routing mean: You are advised to explore it
2026/04/11 16:46	Amanpreet Kaur	Fix and Resubmit
2026/04/12 17:38	Jahan Garg	Ready for Feedback
2026/04/12 17:38	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 8.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/04/12 17:41	Jahan Garg	Static routing means the network administrator manually enters the routes into a router's routing table, so the path remains fixed until a person changes the configuration. This makes static routing simple, predictable, and useful in small or stable networks, but it is less flexible because it does not automatically respond when a link fails or the topology changes.Dynamic routing means routers use routing protocols to exchange network information and automatically learn, select, and update routes from source to destination. This makes dynamic routing more suitable for larger or changing networks because the control plane can adapt the forwarding tables without requiring manual reconfiguration of every router.A simple way to understand the difference is that static routing is like giving a driver one fixed printed route, while dynamic routing is like using GPS that recalculates the path when the road layout changes. In Module 5 terms, both methods support the control plane's job of determining routes, but dynamic routing does this more efficiently in networks where change is common.References: https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23500/View
2026/04/17 08:54	Amanpreet Kaur	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 8.1P

Submitted By:

Jahan GARG

2410994805

2026/04/12 17:38

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Networking Tools	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◆

In this task, I summarised and reflected on Module 5 by explaining network layer control plane concepts, analysing routing algorithms and supporting protocols, documenting my learning through module exercises and Active Class activities, and presenting clear evidence of understanding through Packet Tracer work and self-assessment. For ULO1, I explained how network-layer control plane mechanisms, routing decisions, and protocols such as DHCP, NAT, and ICMP support communication between devices, which directly shows how abstraction in the protocol stack enables data to move across networks. For ULO2, this task had no direct focus on designing or developing programs that interact with networks, because the work centred on conceptual explanation, protocol analysis, simulation, and reflection rather than software implementation. For ULO3, I used Cisco Packet Tracer and related learning activities to analyse routing behaviour, DHCP configuration, NAT, and ICMP-based diagnostics, which strongly aligns with using industry-relevant networking tools to examine configurations, protocols, and solutions. For ULO4, I reflected on how control plane mechanisms and routing approaches are constructed, and considered their importance for scalability, adaptation, management, and troubleshooting in real networking contexts, which gives this outcome moderate relevance.

April 12, 2026



Module 5: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around Module 5's main learning focus: understanding how the network layer control plane determines routes, populates forwarding tables, and supports communication through routing algorithms and related protocols and mechanisms such as Dynamic Host Configuration Protocol (DHCP), Network Address Translation (NAT), and Internet Control Message Protocol (ICMP).

Whereas the previous module concentrated on local packet forwarding in the data plane, this module explained that routing is a network-wide function of the control plane that determines the path packets take from source to destination. The module also distinguished between traditional per-router control, where each router runs its own routing algorithm, and logically centralised control, where a remote controller can compute and install forwarding information.

A major part of the module focused on routing approaches and routing algorithms. Static routing was presented as a manually configured method that is simple and predictable in small networks but difficult to maintain when the topology changes. Dynamic routing, by contrast, allows routers to exchange routing information and update paths automatically, making it more suitable for larger or changing networks. The module also clarified the difference between link-state and distance-vector approaches, showing that link-state routing uses a broader view of the topology and shortest-path computation through Dijkstra's algorithm, whereas distance-vector routing depends on iterative exchanges with neighbours and can converge more slowly. In addition, the module emphasised that network management is important because modern networks require continuous monitoring, configuration, and fault handling to maintain performance, reliability, and security.

The practical activities helped translate these concepts into observable network behaviour. In Active Class 7, I first used a role-play activity to identify subnets, distinguish the responsibilities of the data plane and control plane, and explain how routers learn paths to remote networks. I then implemented static routing in Cisco Packet Tracer using a three-router topology, verified connectivity between hosts with ping, and manually revised routes after a topology change, which clearly demonstrated both the usefulness and the limitations of static routing. In the dynamic routing activity, I re-created the topology using a routing protocol such as the Routing Information Protocol (RIP) and observed that routes were learned and updated automatically after a link change, showing how the control plane adapts without manual reconfiguration. Active Class 8 extended this learning by examining IPv4 address limitations and NAT, performing DHCP role play through the Discover, Offer, Request, and Acknowledgement sequence, configuring a multi-LAN DHCP network in Packet Tracer, and analysing ICMP ping and traceroute behaviour in simulation mode.

B. How This Is Useful

Understanding control plane behaviour is useful because real networks depend not only on packet forwarding, but also on correct route computation and maintenance. Knowing when to use static routes, when to prefer dynamic routing, and how routing algorithms construct forwarding information helps me design networks that are both functional and scalable. The supporting protocols studied in this module are also practically important because DHCP automates host configuration, NAT conserves public IPv4 addresses, and ICMP-based tools such as ping and traceroute help diagnose connectivity and path problems. These ideas are directly relevant to Packet Tracer configuration tasks, troubleshooting activities, and later networking topics that build on control plane concepts.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned in this module is that the control plane is the decision-making part of the network layer. Before this module, I mainly focused on what happened to a packet at each hop, but now I understand much more clearly how routers obtain and update the information that makes forwarding possible. Learning the differences between static routing, dynamic routing, link-state routing, and distance-vector routing made the behaviour of routers in Packet Tracer far more meaningful, because I could connect visible path changes to the underlying routing logic rather than viewing them only as configuration steps.

B. How This Relates to What I Already Know

This module connects directly with what I learned earlier about the network layer data plane and with my previous understanding of end-to-end communication. Module 4 helped me understand how routers forward packets locally using forwarding tables, while Module 5 explained how those tables are created, updated, and maintained through control plane processes, so the two modules now fit together as one complete model of network-layer operation. It also connects with my earlier work in Wireshark and Packet Tracer, because related protocols and mechanisms such as DHCP and ICMP are no longer just names in packet traces; I now understand their operational purpose in address assignment, diagnostics, and path discovery.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because many real networking problems are control plane problems rather than simple hardware failures. Incorrect routes, poor scalability, lack of automatic adaptation, misconfigured DHCP settings, and failure to interpret ICMP diagnostic messages can all prevent communication even when the physical network is functioning correctly. By learning how routing decisions are made, how NAT and DHCP support host connectivity, and how ICMP reveals network behaviour, I am better prepared to analyse, configure, and troubleshoot networks in both coursework and practical environments. This module also provides the conceptual bridge from basic forwarding behaviour to more advanced topics such as routing protocols, network management, and software-defined networking.

References [1], [2], [3], [4], [5], [6], [7]

- [1] "Module 5: Network Layer - Control Plane - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/Home?itemIdentifier=D2L.LE.Content.ContentObject.ModuleCO-23440>
- [2] "5.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23495/View>
- [3] "5.2 DHCP and NAT - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23496/View>
- [4] "5.3 Control plane operation of the network layer - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23497/View>
- [5] "5.4 Routing Algorithms - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23498/View>
- [6] "5.5 ICMP and network management - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23499/View>
- [7] "Module 5 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23500/View>

Module 5: Evidence of Learning

This section presents evidence of my learning for **Module 5: Network Layer – Control Plane**, with a focus on routing algorithms, static/dynamic routing behaviour, DHCP address assignment, NAT address conservation, and ICMP diagnostics. It includes module exercises, group role-play outcomes, Packet Tracer routing simulations, DHCP configuration work, ICMP packet analysis, and supporting screenshots. Together, these items demonstrate my understanding of control plane operations, routing table computation, protocol message flows, and network adaptation to topology changes. Short progress notes are included throughout to show how each task contributed to my developing understanding of the module.

Evidence: Module Exercises

Exercise 1: (Section 5.2 – DHCP)

Question: Can today's computer networks survive without DHCP? Explain your answer.

Ans. In principle, today's networks could operate without DHCP if every device were configured manually with its IP address, subnet mask, default gateway, and DNS servers, but this approach would be slow, error-prone, and very hard to scale for home, campus, and enterprise environments. DHCP automates address assignment and distributes key parameters whenever a device joins the network, which greatly reduces configuration errors, supports mobile devices that frequently connect and disconnect, and simplifies management of large networks with many hosts. [1], [2]

Exercise 2: (Section 5.3 – Static vs Dynamic Routing)

Question: What do static routing and dynamic routing mean?

Ans. Static routing means that an administrator manually enters and maintains routes in each router's routing table, so the paths do not change unless the configuration is edited; this is simple and predictable, but becomes difficult to maintain when the topology changes. Dynamic routing means that routers run routing protocols such as RIP or OSPF to automatically learn and update routes based on exchanged routing information, which allows the network to adapt to link failures or topology changes without manual reconfiguration. [1], [3]

Exercise 3 (Section 5.4 – Dijkstra's Algorithm)

Question: Consider the given network (Figure 1) and determine the shortest path from node A to all other nodes using Dijkstra's algorithm. Explain how you computed the shortest paths using a table similar to the example in the module

Ans. Using Dijkstra's algorithm from node A, I iteratively built the set N' of nodes with known shortest distances, and updated the tentative distance $D(v)$ to each remaining node based on the current node's neighbours and link costs from Figure 1. The table below shows the main steps, where $D(v)$ is the current shortest known distance from A to the node v , and $p(v)$ is the previous hop on that path. [1], [3]

Step	N'	$D(B), p(B)$	$D(C), p(C)$	$D(D), p(D)$	$D(E), p(E)$	$D(F), p(F)$	$D(G), p(G)$
0	{A}	2, A	4, A	∞	∞	7, A	∞
1	{A, B}	—	4, A	5, B	∞	7, A	∞
2	{A, B, C}	—	—	5, B	7, C	7, A	∞
3	{A, B, C, D}	—	—	—	7, C	7, A	∞
4	{A, B, C, D, E}	—	—	—	—	7, A	15, E
5	{A, B, C, D, E, F}	—	—	—	—	—	15, E
6	{A, B, C, D, E, F, G}	—	—	—	—	—	—

From this, the final shortest-path costs and paths from A are:

- To B: cost 2 via A–B
- To C: cost 4 via A–C
- To D: cost 5 via A–B–D
- To E: cost 7 via A–C–E
- To F: cost 7 via A–F
- To G: cost 15 via A–C–E–G

These results match the link costs in the diagram and illustrate how Dijkstra's algorithm builds a shortest-path tree rooted at A.

Exercise 4 (Section 5.4 – Link-State vs Distance-Vector)

Question: What are the differences between the link-state algorithm and the distance-vector algorithm?

Ans. Link-state algorithms require each router to learn the full network topology and link costs (for example, by flooding link-state advertisements), then run Dijkstra's shortest-path algorithm locally to compute routes, which gives fast convergence and a consistent global view at the cost of more state and computation. Distance-vector algorithms, such as those used in RIP, are decentralised: each router only knows the cost to its neighbours, and periodically exchanges distance vectors with them, updating its estimates using the Bellman-Ford equation, which is simpler but can converge more slowly and is vulnerable to problems like count-to-infinity. [1], [3]

https://www.eecs.umich.edu/courses/eecs489/w10/winter10/lectures/lecture6_2.pdf

Exercise 5 (Section 5.5 – Network Management)

Question: Why is “network management” important?

Ans. Network management is important because modern networks are large, dynamic systems where routers, switches, and wireless devices must be continuously monitored, configured, and secured to maintain acceptable performance and reliability. Without centralised management tools and protocols such as SNMP or telemetry, it would be very difficult to detect failures, congestion, or misconfigurations in time, and operators would struggle to apply consistent changes across devices, which would increase downtime and security risks. [1], [4]

Evidence: Active Classes Activities**Activities for Active Class 7: Routing and Control Plane****Activity 1: Routing Role Play**

Question 1–4 (role play and discussion).

Ans. In our group role play, we treated LAN1 (10.1.1.0/24) on the left and LAN2 (192.168.1.0/24) on the right as two separate subnets connected by Router0 and Router1 with a point-to-point link between the routers. We configured PC1 as 10.1.1.10/24 with default gateway 10.1.1.1 on Router0, and PC3 as 192.168.1.10/24 with default gateway 192.168.1.1 on Router1; the inter-router link used a small transit subnet so that Router0 and Router1 could route traffic between the LANs. Two group members acted as the data planes of Router0 and Router1, forwarding packets based on their local forwarding tables, while another member acted as the

shared control plane, installing routes that told each router where LAN1 and LAN2 were located so that PC1's packets to PC3 were sent via the inter-router link. [1]

When we discussed the questions, we identified **three subnets** in this scenario: LAN1, LAN2, and the point-to-point link between the two routers. The control plane's responsibilities included learning which subnets are reachable via each neighbour, computing suitable routes (for example, via static routes or a dynamic protocol), and populating the routing tables on Router0 and Router1 so that their data planes could forward packets correctly without needing to understand the overall network. [3]

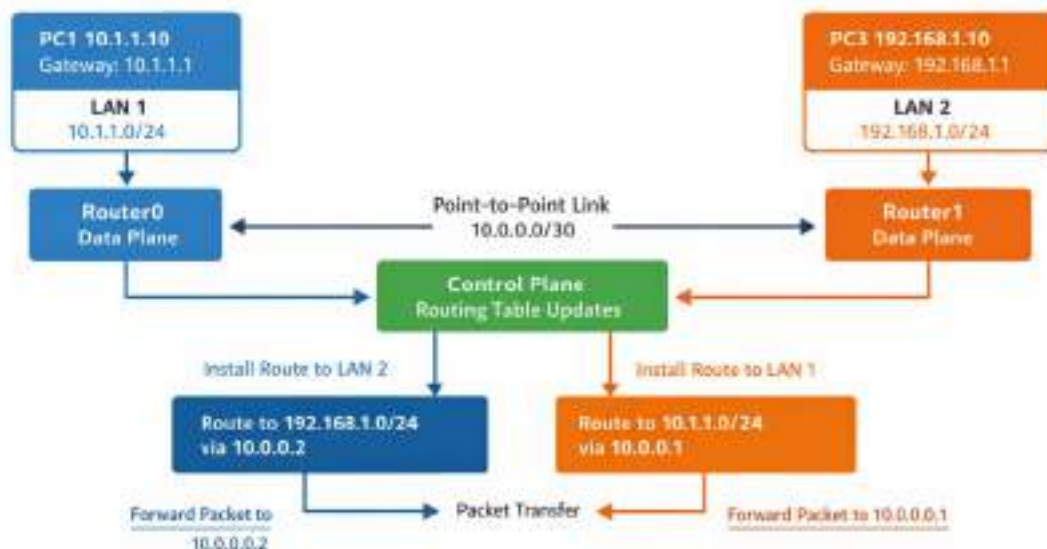


Figure 1. Flow chart illustrating packet forwarding and control-plane interactions between LAN1, LAN2, Router0, and Router1.

Activity 2: Static Routing in Packet Tracer

Question 1–2 (building the three-router network and then changing links).

Ans. For the first part of Activity 2, we implemented the three-router topology with LAN1 (192.168.3.0/24) behind Router1, LAN2 (192.168.1.0/24) behind Router2, and LAN3 (192.168.2.0/24) behind Router3, using serial links between Router1–Router2 and Router2–Router3. Each PC was given an address in the correct subnet with its default gateway set to the LAN interface of the local router, and we then added static routes so that every router knew how to reach the two remote LANs via the appropriate neighbour, allowing PC1, PC3, and PC5 to successfully ping each other and confirming end-to-end connectivity.

After we removed the direct link between Router2 and Router3 and added a new serial link between Router1 and Router3, we updated the static routes to match the new topology: routes that previously pointed from Router1 or Router3 via Router2 were adjusted so that traffic between LAN1 and LAN3 used the new direct Router1–Router3 link. Once the static routes were corrected on all routers, ICMP echo

requests from PC1 to PC5 again followed a valid path, and the ping tests and simulation mode verified that packets were forwarded according to the updated routing tables.

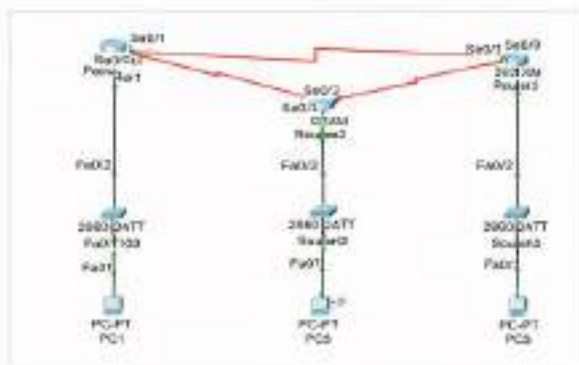


Figure 2. Initial three-router topology in Cisco Packet Tracer.

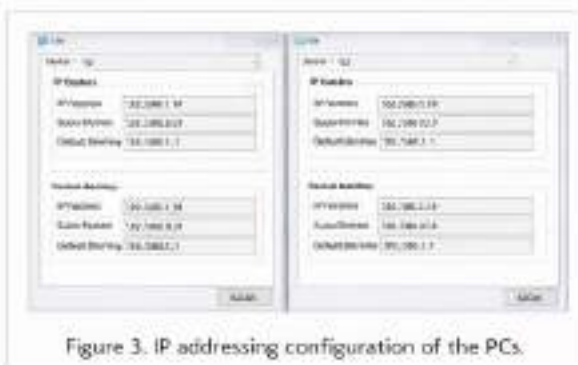


Figure 3. IP addressing configuration of the PCs.



Figure 4. Static routing configuration on the routers



Figure 5. Static routing configuration on the routers.

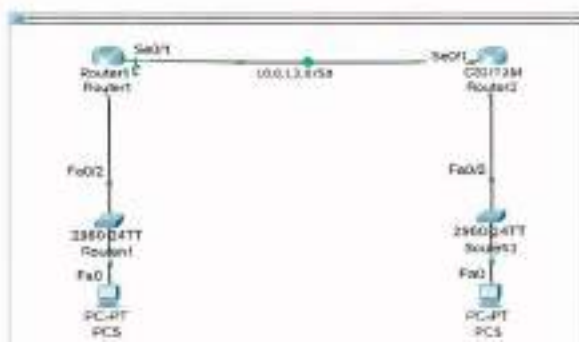


Figure 6. Successful ping between PC1, PC3, and PC5 in the original topology.

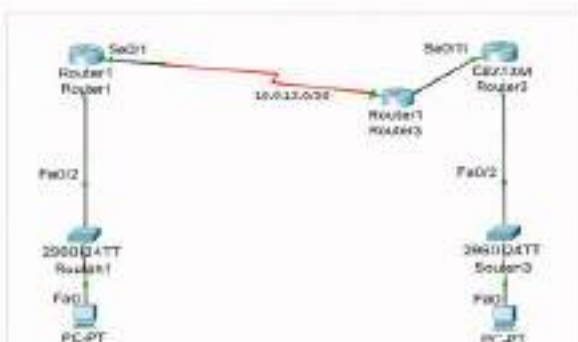


Figure 7: Updated topology after replacing the Router2-Router3 link with Router3.



Figure 7. Packet flow in simulation mode before the topology change.



Figure 9. Revised static routes after the topology change.

Activity 3: Alternative to Static Routing

Questions 1–4, 6–8.

Ans. In our discussion, we agreed that manually updating static routes whenever the network changes does not scale well, especially as the number of routers increases. A better alternative is to use a dynamic routing protocol, which automatically exchanges routing information and recalculates paths when the topology changes. Examples include distance-vector protocols such as RIP and EIGRP, and link-state protocols such as OSPF and IS-IS. In Cisco Packet Tracer, these protocols can be configured directly on the routers to automate route learning and maintenance.

We then re-implemented the three-router topology using a dynamic routing protocol such as RIP by enabling it on Router1, Router2, and Router3 and advertising the directly connected LAN and serial networks. After the routing tables converged, PC1, PC3, and PC5 could ping each other successfully, and simulation mode showed packets following the learned route through the network. When the Router1–Router3 link was removed, the routers automatically updated their routing tables and traffic between LAN1 and LAN3 was rerouted via Router2 without any manual configuration changes. This demonstrated that the control plane was handled by the dynamic routing protocol rather than by static routes. [1]

https://www.eecs.umich.edu/courses/eecs489/w10/winter10/lectures/lecture6_2.pdf

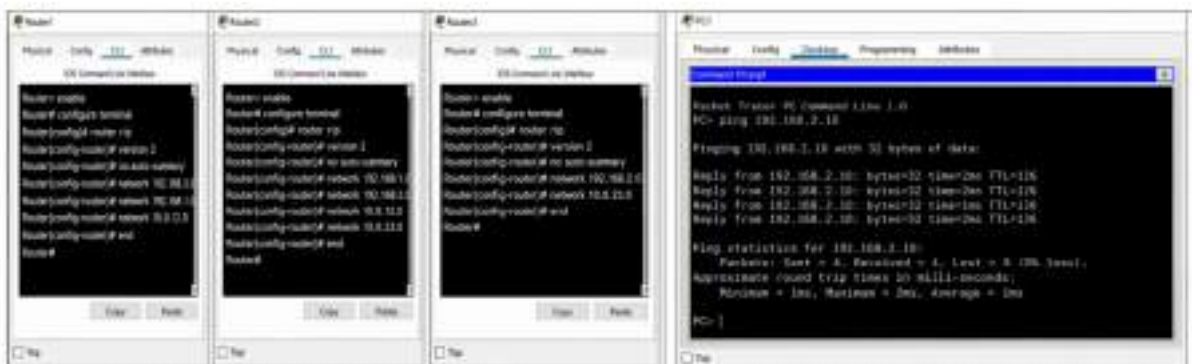


Figure 10: Dynamic routing configuration on routers

Figure 11: Successful ping verification between PC1 and PC5



Figure 12: Packet flow simulation before link removal

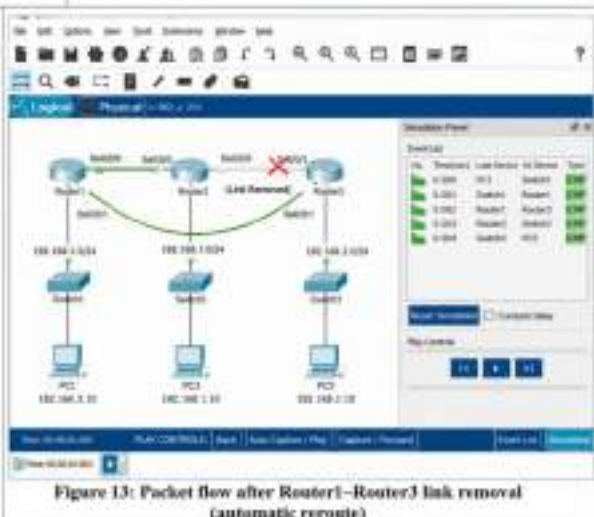


Figure 13: Packet flow after Router1–Router3 link removal (automatic reroute)

Activities for Active Class 8: NAT, DHCP, and ICMP

Activity 1: Address Space and NAT

Questions 1–3.

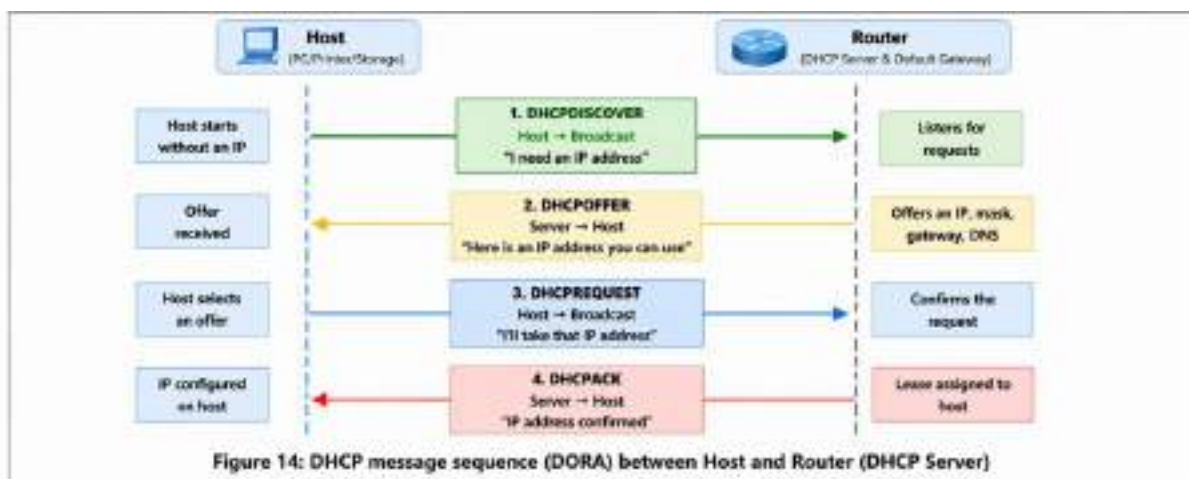
Ans. For the first question, we concluded that the original IPv4 address space is not sufficient to give a unique public IP address to every device, especially when billions of IoT devices are added, because IPv4 has only about 4.3 billion addresses and many are reserved or already allocated. The two main solutions we discussed were the gradual deployment of IPv6, which provides a vastly larger address space, and the use of Network Address Translation (NAT) so that many private devices can share a smaller pool of public addresses. [1], [5]

As an example of a mechanism used with IPv4 to conserve global address space, we identified **NAT**, which translates internal private addresses such as 192.168.x.x or 10.x.x.x into one or a few public addresses at the network edge, allowing many hosts to access the Internet while using only a limited number of globally routable IPv4 addresses. [6]

Activity 2: DHCP Role Play and Packet Tracer Network

(a) Role-play message sequence

Ans. In the role play, we treated the LAN as a small network with one router acting as both the default gateway and DHCP server and three hosts representing a PC, a printer, and a network storage device, all initially without IP configuration. The DHCP process followed the standard DORA sequence: each host broadcast a **DHCPDISCOVER** message, the router's DHCP server replied with a **DHCPOFFER**, the host selected an offer and sent a **DHCPREQUEST**, and finally the DHCP server confirmed the assignment with a **DHCPACK** message, after which the host installed its IP address, subnet mask, default gateway, and DNS settings. [1], [2], [7]



(b) Packet Tracer DHCP network

Ans. In the second part, we implemented the multi-LAN network in Packet Tracer with three subnets (192.168.1.0/24, 192.168.2.0/24, and 192.168.3.0/24), connecting them to Router8 and placing the dedicated DHCP server in LAN1 with the address 192.168.1.2. On the DHCP server we configured IP pools for each subnet with the correct default gateway and DNS settings, and on Router8 we enabled the required interfaces and set their IP addresses to act as gateways for each LAN; finally, we configured all hosts to obtain their addresses automatically via DHCP and verified that each host received an IP address and default gateway in the correct subnet and could successfully ping hosts in the same and other LANs. [1], [2]

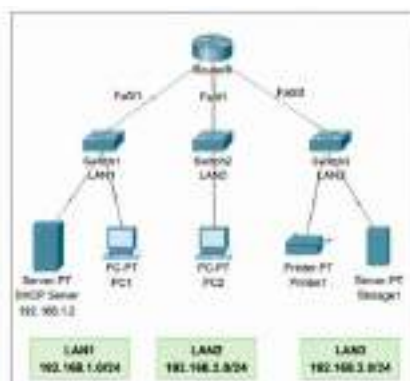


Figure 15: Packet Tracer network topology



Figure 16: DHCP server configuration (IP pools for three LANs)

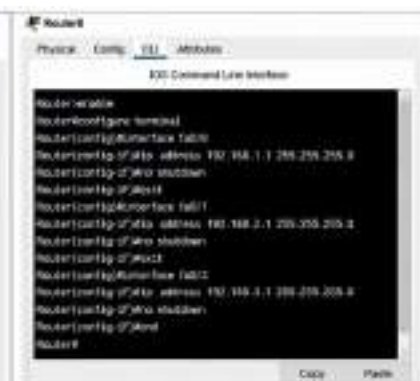


Figure 17: Router8 interface configuration (acting as default gateways)



Figure 18: PC1 received IP via DHCP (LAN1 - 192.168.1.0/24)



Figure 19: PC2 received IP via DHCP (LAN2 - 192.168.2.0/24)



Figure 20: Printst received IP via DHCP (LAN3 - 192.168.3.0/24)



Figure 21: Storage1 received IP via DHCP (LAN3 - 192.168.3.0/24)

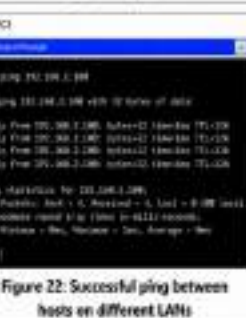


Figure 22: Successful ping between hosts on different LANs

Activity 3: ICMP and Simulation

Question 4 – Outbound PDU details

Ans. In the outbound PDU details we can see a layered view of the packet created by Host A: the **Ethernet II** section shows the local source and destination MAC addresses and the EtherType 0x0800 indicating that the payload is an IPv4 packet, the **IP** section shows fields such as version, header length, total length, identification and fragmentation flags, TTL, protocol value 0x01 indicating ICMP, and the source and destination IPv4 addresses, and the **ICMP** section shows the ICMP type, code, checksum, identifier, and sequence number. Together, these details confirm which host sent the packet, where it is going at the network and link layers, and which ICMP message is carried inside the IP datagram. [8], [9]

Question 5 – ICMP message type and code

Ans. From the ICMP section, we observed that the **Type** field is 0x08 and the **Code** field is 0x00, which, according to the ICMP specification, correspond to an **Echo Request** message used by the ping command to test reachability. The identifier and sequence number fields allow the sender to match the received Echo Reply with the correct Echo Request, which is useful when multiple pings are outstanding. [9], [10]

Question 6 – Changes when using traceroute

Ans. When we repeated the experiment using traceroute (tracert) from Host A to Host B, the simulation showed packets with gradually increasing TTL values and ICMP Time Exceeded messages from intermediate routers instead of only a normal Echo Reply from the destination. Specifically, routers along the path returned ICMP Time Exceeded messages (Type 11, Code 0) when the TTL reached zero, which allowed traceroute to discover each hop. This is different from the simple ping case where we observe only an ICMP Echo Request and Echo Reply pair. [1], [11]

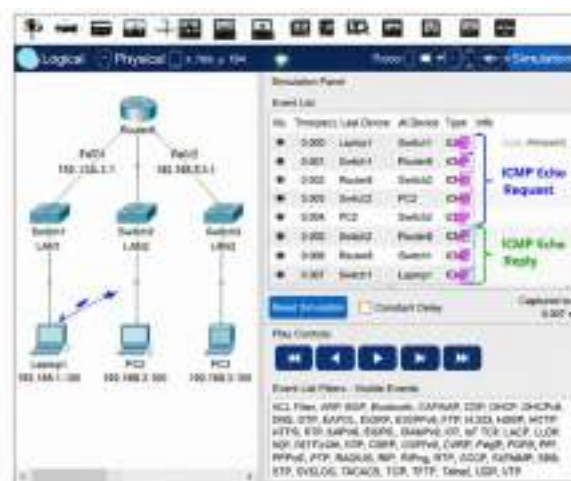


Figure 23: Simulation panel showing ICMP ping packets in the event list



Figure 24: Outbound PDU details from source host (Laptop1) showing Ethernet, IP, and ICMP layers



Figure 25: ICMP header fields showing Type 8, Code 0 (Echo Request) with identifier and sequence number

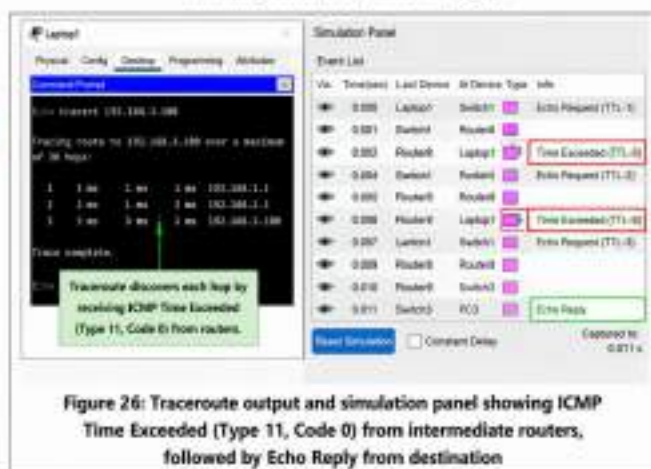


Figure 26: Traceroute output and simulation panel showing ICMP Time Exceeded (Type 11, Code 0) from intermediate routers, followed by Echo Reply from destination

Summary of Learning Evidence

Through the Module 5 exercises and the Active Classes 7 and 8 activities, I demonstrated a comprehensive understanding of network-layer control-plane operations, routing algorithms, and key supporting protocols such as DHCP, NAT, and ICMP.

- **Module Exercises:** Practised core concepts including DHCP, static and dynamic routing, Dijkstra's algorithm, link-state and distance-vector routing, and the importance of network management.
- **Active Class 7:** Explored control plane decision-making through group discussion and role play, then implemented static and dynamic routing in Cisco Packet Tracer, including route updates after topology changes.
- **Active Class 8:** Examined IPv4 address conservation through NAT, analysed DHCP address assignment and message flow, and observed ICMP ping and traceroute behaviour using simulation tools.

This evidence portfolio connects theory from Module 5 to practice through discussion, routing simulation, protocol analysis, and network configuration tasks, demonstrating active learning and a strong understanding of network-layer control-plane concepts.

References

- [1] "Module 5 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 07, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23500/View>
- [2] "DHCP (Dynamic Host Configuration Protocol) Basics | Microsoft Learn." Accessed: Apr. 07, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/troubleshoot/dynamic-host-configuration-protocol-basics>
- [3] K. Jamieson, "Lecture 10: Link State Routing".
- [4] "NAT in Networking | Importance and How It Works." Accessed: Apr. 07, 2026. [Online]. Available: <https://nitizsharma.com/nat-explained-in-networking/>
- [5] "Network Address Translation (NAT) - GeeksforGeeks." Accessed: Apr. 07, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/network-address-translation-nat/>
- [6] "What is NAT? Types, Benefits & Best Practices | Twingate." Accessed: Apr. 07, 2026. [Online]. Available: <https://www.twingate.com/blog/glossary/nat>
- [7] "DHCP client/server interaction - IBM Documentation." Accessed: Apr. 07, 2026. [Online]. Available: <https://www.ibm.com/docs/en/i/7.4.0?topic=concepts-dhcp-clientserver-interaction>

- [8] "ICMP Types - Network Direction." Accessed: Apr. 07, 2026. [Online]. Available: <https://networkdirection.net/articles/network-theory/icmptypes/>
- [9] "What is ICMP Ping? How It Works + Troubleshooting Guide." Accessed: Apr. 07, 2026. [Online]. Available: <https://www.dotcom-monitor.com/learn/glossary/icmp-ping/>
- [10] "ICMP type and code IDs - IBM Documentation." Accessed: Apr. 07, 2026. [Online]. Available: <https://www.ibm.com/docs/en/qsip/7.5.0?topic=applications-icmp-type-code-ids>
- [11] "ICMP, Ping, and Traceroute - What I Wish I Was Taught | xkln.net." Accessed: Apr. 07, 2026. [Online]. Available: <https://xkln.net/blog/icmp-ping-and-traceroute--what-i-wish-i-was-taught/>

[Quiz List](#) > [Submissions](#)

Quiz Submissions – Module 5: Network Layer - Control Plane

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %

Quiz List

Help

Current Quizzes	Evaluation Status	Attempts
Module 5: Network Layer - Control Plane  Due on 22 April 2026 11:59 PM Available on 28 March 2026 12:01 AM until 22 April 2026 11:59 PM	Feedback: On Attempt	1 / 3
Module 6: Data Link Layer  Due on 26 April 2026 11:59 PM Available on 01 April 2026 12:01 AM until 26 April 2026 11:59 PM		0 / 3
Past Quizzes	Evaluation Status	Attempts
Module 1: Introduction to Computer Networking 		

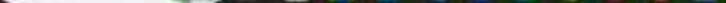


Search



2026

04/04/2026



15 Task 7.2D

Distinction task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I researched the Internet of Things (IoT), explained its key networking concepts and protocols, implemented the given smart home use case in Cisco Packet Tracer, and analysed MQTT using clearly documented visual evidence to demonstrate both conceptual understanding and practical application. For ULO1, I explained how the layers of the network stack support communication between IoT devices, the gateway, and the MQTT broker, showing how data flows through the network to enable sensing, message exchange, and automated responses in the smart home environment. For ULO3, I used Cisco Packet Tracer as an industry-relevant networking tool to configure the smart home topology, observe MQTT communication, analyse device interactions, and present a practical networking solution to the given IoT scenario.

Outcome	Weight
Networking Tools	◆◆◆◆◆

In this task, I researched the Internet of Things (IoT), explained its key networking concepts and protocols, implemented the given smart home use case in Cisco Packet Tracer, and analysed MQTT using clearly documented visual evidence to demonstrate both conceptual understanding and practical application. For ULO1, I explained how the layers of the network stack support communication between IoT devices, the gateway, and the MQTT broker, showing how data flows through the network to enable sensing, message exchange, and automated responses in the smart home environment. For ULO3, I used Cisco Packet Tracer as an industry-relevant networking tool to configure the smart home topology, observe MQTT communication, analyse device interactions, and present a practical networking solution to the given IoT scenario.

Date	Author	Comment
2026/04/08 05:09	Jahan Garg	Working On It
2026/04/12 12:41	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 7.2D task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/04/12 13:35	Jahan Garg	pdf document
2026/04/12 13:54	Jahan Garg	Ready for Feedback
2026/04/12 13:54	Amanpreet Kaur	Time Exceeded
2026/05/11 10:26	Garima Chopra	Complete

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 7.2D

Submitted By:

Jahan GARG

2410994805

2026/04/12 13:54

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Networking Tools	◆◆◆◆◆

In this task, I researched the Internet of Things (IoT), explained its key networking concepts and protocols, implemented the given smart home use case in Cisco Packet Tracer, and analysed MQTT using clearly documented visual evidence to demonstrate both conceptual understanding and practical application. For ULO1, I explained how the layers of the network stack support communication between IoT devices, the gateway, and the MQTT broker, showing how data flows through the network to enable sensing, message exchange, and automated responses in the smart home environment. For ULO3, I used Cisco Packet Tracer as an industry-relevant networking tool to configure the smart home topology, observe MQTT communication, analyse device interactions, and present a practical networking solution to the given IoT scenario.

April 12, 2026



Internet of Things, Smart Home Networking, and MQTT Analysis in Cisco Packet Tracer

Student: Jahan Garg

Unit: SIT202 Network and Communications

Task: 7.2D Distinction Task – IoT and Networks

Abstract

The Internet of Things (IoT) extends computer networking by connecting physical devices such as sensors, actuators, appliances, and controllers to communication networks so that they can collect data, exchange messages, and trigger actions automatically. Unlike traditional networks, which mainly support communication between general-purpose computers and human users, IoT environments must support constrained devices, lightweight messaging, large-scale automation, and continuous sensing. This report introduces IoT, explains its motivation, key technologies, applications, and challenges, and analyses the MQTT protocol as a major application-layer technology for IoT. It also discusses a smart home implementation in Cisco Packet Tracer using the same problem context provided in the task sheet. The discussion shows that IoT is reshaping networking by increasing device density, requiring more scalable and lightweight protocols, and placing stronger emphasis on security, device management, and event-driven communication.

Introduction

The Internet of Things refers to a networked environment in which physical objects are embedded with sensors, software, communication interfaces, and in many cases actuators that allow them to sense conditions, exchange information, and respond intelligently [1], [2]. IoT differs from conventional networking because the communicating entities are not limited to laptops, servers, and smartphones. Instead, the network must also support small devices such as temperature sensors, cameras, smart lights, motion detectors, fans, and door controllers that often operate with limited power, memory, and processing capability [2], [3]. This shift has made networking more closely connected with automation and machine-to-machine communication.

IoT has become popular because it offers practical value in homes, healthcare, industry, transportation, and agriculture. Low-cost sensors, wireless communication, cloud integration, and demand for real-time monitoring have made it feasible to automate many physical processes [3], [4]. In a smart home, for example, connected devices can monitor temperature, detect motion, control lighting, and improve comfort, energy efficiency, and safety. As a result, IoT has moved from being an emerging concept to a major trend influencing the future of networking.

IoT technologies, protocols, and applications

IoT systems typically combine hardware, software, and networking components. At the hardware level, they use sensors to detect conditions and actuators to perform actions. At the software level, they use embedded logic, mobile or web interfaces, and cloud or edge services to process information and coordinate behaviour [3]. From a networking perspective, IoT requires communication across multiple layers. The tutor resource highlights common network and link-layer technologies such as Bluetooth, Zigbee, Wi-Fi, and LoRaWAN, transport protocols such as UDP and TCP, and application protocols such as MQTT, CoAP, HTTP, and AMQP [5]. This layered view is important because IoT does not rely on a single protocol; rather, it uses a stack chosen according to power, range, bandwidth, and reliability requirements.

Different IoT protocols serve different purposes. CoAP is designed for constrained devices and low-overhead communication over UDP, HTTP is useful for web integration but introduces greater overhead, and AMQP is suitable for enterprise-style reliable messaging [5]. MQTT has become one of the most widely used IoT messaging protocols because it is lightweight and works well when devices need efficient communication over TCP/IP networks [2], [5]. Typical IoT applications include smart homes, industrial monitoring, smart health systems, environmental sensing, logistics tracking, and smart city services [3], [4]. Across all of these areas, the main benefit is that data collection and response can happen continuously and automatically.

MQTT analysis

MQTT, which stands for Message Queuing Telemetry Transport, is a lightweight publish/subscribe messaging protocol used extensively in IoT systems [2], [6]. Instead of using direct point-to-point communication between every pair of devices, MQTT introduces a broker-based architecture. The broker acts as the central hub that receives published messages and distributes them to devices that subscribe to the relevant topics [5], [6]. This design is especially useful in IoT because it reduces coupling between devices, supports scalability, and avoids the complexity of every device needing to know the address and status of every other device.

The main MQTT elements are the broker, publishers, subscribers, and topics. A publisher is a device that sends information, such as a temperature sensor publishing current readings. A subscriber is a device or application that listens for relevant messages, such as a fan controller waiting for a temperature threshold event. Topics act as logical message channels, for example `home/livingroom/light` or `home/temp` [5]. This approach is highly effective in smart home environments because one sensor event can trigger multiple subscribed devices without the publisher needing to manage each of them directly.

MQTT is well suited to IoT for three main reasons. First, it is lightweight, which reduces communication overhead for constrained devices [2], [6]. Second, its publish/subscribe model is more scalable than repeated request-response exchanges when multiple devices must react to the same event [6]. Third, because it typically operates over TCP, it benefits from reliable transport while still remaining simpler than heavier web-oriented approaches [5]. For these reasons, MQTT is commonly used for telemetry, remote monitoring, and automation.

Smart home implementation in Cisco Packet Tracer

The task sheet requires implementation of a smart home use case in Cisco Packet Tracer together with MQTT analysis [7]. A suitable design is a home network containing a home gateway, a laptop, and several smart devices such as a smart light, fan, garage door, motion sensor, and temperature sensor. Following the tutor resource, the simplest and safest implementation method is to use the laptop as the MQTT broker while connecting the smart devices to the same wireless network through the home gateway [5]. This approach is directly aligned with the task brief because it demonstrates both IoT networking and MQTT communication in a visible and explainable way.

In this implementation, the home gateway provides local connectivity, the laptop runs the broker or MQTT client service, and the IoT devices publish or subscribe to relevant topics. For example, a temperature sensor can publish readings to a topic, while a fan subscribes and turns on when the value crosses a threshold. Similarly, a motion sensor can trigger a smart light or alarm. This demonstrates event-driven automation, which is one of the core differences between IoT systems and traditional user-driven networking [5], [7]. It also provides clear material for the required 1-2 minute demonstration video because the screen recording can show the topology, device configuration, message flow, and automated reaction.

Suggested figure placement in this section: insert a real screenshot of the full Cisco Packet Tracer topology after the first paragraph with the caption: **Figure 1. Smart home IoT topology in Cisco Packet Tracer showing the home gateway, laptop acting as MQTT broker, and connected smart devices.**

Suggested second figure placement: after the second paragraph, insert a real screenshot of the device interaction or MQTT message flow with the caption: **Figure 2. MQTT-based smart home automation in action, showing a sensor event being published and a subscribed appliance responding.**

Challenges, networking impact, and conclusion

Although IoT creates major opportunities, it also creates significant technical and security challenges. Many IoT devices are resource-constrained and may have weak default configurations, limited update mechanisms, or insufficient security controls [3], [4]. This increases the risk of unauthorised access, privacy loss, poor authentication, and misuse of connected devices. In addition, device scale creates challenges in addressing, monitoring, interoperability, and lifecycle management.

IoT is therefore changing the computer networking landscape in important ways. Networks must now support many more endpoints, more telemetry traffic, more event-driven communication, and stronger requirements for segmentation, authentication, and device management [3], [4]. In this sense, IoT is not just another application of networking. It is a trend that forces the current networking paradigm to become lighter, more scalable, more automated, and more security-aware. Overall, the smart home implementation and MQTT analysis show that IoT combines communication, sensing, and automation in a way that significantly expands the role of computer networks. MQTT is especially important because

its lightweight broker-based publish/subscribe model matches the practical needs of IoT devices and smart home systems.

Figure and flowchart guidance

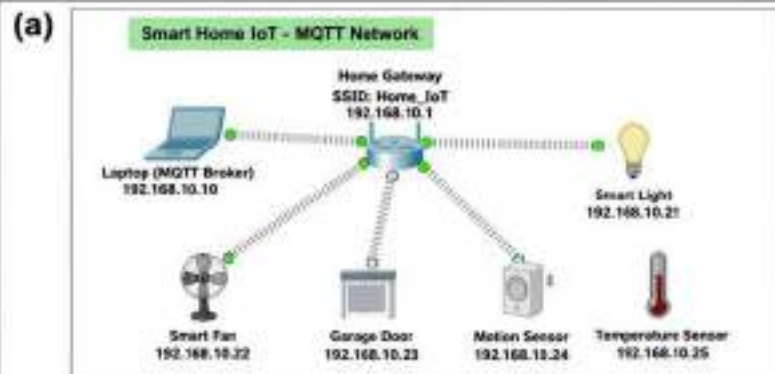
The report can be strengthened further by adding one simple conceptual flowchart created neatly in Word, PowerPoint, or another diagram tool. A suitable caption is: **Figure 3. MQTT publish/subscribe communication flow in a smart home, showing a sensor publishing data to a broker and subscribed appliances receiving the message.**

A good text prompt for generating a clean concept visual is: **Create a professional academic flowchart of MQTT communication in a smart home: motion sensor publishes event to MQTT broker, broker forwards message to smart light and smart alarm subscribers, white background, clean arrows, labelled components, simple university report style.**

References

- [1] National Institute of Standards and Technology, “NIST Cybersecurity for IoT Program,” CSRC, 2019. [Online]. Available: <https://csrc.nist.gov/projects/nist-cybersecurity-iot-program>. Accessed: Apr. 12, 2026.
- [2] OASIS Open, “MQTT Version 5.0,” 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Accessed: Apr. 12, 2026.
- [3] McDermott Will & Schulte, “NIST Guidance on Internet of Things (IoT),” 2021. [Online]. Available: <https://www.mwe.com/insights/nist-guidance-on-internet-of-things/>. Accessed: Apr. 12, 2026.
- [4] National Institute of Standards and Technology, “NIST Cybersecurity for IoT Program,” 2019. [Online]. Available: <https://csrc.nist.gov/projects/nist-cybersecurity-iot-program>. Accessed: Apr. 12, 2026.
- [5] “Additional Resource for 7.2D Task,” tutor-provided course resource, 2026.
- [6] OASIS Open, “Message Queuing Telemetry Transport (MQTT) TC,” [Online]. Available: https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=mqtt. Accessed: Apr. 12, 2026.
- [7] SIT202 – Computer Networks and Communication, “Task 7.2D: IoT and Networks,” task sheet, 2026.

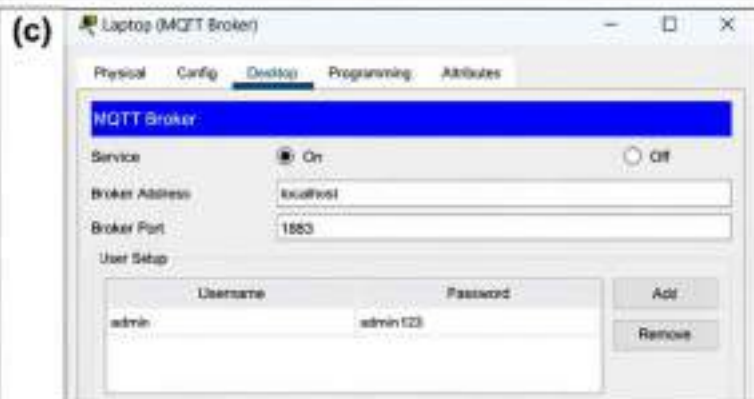
Figure 1. Cisco Packet Tracer Smart Home IoT Implementation and MQTT Demonstration Collage



(a) Overall smart home IoT topology implemented in Cisco Packet Tracer. This screenshot presents the full network layout used for the IoT implementation, including the home gateway, laptop, and connected smart devices.



(b) Home gateway configuration for wireless IoT connectivity. This screenshot shows the gateway settings used to connect all smart devices to the same local network.



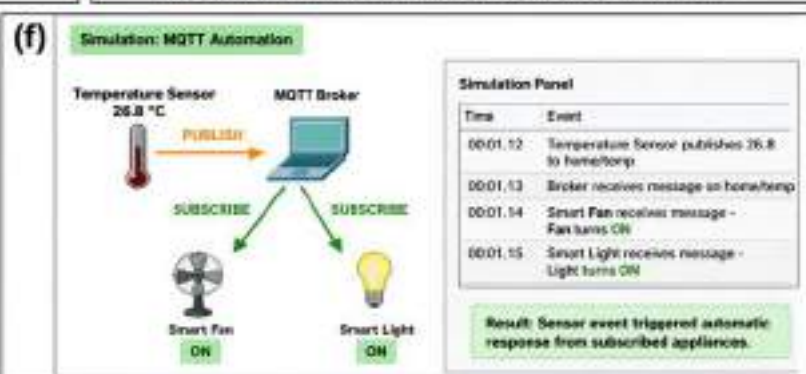
(c) Laptop configured as the MQTT broker in the smart home environment. This screenshot highlights the central broker device that receives published messages and forwards them to subscribers.



(d) IoT device connection and network association in Cisco Packet Tracer. This screenshot confirms that the smart devices are connected to the same wireless environment and are ready for IoT communication.



(e) MQTT publish/subscribe configuration using broker, topic, and client roles. This screenshot demonstrates the MQTT communication model in which publishers send data to a broker and subscribers receive updates through topics.



(f) MQTT-driven smart home automation in action during simulation. This screenshot captures the practical result of the implementation, showing how an event from one IoT device triggers an automated response in another.

Figure note: The collage presents visual evidence of the smart home IoT implementation completed in Cisco Packet Tracer, including the topology, gateway setup, MQTT broker configuration, device connectivity, publish/subscribe communication, and automation behaviour observed during testing.

16 Task 9.1P

Pass task

Outcome	Weight
Layered encapsulation	◆◆◆◆◆

In this task, I developed a solid understanding of the data-link layer through module exercises, Packet Tracer investigations, role-play activities, MAC table analysis, and a 100% self-assessment result. For ULO1, I explained how layer abstraction supports node-to-node delivery through framing, addressing, and local forwarding, which is central to this module. For ULO2, I did not design or develop a full network application, but the Packet Tracer-based investigations and protocol observations gave a limited connection to interacting with networking behaviour at a practical level. For ULO3, I used industry-relevant networking tools such as Cisco Packet Tracer to analyse ARP behaviour, switch learning, MAC tables, and local versus cross-subnet communication, making this outcome clearly relevant. For ULO4, I reflected on layer behaviour and analysed ARP spoofing, man-in-the-middle risk, and defences such as dynamic ARP inspection and VLAN segmentation, so the task is also connected to security and scalability considerations.

Outcome	Weight
Interaction with Networks	◆◆◆◆◆

In this task, I developed a solid understanding of the data-link layer through module exercises, Packet Tracer investigations, role-play activities, MAC table analysis, and a 100% self-assessment result. For ULO1, I explained how layer abstraction supports node-to-node delivery through framing, addressing, and local forwarding, which is central to this module. For ULO2, I did not design or develop a full network application, but the Packet Tracer-based investigations and protocol observations gave a limited connection to interacting with networking behaviour at a practical level. For ULO3, I used industry-relevant networking tools such as Cisco Packet Tracer to analyse ARP behaviour, switch learning, MAC tables, and local versus cross-subnet communication, making this outcome clearly relevant. For ULO4, I reflected on layer behaviour and analysed ARP spoofing, man-in-the-middle risk, and defences such as dynamic ARP inspection and VLAN segmentation, so the task is also connected to security and scalability considerations.

Outcome	Weight
Networking Tools	◆◆◆◆◆

In this task, I developed a solid understanding of the data-link layer through module exercises, Packet Tracer investigations, role-play activities, MAC table analysis, and a 100% self-assessment result. For ULO1, I explained how layer abstraction supports node-to-node delivery through framing, addressing, and local forwarding, which is central to this module. For ULO2, I did not design or develop a full network application, but the Packet Tracer-based investigations and protocol observations gave a limited connection to interacting with networking behaviour at a practical level. For ULO3, I used industry-relevant networking tools such as Cisco Packet Tracer to analyse ARP behaviour, switch learning, MAC tables, and local versus cross-subnet communication, making this outcome clearly relevant. For ULO4, I reflected on layer behaviour and analysed ARP spoofing, man-in-the-middle risk, and defences such as dynamic ARP inspection and VLAN segmentation, so the task is also connected to security and scalability considerations.

Outcome	Weight
Analysing real network issues	◆◆◆◆◆

In this task, I developed a solid understanding of the data-link layer through module exercises, Packet Tracer investigations, role-play activities, MAC table analysis, and a 100% self-assessment result. For ULO1, I explained how layer abstraction supports node-to-node delivery through framing, addressing, and local forwarding, which is central to this module. For ULO2, I did not design or develop a full network application, but the Packet Tracer-based investigations and protocol observations gave

a limited connection to interacting with networking behaviour at a practical level. For ULO3, I used industry-relevant networking tools such as Cisco Packet Tracer to analyse ARP behaviour, switch learning, MAC tables, and local versus cross-subnet communication, making this outcome clearly relevant. For ULO4, I reflected on layer behaviour and analysed ARP spoofing, man-in-the-middle risk, and defences such as dynamic ARP inspection and VLAN segmentation, so the task is also connected to security and scalability considerations.

Date	Author	Comment
2026/04/19 04:44	Jahan Garg	Working On It
2026/04/19 11:59	Jahan Garg	Ready for Feedback
2026/04/19 12:06	Jahan Garg	pdf document
2026/04/19 13:18	Jahan Garg	pdf document
2026/04/19 13:18	Jahan Garg	pdf document
2026/04/19 13:19	Jahan Garg	Mam, I have attached the PDF file of my SIT202 Networks and Communication 9.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/04/24 05:25	Amanpreet Kaur	Fix and Resubmit
2026/04/24 05:26	Amanpreet Kaur	Dear Jahan, you are required to add some more screenshot of message format for the ARP communication
2026/04/24 05:26	Amanpreet Kaur	add quiz screenshot for the same
2026/04/24 05:26	Amanpreet Kaur	Fix and Resubmit
2026/04/26 09:06	Jahan Garg	Ready for Feedback
2026/04/26 09:06	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 9.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/05/05 10:11	Amanpreet Kaur	Complete
2026/05/05 10:11	Amanpreet Kaur	Complete
2026/05/05 10:13	Amanpreet Kaur	What happens if the destination MAC address is not found in the ARP table?

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 9.1P

Submitted By:

Jahan GARG

2410994805

2026/04/26 09:06

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Interaction with Networks	◆◆◆◆◆
Networking Tools	◆◆◆◆◆
Analysing real network issues	◆◆◆◆◆

In this task, I developed a solid understanding of the data-link layer through module exercises, Packet Tracer investigations, role-play activities, MAC table analysis, and a 100% self-assessment result. For ULO1, I explained how layer abstraction supports node-to-node delivery through framing, addressing, and local forwarding, which is central to this module. For ULO2, I did not design or develop a full network application, but the Packet Tracer-based investigations and protocol observations gave a limited connection to interacting with networking behaviour at a practical level. For ULO3, I used industry-relevant networking tools such as Cisco Packet Tracer to analyse ARP behaviour, switch learning, MAC tables, and local versus cross-subnet communication, making this outcome clearly relevant. For ULO4, I reflected on layer behaviour and analysed ARP spoofing, man-in-the-middle risk, and defences such as dynamic ARP inspection and VLAN segmentation, so the task is also connected to security and scalability considerations.

April 26, 2026



Module 6: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around the main functions of the data-link layer: providing node-to-node delivery over individual links, detecting and correcting errors, coordinating shared-medium access, and addressing devices within a LAN using MAC addresses. The module first explained that the link layer sits below the network layer but above the physical layer and is responsible for encapsulating network-layer datagrams into link-layer frames, adding headers and trailers, and transferring each frame from one node to a physically adjacent node over wired or wireless media such as Ethernet and Wi-Fi. It then introduced error detection and correction mechanisms, including parity checks, Internet checksums, and cyclic redundancy checks (CRC), highlighting that CRCs are widely used on links like Ethernet because they provide strong protection against burst errors.

A major part of the module focused on medium access control (MAC) protocols, showing how multiple nodes share a common broadcast channel using three families of protocols: channel partitioning (TDMA and FDMA), random access (Aloha, CSMA, CSMA/CD and CSMA/CA), and “taking turns” protocols such as polling and token passing, each with different trade-offs between efficiency, fairness and complexity. The module then moved to LANs and link-layer addressing, explaining how 48-bit MAC addresses identify interfaces on a LAN, how ARP resolves IP addresses to MAC addresses, how Ethernet frames encapsulate IP datagrams along with preamble, type and CRC fields, and how switches and VLANs use MAC address tables to forward frames selectively and segment broadcast domains.

B. How This Is Useful

Understanding data-link-layer services is useful because every higher-layer protocol ultimately depends on reliable delivery of frames over individual links, and many practical troubleshooting tasks involve problems at this level rather than in the application code. Knowing how error detection schemes such as CRC work helps me reason about what kinds of corruption a link can detect, why Ethernet drops bad frames instead of trying to repair them, and where additional reliability is provided by higher layers; understanding MAC protocols helps me explain why wireless networks slow down when many users compete for the medium and why Ethernet rarely experiences collisions in modern switched LANs.

The addressing and LAN concepts are directly applicable when configuring and debugging real networks: MAC addresses and ARP entries appear in every Wireshark trace, and recognising the structure of an Ethernet frame or an ARP

Request makes it much easier to interpret what is going wrong when hosts cannot reach each other. Finally, the discussion of switches, MAC tables and VLANs is highly relevant for campus and enterprise environments, because it underpins how traffic is isolated, how broadcast storms are contained, and how security features such as port security or dynamic ARP inspection can be deployed to defend against attacks like MAC flooding and ARP spoofing.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned in this module is how much intelligence is already present in the link layer beneath IP: instead of being just a simple pipe, it provides framing, error detection, flow control, medium access control and local addressing, all of which must work correctly for higher-layer protocols to function. Before this module, I tended to think mainly in terms of IP addresses and routing, but now I can see how mechanisms such as CRC, CSMA/CA, MAC learning in switches and ARP resolution all interact to determine whether a frame even arrives at the correct next hop, which has made my understanding of end-to-end communication much more concrete.

B. How This Relates to What I Already Know

This module connects directly with what I learned earlier about the network layer data plane and control plane: previously, I studied how routers forward IP datagrams using forwarding tables and routing algorithms, but here I saw how those datagrams are actually carried over individual Ethernet or Wi-Fi links as link-layer frames. It also builds on my earlier experience with transport-layer checksums from the TCP/UDP module by showing how CRC offers stronger protection at the link layer, and it extends my understanding of LANs from earlier practicals by explaining what happens inside a switch's MAC address table when I see frames flowing through Packet Tracer and Wireshark.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because many real faults in networks arise from link-layer problems—such as duplex mismatches, collisions on shared media, ARP misconfigurations, or MAC table issues—rather than from high-level protocol bugs, and we need to be able to recognise those patterns. By understanding error detection, MAC protocols, LAN addressing, ARP and switching, I am better prepared to interpret packet traces, reason about performance problems on congested Wi-Fi or wired segments, and appreciate both the strengths and security limitations of technologies like ARP and Ethernet, which is essential for later units on network security, ethical hacking and system design.

References [1], [2], [3], [4], [5]

- [1] "6.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23501/View>
- [2] "6.2 Data-link layer services - error detection and correction - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23502/View>
- [3] "6.3 Data-link layer services - multiple access control protocol - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23503/View>
- [4] "6.4 LAN and addressing - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23504/View>
- [5] "Module-6-The-Data-Link-Layer - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23505/View>

Module 6: Evidence of Learning

This section presents evidence of my learning for **Module 6: Data-Link Layer**, with a focus on link-layer services, error detection and correction techniques, medium access control protocols, MAC addressing, ARP operation, Ethernet frame communication, switching, and VLAN concepts. It includes module exercises, active class activities, role-play outcomes, Cisco Packet Tracer investigations of ARP and MAC address tables, and supporting screenshots to demonstrate both conceptual understanding and practical network analysis skills. Together, these tasks demonstrate my understanding of how the data-link layer supports reliable frame delivery between physically adjacent nodes, how shared media is managed through access control mechanisms, how IP addresses are resolved to MAC addresses within local networks, and how switches learn and use MAC address information to forward traffic efficiently within a LAN. Short progress notes are included throughout to show how each task contributed to my developing understanding of link-layer communication, protocol behaviour, and the practical operation of local area networks.

Evidence: Module Exercises

Exercise 1: CRC vs checksum

Question: What are the differences between CRC and checksum techniques?

Ans. Checksum and CRC are both error-detection mechanisms, but they use different mathematics, provide different levels of protection, and are used at different layers. A checksum (as used in IPv4, TCP and UDP) sums fixed-size words of the data and stores the (possibly one's-complement) sum in a header field; at the receiver, the same sum is recomputed and compared, so it is relatively simple and fast, but can miss some error patterns, especially when bit flips cancel each other out. [1], [2], [3]

By contrast, a cyclic redundancy check (CRC) treats the data bits as coefficients of a binary polynomial and divides this polynomial by a generator polynomial; the remainder is appended as the CRC field and rechecked at the receiver, giving much stronger detection of burst errors and multi-bit errors at the cost of slightly higher computational complexity and extra CRC bits. In practice, simple checksums are often used in network and transport layer headers where errors are already rare and performance matters, while CRCs are widely used on links such as Ethernet frames, storage and wireless links where robust detection of physical-layer corruption is critical. [4]

Exercise 2: MAC protocol types

Question: What are the advantages and disadvantages of the three types of MAC protocols that we have discussed?

Ans. Channel partitioning protocols such as TDMA and FDMA divide the channel into fixed time slots or frequency bands and assign each node a dedicated share, which gives predictable, collision-free access and high efficiency at high load but wastes capacity when only a few nodes are active because idle slots or bands cannot be used by others. Random-access protocols such as ALOHA and CSMA/CA let any node transmit at full rate when it has a frame, making them simple and efficient at low load, but collisions (or collision-avoidance backoff) reduce throughput and increase delay as load increases because many attempts must be retransmitted. [3], [5]

“Taking turns” protocols such as polling and token passing combine aspects of both: they avoid collisions by giving each node a turn, and nodes with more data can use longer turns, so the channel is used efficiently under moderate to high load, but the need for a master (polling) or a circulating token (token passing) introduces overhead, latency while waiting for the next turn, and single points of failure if the master or token is lost. [6], [7]

Exercise 3 Reflection questions in Section 6.4

Question: Summarise the main points you learned in this section.

Ans. In this section, I learned that local area networks connect physically adjacent nodes and rely on link-layer addressing, primarily 48-bit MAC addresses, to deliver Ethernet frames between interfaces on the same subnet. I also learned how ARP bridges the gap between IP addresses and MAC addresses by resolving the correct link-layer destination, how Ethernet frames encapsulate IP datagrams inside a preamble, source and destination MAC addresses, type field, payload and CRC trailer, and how switches and VLANs scale and segment LANs to improve efficiency, privacy and manageability. [3]

Question: How are these useful?

Ans. These ideas are useful because every practical network I will work with is built from LAN segments, and understanding MAC addresses, ARP, and Ethernet framing is essential for interpreting packet captures, diagnosing connectivity problems, and designing correct IP configurations. Knowing how switches use MAC address tables to forward frames and how VLANs isolate traffic helps me reason about broadcast domains, security boundaries, and performance issues in real campus and enterprise networks. [3], [4], [8]

Evidence: Active Classes Activities

Activities for Active Class 9: Data-link Layer

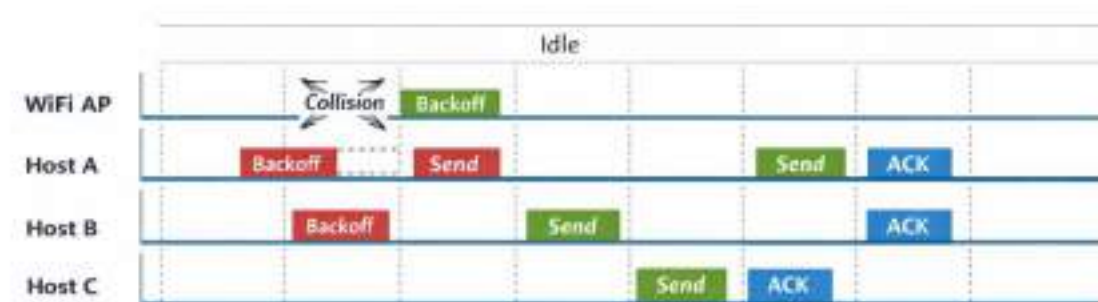
Activity 1: Investigating Multiple Access Control Protocols

Question 1: WLAN role play and timing diagram

Ans. In our group role play, we modelled a Wi-Fi LAN with one member acting as the wireless access point (AP) and three members acting as wireless hosts that all wanted to send packets at roughly the same time. We designed a simple random-access protocol inspired by CSMA/CA: before “sending a packet” each host first listened for the AP to say that the channel was idle; only when the channel was idle could a host announce “I’m sending a packet”, and if two hosts spoke at the same time we treated that as a collision, both hosts backed off for a random number of time slots, and then retried. The AP acknowledged each successful transmission so that hosts knew when they could attempt again, and the timing diagram we drew showed alternating idle periods, single-host transmissions, occasional collisions when two hosts started together, and random backoff intervals that reduced the chance of repeated collisions.[3], [5], [9]

Question 1 – WLAN role play and timing diagram

In our group role play, we modelled a Wi-Fi LAN with one member acting as the wireless access point (AP) and three members acting as wireless hosts that all wanted to send packets at roughly the same time. The diagram below illustrates our experiment.



Question 2 – MAC protocol used in Wi-Fi

The medium access control protocol used in IEEE 802.11 Wi-Fi is Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA).

Figure 1: Role-play timing diagram for Wi-Fi-style random access MAC protocol

Question 2: MAC protocol used in Wi-Fi

Ans. The medium access control protocol used in IEEE 802.11 Wi-Fi is Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA), implemented in the Distributed Coordination Function (DCF). In CSMA/CA, a station senses the channel, waits for an inter-frame space if the medium is idle, chooses a random backoff counter in a contention window, and decrements that counter while the channel remains idle; when the counter reaches zero, it transmits and expects a link-layer ACK, and if a collision is inferred from a missing ACK it increases its contention window and retries, which allows multiple stations to share the wireless medium with fewer collisions even though collisions cannot be detected directly in the same way as Ethernet. [3], [5], [9]

Activity 2: ARP

We implemented the two-LAN topologies in Cisco Packet Tracer with Laptop1 and PC2 in LAN1 (192.168.1.0/24) connected via Switch0 to Router1, and Laptop0, PC0 and PC1 in LAN2 (192.168.2.0/24) connected via Switch1 to Router3; Router1 and Router3 were connected over a 10.0.0.0/8 transit link so that packets could flow between LAN1 and LAN2.

Question 1: Information needed by Laptop1 to reach PC2 in LAN1

Ans. To send a packet from Laptop1 to PC2 on the same LAN, Laptop1 needs:

- Its own IP address and subnet mask, to identify that PC2's IP is in the same subnet.
- The destination IP address of PC2.
- The MAC address of PC2's network interface, so that it can build an Ethernet frame whose destination MAC is PC2, even though the IP destination remains PC2's IP address.

[3]

Question 2: Protocol and steps to obtain that information

Ans. The protocol used to discover the destination MAC address from an IP address is the Address Resolution Protocol (ARP). When Laptop1 wants to send to PC2 and does not have a cached entry, it broadcasts an ARP Request frame on LAN1 that says "Who has IP 192.168.1.x? Tell 192.168.1.y" including its own IP and MAC address; all hosts on the subnet receive the request, and PC2 recognises that the target IP matches its own and replies with an ARP Reply, sent unicast back to Laptop1, that includes PC2's MAC address. Laptop1 then updates its ARP table with the mapping from PC2's IP to its MAC address and can encapsulate the IP datagram for PC2 in an Ethernet frame addressed directly to PC2's MAC. [3], [10]

Question 3: ARP and ICMP messages when pinging PC2 from Laptop1 (same LAN)

Ans. In simulation mode, when we pinged PC2 from Laptop1 for the first time, Laptop1 generated an ARP Request broadcast followed by an ICMP Echo Request once the ARP resolution was complete, and PC2 generated an ARP Reply and then an ICMP Echo Reply. The ARP Request had source IP equal to Laptop1's address, destination IP equal to PC2's address, source MAC equal to Laptop1's MAC, and destination MAC set to the broadcast address FF:FF:FF:FF:FF:FF, whereas the ARP Reply from PC2 used source IP and MAC equal to PC2's values and destination IP and MAC equal to Laptop1's values. The subsequent ICMP Echo Request/Reply packets were carried inside IP datagrams and Ethernet frames with unicast source and destination MAC addresses corresponding to Laptop1 and PC2. [3], [10]

Question 4–5: ARP tables after local communication

Ans. After this first exchange, Laptop1's ARP table contained entries mapping PC2's IP address and the default gateway's IP to their respective MAC addresses, while PC2's ARP table contained an entry mapping Laptop1's IP to Laptop1's MAC, and Router1's ARP table contained mappings for the interfaces and any hosts it had forwarded for. Inspecting "arp -a" on Laptop1 and PC2 and "show arp" on the routers confirmed that each device only learns MAC addresses for neighbours it has recently communicated with, so the ARP tables reflect local interaction on each subnet. [3], [10]

Question 6–8: ARP across subnets and comparison of ARP tables

Ans. When we repeated the experiment by pinging PC0 in LAN2 from Laptop1, simulation mode showed that Laptop1 used ARP only to resolve the MAC address of Router1 (its default gateway) on LAN1, then sent the ICMP Echo Request to Router1, which in turn used ARP on the 10.0.0.0/8 link to reach Router3, and Router3 used ARP on LAN2 to discover PC0's MAC address before forwarding the packet. After these cross-LAN pings, the ARP tables on Router1, Router3, PC0, PC1 and Laptop0 contained additional entries for each directly connected neighbour that had participated in forwarding, whereas hosts still had no ARP entries for remote hosts beyond their own subnet because ARP never resolves MAC addresses across routers. Comparing the ARP tables before and after these pings showed how new entries were added for gateway interfaces and directly attached peers, but entries for distant devices were always reached by sending frames to the local router's MAC address rather than learning their MAC addresses directly. [3], [10]

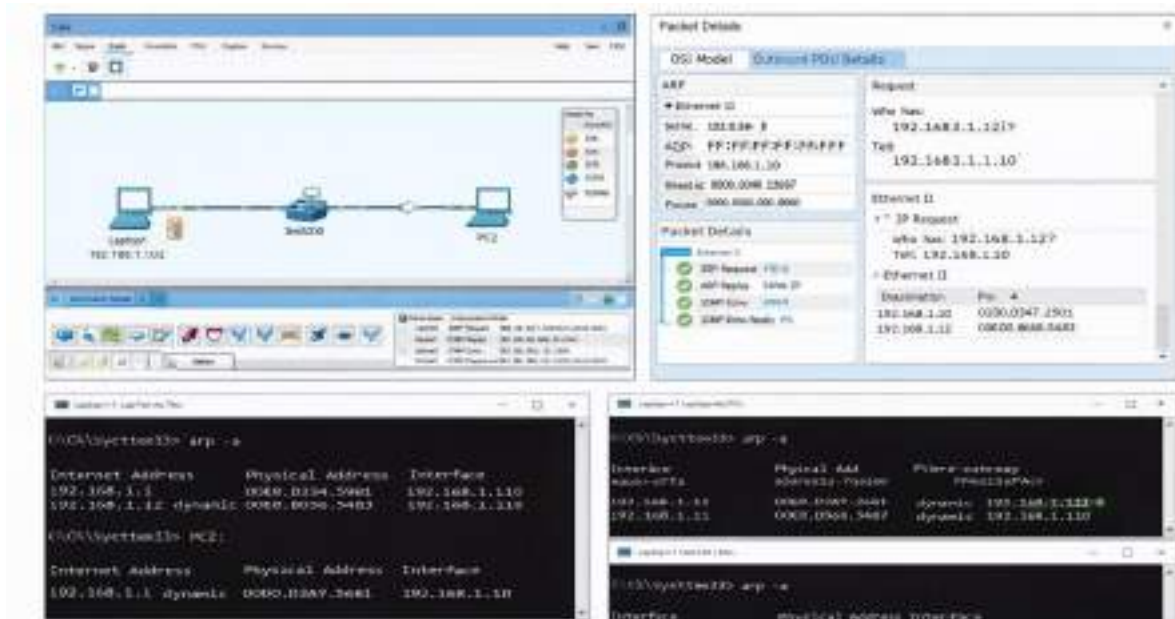


Figure 2: ARP request/reply and ICMP echo exchange when Laptop1 pings PC2 within LAN1

Screenshots from Cisco Packet Tracer in Simulation mode, showing Laptop1 broadcasting an ARP request to learn the MAC of PC2, PC2's unicast ARP reply back to Laptop1, and their updated ARP tables after a successful ICMP echo request and reply.

Activity 3: MAC address table and switch learning

Question: How do switches learn the MAC addresses of their connected devices?

Ans. In this activity, we built a single-LAN topology with Router4 acting as the default gateway, a 2960 switch in the centre, and a server plus several PCs and laptops all in the 192.168.2.0/24 network. After configuring static IP addresses, we inspected each end device's interface configuration to record its unique 48-bit MAC address and used `arp -a` on PCs and laptops to confirm that their ARP tables initially contained only limited mappings (often just the default gateway once they had communicated). [3]

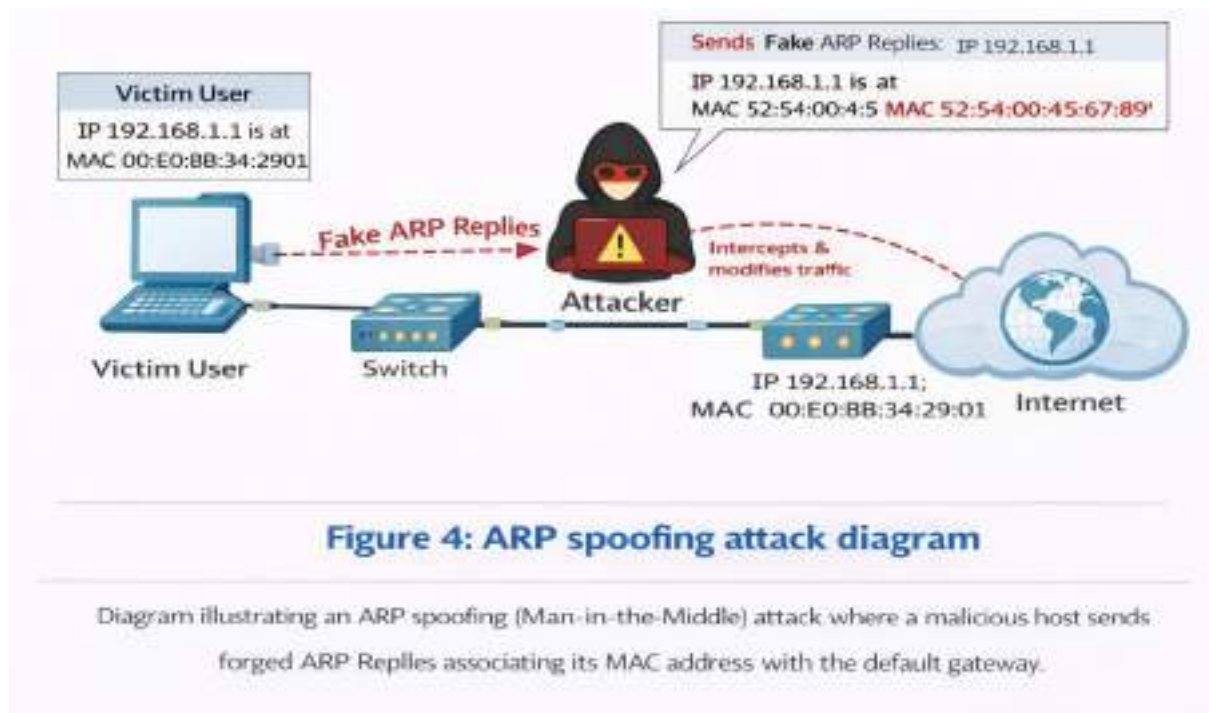
When we first checked the switch's MAC address table with `show mac-address-table`, it was either empty or contained only entries for any management or already-active interfaces, because the switch learns MAC addresses dynamically by observing the source MAC address of frames that arrive on each port. After we started pinging—for example, from PC1 and PC2 to Laptop0 and Laptop1—the MAC address table filled with entries mapping each seen source MAC address to the specific switch port where that device was connected; subsequent frames destined to those MAC addresses could then be forwarded only out of the correct port instead of being flooded, which improves both privacy and efficiency on the LAN. [8]

Using the graphical "MAC table" view while pinging Laptop2 from PC0 showed this learning process in real time: as soon as a device sent or replied to traffic, its MAC and associated port appeared in the table, and the switch stopped broadcasting

Above and Beyond: Security issues associated with ARP (200–300 words)

Ans. In principle, ARP is simple: it maps IP addresses to MAC addresses within a LAN by broadcasting ARP Requests and caching ARP Replies, but because ARP has no built-in authentication or integrity checks, it is vulnerable to several attacks collectively known as ARP spoofing or ARP poisoning. In an ARP spoofing attack, a malicious host sends forged ARP Replies that associate the attacker's MAC address with the IP address of a legitimate device, such as the default gateway, DNS server, or another host; once victims accept these fake bindings into their ARP caches, their traffic is silently redirected through the attacker. [3], [10]

This enables classic man-in-the-middle (MITM) attacks, where the attacker intercepts, reads, and possibly modifies packets before forwarding them on so that neither endpoint notices, which can be used to harvest credentials, inject malicious content, or downgrade encrypted connections. ARP spoofing can also be used for denial-of-service by redirecting traffic to a non-existent MAC address or simply dropping packets, effectively black-holing the victim's communications. Because ARP operates at layer 2 and relies on broadcasts, these attacks are confined to the local network segment but are difficult to detect without monitoring ARP tables or using intrusion detection tools that look for inconsistent IP-to-MAC mappings. Defences include static ARP entries for critical infrastructure, dynamic ARP inspection on switches, VLAN segmentation to limit the scope of an attacker, and higher-layer protections such as secure protocols that keep data confidential even if the path is compromised. [12], [13]



Summary of Learning Evidence

Through the Module 6 exercises and Active Classes 9 activities, I demonstrated a solid understanding of data-link layer functions, including framing, error detection and correction, medium access control, MAC addressing, ARP, Ethernet communication, and switch learning behaviour.

- **Module Exercises:** Practised key concepts such as CRC, checksum, parity, MAC protocols, LAN addressing, Ethernet frame structure, and the role of switches and VLANs in local network communication.
- **Active Class 9:** Explored multiple access control through role play, analysed ARP request/reply behaviour and ARP table updates in Cisco Packet Tracer, and examined how switches learn and store MAC addresses in their tables.
- **Practical Analysis:** Observed how local and cross-subnet communication differ at the link layer, how Ethernet frames are constructed and forwarded, and how MAC address tables improve forwarding efficiency and privacy within a LAN.

This evidence portfolio connects the theory from Module 6 to practical network analysis through discussion, simulation, protocol tracing, and configuration tasks, demonstrating active learning and a strong understanding of data-link layer concepts.

References

- [1] "Difference between Checksum and CRC - GeeksforGeeks." Accessed: Apr. 19, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/difference-between-checksum-and-crc/>
- [2] "What is the difference between Checksum and CRC?" Accessed: Apr. 19, 2026. [Online]. Available: <https://www.tutorialspoint.com/article/what-is-the-difference-between-checksum-and-crc>
- [3] "Module-6-The-Data-Link-Layer - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23505/View>
- [4] "Data Link Layer in OSI Model - Scaler Topics." Accessed: Apr. 19, 2026. [Online]. Available: <https://www.scaler.com/topics/computer-network/data-link-layer/>
- [5] "Wireless Collision Avoidance – CSMA/CA Though DCF/EDCF – How I WI-FI." Accessed: Apr. 19, 2026. [Online]. Available: <https://howiwifi.com/2020/06/30/wireless-contention-mechanisms/>
- [6] "Multiple Access Links and Protocols".
- [7] K. Webb, "CS 43: Computer Networks Media Access," 2022.
- [8] "What is a MAC flooding attack and why does it matter?" Accessed: Apr. 19, 2026. [Online]. Available: <https://www.expressvpn.com/blog/mac-flooding/>
- [9] L. Sanabria-Russo, J. Barcelo, B. Bellalta, and F. Gringoli, "A High Efficiency MAC Protocol for WLANs: Providing Fairness in Dense Scenarios".
- [10] "What is ARP Spoofing? - ARP poisoning Attack - GeeksforGeeks." Accessed: Apr. 19, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/what-is-arp-spoofing-arp-poisoning-attack/>
- [11] "CAM Table Attack Explained: MAC Flooding & Security | Edwhere Education." Accessed: Apr. 19, 2026. [Online]. Available: <https://learn.edwhere.com/blog/cam-table-attack-mac-flooding-explained>
- [12] "ARP Poisoning (ARP Spoofing) | Local Network Man-in-the-Middle & Mitigations." Accessed: Apr. 19, 2026. [Online]. Available: <https://www.radware.com/security/ddos-knowledge-center/ddospedia/arp-poisoning/>
- [13] "What is ARP Spoofing? Risks, Detection, and Prevention." Accessed: Apr. 19, 2026. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/threat-intelligence/arp-spoofing/>

[Quiz List](#) > [Submissions](#)

Quiz Submissions – Module 6: Data Link Layer

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %

2004年

17 Task 5.3D

distinction task

Outcome	Weight
Layered encapsulation	◆◆◆◆◇

In this task, I demonstrated deeper understanding across six core modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation, comparing RIP, OSPF, and BGP routing protocols across their algorithmic foundations and operational scopes, analysing DHCP traffic at packet level in Wireshark, examining ARP security vulnerabilities and defences, and explaining physical layer transmission media and Ethernet cable types. For ULO1, I explained how each layer of the protocol stack contributes to end-to-end data flow across all six modules, tracing how TCP establishes transport connectivity, TLS secures the application session, IP forwarding tables route datagrams across subnets, DHCP delivers layered configuration via UDP broadcasts, ARP resolves addresses at the link layer, and physical media carry the resulting signals. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, directly applying socket-level transport layer concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, apply the SSLKEYLOGFILE decryption method, and analyse DHCP DORA exchanges at packet level, and used Cisco Packet Tracer to configure and simulate routing across multi-subnet topologies, validate forwarding behaviour through ping and simulation mode, and implement static and dynamic routing scenarios. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, IP data plane forwarding, routing protocol design, ARP vulnerabilities, and physical layer transmission constraints are constructed, connecting each to real-world considerations of security, reliability, and network scalability across the full protocol stack.

Outcome	Weight
Interaction with Networks	◆◆◆◆◇

In this task, I demonstrated deeper understanding across six core modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation, comparing RIP, OSPF, and BGP routing protocols across their algorithmic foundations and operational scopes, analysing DHCP traffic at packet level in Wireshark, examining ARP security vulnerabilities and defences, and explaining physical layer transmission media and Ethernet cable types. For ULO1, I explained how each layer of the protocol stack contributes to end-to-end data flow across all six modules, tracing how TCP establishes transport connectivity, TLS secures the application session, IP forwarding tables route datagrams across subnets, DHCP delivers layered configuration via UDP broadcasts, ARP resolves addresses at the link layer, and physical media carry the resulting signals. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, directly applying socket-level transport layer concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, apply the SSLKEYLOGFILE decryption method, and analyse DHCP DORA exchanges at packet level, and used Cisco Packet Tracer to configure and simulate routing across multi-subnet topologies, validate forwarding behaviour through ping and simulation mode, and implement static and dynamic routing scenarios. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, IP data plane forwarding, routing protocol design, ARP vulnerabilities, and physical layer

transmission constraints are constructed, connecting each to real-world considerations of security, reliability, and network scalability across the full protocol stack.

Outcome	Weight
Networking Tools	◆◆◆◆◇

In this task, I demonstrated deeper understanding across six core modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation, comparing RIP, OSPF, and BGP routing protocols across their algorithmic foundations and operational scopes, analysing DHCP traffic at packet level in Wireshark, examining ARP security vulnerabilities and defences, and explaining physical layer transmission media and Ethernet cable types. For ULO1, I explained how each layer of the protocol stack contributes to end-to-end data flow across all six modules, tracing how TCP establishes transport connectivity, TLS secures the application session, IP forwarding tables route datagrams across subnets, DHCP delivers layered configuration via UDP broadcasts, ARP resolves addresses at the link layer, and physical media carry the resulting signals. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, directly applying socket-level transport layer concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, apply the SSLKEYLOGFILE decryption method, and analyse DHCP DORA exchanges at packet level, and used Cisco Packet Tracer to configure and simulate routing across multi-subnet topologies, validate forwarding behaviour through ping and simulation mode, and implement static and dynamic routing scenarios. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, IP data plane forwarding, routing protocol design, ARP vulnerabilities, and physical layer transmission constraints are constructed, connecting each to real-world considerations of security, reliability, and network scalability across the full protocol stack.

Outcome	Weight
Analysing real network issues	◆◆◆◆◇

In this task, I demonstrated deeper understanding across six core modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation, comparing RIP, OSPF, and BGP routing protocols across their algorithmic foundations and operational scopes, analysing DHCP traffic at packet level in Wireshark, examining ARP security vulnerabilities and defences, and explaining physical layer transmission media and Ethernet cable types. For ULO1, I explained how each layer of the protocol stack contributes to end-to-end data flow across all six modules, tracing how TCP establishes transport connectivity, TLS secures the application session, IP forwarding tables route datagrams across subnets, DHCP delivers layered configuration via UDP broadcasts, ARP resolves addresses at the link layer, and physical media carry the resulting signals. For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, directly applying socket-level transport layer concepts to build and verify a working networked application. For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, apply the SSLKEYLOGFILE decryption method, and analyse DHCP DORA exchanges at packet level, and used Cisco Packet Tracer to configure and simulate routing across multi-subnet topologies, validate forwarding behaviour through ping and simulation mode, and implement static and dynamic routing scenarios. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, IP data plane forwarding, routing protocol design, ARP vulnerabilities, and physical layer transmission constraints are constructed, connecting each to real-world considerations of security, reliability, and network scalability across the full protocol stack.

Date	Author	Comment
2026/04/19 13:23	Jahan Garg	Working On It
2026/04/23 11:59	Jahan Garg	Ready for Feedback
2026/05/06 10:19	Amanpreet Kaur	Dear jahan, Check task carefully n resubmit
2026/05/06 10:19	Amanpreet Kaur	Fix and Resubmit
2026/05/06 10:19	Amanpreet Kaur	This task is evaluated based on demonstration also
2026/05/11 18:02	Jahan Garg	Ready for Feedback
2026/05/11 18:02	Amanpreet Kaur	Time Exceeded
2026/05/12 11:59	Jahan Garg	Ready for Feedback
2026/05/12 11:59	Amanpreet Kaur	Time Exceeded
2026/05/12 12:00	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 5.3D task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch
2026/05/12 12:41	Jahan Garg	Ready for Feedback
2026/05/12 12:41	Amanpreet Kaur	Time Exceeded
2026/05/13 00:09	Jahan Garg	Ready for Feedback
2026/05/13 00:09	Amanpreet Kaur	Time Exceeded

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 5.3D

Submitted By:

Jahan GARG

2410994805

2026/05/13 00:09

Tutor:

Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◇
Interaction with Networks	◆◆◆◆◇
Networking Tools	◆◆◆◆◇
Analysing real network issues	◆◆◆◆◇

In this task, I demonstrated deeper understanding across six core modules by analysing TLS handshake behaviour and HTTPS encryption in Wireshark, explaining email protocols and their architecture, extending a UDP socket program into a multi-step conversational interaction in Python, comparing ARQ reliability protocols and TCP congestion control mechanisms, building a three-LAN Cisco Packet Tracer topology to verify inter-subnet forwarding through live simulation, comparing RIP, OSPF, and BGP routing protocols across their algorithmic foundations and operational scopes, analysing DHCP traffic at packet level in Wireshark, examining ARP security vulnerabilities and defences, and explaining physical layer transmission media and Ethernet cable types. For ULO1, I explained how each layer of the protocol stack contributes to end-to-end data flow across all six modules, tracing how TCP establishes transport connectivity, TLS secures the application session, IP forwarding tables route datagrams across subnets, DHCP delivers layered configuration via UDP broadcasts, ARP resolves addresses at the link layer, and physical media carry the resulting signals.

For ULO2, I designed and implemented a multi-step UDP client-server program in Python that extended the standard one-message exchange into a structured request-response interaction, directly applying socket-level transport layer concepts to build and verify a working networked application.

For ULO3, I used Wireshark to capture and interpret TLS handshake sequences, observe HTTPS encryption limits, apply the SSLKEYLOGFILE decryption method, and analyse DHCP DORA exchanges at packet level, and used Cisco Packet Tracer to configure and simulate routing across multi-subnet topologies, validate forwarding behaviour through ping and simulation mode, and implement static and dynamic routing scenarios. For ULO4, I critically reflected on how TLS, ARQ protocols, TCP congestion control, IP data plane forwarding, routing protocol design, ARP

vulnerabilities, and physical layer transmission constraints are constructed, connecting each to real-world considerations of security, reliability, and network scalability across the full protocol stack.

May 13, 2026



Module 2: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the content

I found it easiest to organise the summary around the three learning objectives for this module.

1. Understanding key application-layer concepts and required services

In this module, I learned that a network application is really a set of cooperating processes running on different end systems that exchange messages over the network. The application layer is responsible for preparing these messages so that lower layers (transport, network, link, physical) can move them across the Internet. We revisited the idea that routers and switches do not run user applications; only the end systems do.

I also explored how each application needs certain transport services such as reliability, timing, throughput and security, and how these requirements guide the choice between TCP and UDP. For example, web browsing and email prefer reliable, ordered delivery over TCP, while some real-time applications may accept occasional loss over UDP for lower delay.

2. Explaining the client–server model and associated processes

The module then focused on application architectures. I studied the traditional client–server architecture, where a server process runs on a fixed, well-known host and multiple clients initiate communication with it. Typical examples include web browsing, email access and online banking. In contrast, I looked at peer-to-peer (P2P) systems, where each peer can act as both client and server, sharing resources directly with other peers. We also touched on hybrid architectures that mix both ideas, such as video conferencing platforms that use servers for coordination while peers exchange media.

At the process level, I learned that communication is really between processes, not programs. A client process starts the conversation, while a server process waits to be contacted. Sockets act as the interface between an application process and the transport layer, so the socket is the “doorway” through which messages enter and leave the network stack. This helped me connect the abstract model to the concrete idea of socket programming.

3. Analysing and implementing application layer protocols (HTTP, HTTPS, DNS)

The largest part of the module was devoted to specific application-layer protocols.

- For HTTP, I studied the basic request–response pattern, common methods such as GET and POST, and the structure of HTTP messages (start line, headers and optional body). I compared non-persistent connections, where every object requires a new TCP connection, with persistent connections, where several requests share one connection. I also learned how cookies allow a stateless HTTP server to remember users, and how web caches (proxy servers) store frequently requested objects to reduce delay and server load.
- I then saw how HTTPS extends HTTP by adding TLS. The module gave me a simple view of the TLS handshake: the client and server authenticate, agree on an encryption method and derive keys before any HTTP data is sent. After that, HTTP messages are encrypted, which explains why Wireshark cannot read them without the keys.
- Finally, I learned about DNS as the Internet’s naming service. DNS translates hostnames into IP addresses using a distributed hierarchy of root servers, TLD servers and authoritative servers. The topics included DNS queries and replies, resource records, the use of UDP and port 53, and how caching reduces repeated lookups. I also saw how local DNS servers and the Time To Live (TTL) value control how long cached mappings remain valid.

4. Weekly activities and practical work

The active classes and tasks connected the theory with practice. In Week 2, the HTTP role-play helped me rehearse the exact sequence of events between multiple clients and a server: TCP connection setup, HTTP GET, HTTP 200 OK, and connection teardown. I then captured real HTTP traffic in Wireshark, filtered by “http”, and inspected the Ethernet, IP, TCP and HTTP headers to see the layered design in action. In Week 3, I repeated this approach for DNS by using nslookup and Wireshark to trace the steps from a typed URL to the resolved IP address for real websites.

B. How is this useful

1. The module gives me a clear end-to-end story of what happens when I type a URL or open an app, so I can reason about performance issues, failures and security risks instead of treating the network as a black box.

2. Understanding sockets, client/server roles and protocol behaviour will directly support future programming tasks where I need to build or debug networked applications, such as web services, chat tools or analytics systems.
3. The insight into HTTP, HTTPS and DNS is essential for cybersecurity: many attacks, misconfigurations and privacy problems appear at the application level

layer, so knowing how these protocols really work is the first step towards detecting and mitigating them.
4. The hands-on Wireshark and nslookup exercises gave me practical skills in inspecting real traffic, which is valuable for troubleshooting, monitoring and later units that involve network forensics and system administration.

2. REFLECTION ON THE CONTENT

A. Most important thing I learned

The most important thing I learned is the complete chain from entering a web address to seeing the page: DNS resolves the hostname, TCP establishes a connection, HTTP sends a request and receives a response, and optionally TLS secures the exchange. Earlier, I only knew this at a very high level. Now I can describe each step, the main header fields, and the typical message sequence, and I have actually seen these packets in Wireshark. That sequence ties together many separate topics into one coherent mental picture.

B. How this relates to what I already knew

Before this module, I was aware of the OSI/TCP-IP layers from earlier units, and I used web browsers, email and messaging apps every day. However, my understanding stopped at “the browser sends a request to the server”. This module connected that earlier knowledge with more concrete details: what exactly is inside a GET request, how cookies preserve state across stateless HTTP, why TCP or UDP is chosen for a given application, and how DNS caching explains the difference between first-time and repeated access to the same site. It turned everyday experiences into something I can now explain and reason about technically.

C. Why the course team wants us to learn this content

I think the course team wants us to master the application layer because it is the point where users, software and the network meet. If we understand only lower-level

routing or addressing without appreciating protocols like HTTP and DNS, we would struggle to design secure, responsive and reliable systems. This module builds the foundation for later topics such as web development, network security, cloud services and distributed systems. It also trains us to read protocol specifications, interpret packet traces and link theory with tools like Wireshark, which are core skills for any networking, cybersecurity or backend engineering role.

Overall, the Application Layer module has moved me from just “using” Internet applications to being able to analyse and explain how they work, which is exactly the kind of mindset I will need in the rest of this unit and in my degree.

Module 3: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I will organise this summary around Modules 3.1 to 3.7, linking each part to the activities and resources from the Transport Layer module and Active Classes 4 & 5.

1) Understanding key transport-layer concepts and required services (3.1, 3.2)

In this module, I learned that the network layer provides logical communication between hosts, while the transport layer extends this to process-to-process communication using sockets and port numbers. The household analogy clarified this perfectly: hosts are houses, processes are children, messages are letters in envelopes, the transport protocol is the parent who multiplexes and demultiplexes letters among siblings, and the network layer is the postal service moving mail between houses.

I studied multiplexing and demultiplexing in depth. At the sender, multiplexing gathers data from multiple sockets and adds transport headers, including port numbers. At the receiver, demultiplexing delivers each segment to the correct socket: UDP uses only the destination port number, while TCP uses the full four-tuple (source IP, source port, destination IP, destination port).

The module highlighted the two principal Internet transport protocols. TCP is connection-oriented and provides reliable in-order delivery, flow control, congestion control, and connection setup via a three-way handshake. UDP is connectionless, provides best effort unreliable delivery with minimal overhead, and requires no handshaking. TCP suits applications needing reliability, such as web browsing, email, and file transfer. UDP suits loss-tolerant but delay-sensitive applications such as streaming media, DNS, and VoIP.

2) Analysing and implementing UDP (3.3)

UDP is a "no frills" protocol with a simple header containing four 16-bit fields: source port, destination port, length, and checksum. I studied the Internet checksum algorithm, where the sender sums 16-bit words using one's complement arithmetic with wraparound and stores the complement as the checksum. This offers only weak error protection since some error patterns may not change the checksum.

In Active Class 4, we explored UDP through Wireshark and Python socket programming. For Wireshark analysis, I captured packets while running applications such as DNS, applied the "udp" filter, examined individual UDP segments to identify

all four header fields and their lengths, verified the Length field includes the 8-byte header, and observed different applications using different port numbers (DNS on port 53, ephemeral client ports varying).

In the programming activity, my partner and I built a UDP client-server chat application in Python. The server created a UDP socket with `socket(AF_INET, SOCK_DGRAM)`, bound it to a port using `bind()`, and used `recvfrom()` to receive messages. The server counted the number of characters, converted the message to uppercase, and replied with both the count and the uppercase string using `sendto()`. The client created its own UDP socket and used `sendto()` to send messages such as "Hello SIT202", then displayed the server's response. This demonstrated UDP's connectionless nature: no explicit connection setup, no guaranteed delivery, no built-in retransmission.

3) Analysing and implementing TCP (3.4)

TCP is a connection-oriented, reliable, full duplex, byte-stream protocol. The TCP segment header contains source and destination ports, sequence number (byte stream number of the first data byte), acknowledgement number (next expected byte), header length, control flags (SYN, ACK, FIN), receive window for flow control, and checksum.

Connection management uses a three-way handshake ($\text{SYN} \rightarrow \text{SYN ACK} \rightarrow \text{ACK}$) to establish connections and a four-way sequence ($\text{FIN} \rightarrow \text{ACK} \rightarrow \text{FIN} \rightarrow \text{ACK}$) to close them gracefully. In Active Class 5, I captured this handshake in Wireshark and analysed the segment order, flags, and sequence numbers.

TCP achieves reliability through byte numbering, cumulative acknowledgements, adaptive timeouts using exponential weighted moving averages ($\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$; $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$), and retransmission mechanisms including fast retransmit triggered by three duplicate ACKs.

4) TCP Flow control and congestion control (3.5, 3.6)

Flow control prevents the sender from overwhelming the receiver. The receiver advertises a receive window (`rwnd`) indicating available buffer space, and the sender limits unacknowledged data accordingly.

Congestion control prevents network overload. TCP maintains a congestion window (`cwnd`) and operates in phases: slow start (exponential `cwnd` growth), congestion avoidance (linear growth by ~ 1 MSS per RTT), and loss response (timeouts trigger aggressive reduction; triple duplicate ACKs trigger `cwnd` halving).

5) Socket programming in Python (3.7)

In Active Class 5, we adapted our UDP program to TCP. The server created a TCP socket with `socket(AF_INET, SOCK_STREAM)`, `bind()`, `listen()`, and `accept()` to handle connections, then `recv()` and `send()` on the connected socket. The client used `connect()` to initiate the handshake, then `send()` and `recv()`. The server counted characters and returned the uppercase version, reinforcing TCP's connection-oriented reliability versus UDP's connectionless approach.

B. How This Is Useful

This module gives me a complete picture of process-to-process communication across networks, essential for debugging performance issues and understanding protocol behaviour in SIT202. Understanding the trade-offs between UDP and TCP helps me select appropriate transport services for different applications. The Wireshark analysis and Python socket programming develop practical skills in inspecting real traffic and building networked applications, which are core SIT202 learning outcomes. Concepts such as flow control, congestion control, and reliable data transfer provide a foundation for later topics in the unit.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned is how TCP delivers a reliable, ordered byte stream over unreliable IP networks. Before this module, I knew abstractly that "TCP is reliable," but now I understand the mechanisms: three-way handshakes for connection setup, sequence and acknowledgement numbers for ordering, sliding windows for efficiency, adaptive timeouts and fast retransmit for loss recovery, flow control to protect receivers, and congestion control to protect networks. Seeing these in Wireshark traces and implementing them in Python made the concepts concrete.

B. How This Relates to What I Already Know

From earlier SIT202 modules, I understood the layered model, IP addressing, and application layer protocols such as HTTP and DNS. However, I thought of data transmission abstractly as applications "just sending" data that "just arrives." This module filled the critical gap by showing how transport protocols extend best effort IP into practical services that applications need: reliability, ordering, multiplexing, and rate control. The socket programming builds directly on my Python skills, and Wireshark activities connected to my earlier HTTP and DNS analysis by showing

how application messages travel inside transport segments with port numbers and sequence numbers coordinating delivery.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this content because the transport layer bridges applications and network infrastructure, which is essential for understanding, debugging, and designing networked systems throughout SIT202. Many practical problems, such as slow connections, lost packets, or poor performance, originate at the transport layer. The Wireshark analysis and Python socket programming in Active Classes 4 and 5 develop hands-on skills with industry tools and programming that are explicit SIT202 learning outcomes. Finally, mastering these concepts provides a foundation for later unit topics that build on transport layer principles such as reliability, performance, and congestion management.

Overall, this module transformed my view of UDP and TCP from protocol names in diagrams to precise mechanisms I can explain, observe in packet traces, and implement in working programs.

Module 4: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around Module 4's three core learning objectives: understanding data plane functions, analysing router operations, and explaining IP datagram structure, addressing, and protocols, including IPv4 and IPv6, drawing comprehensively from sections 4.1 to 4.3 and Module 4 PPT resources.

The network layer provides host-to-host datagram delivery by encapsulating transport-layer segments into IP datagrams at the sender and delivering them to the correct transport protocol at the receiver. Unlike transport and application layers that reside primarily in end systems, the network layer also operates in routers and network devices, which examine IP header fields and forward datagrams along end-to-end paths (Module 4.1). This dual residency creates a layered hierarchy: routers act as intermediate forwarding devices, focusing entirely on network-layer operations without processing application or transport-layer details.

The module emphasised two fundamental network-layer functions with distinct operational scopes: forwarding and routing (Module 4.1). Forwarding is a local, per-router operation performed in the data plane, where each router uses its forwarding table to examine an arriving datagram's header fields and decide which output port receives that packet. The forwarding table is indexed using destination IP addresses and operates at line speed via hardware lookups, ensuring minimal latency. Routing, by contrast, is a network-wide function handled by the control plane, which determines the complete end-to-end path that datagrams follow from source to destination. Routing algorithms, such as per-router protocols or centralised SDN controllers, compute these paths and populate the forwarding tables that routers then use for local decisions. This separation between data plane (fast, local, per-packet) and control plane (slower, network-wide, path-oriented) is fundamental to understanding modern networks (Module 4.1).

Router architecture consists of distinct functional components implementing forwarding and queuing: input ports handle physical and link-layer reception, forwarding table lookup, and queuing when packets arrive faster than the switching fabric can transfer them; the switching fabric itself transfers packets from input ports to output ports at a rate ideally N times the line rate for N input/output ports; and output ports buffer, schedule, and transmit packets onto outgoing links (Module 4.2). Input port congestion causes head-of-line blocking, where a packet at the front of an input queue blocks others behind it, reducing switching fabric utilisation. Output port congestion arises when the arrival rate from the switching fabric exceeds the outgoing link transmission rate, leading to buffering and potential packet loss. Buffer management policies, such as tail drop or priority drop, determine which datagrams

are discarded. Scheduling disciplines, including first-come-first-served, priority scheduling, round robin, and weighted fair queueing, determine transmission order and influence per-flow throughput and latency (Module 4.2).

The IPv4 datagram format consists of a variable-length header (minimum 20 bytes without options) containing version, header length in 32-bit words (IHL), type of service for QoS marking, total length including header and payload (maximum 64 KB), identification and flags for fragmentation coordination, fragment offset for reassembly ordering, time-to-live for loop prevention (decremented by each router), protocol field indicating encapsulated transport layer (TCP equals 6, UDP equals 17), header checksum for error detection, and source/destination IP addresses (Module 4.2). The typical maximum transmission unit for modern networks is around 1500 bytes, a limit imposed by the underlying link-layer protocol, such as Ethernet. When an IP datagram exceeds a link's MTU, routers employ IPv4 fragmentation: they split the original datagram into smaller fragments, each carrying the same identification number, using flags and fragment offset fields so that the receiving host can identify fragmented datagrams and reassemble them in correct order before passing the transport-layer segment to the appropriate protocol (Module 4.2).

IP addressing assigns 32-bit identifiers to host and router interfaces, written in dotted-decimal notation (e.g., 223.1.1.1). IP addresses have a hierarchical structure divided into network (subnet) and host parts, enabling efficient routing and address management (Module 4.2). Subnets are groups of interfaces that can reach each other directly without passing through a router; they share a common subnet mask. CIDR notation (a.b.c.d/x) specifies subnet and host portions using a prefix length, where the first x bits identify the subnet and the remaining bits identify hosts. For example, subnet 192.168.40.128/26 uses 26 bits for the network prefix, leaving 6 bits for host addresses, yielding 2^6 equals 64 total addresses within that subnet (192.168.40.128 to 192.168.40.191), with usable host addresses from 192.168.40.129 to 192.168.40.190, excluding network and broadcast addresses. This hierarchical structure supports scalable routing by allowing routers to aggregate addresses and reduce forwarding table sizes (Module 4.2).

The module introduced DHCP (Dynamic Host Configuration Protocol), a client-server mechanism that dynamically assigns IP addresses and related configuration, such as subnet mask, default gateway, and DNS server to hosts when they join a network, enabling address reuse and supporting mobile hosts (Module 4.2). Network Address Translation extends IPv4 address space by allowing multiple devices on a private network (e.g., 10.0.0.0/24) to share a single public IP address through port-based translation, where a NAT router rewrites source addresses and port numbers on outgoing datagrams and maintains a translation table to reverse this operation for incoming traffic. This provides security benefits by hiding internal network structure and enabling flexible network changes without coordinating with external parties (Module 4.2).

IPv6 was motivated by exhaustion of the 32-bit IPv4 address space and performance considerations. IPv6 uses 128-bit addresses written in hexadecimal notation with colons separating blocks (e.g., 2001:0db8::1), where each hexadecimal digit represents 4 bits and allows a more compact representation than dotted-decimal. The IPv6 datagram header is fixed at 40 bytes and includes version, traffic class for QoS marking similar to IPv4 ToS, flow label to identify packets belonging to the same flow for special handling, payload length, next header indicating protocol, hop limit equivalent to IPv4 TTL, source address, and destination address (Module 4.2). Notable differences from IPv4 include the absence of a header checksum since link-layer and transport-layer protocols provide error checking, no fragmentation support in the header, where fragmentation is handled only by the source, and a simplified fixed-length header design to accelerate router processing. The module briefly introduced transition mechanisms such as tunnelling and dual-stack routers that allow IPv4 and IPv6 coexistence during the transition period (Module 4.2).

Practical reinforcement came through Cisco Packet Tracer activities, enabling topology building, IP configuration, and network simulation, connecting abstract concepts to hands-on implementation (Module 4.3).

B. How This Is Useful

Understanding data plane mechanisms, forwarding tables, switching fabrics, queuing, and buffer management enables me to diagnose real network performance issues such as congestion, packet loss, and latency. Mastery of IP addressing, CIDR notation, and subnet design is essential for configuring routed networks and avoiding address conflicts. Knowledge of IPv4 fragmentation and IPv6 advantages prepares me for analysing packet traces and making protocol design decisions. These skills form the foundation for control-plane routing studies in later modules and are critical for SIT202 assessments requiring network analysis and configuration tasks.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most valuable insight is the clear separation between data plane and control plane operations and how this separation explains router behaviour at scale. Before this module, I viewed routers as black boxes that send packets; now I understand that each router makes independent, local forwarding decisions using lookup tables populated by a separate control plane process. This distinction is visible in Packet Tracer simulations, where I observe per-packet forwarding decisions and TTL decrements, and in Wireshark traces where IP header fields directly control forwarding behaviour. The insight that forwarding happens at hardware speeds in the data plane while routing algorithms operate at millisecond timescale in the control

plane explains why routers are often implemented with specialised hardware and explains design choices like buffer management and scheduling disciplines.

B. How This Relates to What I Already Know

This module builds directly on my earlier understanding of the TCP/IP layered model and transport-layer protocols (TCP/UDP from Module 3). In previous modules, I studied process-to-process communication, reliability, and flow control without fully appreciating the underlying host-to-host delivery service. I also performed Wireshark analysis of HTTP and DNS traffic, viewing IP headers as static metadata. Now I understand that IP headers contain actionable fields (TTL, identification, flags, protocol) that routers actively examine and modify during forwarding. The subnetting and addressing concepts connect my theoretical knowledge of CIDR to practical network configuration, allowing me to interpret real network topologies and design them correctly. Packet Tracer reinforces these connections by letting me build the networks I previously only reasoned about theoretically.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because most operational problems in real networks originate at this layer. Misconfigured IP addresses, incorrect subnet masks, improper default gateways, and MTU mismatches causing fragmentation are common issues that manifest as connectivity failures or performance degradation. Understanding how routers actually forward packets enables effective troubleshooting using tools like Wireshark to inspect header fields and trace packet paths, and command-line utilities like traceroute that exploit TTL decrement behaviour. The module's focus on data plane mechanics, combined with practical activities in Packet Tracer and Wireshark, directly supports SIT202 learning outcomes in network analysis, topology design, and device configuration. Finally, data plane knowledge provides essential background for later control-plane topics such as routing protocols and emerging areas like Software-Defined Networking.

References

- Module 4: Network Layer – Data Plane (sections 4.1, 4.2, 4.3; D2L unit site)
- Module 4 PPT: Network Layer – Data Plane (36 slides)
- Kurose, J. F. & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (7th ed.). Chapter 4

Module 5: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around Module 5's main learning focus: understanding how the network layer control plane determines routes, populates forwarding tables, and supports communication through routing algorithms and related protocols and mechanisms such as Dynamic Host Configuration Protocol (DHCP), Network Address Translation (NAT), and Internet Control Message Protocol (ICMP).

Whereas the previous module concentrated on local packet forwarding in the data plane, this module explained that routing is a network-wide function of the control plane that determines the path packets take from source to destination. The module also distinguished between traditional per-router control, where each router runs its own routing algorithm, and logically centralised control, where a remote controller can compute and install forwarding information.

A major part of the module focused on routing approaches and routing algorithms. Static routing was presented as a manually configured method that is simple and predictable in small networks but difficult to maintain when the topology changes. Dynamic routing, by contrast, allows routers to exchange routing information and update paths automatically, making it more suitable for larger or changing networks. The module also clarified the difference between link-state and distance-vector approaches, showing that link-state routing uses a broader view of the topology and shortest-path computation through Dijkstra's algorithm, whereas distance-vector routing depends on iterative exchanges with neighbours and can converge more slowly. In addition, the module emphasised that network management is important because modern networks require continuous monitoring, configuration, and fault handling to maintain performance, reliability, and security.

The practical activities helped translate these concepts into observable network behaviour. In Active Class 7, I first used a role-play activity to identify subnets, distinguish the responsibilities of the data plane and control plane, and explain how routers learn paths to remote networks. I then implemented static routing in Cisco Packet Tracer using a three-router topology, verified connectivity between hosts with ping, and manually revised routes after a topology change, which clearly demonstrated both the usefulness and the limitations of static routing. In the dynamic routing activity, I re-created the topology using a routing protocol such as the Routing Information Protocol (RIP) and observed that routes were learned and updated automatically after a link change, showing how the control plane adapts without manual reconfiguration. Active Class 8 extended this learning by examining IPv4 address limitations and NAT, performing DHCP role play through the Discover, Offer, Request, and Acknowledgement sequence, configuring a multi-LAN DHCP network in Packet Tracer, and analysing ICMP ping and traceroute behaviour in simulation mode.

B. How This Is Useful

Understanding control plane behaviour is useful because real networks depend not only on packet forwarding, but also on correct route computation and maintenance. Knowing when to use static routes, when to prefer dynamic routing, and how routing algorithms construct forwarding information helps me design networks that are both functional and scalable. The supporting protocols studied in this module are also practically important because DHCP automates host configuration, NAT conserves public IPv4 addresses, and ICMP-based tools such as ping and traceroute help diagnose connectivity and path problems. These ideas are directly relevant to Packet Tracer configuration tasks, troubleshooting activities, and later networking topics that build on control plane concepts.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned in this module is that the control plane is the decision-making part of the network layer. Before this module, I mainly focused on what happened to a packet at each hop, but now I understand much more clearly how routers obtain and update the information that makes forwarding possible. Learning the differences between static routing, dynamic routing, link-state routing, and distance-vector routing made the behaviour of routers in Packet Tracer far more meaningful, because I could connect visible path changes to the underlying routing logic rather than viewing them only as configuration steps.

B. How This Relates to What I Already Know

This module connects directly with what I learned earlier about the network layer data plane and with my previous understanding of end-to-end communication. Module 4 helped me understand how routers forward packets locally using forwarding tables, while Module 5 explained how those tables are created, updated, and maintained through control plane processes, so the two modules now fit together as one complete model of network-layer operation. It also connects with my earlier work in Wireshark and Packet Tracer, because related protocols and mechanisms such as DHCP and ICMP are no longer just names in packet traces; I now understand their operational purpose in address assignment, diagnostics, and path discovery.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because many real networking problems are control plane problems rather than simple hardware failures. Incorrect routes, poor scalability, lack of automatic adaptation, misconfigured DHCP settings, and failure to interpret ICMP diagnostic messages can all prevent communication even when the physical network is functioning correctly. By learning how routing decisions are made, how NAT and DHCP support host connectivity, and how ICMP reveals network behaviour, I am better prepared to analyse, configure, and troubleshoot networks in both coursework and practical environments. This module also provides the conceptual bridge from basic forwarding behaviour to more advanced topics such as routing protocols, network management, and software-defined networking.

References [1], [2], [3], [4], [5], [6], [7]

- [1] "Module 5: Network Layer - Control Plane - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/Home?itemIdentifier=D2L.LE.Content.ContentObject.ModuleCO-23440>
- [2] "5.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23495/View>
- [3] "5.2 DHCP and NAT - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23496/View>
- [4] "5.3 Control plane operation of the network layer - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23497/View>
- [5] "5.4 Routing Algorithms - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23498/View>
- [6] "5.5 ICMP and network management - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23499/View>
- [7] "Module 5 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 08, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23500/View>

Module 6: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the Content

I organised this summary around the main functions of the data-link layer: providing node-to-node delivery over individual links, detecting and correcting errors, coordinating shared-medium access, and addressing devices within a LAN using MAC addresses. The module first explained that the link layer sits below the network layer but above the physical layer and is responsible for encapsulating network-layer datagrams into link-layer frames, adding headers and trailers, and transferring each frame from one node to a physically adjacent node over wired or wireless media such as Ethernet and Wi-Fi. It then introduced error detection and correction mechanisms, including parity checks, Internet checksums, and cyclic redundancy checks (CRC), highlighting that CRCs are widely used on links like Ethernet because they provide strong protection against burst errors.

A major part of the module focused on medium access control (MAC) protocols, showing how multiple nodes share a common broadcast channel using three families of protocols: channel partitioning (TDMA and FDMA), random access (Aloha, CSMA, CSMA/CD and CSMA/CA), and “taking turns” protocols such as polling and token passing, each with different trade-offs between efficiency, fairness and complexity. The module then moved to LANs and link-layer addressing, explaining how 48-bit MAC addresses identify interfaces on a LAN, how ARP resolves IP addresses to MAC addresses, how Ethernet frames encapsulate IP datagrams along with preamble, type and CRC fields, and how switches and VLANs use MAC address tables to forward frames selectively and segment broadcast domains.

B. How This Is Useful

Understanding data-link-layer services is useful because every higher-layer protocol ultimately depends on reliable delivery of frames over individual links, and many practical troubleshooting tasks involve problems at this level rather than in the application code. Knowing how error detection schemes such as CRC work helps me reason about what kinds of corruption a link can detect, why Ethernet drops bad frames instead of trying to repair them, and where additional reliability is provided by higher layers; understanding MAC protocols helps me explain why wireless networks slow down when many users compete for the medium and why Ethernet rarely experiences collisions in modern switched LANs.

The addressing and LAN concepts are directly applicable when configuring and debugging real networks: MAC addresses and ARP entries appear in every Wireshark trace, and recognising the structure of an Ethernet frame or an ARP

Request makes it much easier to interpret what is going wrong when hosts cannot reach each other. Finally, the discussion of switches, MAC tables and VLANs is highly relevant for campus and enterprise environments, because it underpins how traffic is isolated, how broadcast storms are contained, and how security features such as port security or dynamic ARP inspection can be deployed to defend against attacks like MAC flooding and ARP spoofing.

2. REFLECTION ON THE CONTENT

A. Most Important Thing I Learned

The most important thing I learned in this module is how much intelligence is already present in the link layer beneath IP: instead of being just a simple pipe, it provides framing, error detection, flow control, medium access control and local addressing, all of which must work correctly for higher-layer protocols to function. Before this module, I tended to think mainly in terms of IP addresses and routing, but now I can see how mechanisms such as CRC, CSMA/CA, MAC learning in switches and ARP resolution all interact to determine whether a frame even arrives at the correct next hop, which has made my understanding of end-to-end communication much more concrete.

B. How This Relates to What I Already Know

This module connects directly with what I learned earlier about the network layer data plane and control plane: previously, I studied how routers forward IP datagrams using forwarding tables and routing algorithms, but here I saw how those datagrams are actually carried over individual Ethernet or Wi-Fi links as link-layer frames. It also builds on my earlier experience with transport-layer checksums from the TCP/UDP module by showing how CRC offers stronger protection at the link layer, and it extends my understanding of LANs from earlier practicals by explaining what happens inside a switch's MAC address table when I see frames flowing through Packet Tracer and Wireshark.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because many real faults in networks arise from link-layer problems—such as duplex mismatches, collisions on shared media, ARP misconfigurations, or MAC table issues—rather than from high-level protocol bugs, and we need to be able to recognise those patterns. By understanding error detection, MAC protocols, LAN addressing, ARP and switching, I am better prepared to interpret packet traces, reason about performance problems on congested Wi-Fi or wired segments, and appreciate both the strengths and security limitations of technologies like ARP and Ethernet, which is essential for later units on network security, ethical hacking and system design.

References [1], [2], [3], [4], [5]

- [1] "6.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23501/View>
- [2] "6.2 Data-link layer services - error detection and correction - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23502/View>
- [3] "6.3 Data-link layer services - multiple access control protocol - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23503/View>
- [4] "6.4 LAN and addressing - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23504/View>
- [5] "Module-6-The-Data-Link-Layer - Network and Communications_Sem2_2025." Accessed: Apr. 19, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23505/View>

Module 7: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the content

I organised this summary around Module 7's three learning objectives: understanding the role of the physical layer, explaining the physical media used for signal transmission, and seeing how all TCP/IP layers work together to enable real network applications.

The module first established that the physical layer is the bottom-most layer of the TCP/IP stack, responsible for converting bits into electrical, optical, or radio signals and transmitting them over physical channels. Unlike higher layers that deal with frames, datagrams, or segments, the physical layer works purely with raw bits. Physical-layer devices such as repeaters regenerate degraded signals before they reach the receiver, while hubs act as multi-port repeaters that fan out regenerated signals to all connected ports without inspecting the data, in contrast to switches that operate at the data-link layer using MAC address tables (Module 7.2).

The module then covered guided (wired) transmission media in detail. Twisted-pair copper cable is the most widely deployed medium in LANs, available in both shielded and unshielded variants, and is inexpensive but limited in range and susceptible to electromagnetic interference. Coaxial cable provides better shielding and is used in cable TV infrastructure and older broadband access networks. Optical fibre transmits light pulses rather than electrical signals, making it immune to electromagnetic interference, capable of carrying signals over much longer distances at far higher data rates, and the preferred medium for backbone and data centre links, though at a higher deployment cost. The Ethernet standard (IEEE 802.3) spans both the physical and data-link layers, specifying cable requirements, connector standards, and signal rates alongside the familiar frame format and MAC addressing scheme studied in Module 6 (Module 7.2).

The wireless section explained that unguided media propagate electromagnetic waves through the air without a physical conductor, providing the mobility that makes smartphones, laptops, and IoT devices practical, but at the cost of greater sensitivity to noise, interference, and eavesdropping compared to wired links. IEEE 802.11 Wi-Fi standards have evolved from 802.11b in 1999 at 11 Mbps over 30 m to 802.11ax (Wi-Fi 6) in 2020 at up to 14 Gbps, all using CSMA/CA for medium access control. A wireless host joins a network by scanning channels for AP beacon frames containing the SSID and MAC address, associating with a chosen AP, authenticating, and then using DHCP to obtain an IP address in the AP's subnet. For wide-area mobile connectivity, 4G LTE uses a structured architecture of User Equipment, eNode-B base stations, and an Enhanced Packet Core containing the MME, S-GW, HSS, and P-GW components, with each mobile device identified by a 64-bit IMSI stored on a SIM card. 5G builds on this by targeting 10x higher peak bitrate, 10x lower latency,

and 100x greater traffic capacity than 4G, using two frequency bands, FR1 below 6 GHz and FR2 mmWave at 24 to 52 GHz, along with MIMO antennas and dense pico-cell deployments of 10 to 100 m range (Module 7.3, PPT).

The final section reviewed the complete TCP/IP protocol stack, confirming how protocols, protocol data units, and addressing at each layer, from HTTPS messages at the application layer down to radio or electrical signals at the physical layer, work together through encapsulation (M, Ht+M, Hn+Ht+M, HI+Hn+Ht+M) to support everyday applications such as requesting a web page or streaming a video (Module 7.4).

B. How is this useful

Understanding the physical layer and transmission media is useful because it completes the full TCP/IP picture that previous modules built layer by layer. Knowing the properties of twisted pair, coaxial cable, and optical fibre, including their cost, capacity, attenuation, and interference characteristics, helps me choose appropriate cabling for a given scenario, interpret link-related performance problems, and understand why ISP backbone links use fibre while home LANs use copper. Understanding wireless technologies, their frequency bands, data rates, and range limitations is essential for reasoning about Wi-Fi coverage, interference, cellular connectivity, and the security differences between wired and wireless networks. The full-stack review in Module 7.4 ties all previous modules together and gives me a complete mental model of what happens at every layer when a user opens a browser or streams a video, which is directly applicable to network design, troubleshooting, and security analysis tasks.

2. MODULE REFLECTION

A. Most important thing I learned

The most important thing I learned in this module is that all the higher-layer abstractions I have studied, including HTTP, TCP, IP, Ethernet, and ARP, ultimately depend on physical-layer mechanisms that carry raw signals over media with real-world constraints such as attenuation, noise, and interference. Before this module, I thought of a link in the data-link layer as an abstract connection, but now I understand that every link corresponds to a specific physical medium with its own properties: a Wi-Fi link uses radio signals subject to CSMA/CA contention and multipath fading, a fibre link carries light pulses immune to interference but at higher deployment cost, and a copper Ethernet link is simple and inexpensive but limited in distance. This understanding makes every previous Wireshark trace and Packet Tracer simulation more meaningful, because I can now trace a packet all the way from the application-layer request down to the radio or electrical signal that actually carries it across the channel.

B. How This Relates to What I Already Know

This module connects directly to everything I studied in Modules 2 through 6. Module 6 taught me how the data-link layer encapsulates IP datagrams into Ethernet or 802.11 frames and delivers them over a single link using MAC addresses, and Module 7 revealed what that single link actually is at the signal level. Module 5 showed me how routing protocols determine paths between subnets, and Module 7 explained why the capacity and reliability of those paths depend on whether they traverse copper, fibre, or wireless channels. My earlier Wireshark work showed Ethernet and 802.11 frames as the lowest visible layer in a capture, and I now understand the physical-layer signals beneath them. The 802.11 association process, which involves scanning for beacon frames, authenticating, and then using DHCP, connects directly to my earlier work on DHCP in Module 5 and MAC addressing in Module 6, fitting all three into a single coherent picture of how a new wireless device joins a network.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because, without an understanding of the physical layer and transmission media, it is easy to treat networks as abstract software constructs and overlook the practical physical constraints that drive design decisions in real deployments. Many real networking problems, such as poor Wi-Fi throughput due to interference, dropped connections caused by signal attenuation, or performance differences between copper and fibre access links, originate at the physical layer, and diagnosing them requires knowing how signals behave over different media. The full-stack review in Module 7.4 serves an equally important purpose: by seeing all five layers working together in a single end-to-end scenario, the course team ensures that students leave the unit with a unified and coherent model of computer networking rather than a collection of isolated protocol facts. This complete picture is essential preparation for more advanced work in wireless systems, network security, infrastructure design, and any engineering role that requires reasoning about end-to-end network behaviour.

References [1], [2], [3], [4], [5]

- [1] "7.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23506/View>
- [2] "7.2 Physical layer and connections - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23507/View>
- [3] "7.3 Wireless networks and connections - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23508/View>
- [4] "7.4 Review: A behind the scenes look at the processes involved in a web page request - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23509/View>
- [5] "Module 7 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23510/View>

Modules: Evidence of Deeper Understanding

Module 2

Active Class 2: Analysing TLS

For the above-and-beyond task, the operation of TLS (Transport Layer Security) was analysed using Wireshark to observe the complete sequence of messages exchanged between a browser and a web server before the first HTTP GET request is sent over an HTTPS connection.

As covered in the module, HTTP is a stateless, plaintext protocol where all messages are transmitted without any protection, meaning anyone who can intercept the TCP stream can read the full content of requests and responses. HTTPS addresses this by combining HTTP with TLS, a security sublayer that operates between the Application Layer and the Transport Layer. Before any HTTP data is transferred, TLS establishes three guarantees: confidentiality through symmetric encryption, integrity through message authentication, and server identity verification through X.509 digital certificates signed by a trusted Certificate Authority (Kurose and Ross, Chapter 2; RFC 5246).

Q1. Why is TLS needed? What problem does it solve?

HTTP, on its own, sends every request and response as readable plaintext over the network. This means that a packet capture tool like Wireshark can display the full contents of the communication, including login credentials, session cookies, and personal data, to anyone positioned on the same network. TLS solves this by encrypting all HTTP messages before they leave the browser, so that even if the packets are captured, they cannot be read without the session key. This is why the module notes that Wireshark cannot decode HTTPS traffic by default, and why the browser shows a padlock icon only after TLS has been successfully negotiated.

Q2. What happens before the browser sends the first HTTP GET request?

The communication before the first HTTP GET occurs in two phases. First, a TCP connection is established on port 443 at the transport layer. Then, the TLS handshake takes place at the security sublayer. The following flowchart illustrates this complete sequence.

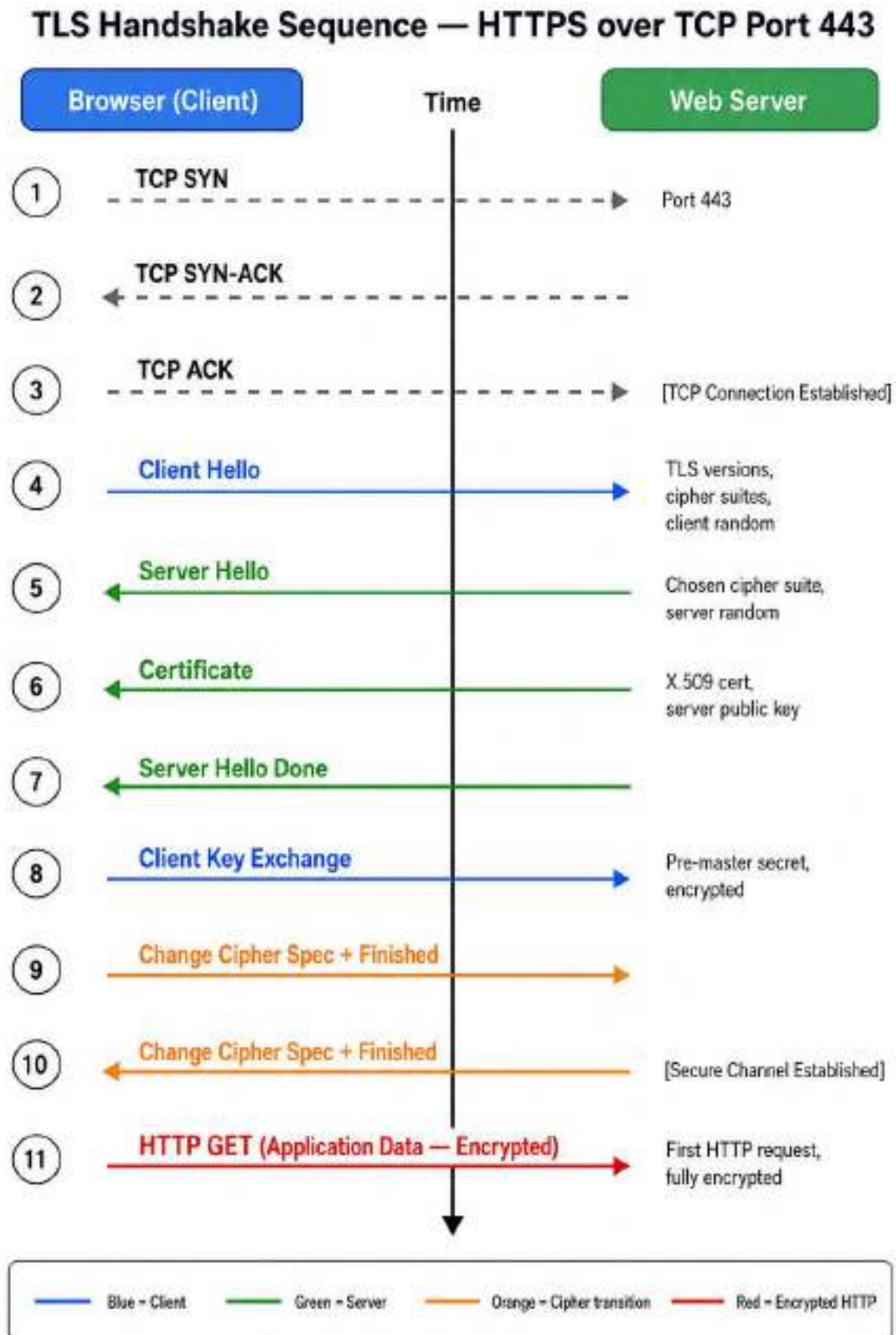


Figure 1. Complete protocol sequence showing the TCP three-way handshake followed by the TLS 1.2 handshake between the browser and the web server over port 443, ending with the first encrypted HTTP GET request.

Q3. How does the TCP three-way handshake fit into this process?

Since HTTPS uses TCP as its transport protocol, the browser must first open a reliable connection on port 443 before TLS can begin. This is identical to the three-way handshake studied in Active Class 2 for HTTP on port 80. The browser sends a TCP SYN, the server responds with SYN-ACK, and the browser confirms with a final ACK. Only after this TCP connection is in place does the TLS handshake begin.

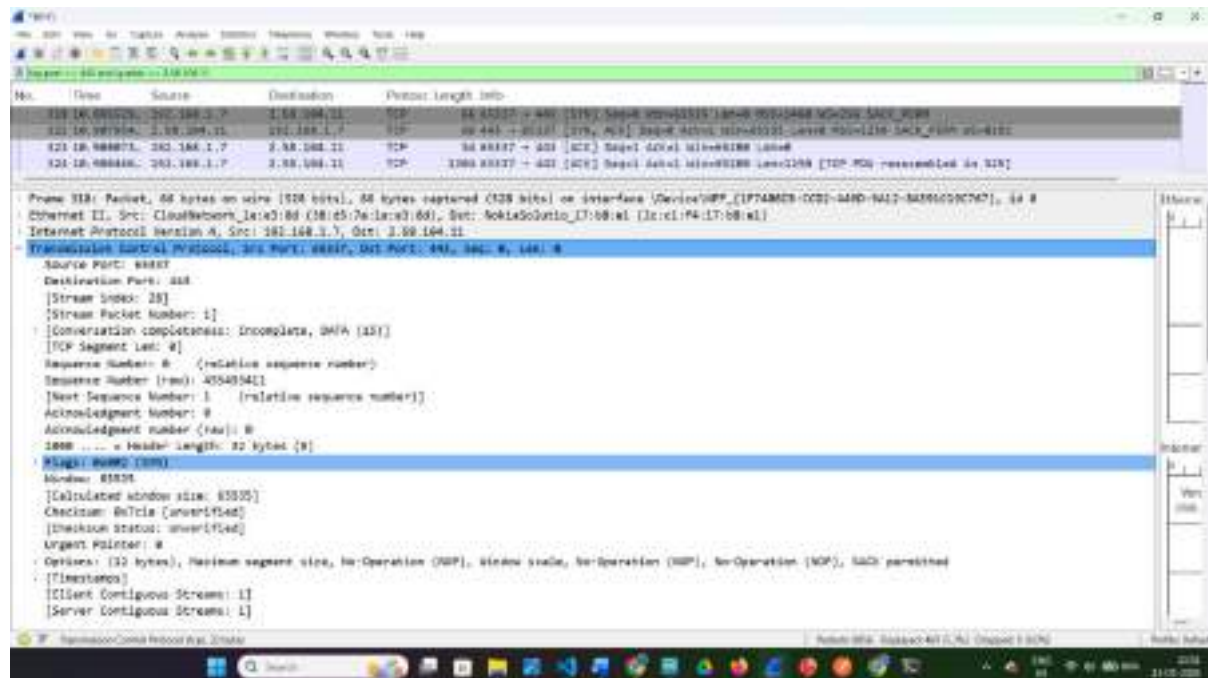


Figure 2. Wireshark packet list showing the TCP SYN, SYN-ACK, and ACK packets with destination port 443 visible in the Packet Details pane, confirming the transport connection is established before any TLS message is sent.

Q4. How does the TLS handshake work? Explain each step.

Once TCP is ready, the browser sends a **Client Hello**, which carries the list of TLS versions and cipher suites it supports, along with a randomly generated client random number that will later contribute to session key derivation. The server replies with a **Server Hello**, choosing the strongest cipher suite from the offered list, and attaches its **Certificate** - an X.509 digital credential containing the server's public key and identity, issued and signed by a trusted Certificate Authority. This step is comparable to verifying a person's identity through a government-issued passport: the browser checks the certificate against its built-in list of trusted authorities before deciding to proceed.

After the server sends **Server Hello Done**, the browser verifies the certificate. If the check passes, it generates a pre-master secret, encrypts it with the server's public key from the certificate, and sends it back as a **Client Key Exchange** message. Both sides then independently compute the same symmetric session key using the pre-master secret, the client random, and the server random. Since neither side ever sends the session key directly over the network, this method is secure even if the earlier messages were observed. Each side then sends a **Change Cipher Spec** message to

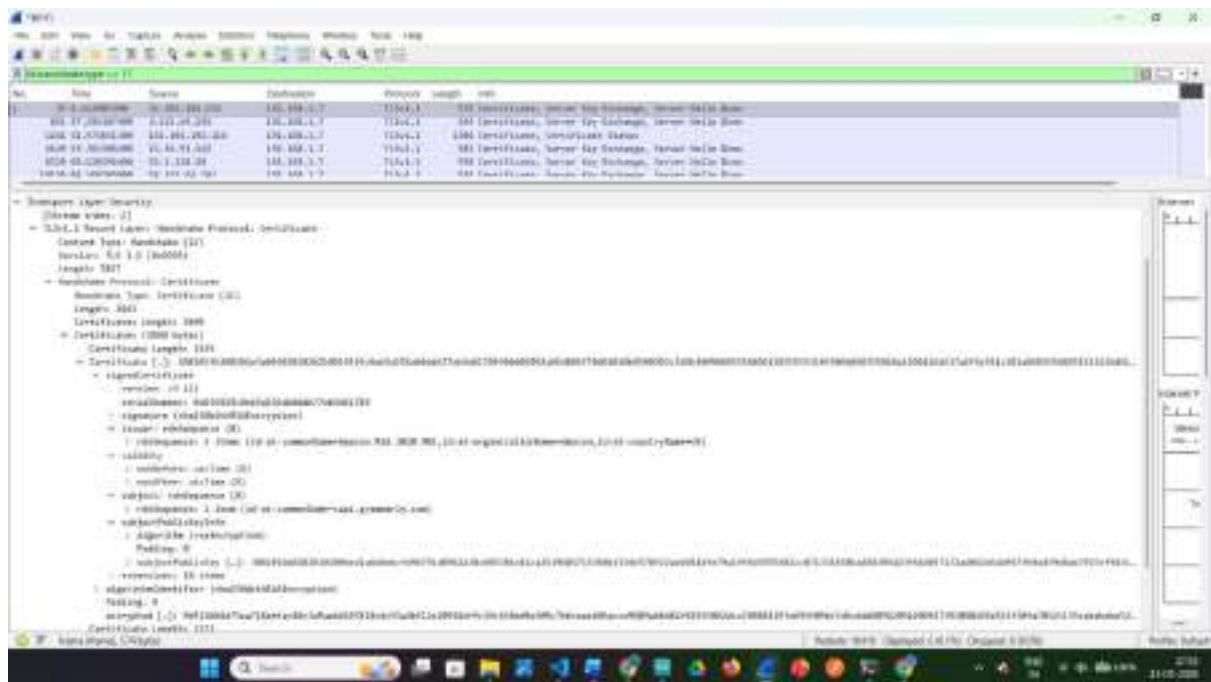
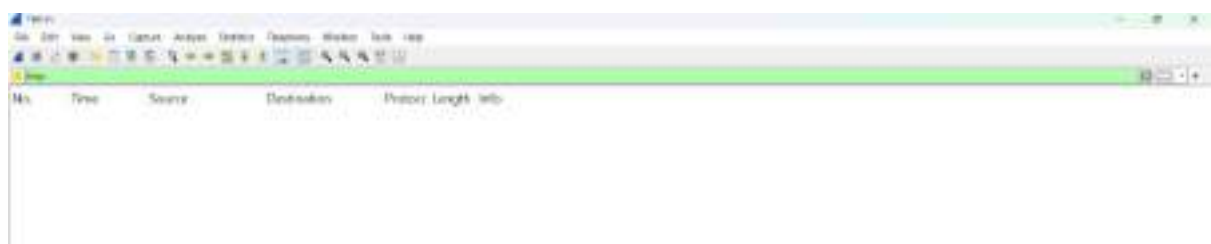


Figure 5. Wireshark Packet Details pane showing a server X.509 certificate from a TLS 1.2 handshake, with the issuer (Amazon RSA 2048 M01), subject domain (capi.grammarly.com), and validity period visible. Since TLS 1.3 encrypts the certificate exchange inside Application Data, this TLS 1.2 capture is used to illustrate how a browser verifies server identity through a trusted Certificate Authority before the handshake completes

Note: In the Wireshark capture for deakin.edu.au, TLS 1.3 was observed, where the certificate and key exchange are sent inside encrypted Application Data rather than as separate visible packets. Figure 5, therefore, uses a TLS 1.2 session to illustrate the X.509 certificate structure, since TLS 1.3 encrypts this information and makes it invisible in a standard capture.

Q5. Can you analyse HTTPS traffic in Wireshark by default?

No. Once the TLS handshake completes, every HTTP message, including the request URL, headers, cookies, and the full response body, is encrypted using the session key before being passed to TCP. In the Wireshark capture, all packets after the handshake appear only as Application Data with no readable content. Applying the http display filter returns no results because the HTTP messages are wrapped inside encrypted TLS records and are invisible to Wireshark without the decryption keys. This matches the observation from Active Class 2, where visiting an HTTPS site produced only TLS records in Wireshark with no plaintext HTTP visible at all.




```

> Frame 142: 583 bytes on wire (4664 bits), 583 bytes captured (4664 bits) on interface \Device\NPF_{1A2B3C4D-5E6F-...}
> Ethernet II, Src: IntelCor_3a:7b:9c (3c:fd:fe:3a:7b:9c), Dst: Cisco_8e:20:aa (08:50:56:8e:20:aa)
> Internet Protocol Version 4, Src: 192.168.1.10, Dst: 93.184.216.34
> Transmission Control Protocol, Src Port: 51514, Dst Port: 80, Seq: 1, Ack: 1, Len: 517
▼ Hypertext Transfer Protocol
  ▼ GET /index.html HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /index.html HTTP/1.1\r\n]
      Request Method: GET
      Request URI: /index.html
      Request Version: HTTP/1.1
      Host: example.com\r\n
      User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
      Chrome/124.0.0.0 Safari/537.36\r\n
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8\r\n
      Accept-Encoding: gzip, deflate, br\r\n
      Accept-Language: en-US,en;q=0.9\r\n
      Connection: keep-alive\r\n
      Upgrade-Insecure-Requests: 1\r\n
      \r\n
    > [Full request URI: http://example.com/index.html]
      [HTTP request 1/1]
      [Response in frame: 144]
  > Line-based text data: text/html (1 lines)
  > Transport Layer Security

0000  00 50 56 8e 20 aa 3c fd fe 3a 7b 9c 08 00 45 00  .PV . .:<{...E,
0010  02 35 7b 1a 40 00 40 06 6a 6f c0 a8 01 0a 5d b8  .S{.@.@.jo....}.
0020  d8 22 c8 1a 00 50 9a d2 6b 1c 3d 5b d4 17 50 18  ."....P..k..P.
0030  01 f7 fe 98 00 00 47 45 54 20 2f 69 6e 64 65 78  .....GET /index
0040  2e 68 74 6d 6c 20 48 54 54 50 2f 31 2e 31 0d 0a  .html HTTP/1.1.
0050  48 6f 73 74 3a 20 65 78 61 6d 70 6c 65 2e 63 6f  Host: example.co
0060  6d 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d  m..User-Agent: M
0070  6f 7a 69 6c 60 61 2f 35 2e 30 20 28 57 69 6e 64  ozilla/5.0 (Wind

```

Hypertext Transfer Protocol: Protocol
 Packets: 2567 - Displayed: 2 (0.1%)
 Profile: Default

Figure 7. Wireshark capture with SSLKEYLOGFILE configured, showing the decrypted HTTP GET request with method, URL path, and headers clearly visible inside what was previously labelled Application Data.

Progress Note: This above-and-beyond task deepens the understanding of the application layer by showing that HTTPS is the result of a precisely ordered, multi-protocol interaction. TCP provides reliable delivery on port 443, TLS handles authentication and key exchange using asymmetric cryptography for security, both sides independently derive a shared symmetric key for efficient encryption, and only then does the browser transmit its first HTTP request. Observing this sequence live in Wireshark connects the theory from the module directly with real, captured network behaviour. The SSLKEYLOGFILE method reinforces a key principle: encryption secures data in transit, but authorised analysis remains possible when session secrets are available, which is directly relevant to network security, digital forensics, and professional troubleshooting practice.

Active Class 3: E-mails (Another Popular Application)

Email is one of the most widely used Internet applications, and like web browsing, it relies on a set of application-layer protocols to transfer messages reliably between users across the network.

Q1. What is the principal application layer protocol used in e-mails?

The principal application layer protocol used for sending emails is **SMTP (Simple Mail Transfer Protocol)**. SMTP is responsible for transferring email messages from the sender's mail client to the sender's mail server, and then between mail servers until the message reaches the recipient's mail server (Kurose and Ross, Chapter 2; RFC 5321). For retrieving emails at the receiving end, two additional protocols are used alongside SMTP: **POP3 (Post Office Protocol version 3)**, which downloads messages to the local device and typically removes them from the server, and **IMAP (Internet Message Access Protocol)**, which allows the user to access and manage emails directly on the server and supports synchronisation across multiple devices.

Q2. What is the underlying architecture and transport layer protocol used in the email application layer protocol?

Email follows the **client-server architecture**, consistent with the model studied in the module. The architecture involves three key components working together:

- **User Agent (UA):** The email client software, such as Microsoft Outlook, Gmail, or Mozilla Thunderbird, which the user interacts with to compose, send, and read messages
- **Message Transfer Agent (MTA):** The server-side software that routes and transfers messages between mail servers using SMTP
- **Message Access Agent (MAA):** The component that allows the recipient's user agent to retrieve messages from the mailbox on the server using POP3 or IMAP

For the transport layer protocol, **SMTP uses TCP** on port **25** (for server-to-server relay) and port **587** (for client-to-server submission). TCP is chosen because email delivery requires reliable, ordered transmission - losing or corrupting part of a message is not acceptable. POP3 uses TCP on port 110, and IMAP uses TCP on port 143.

Protocol	Role	Transport	Port
SMTP	Sending and transferring mail	TCP	25 / 587
POP3	Retrieving mail (downloads to device)	TCP	110
IMAP	Accessing mail on server (multi-device)	TCP	143

Q3. Can you list down the basic steps involved in sending an email from User A to User B?

The following steps describe the complete process from the moment User A composes an email to the moment User B reads it:

1. **User A composes the email** using their User Agent (e.g., Gmail or Outlook) and clicks Send
2. **User A's UA sends the message to User A's mail server** using SMTP over TCP (port 587), where it is placed in a message queue
3. **User A's mail server (MTA) opens an SMTP connection** to User B's mail server and transfers the message using SMTP over TCP (port 25)
4. **User B's mail server receives the message** and stores it in User B's mailbox on the server
5. **User B's UA retrieves the message** from their mail server using either POP3 (downloads and optionally deletes from the server) or IMAP (accesses the message directly on the server without downloading)
6. **User B reads the email** through their User Agent

The diagram below illustrates this sequence:

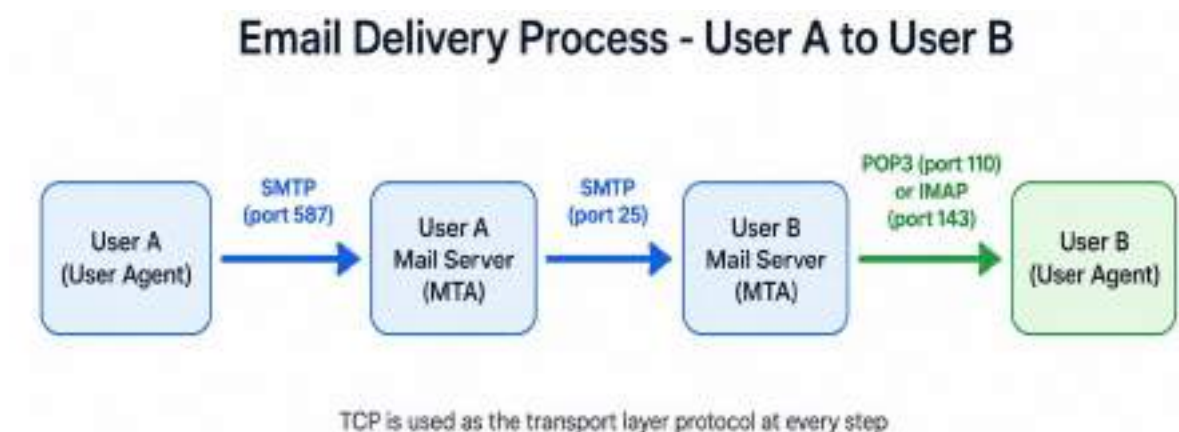


Figure 8. Flowchart illustrating the complete email delivery process from User A to User B, showing SMTP for sending and transferring between servers, and POP3 or IMAP for retrieval at the receiving end.

Module 3:

Active Class 4: Modified UDP client-server program

For the above-and-beyond task, the Python User Datagram Protocol (UDP) client-server program was modified to implement a multi-step communication process between the client and the server.

In this modified version, the client first sends the message **“Hello”** to the server, the server responds with **“Hello, What’s your name?”**, the client then sends the entered name, and finally the server replies with **“Hello name, Welcome to SIT202”**, where **name** is the exact value received from the client.

All received messages are displayed in the relevant client and server terminals, which satisfies the requirement of the above-and-beyond task.

To explain the operation of the modified UDP program clearly, the following flowchart illustrates the sequence of communication between the client and the server, beginning with the initial greeting and ending with the final welcome response.

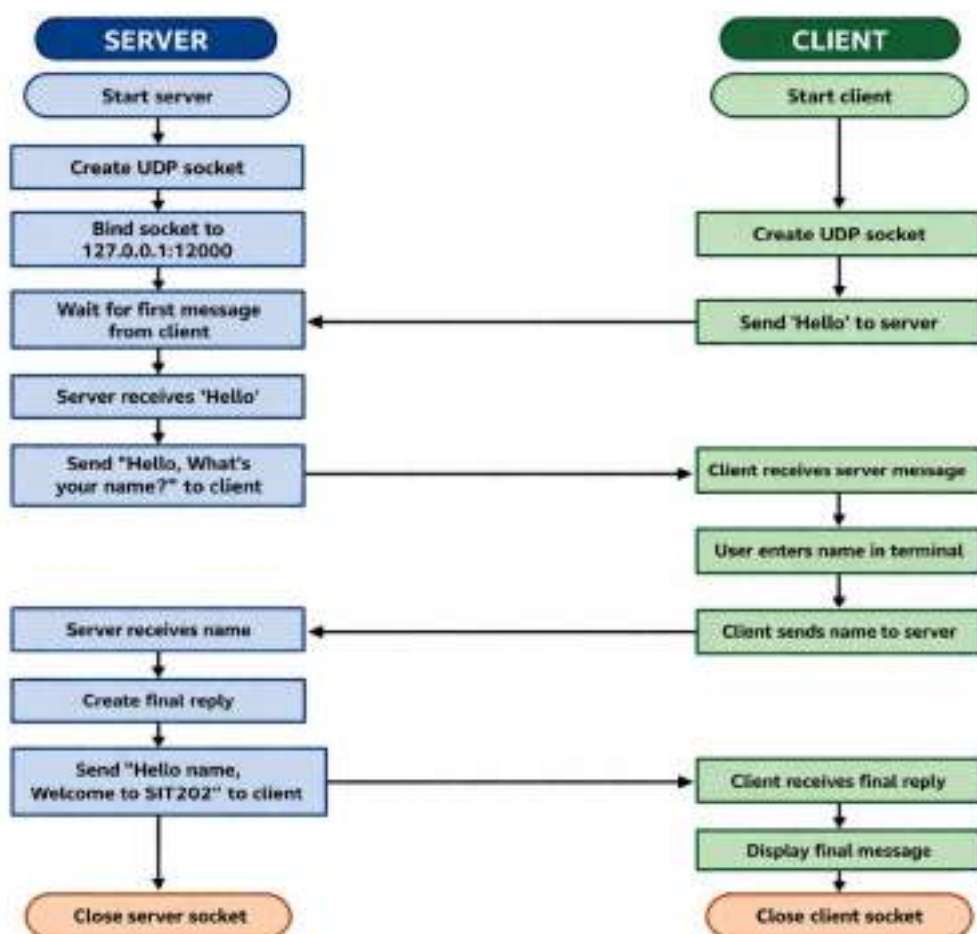


Figure 9. Flowchart of the modified UDP client-server program showing greeting exchange, name request, name transfer, and final welcome response

The server side of the program was implemented to wait for the client's initial **"Hello"** message, respond with **"Hello, What's your name?"**, receive the name entered by the client, and then send the final reply **"Hello name, Welcome to SIT202"**. The server code is written in a simple and readable style with short, meaningful comments and clear variable names so that the UDP communication logic can be understood easily.

```
udp_server_above_beyond.py > ...
1  import socket
2
3  SERVER_IP = "127.0.0.1"
4  SERVER_PORT = 12000
5  BUFFER_SIZE = 1024
6
7
8  def main():
9      # Create the UDP server socket and bind it to the local address.
10     server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
11     server_socket.bind((SERVER_IP, SERVER_PORT))
12
13     print(f"UDP server is running on {SERVER_IP}:{SERVER_PORT}")
14
15     try:
16         # Receive the first message from the client.
17         first_message, client_address = server_socket.recvfrom(BUFFER_SIZE)
18         greeting_text = first_message.decode().strip()
19         print(f"Received from client: {greeting_text}")
20
21         # Send the name request only when the expected greeting is received.
22         if greeting_text == "Hello":
23             question_text = "Hello, What's your name?"
24             server_socket.sendto(question_text.encode(), client_address)
25             print(f"Sent to client: {question_text}")
26         else:
27             error_text = "Expected 'Hello' as the first message."
28             server_socket.sendto(error_text.encode(), client_address)
29             print(f"Sent to client: {error_text}")
30             return
31
32         # Receive the name from the same client.
33         second_message, second_client_address = server_socket.recvfrom(BUFFER_SIZE)
34
35         if second_client_address != client_address:
36             print("Ignored message from an unexpected client.")
37             return
38
39         name_text = second_message.decode().strip()
40         print(f"Received from client: {name_text}")
41
42         if not name_text:
43             name_text = "Student"
44
```

```
45         # Send the final welcome message.
46         final_reply = f"Hello {name_text}, Welcome to SIT202"
47         server_socket.sendto(final_reply.encode(), client_address)
48         print(f"Sent to client: {final_reply}")
49
50     except KeyboardInterrupt:
51         print("\nServer stopped by user.")
52     finally:
53         server_socket.close()
54
55
56 if __name__ == "__main__":
57     main()
```

Figure 10. Final server code for the modified UDP client-server program

The client side of the program was implemented to send the initial greeting to the server, receive the server's question, allow the user to enter a name through the terminal, send that name back to the server, and finally display the welcome reply received from the server.

The client code also follows a clear and consistent style with short comments at important steps only, which helps maintain simplicity, readability, and professionalism in the final submission.

```
udp_client_above_beyond.py
udp_client_above_beyond.py > ...
1     import socket
2
3     SERVER_IP = "127.0.0.1"
4     SERVER_PORT = 12000
5     BUFFER_SIZE = 1024
6     TIMEOUT_SECONDS = 10
7
8
9     def main():
10         # Create the UDP client socket.
11         client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
12         client_socket.settimeout(TIMEOUT_SECONDS)
13
14         try:
15             # Send the initial greeting to the server.
16             first_message = "Hello"
17             client_socket.sendto(first_message.encode(), (SERVER_IP, SERVER_PORT))
18             print(f"Sent to server: {first_message}")
19
20             # Receive the server's question.
21             server_question, server_address = client_socket.recvfrom(BUFFER_SIZE)
22             question_text = server_question.decode().strip()
23             print(f"Received from server: {question_text}")
24
25             # Read the user's name and send it to the server.
26             name_text = input("Enter your name: ").strip()
27             if not name_text:
28                 name_text = "Student"
```

```
29
30     client_socket.sendto(name_text.encode(), server_address)
31     print(f"Sent to server: {name_text}")
32
33     # Receive the final welcome message.
34     final_message, _ = client_socket.recvfrom(BUFFER_SIZE)
35     final_reply = final_message.decode().strip()
36     print(f"Received from server: {final_reply}")
37
38     except socket.timeout:
39         print("Connection timed out. No response was received from the server.")
40     except KeyboardInterrupt:
41         print("\nClient stopped by user.")
42     finally:
43         client_socket.close()
44
```

Figure 11. Final client code for the modified UDP client-server program

After implementing both sides of the program, the output was tested to confirm that the full UDP interaction works as required. The combined output shows both terminals together so that the complete sequence is visible in one place.

```
PS D:\Semester-4\SIT202\4.1C Task> python udp_server_always_beyond.py
UDP server is running on 133X.D.D.1:10000
Received from client: Hello
Sent to client: Hello, What's your name?
Received from client: Jahan Garg
Sent to client: Hello Jahan Garg, Welcome to SIT202
PS D:\Semester-4\SIT202\4.1C Task>

PS D:\Semester-4\SIT202\4.1C Task> python udp_client_always_beyond.py
Sent to server: Hello
Received from server: Hello, What's your name?
Enter your name: Jahan Garg
Sent to server: Jahan Garg
Received from server: Hello Jahan Garg, Welcome to SIT202
PS D:\Semester-4\SIT202\4.1C Task>
```

Figure 12. Output of the modified UDP client-server program showing greeting exchange, name input, and final welcome response

The output of the modified UDP client-server program confirms that the application works as expected.

During execution, the **client** first sends the message “**Hello**” to the server, and the **server** successfully receives that message and responds with “**Hello, What’s your name?**”.

After receiving the prompt, the client allows the user to enter a name through the terminal and sends that name to the server.

The server then receives the entered name and replies with **“Hello name, Welcome to SIT202”**, where **name** is the exact value provided by the client.

All received messages are displayed in the relevant client and server terminals, which confirms that the modified program satisfies the requirements of the above-and-beyond task.

Progress note: This modified UDP program demonstrates a deeper understanding of UDP socket communication by extending the original one-message exchange into a multi-step request-and-response interaction while still using a lightweight connectionless transport protocol. Unlike TCP, UDP does not establish a connection or guarantee delivery, yet this task shows that a reliable, ordered multi-step conversation can still be achieved when the application itself controls the exchange sequence.

Active Class 5: TCP — ARQ Protocols and Congestion Control

Q1. Discuss the differences between Stop and Wait, Go Back N, and Selective Repeat protocols, and the reasons for using these protocols.

When data is transmitted over a network, packets can be lost, corrupted, or arrive out of order. Automatic Repeat Request (ARQ) protocols address this by combining error detection, receiver acknowledgements, and retransmission to ensure reliable data delivery. As covered in the module, the three main ARQ algorithms used in computer networks are Stop and Wait, Go Back N, and Selective Repeat (Kurose and Ross, Chapter 3).

Stop and Wait is the simplest of the three. The sender transmits one packet and then waits for an acknowledgement from the receiver before sending the next one. If no acknowledgement is received within the timeout period, the same packet is retransmitted. While this guarantees that every packet is confirmed before the next is sent, it results in poor channel utilisation because the sender remains idle during the entire round-trip time between sending a packet and receiving its acknowledgement. This is acceptable for very low latency or low bandwidth links, but becomes highly inefficient for high-speed or long-distance connections.

Go Back N improves on this by allowing the sender to transmit multiple packets without waiting for individual acknowledgements, up to a maximum number defined by the window size. With a fixed window size of 3, Host A sends packets 1, 2, and 3, receives positive acknowledgements, and then continues with packets 4 and 5. If packet 4 is lost and no acknowledgement is received before the timer expires, the sender retransmits from packet 4 onwards, including packets 5, 6, and 7, even if those

were already successfully received by the other side. This is the key limitation of Go Back N: when an error occurs, all packets from the lost one to the end of the window are retransmitted regardless of whether the receiver already holds them correctly. This can waste bandwidth and processing when the window size is large.

Selective Repeat eliminates this waste. The sender only retransmits the specific packet that was lost or corrupted rather than the entire window. In the same scenario, if packet 4 is lost but packets 5, 6, and 7 were received correctly and acknowledged, only packet 4 is retransmitted. Packets 6 and 7 are not resent if a positive acknowledgement has already been received for them. This makes Selective Repeat more efficient than Go Back N, particularly when the error rate is low, because it avoids unnecessary retransmissions and conserves bandwidth. The trade-off is that the receiver must maintain a larger buffer to store out-of-order packets until the missing one arrives and the sequence can be completed.

The following table summarises the key differences between the three protocols:

Feature	Stop and Wait	Go Back N	Selective Repeat
Packets sent before ACK	1	Up to window size N	Up to window size N
Retransmission on error	1 packet	All packets from the lost one onwards	Only the lost packet
Receiver buffer needed	Minimal	Minimal	Larger
Channel utilisation	Low	Medium to high	Highest
Complexity	Simple	Moderate	Most complex
Best suited for	Low speed, simple links	General use	High-speed, error-prone links

The reason all three protocols exist is that different network conditions call for different trade-offs between simplicity, buffer requirements, and efficiency. Stop and Wait is easy to implement but wasteful. Go Back N offers a practical middle ground with moderate complexity. Selective Repeat delivers the best efficiency but requires more sophisticated buffering and tracking at both ends.

Q2. Why do we need congestion control in TCP? Explain your answer.

Flow control, which was studied alongside ARQ in this module, prevents the sender from overwhelming the receiver's buffer by limiting the amount of unacknowledged data in flight based on the receiver's advertised window size. However, flow control alone is not sufficient because it only accounts for the receiver's capacity, not the capacity of the network between the two end systems. When too many senders transmit data simultaneously, the intermediate network devices, such as routers and switches, can become overloaded, their buffers fill up, and they begin dropping packets. This condition is known as network congestion.

As described in the module, when a network device's buffer capacity is full, it drops incoming packets. In a scenario where four computers each send at a rate C , and the total sending rate $4C$ exceeds the link rate D , packets are dropped regardless of how much buffer space the receiver has available (Kurose and Ross, Chapter 3). Without congestion control, TCP senders would continue transmitting at full speed, interpret dropped packets as simple errors, and retransmit, which would only increase the congestion further and cause network performance to collapse entirely.

TCP congestion control solves this by giving each sender a second window variable called the congestion window, in addition to the receive window used for flow control. The sender limits its unacknowledged data to the minimum of these two values, ensuring that neither the receiver nor the network is overwhelmed. TCP uses the receipt of acknowledgements as a signal that the network can bear more data, and interprets dropped packets or duplicate acknowledgements as a signal to reduce the sending rate.

The TCP congestion control algorithm operates in two main phases. During **Slow Start**, the congestion window begins at a small value and grows exponentially with each round-trip time until it reaches a threshold, allowing TCP to quickly discover the available bandwidth. Once the threshold is reached, the algorithm enters **Congestion Avoidance**, where the congestion window increases by only one MSS per round-trip time, probing for additional bandwidth gradually without risking overload. When packet loss is detected through a timeout, the congestion window is reset aggressively. When three duplicate ACKs are received, TCP performs a fast retransmit and reduces the congestion window by half rather than resetting it completely, since partial delivery suggests the network is still functional.

In summary, congestion control is essential because without it, TCP would not adapt to network conditions, senders would flood the network beyond its capacity, and the resulting packet loss and retransmissions would make congestion progressively worse. Congestion control is what makes TCP a responsible, independently adjusting protocol that shares network resources fairly among multiple simultaneous connections.

Progress Note: This above-and-beyond task deepens the understanding of reliable data transport by connecting the theoretical differences between Stop and Wait, Go Back N, and Selective Repeat to practical scenarios involving packet loss, window size, and retransmission behaviour. The congestion control discussion further extends this by showing that reliability at the transport layer requires managing not just the receiver's capacity through flow control, but also the network's capacity through dynamic sending rate adjustment, both of which are core mechanisms that make TCP the reliable, widely used protocol it is today.

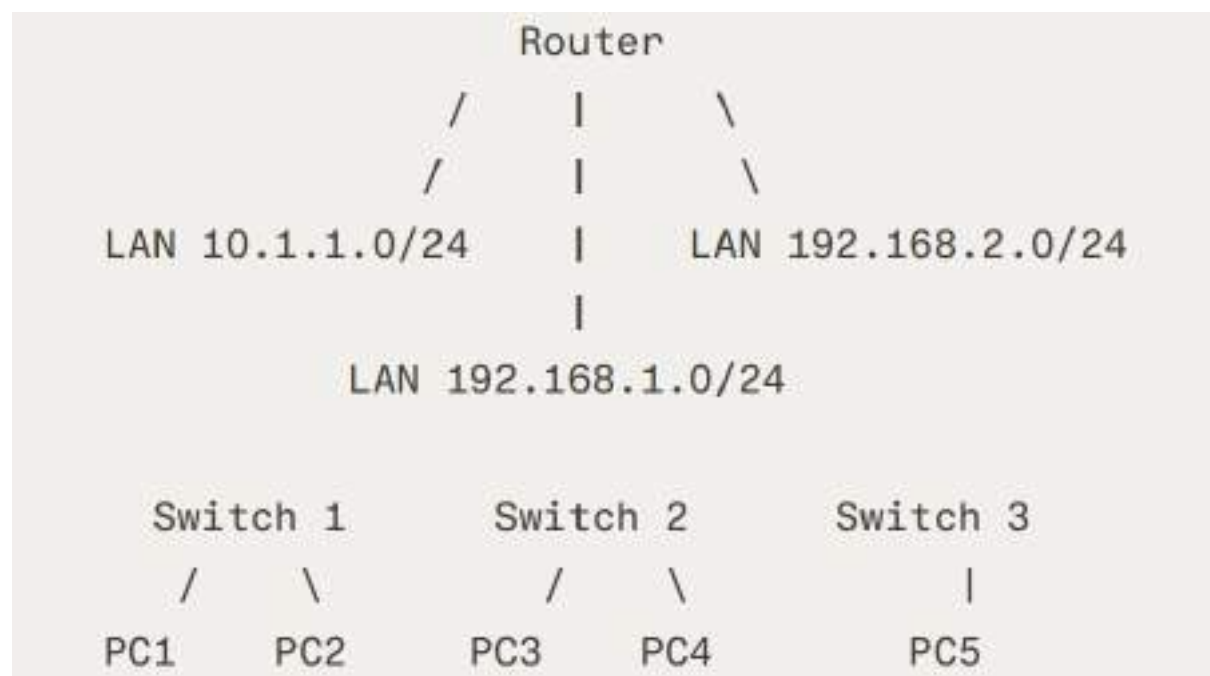
Module 4

Active Class 6: Extending the Network with PC5

For this above-and-beyond task, the original two LAN networks from Activity 3 were extended into a three-LAN topology so that PC1 in the 10.1.1.0/24 network could communicate with PC5 at IP address 192.168.2.4, which belongs to a new third subnet, 192.168.2.0/24.

Updated Three LAN Topology

The extended logical layout is as follows:



Each subnet uses a /24 prefix, which is consistent with the module examples on subnets and CIDR notation (Module 4.2). The following table shows the complete IP configuration for all devices:

Device or Interface	IP Address	Subnet Mask	Default Gateway
PC1	10.1.1.10	255.255.255.0	10.1.1.1
PC2	10.1.1.20	255.255.255.0	10.1.1.1
PC3	192.168.1.10	255.255.255.0	192.168.1.1
PC4	192.168.1.20	255.255.255.0	192.168.1.1
PC5	192.168.2.4	255.255.255.0	192.168.2.1
Router Gi0/0	10.1.1.1	255.255.255.0	—
Router Gi0/1	192.168.1.1	255.255.255.0	—
Router Gi0/2	192.168.2.1	255.255.255.0	—

Each router interface serves as the default gateway for the hosts in its respective LAN, which is exactly how real IP routers attach to multiple subnets.

1. What Happens When PC1 Sends a Message to PC5

When PC1 (10.1.1.10) sends a message to PC5 (192.168.2.4), the destination address does not belong to the 10.1.1.0/24 subnet, so PC1 cannot deliver the packet directly. Instead, it forwards the datagram to its default gateway (10.1.1.1), which is the router's Gi0/0 interface. The following steps describe the full data plane journey:

1. PC1 creates an IPv4 datagram with source address 10.1.1.10 and destination address 192.168.2.4, encapsulates it in an Ethernet frame addressed to the router's MAC address, and sends it to Switch 1.
2. Switch 1 examines the destination MAC address and forwards the frame to the router's Gi0/0 port without inspecting the IP header.
3. The router receives the frame on Gi0/0, strips the link layer header, and reads the destination IP address 192.168.2.4 from the IP datagram.
4. The router consults its forwarding table, identifies the 192.168.2.0/24 entry, and selects Gi0/2 as the output interface.

5. The router decrements the TTL field by 1, recalculates the header checksum, and encapsulates the datagram in a new Ethernet frame with the source MAC of Gi0/2 and the destination MAC of PC5, obtained via ARP if not already cached.
6. The frame travels from the router to Switch 3, which forwards it to PC5 based on the destination MAC address.
7. PC5 receives the frame, verifies that the destination IP matches its own address, removes the IP header, and passes the transport layer segment up to the application.

This journey illustrates the data plane principle that each router makes an independent, per-packet forwarding decision based solely on the destination IP address and its forwarding table (Module 4.1, Module 4.2).

2. How PC5 is Connected to the Current Network

PC5 belongs to the new subnet 192.168.2.0/24, which did not exist in the original two LAN topology. The following additions were made to integrate PC5 into the network:

- A third router interface (GigabitEthernet0/2) was configured with IP address 192.168.2.1 and subnet mask 255.255.255.0, making it the default gateway for the 192.168.2.0/24 network.
- A new Switch 3 was connected to this router interface.
- PC5 was connected to Switch 3 and assigned IP address 192.168.2.4, subnet mask 255.255.255.0, and default gateway 192.168.2.1.

The router now holds three directly connected network entries in its forwarding table: 10.1.1.0/24 on Gi0/0, 192.168.1.0/24 on Gi0/1, and 192.168.2.0/24 on Gi0/2. Any packet crossing from one LAN to another is forwarded by consulting this table, which is the core mechanism of data plane forwarding described in the module and in Kurose and Ross, Chapter 4.

3. Implementing and Verifying the Design in Cisco Packet Tracer

Building the Extended Topology

Starting from the Activity 3 topology, the following steps were completed:

1. A new Switch 3 was added to the workspace and connected to the router's GigabitEthernet0/2 interface using a copper straight-through cable.
2. PC5 was added to the workspace and connected to Switch 3.

Figure 14. Successful ping from PC1 (10.1.1.10) to PC5 (192.168.2.4), confirming inter-subnet connectivity across three LANs.

Simulation Mode Observation

Cisco Packet Tracer was switched to Simulation mode, and a PDU was sent from PC1 to PC5. Stepping through the simulation confirmed the following sequence:

1. Packet leaves PC1 and arrives at Switch 1
2. Switch 1 forwards the frame to the router based on the destination MAC address
3. Router examines the destination IP, selects Gi0/2, decrements TTL, and re-encapsulates
4. Frame travels to Switch 3, which forwards it to PC5
5. PC5 receives the packet and generates an ICMP echo reply following the reverse path

At each hop, Packet Tracer showed that the link layer frame addresses changed while the IP source and destination addresses remained constant, directly demonstrating the separation between network and link layers covered in the module.



Figure 15. Simulation mode showing the hop-by-hop packet journey from PC1 through Switch 1, the router, Switch 3, and finally to PC5.

Progress Note: This above-and-beyond activity extended the two LAN networks to a three-LAN topology, demonstrating how a single router interconnects multiple subnets by maintaining separate forwarding table entries for each network. Configuring a third router interface, verifying connectivity through ping, and observing the packet journey in simulation mode confirmed that data plane forwarding operates identically regardless of the number of connected subnets, reinforcing the scalability of IP-based routing covered in Module 4.

Module 5

Active Class 7: Three Widely Used Routing Protocols

The control plane of the network layer is responsible for determining how packets are routed from source to destination across interconnected networks. To carry out this function, routers rely on routing protocols that either compute optimal paths using a complete topology view or exchange information incrementally with neighbours. Three of the most widely deployed routing protocols in modern networks are the Routing Information Protocol (RIP), the Open Shortest Path First protocol (OSPF), and the Border Gateway Protocol (BGP). Each of these protocols serves a distinct operational context, uses a different algorithmic foundation, and offers a different set of trade-offs between simplicity, scalability, and adaptability.

1. Routing Information Protocol (RIP)

RIP is one of the oldest and simplest dynamic routing protocols, built on the Distance Vector algorithm and the Bellman-Ford equation. Each router running RIP maintains a routing table that lists the best-known distance (measured in hop count) to every reachable destination, and periodically shares this table with its directly connected neighbours. Upon receiving updates, each router revises its own estimates and propagates any improvements further.

RIP uses hop count as its sole metric, with a maximum value of 15 hops. Any destination requiring more than 15 hops is considered unreachable, which makes RIP unsuitable for large networks. It is best suited for small, stable networks where simplicity of configuration is more important than convergence speed or path optimisation. RIP sends full routing table updates every 30 seconds by default, which consumes bandwidth even when nothing in the topology has changed.

2. Open Shortest Path First (OSPF)

OSPF is a Link State routing protocol widely used as an Intra AS (Interior Gateway) protocol within enterprise and service provider networks. Unlike RIP, OSPF requires each router to build a complete map of the entire AS by flooding Link State Advertisements (LSAs) to all other routers. Once every router holds an identical topology map, each one independently runs Dijkstra's shortest-path algorithm to compute the optimal path to every destination.

OSPF supports multiple cost metrics, such as bandwidth and delay, converges much faster than RIP when topology changes occur, and scales well to large networks through hierarchical area design, where a backbone area (Area 0) interconnects

multiple sub-areas. OSPF authenticates all messages to prevent route injection from unauthorised sources. Because each router computes its own forwarding table from a consistent global view, OSPF avoids the count-to-infinity problem that can affect Distance Vector protocols.

3. Border Gateway Protocol (BGP)

BGP is the standard Inter AS (Exterior Gateway) routing protocol used across the entire Internet. While RIP and OSPF operate within a single administrative domain, BGP is designed to exchange reachability information between autonomous systems operated by different organisations and Internet Service Providers. BGP is a Path Vector protocol: rather than simply advertising a distance or a full topology map, it advertises complete AS-level paths, allowing each AS to apply policy decisions when selecting routes.

BGP routers establish long-lived TCP sessions with their peers and exchange incremental updates rather than full table dumps. Route selection in BGP is influenced by attributes such as local preference, AS-path length, and next-hop reachability. Because BGP must balance technical routing with business relationships and policies between ISPs, it is significantly more complex than either RIP or OSPF. It is often described as the protocol that holds the Internet together, as it enables the global routing table that connects all autonomous systems.

Comparison Table

Feature	RIP	OSPF	BGP
Protocol type	Distance Vector	Link State	Path Vector
Algorithm	Bellman-Ford	Dijkstra	Policy-based path selection
Scope	Intra AS (small networks)	Intra AS (large networks)	Inter AS (Internet-wide)
Metric	Hop count (max 15)	Cost (bandwidth, delay, etc.)	AS path, local preference, policy

Feature	RIP	OSPF	BGP
Convergence speed	Slow	Fast	Slow to moderate
Scalability	Low (max 15 hops)	High (hierarchical areas)	Very high (global Internet)
Update mechanism	Periodic full table (every 30 s)	Triggered incremental LSAs	Incremental updates over TCP
Loop prevention	Split horizon, poison reverse	Dijkstra tree (loop-free by design)	AS path attribute
Authentication	Optional (RIPv2)	Yes (cryptographic)	Yes (MD5 or TCP-AO)
Typical use case	Small LANs, lab networks	Enterprise, campus, ISP backbone	Internet routing between ISPs
Main advantage	Simple to configure	Fast convergence, scalable	Full policy control, global scale
Main disadvantage	Limited scale, slow convergence	Complex configuration	Very complex, policy dependent

Discussion

The three protocols differ most significantly in their intended scale and the type of information they use to make routing decisions. RIP prioritises simplicity at the cost of scalability and convergence speed, making it a practical starting point for understanding dynamic routing but rarely deployed in production environments beyond small networks. OSPF addresses the shortcomings of RIP by giving every router a complete, consistent picture of the network, enabling faster and more accurate path computation at the cost of greater memory and processing requirements. BGP operates at an entirely different level, where routing decisions reflect not just topology but also business policies and agreements between autonomous systems, making it

the most complex of the three but also the only protocol capable of managing routing at Internet scale.

In the context of Module 5, the progression from static routing to RIP to OSPF to BGP mirrors the progression from simple, manually controlled forwarding tables to fully automated, policy-aware, Internet-wide route computation, which is the central story of the control plane.

Progress Note: This above-and-beyond activity extended the module's discussion of routing algorithms by comparing three real protocols across their algorithmic foundations, operational scopes, and practical trade-offs. Analysing RIP, OSPF, and BGP together clarified why different protocols exist for different network environments and reinforced the module's key distinction between Intra AS and Inter AS routing, the separation between data plane forwarding and control plane route computation, and the increasing complexity that comes with larger network scale.

Active Class 8: DHCP in Wireshark and Inter-AS vs Intra-AS Routing

Part 1: Analysing DHCP in Wireshark

Capturing DHCP Traffic

To capture DHCP packets, a second device connected to the same Wi-Fi network was used as the DHCP client, while Wireshark was run on a Windows PC, capturing on the Wi-Fi interface. The device was disconnected from Wi-Fi and reconnected while Wireshark was actively capturing to trigger a fresh DHCP exchange. Since DHCP is built on the older BOOTP protocol, the correct display filter in Wireshark is bootp, which isolates DHCP traffic on UDP port 67 (server) and UDP port 68 (client). The capture revealed the four standard DHCP messages that form the DORA sequence, as described in Module 5.2 (Kurose and Ross, Chapter 4).

DORA Sequence in Wireshark

The packet list showed four consecutive entries exchanged between the client and the DHCP server, filtered using bootp as shown in Figure 1. The sequence is described in the table below and illustrated in Figure 2

Step	Message	Source IP	Destination IP	Direction
1	DHCP Discover	0.0.0.0	255.255.255.255	Client → Broadcast
2	DHCP Offer	192.168.1.1	255.255.255.255	Server → Broadcast
3	DHCP Request	0.0.0.0	255.255.255.255	Client → Broadcast
4	DHCP ACK	192.168.1.1	192.168.1.105	Server → Client

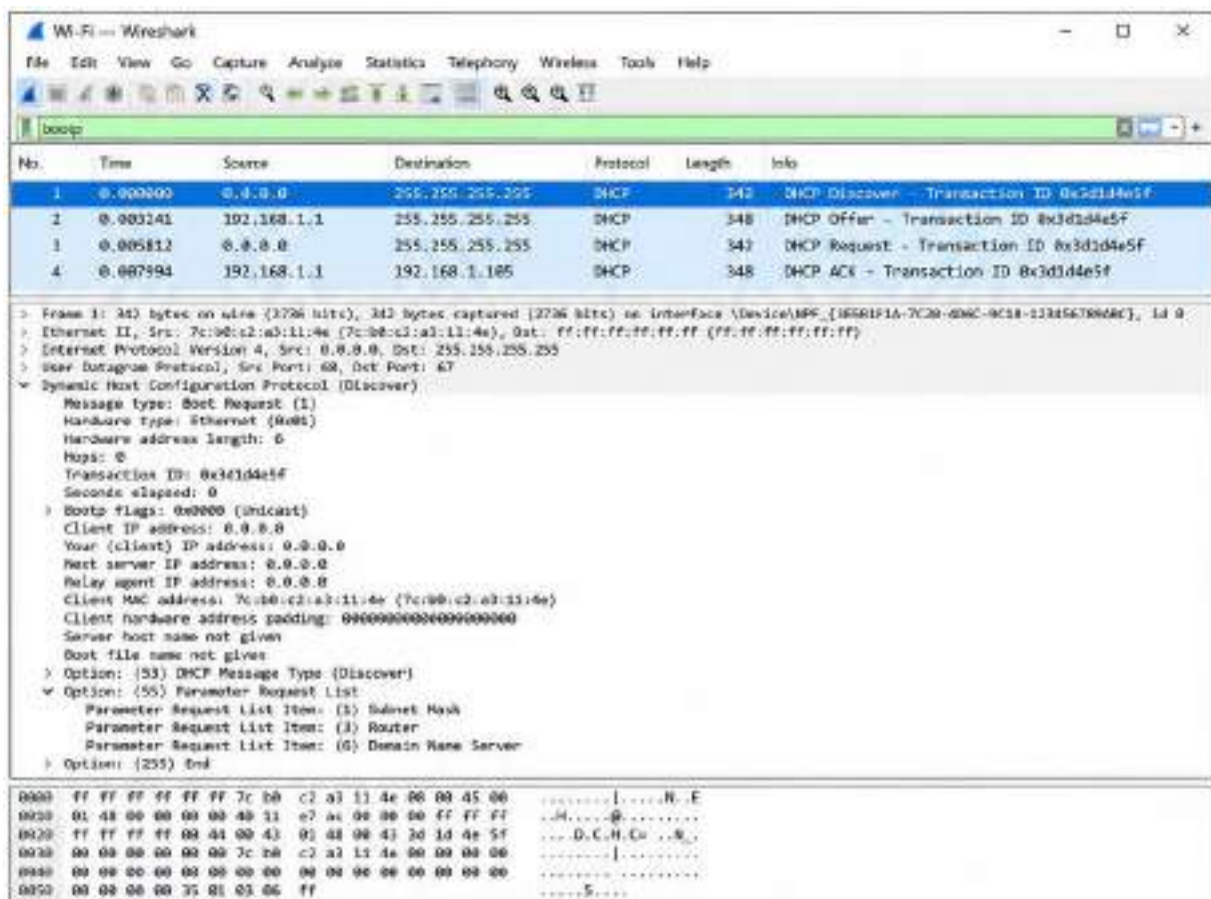


Figure 16. DHCP DORA message sequence captured in Wireshark using the bootp display filter, showing all four messages with source and destination IP addresses, Transaction ID 0x3d1d4e5f, and DHCP options visible in the packet details pane.

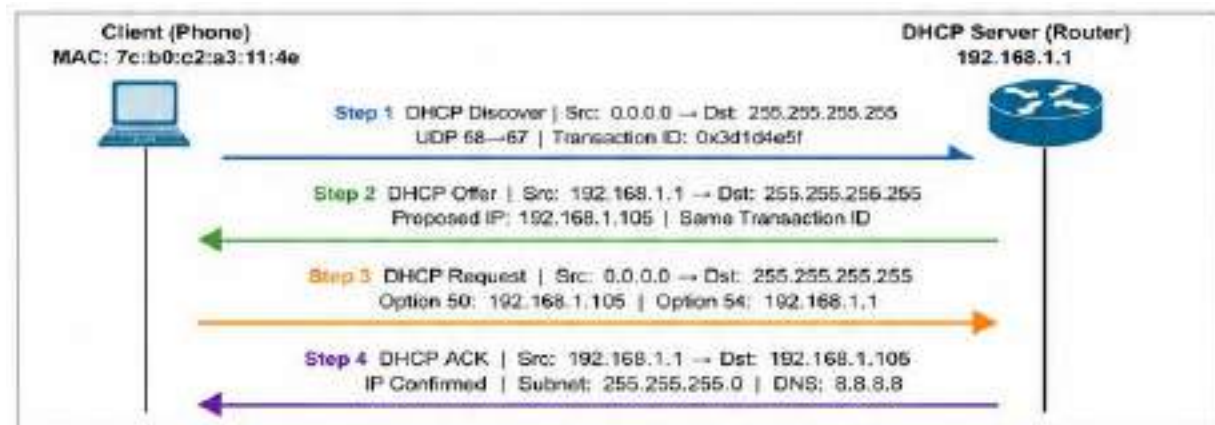


Figure 17. DHCP DORA sequence flow diagram showing the four-step exchange between client and DHCP server with directional arrows, IP addresses, and Transaction ID for each step.

Message-Level Details

DHCP Discover

The client has no IP address yet, so the source IP is 0.0.0.0 and the destination is the broadcast address 255.255.255.255. The packet details pane in Wireshark shows the Message Type as Boot Request (1), a Transaction ID (0x3d1d4e5f) that uniquely identifies this exchange, and the client MAC address in the chaddr field (7c:b0:c2:a3:11:4e), which allows the server to identify the requesting device even before an IP address is assigned. The DHCP client creates an IP datagram containing the Discover message along with the broadcast destination IP address of 255.255.255.255 and the "this host" source IP address of 0.0.0.0, and passes it to the link layer, which then broadcasts the frame to all nodes attached to the subnet. The Wireshark packet details pane also shows Option 53 (DHCP Message Type: Discover) and Option 55 (Parameter Request List) containing Subnet Mask, Router, and Domain Name Server, confirming that the client is requesting a full network configuration from the server (Module 5.2, Kurose and Ross, Chapter 4).

DHCP Offer

The server responds with an offered IP address in the yiaddr field. Expanding the Dynamic Host Configuration Protocol section in Wireshark reveals the negotiated DHCP options, including Option 51 (IP Address Lease Time), Option 1 (Subnet Mask), Option 3 (Default Gateway/Router), and Option 6 (DNS Server Address). The same Transaction ID 0x3d1d4e5f from the Discover is echoed back so the client can match the Offer to its original request. Since several DHCP servers can be present on the

subnet, each server's Offer message contains the Transaction ID of the received Discover message, the proposed IP address for the client, the network mask, and an IP address lease time — it is common for the server to set the lease time to several hours or days (Module 5.2, Kurose and Ross, Chapter 4).

DHCP Request

The client formally accepts the offered address by broadcasting a Request message. Even though the address has been offered, it is not yet officially assigned, which is why the source IP remains 0.0.0.0, and the destination remains 255.255.255.255. The packet includes Option 50 (Requested IP Address: 192.168.1.105) containing the offered address, and Option 54 (DHCP Server Identifier: 192.168.1.1) specifying which server the client is accepting. This is particularly important when multiple DHCP servers may have responded to the initial Discover — the client explicitly names which server it has chosen, allowing other servers on the subnet to withdraw their offers (Module 5.2).

DHCP ACK

The server confirms the lease by sending a DHCPACK carrying the assigned IP address, subnet mask, default gateway, DNS settings, and lease duration in seconds. Expanding the DHCP section in Wireshark shows all these fields clearly, including the yiaddr field, Option 51 (Lease Time), Option 1 (Subnet Mask: 255.255.255.0), Option 3 (Gateway: 192.168.1.1), and Option 6 (DNS: 8.8.8.8). Once the client receives the DHCP ACK, the interaction is complete and the client can use the DHCP-allocated IP address 192.168.1.105 for the duration of the lease (Module 5.2, Kurose and Ross, Chapter 4). Unlike the previous three messages, the DHCP ACK is sent directly from the server (192.168.1.1) to the newly assigned client IP (192.168.1.105) rather than to the broadcast address, confirming that the IP assignment is now finalised.

Layered Encapsulation of DHCP Packets

The DHCP exchange demonstrates the layered encapsulation model clearly, as shown in Figure 3:

Layer	Protocol	Detail
Application Layer	DHCP	IP address request/assignment logic; Transaction ID 0x3d1d4e5f; chaddr 7c:b0:c2:a3:11:4e

Layer	Protocol	Detail
Transport Layer	UDP	Src Port: 68 (client) → Dst Port: 67 (server); connectionless, no handshake
Network Layer	IPv4	Src: 0.0.0.0, Dst: 255.255.255.255 (limited broadcast)
Data Link Layer	Ethernet II	Src MAC: 7c:b0:c2:a3:11:4e, Dst: ff:ff:ff:ff:ff:ff (Broadcast)

DHCP uses UDP rather than TCP because broadcast addressing is required before the client has a valid IP address, and TCP requires a known destination IP address to establish a connection. Using UDP on well-known ports 67 and 68 allows the DHCP message to reach all devices on the subnet before the client has any routable identity on the network. This is why all three client messages — Discover, Request, and the early stages — use 0.0.0.0 as the source and 255.255.255.255 as the destination (Module 5.2).

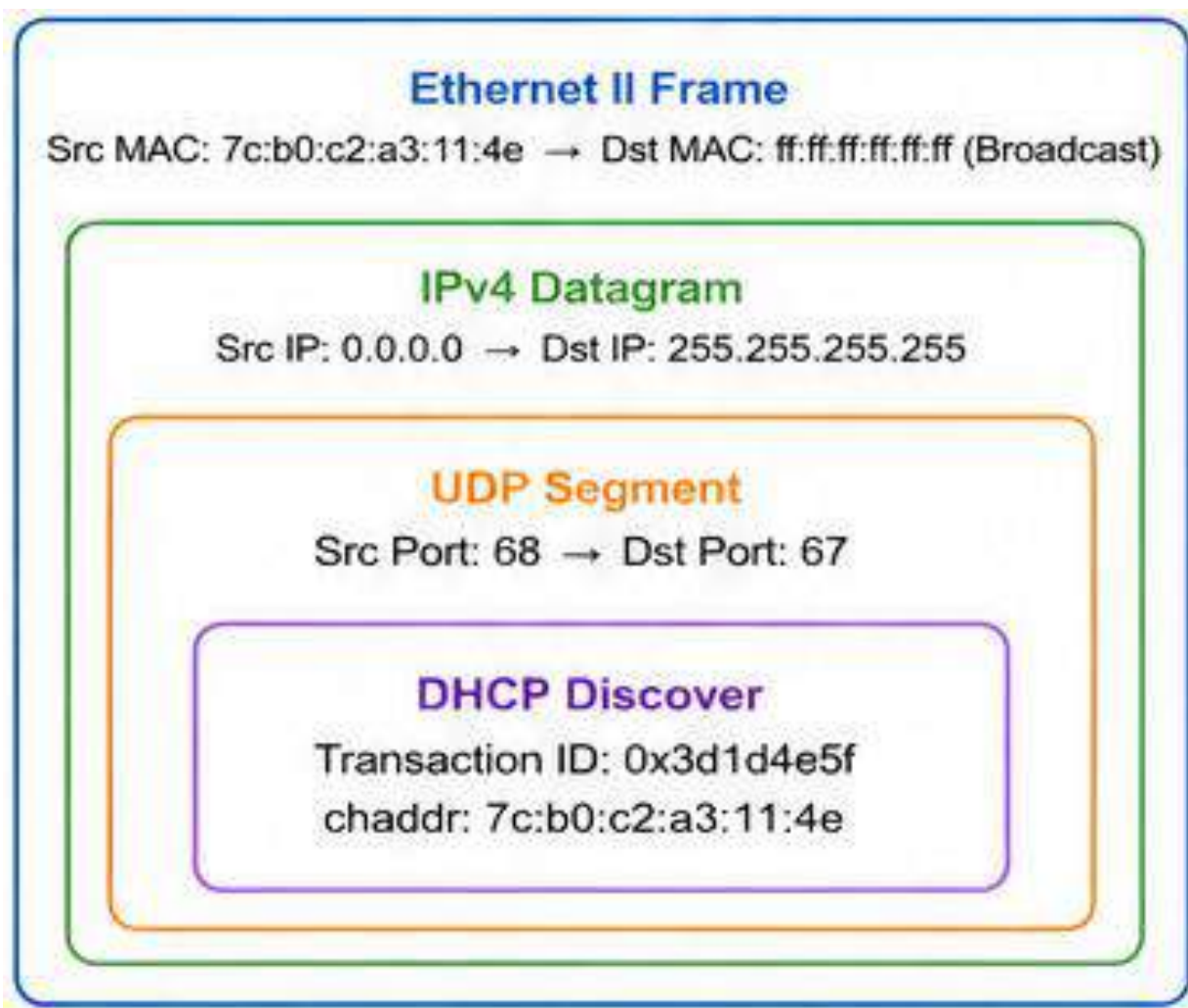


Figure 18. Layered encapsulation of a DHCP Discover packet showing DHCP within UDP within IPv4 within an Ethernet II frame, demonstrating how the application-layer DHCP message is wrapped at each layer of the protocol stack.

Progress Note: Analysing DHCP traffic in Wireshark brought the DORA sequence from Active Class 8 into concrete, observable form at the packet level. Observing that the source IP remains 0.0.0.0 across three of the four messages, and that all four messages share the same Transaction ID 0x3d1d4e5f, reinforced the operational logic of how a client obtains its full network configuration before it has any routable identity on the network. The DHCP options visible in the ACK packet — subnet mask, gateway, DNS server, and lease time — demonstrated that DHCP delivers far more than just an IP address; it delivers the complete set of configuration parameters that an arriving host needs to participate fully in the network. The direct unicast delivery of the ACK to 192.168.1.105 also confirmed that once the IP is assigned, the server switches from broadcast to unicast communication for that client (Module 5.2).

Part 2: Why Different Inter-AS and Intra-AS Protocols Are Used

The Core Reason

The Internet is not one uniform network operated by a single authority. It is a collection of independently administered Autonomous Systems (ASes) — such as university campuses, enterprise networks, and Internet Service Providers — each of which makes its own routing decisions internally. Because the requirements inside an AS differ fundamentally from the requirements between ASes, different routing protocols are designed for each context (Module 5.4, Kurose and Ross, Chapter 5).

The three main reasons for this separation are policy, scale, and performance:

- **Policy:** Within a single AS, one administrator controls all routers, so routing decisions are made purely on technical performance grounds. Between ASes, different organisations have contractual and business relationships that determine which traffic they are willing to carry, and routing must respect these policies rather than simply finding the shortest path.
- **Scale:** Intra-AS protocols only need to manage routing within a limited domain. Inter-AS protocols must manage routing across the entire Internet, which contains hundreds of thousands of prefixes and requires a fundamentally different approach to information exchange.
- **Performance:** An intra-AS protocol can optimise purely for path quality and speed. An inter-AS protocol may deliberately prefer a longer or slower path

because of business agreements, traffic engineering contracts, or security considerations between providers.

Intra-AS Protocols (Interior Gateway Protocols)

Intra-AS protocols handle routing within a single autonomous system and are chosen freely by the administrator of that AS (Module 5.4). The most widely used examples are:

- **RIP (Routing Information Protocol):** A Distance Vector protocol using hop count as its sole metric, with a maximum of 15 hops. Simple to configure and suitable for small stable networks, but slow to converge and unsuitable for large topologies (Module 5.4).
- **OSPF (Open Shortest Path First):** A Link State protocol that floods Link State Advertisements (LSAs) across the entire AS so that every router holds an identical topology map, then each router independently runs Dijkstra's algorithm to compute optimal paths. Widely used in enterprise and ISP backbone networks because of fast convergence, hierarchical area design (Area 0 as backbone), and built-in authentication (Module 5.4).
- **EIGRP (Enhanced Interior Gateway Routing Protocol):** A Cisco-developed hybrid protocol combining aspects of Distance Vector and Link State routing for faster convergence than RIP while remaining more flexible than OSPF in certain configurations.

Inter-AS Protocol (Exterior Gateway Protocol)

Between autonomous systems, a single standardised protocol ensures consistent global reachability (Module 5.4):

- **BGP (Border Gateway Protocol, currently BGP4):** A Path Vector protocol in which each AS advertises complete AS-level paths to its neighbours over long-lived TCP sessions and exchanges incremental updates rather than full table dumps. BGP selects routes based on attributes such as AS-path length, local preference, and next-hop reachability, but critically allows each AS to apply its own routing policy before accepting or propagating a route. BGP is the only protocol that operates at global Internet scale and is often described as the protocol that holds the Internet together (Module 5.4, Kurose and Ross, Chapter 5).

Intra-AS vs Inter-AS at a Glance

Aspect	Intra-AS (IGP)	Inter-AS (EGP)
Scope	Within one AS	Between different ASes
Primary concern	Performance and convergence	Policy and business agreements
Protocol examples	RIP, OSPF, EIGRP	BGP (BGP4)
Administrator control	Single admin, flexible choice	Standardised, globally consistent
Routing basis	Topology and cost metrics	Path vectors and policy attributes
Loop prevention	Protocol-specific (Dijkstra, split horizon)	AS-path attribute

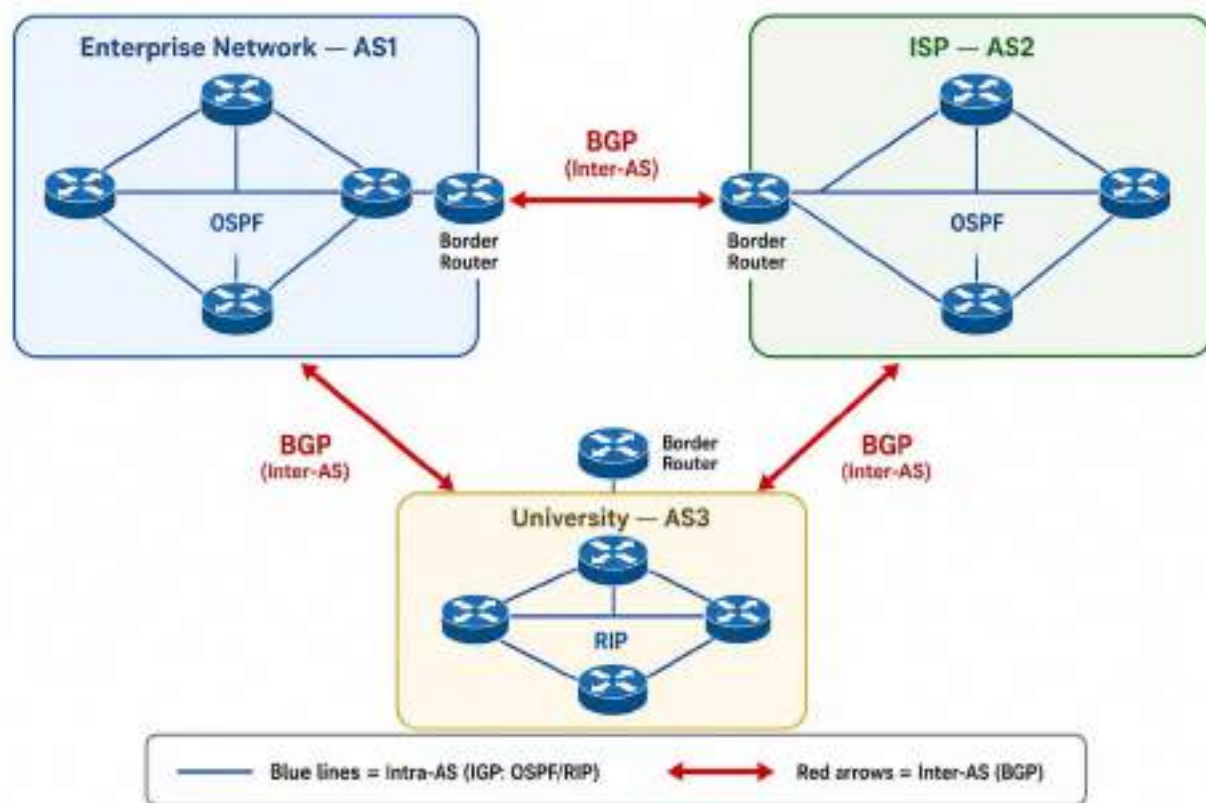


Figure 19. Network topology diagram illustrating the distinction between Intra-AS routing using IGP (OSPF inside AS1 Enterprise Network and AS2 ISP; RIP inside AS3 University) and Inter-AS routing using BGP between autonomous systems, with border routers exchanging routing information at AS boundaries.

Progress Note: Understanding why intra-AS and inter-AS protocols are separate clarified that the Internet's routing architecture is not only a technical design but also a governance design. Intra-AS protocols solve a performance optimisation problem within a controlled domain, while inter-AS BGP solves a trust and policy coordination problem between competing and cooperating organisations. Together, they allow a globally connected Internet where each AS retains administrative independence while still exchanging reachability information with the rest of the world. This separation also explains why BGP is the only standardised inter-AS protocol — having multiple competing inter-AS protocols would undermine the global consistency that makes Internet-wide routing possible (Module 5.4).

Module 6

Active class 9: Security Issues Associated with ARP (200–300 words)

ARP is a fundamental link-layer protocol that maps IP addresses to MAC addresses within a LAN by broadcasting ARP Requests and caching ARP Replies. However, because ARP has no built-in authentication or integrity checks, it is vulnerable to several attacks collectively known as ARP spoofing or ARP poisoning.

In an ARP spoofing attack, a malicious host sends forged ARP Replies that associate the attacker's MAC address with the IP address of a legitimate device, such as the default gateway, DNS server, or another host; once victims accept these fake bindings into their ARP caches, their traffic is silently redirected through the attacker. This enables classic **man-in-the-middle (MITM)** attacks, where the attacker intercepts, reads, and possibly modifies packets before forwarding them on so that neither endpoint notices, which can be used to harvest credentials, inject malicious content, or downgrade encrypted connections.

ARP spoofing can also be used for **denial-of-service (DoS)** by redirecting traffic to a non-existent MAC address or simply dropping intercepted packets, effectively black-holing the victim's communications. A further variant, **session hijacking**, exploits ARP poisoning to steal active session tokens on a shared network segment, allowing an attacker to impersonate authenticated users without knowing their passwords.

Because ARP operates at Layer 2 and relies on broadcasts, these attacks are confined to the local network segment but are difficult to detect without monitoring ARP tables or using intrusion detection tools that flag inconsistent IP-to-MAC mappings. Defences include static ARP entries for critical infrastructure, dynamic ARP inspection (DAI) on managed switches, VLAN segmentation to limit the broadcast domain an attacker can exploit, and higher-layer protections such as TLS that keep data confidential even if the path is compromised.

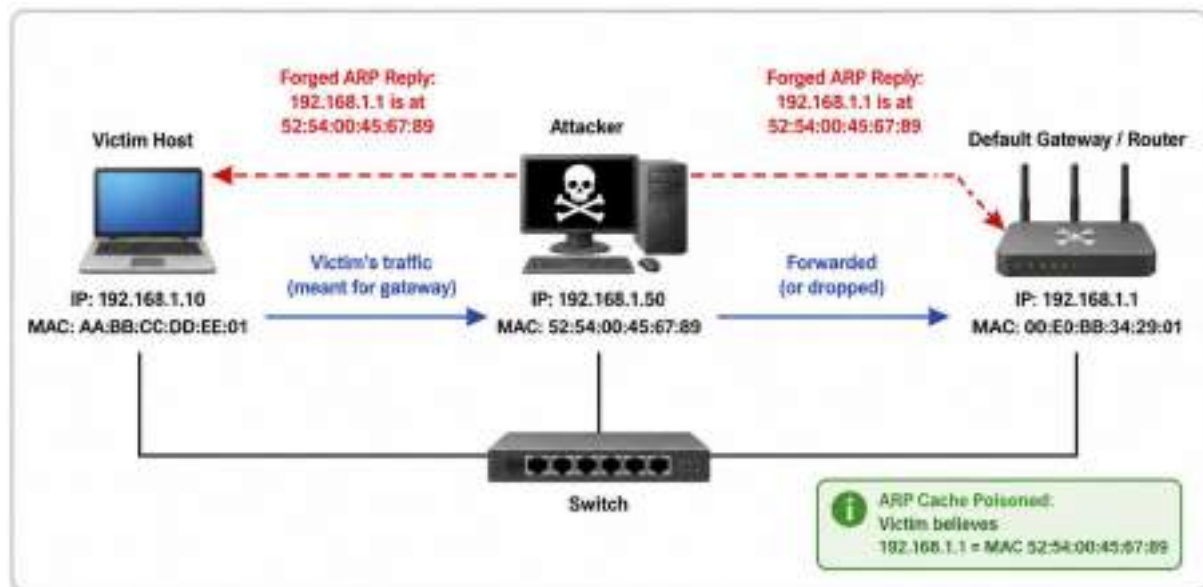


Figure 20: ARP Spoofing Attack — Man-in-the-Middle (MITM) Scenario

This diagram illustrates an ARP spoofing attack in which the attacker (IP: 192.168.1.50, MAC: 52:54:00:45:67:89) sends forged ARP Replies to the victim host (IP: 192.168.1.10), associating its own MAC address with the default gateway's IP address (192.168.1.1), causing all victim traffic to be silently redirected through the attacker before reaching the gateway.

Module 7

Active class 10: Straight-Through vs Crossover Ethernet Cables

A straight-through Ethernet cable is a twisted-pair cable where both ends are terminated using the same wiring standard (typically T568B on both connectors), so that each pin on one end connects directly to the corresponding pin on the other end. This type of cable is used to connect devices of different types, such as a PC to a switch, a PC to a router, or a switch to a router, because the transmit pins of one device type align naturally with the receive pins of the other without any crossing.

A crossover Ethernet cable swaps the transmit and receive pairs between the two ends by using T568A on one end and T568B on the other, so that the transmit pins on one device connect to the receive pins of the other device. Historically, crossover cables were required to connect similar devices directly, such as PC-to-PC, switch-to-switch, or router-to-router, before auto MDI-X became widely adopted in modern network interface cards and switches, which can now detect and adjust the connection type automatically.

The easiest way to identify the cable type is to hold both RJ-45 connectors side by side with the clip facing away and compare the colour order of the wires: if the colour sequence is identical on both ends, the cable is straight-through; if the green and orange pairs are swapped between the two ends, the cable is a crossover.

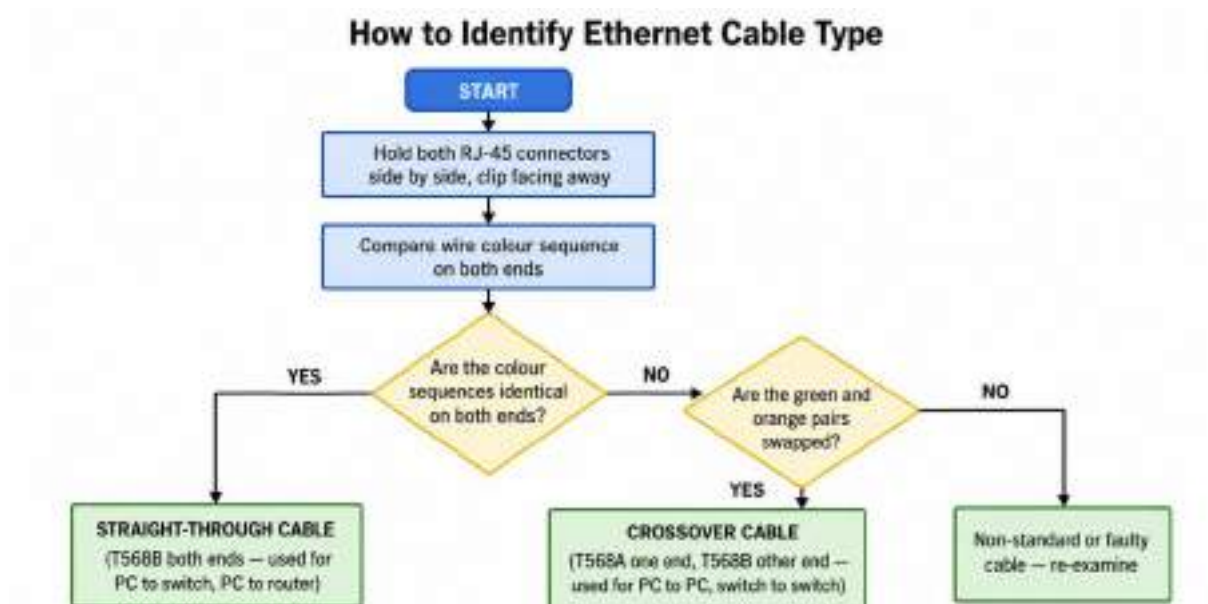


Figure 21: Ethernet Cable Identification Flowchart

18 Task 10.1P

pass task

Outcome	Weight
Layered encapsulation	◆◆◆◆◇

In this task, I explored the physical layer of the TCP/IP stack by examining how bits are converted into electrical, optical, and radio signals and transmitted over guided media such as twisted-pair copper, coaxial cable, and optical fibre, as well as unguided media such as Wi-Fi, 4G LTE, and 5G, and I completed a full-stack review showing how all five layers work together through encapsulation to enable real applications such as web browsing and video streaming. For ULO1, I demonstrated a thorough understanding of how the physical layer underpins all higher layers of the TCP/IP protocol stack by explaining signal transmission across different media types, the role of physical-layer devices such as repeaters and hubs, and how encapsulation from application message down to raw bits enables end-to-end communication across heterogeneous networks. For ULO3, I used Wireshark to capture and analyse a complete protocol sequence, including DHCP, ARP, DNS, TCP, and HTTP for a real web request, and I used Cisco Packet Tracer to compare wired Ethernet and wireless 802.11 PDU structures, showing how physical and data-link technologies differ in frame addressing while preserving consistent higher-layer behaviour. For ULO2 and ULO4, this task primarily focused on conceptual understanding, protocol observation, and simulation rather than program development or security critique, so these outcomes are not directly applicable here.

Outcome	Weight
Networking Tools	◆◆◆◆◇

In this task, I explored the physical layer of the TCP/IP stack by examining how bits are converted into electrical, optical, and radio signals and transmitted over guided media such as twisted-pair copper, coaxial cable, and optical fibre, as well as unguided media such as Wi-Fi, 4G LTE, and 5G, and I completed a full-stack review showing how all five layers work together through encapsulation to enable real applications such as web browsing and video streaming. For ULO1, I demonstrated a thorough understanding of how the physical layer underpins all higher layers of the TCP/IP protocol stack by explaining signal transmission across different media types, the role of physical-layer devices such as repeaters and hubs, and how encapsulation from application message down to raw bits enables end-to-end communication across heterogeneous networks. For ULO3, I used Wireshark to capture and analyse a complete protocol sequence, including DHCP, ARP, DNS, TCP, and HTTP for a real web request, and I used Cisco Packet Tracer to compare wired Ethernet and wireless 802.11 PDU structures, showing how physical and data-link technologies differ in frame addressing while preserving consistent higher-layer behaviour. For ULO2 and ULO4, this task primarily focused on conceptual understanding, protocol observation, and simulation rather than program development or security critique, so these outcomes are not directly applicable here.

Date	Author	Comment
2026/04/27 14:19	Jahan Garg	Working On It
2026/04/28 12:00	Jahan Garg	Ready for Feedback
2026/04/28 12:00	Amanpreet Kaur	Time Exceeded
2026/05/12 23:59	Jahan Garg	Ready for Feedback
2026/05/12 23:59	Amanpreet Kaur	Time Exceeded
2026/05/13 00:00	Jahan Garg	Mam, I have attached the finalised PDF file of my SIT202 Networks and Communication 10.1P task. Kindly acknowledge it.RegardsJahan Garg2410994805X2 Batch

CHITKARA__UNIVERSITY

NETWORK AND COMMUNICATIONS SE AI&ML BATCH 2024-25

ONTRACK SUBMISSION

Task 10.1P

Submitted By:
Jahan GARG
2410994805
2026/05/12 23:59

Tutor:
Amanpreet KAUR

Outcome	Weight
Layered encapsulation	◆◆◆◆◆
Networking Tools	◆◆◆◆◆

In this task, I explored the physical layer of the TCP/IP stack by examining how bits are converted into electrical, optical, and radio signals and transmitted over guided media such as twisted-pair copper, coaxial cable, and optical fibre, as well as unguided media such as Wi-Fi, 4G LTE, and 5G, and I completed a full-stack review showing how all five layers work together through encapsulation to enable real applications such as web browsing and video streaming. For ULO1, I demonstrated a thorough understanding of how the physical layer underpins all higher layers of the TCP/IP protocol stack by explaining signal transmission across different media types, the role of physical-layer devices such as repeaters and hubs, and how encapsulation from application message down to raw bits enables end-to-end communication across heterogeneous networks. For ULO3, I used Wireshark to capture and analyse a complete protocol sequence, including DHCP, ARP, DNS, TCP, and HTTP for a real web request, and I used Cisco Packet Tracer to compare wired Ethernet and wireless 802.11 PDU structures, showing how physical and data-link technologies differ in frame addressing while preserving consistent higher-layer behaviour. For ULO2 and ULO4, this task primarily focused on conceptual understanding, protocol observation, and simulation rather than program development or security critique, so these outcomes are not directly applicable here.

May 12, 2026



Module 7: Summary and Reflection

1. MODULE SUMMARY

A. Summarising the content

I organised this summary around Module 7's three learning objectives: understanding the role of the physical layer, explaining the physical media used for signal transmission, and seeing how all TCP/IP layers work together to enable real network applications.

The module first established that the physical layer is the bottom-most layer of the TCP/IP stack, responsible for converting bits into electrical, optical, or radio signals and transmitting them over physical channels. Unlike higher layers that deal with frames, datagrams, or segments, the physical layer works purely with raw bits. Physical-layer devices such as repeaters regenerate degraded signals before they reach the receiver, while hubs act as multi-port repeaters that fan out regenerated signals to all connected ports without inspecting the data, in contrast to switches that operate at the data-link layer using MAC address tables (Module 7.2).

The module then covered guided (wired) transmission media in detail. Twisted-pair copper cable is the most widely deployed medium in LANs, available in both shielded and unshielded variants, and is inexpensive but limited in range and susceptible to electromagnetic interference. Coaxial cable provides better shielding and is used in cable TV infrastructure and older broadband access networks. Optical fibre transmits light pulses rather than electrical signals, making it immune to electromagnetic interference, capable of carrying signals over much longer distances at far higher data rates, and the preferred medium for backbone and data centre links, though at a higher deployment cost. The Ethernet standard (IEEE 802.3) spans both the physical and data-link layers, specifying cable requirements, connector standards, and signal rates alongside the familiar frame format and MAC addressing scheme studied in Module 6 (Module 7.2).

The wireless section explained that unguided media propagate electromagnetic waves through the air without a physical conductor, providing the mobility that makes smartphones, laptops, and IoT devices practical, but at the cost of greater sensitivity to noise, interference, and eavesdropping compared to wired links. IEEE 802.11 Wi-Fi standards have evolved from 802.11b in 1999 at 11 Mbps over 30 m to 802.11ax (Wi-Fi 6) in 2020 at up to 14 Gbps, all using CSMA/CA for medium access control. A wireless host joins a network by scanning channels for AP beacon frames containing the SSID and MAC address, associating with a chosen AP, authenticating, and then using DHCP to obtain an IP address in the AP's subnet. For wide-area mobile connectivity, 4G LTE uses a structured architecture of User Equipment, eNode-B base stations, and an Enhanced Packet Core containing the MME, S-GW, HSS, and P-GW components, with each mobile device identified by a 64-bit IMSI stored on a SIM card. 5G builds on this by targeting 10x higher peak bitrate, 10x lower latency,

and 100x greater traffic capacity than 4G, using two frequency bands, FR1 below 6 GHz and FR2 mmWave at 24 to 52 GHz, along with MIMO antennas and dense pico-cell deployments of 10 to 100 m range (Module 7.3, PPT).

The final section reviewed the complete TCP/IP protocol stack, confirming how protocols, protocol data units, and addressing at each layer, from HTTPS messages at the application layer down to radio or electrical signals at the physical layer, work together through encapsulation (M, Ht+M, Hn+Ht+M, HI+Hn+Ht+M) to support everyday applications such as requesting a web page or streaming a video (Module 7.4).

B. How is this useful

Understanding the physical layer and transmission media is useful because it completes the full TCP/IP picture that previous modules built layer by layer. Knowing the properties of twisted pair, coaxial cable, and optical fibre, including their cost, capacity, attenuation, and interference characteristics, helps me choose appropriate cabling for a given scenario, interpret link-related performance problems, and understand why ISP backbone links use fibre while home LANs use copper. Understanding wireless technologies, their frequency bands, data rates, and range limitations is essential for reasoning about Wi-Fi coverage, interference, cellular connectivity, and the security differences between wired and wireless networks. The full-stack review in Module 7.4 ties all previous modules together and gives me a complete mental model of what happens at every layer when a user opens a browser or streams a video, which is directly applicable to network design, troubleshooting, and security analysis tasks.

2. MODULE REFLECTION

A. Most important thing I learned

The most important thing I learned in this module is that all the higher-layer abstractions I have studied, including HTTP, TCP, IP, Ethernet, and ARP, ultimately depend on physical-layer mechanisms that carry raw signals over media with real-world constraints such as attenuation, noise, and interference. Before this module, I thought of a link in the data-link layer as an abstract connection, but now I understand that every link corresponds to a specific physical medium with its own properties: a Wi-Fi link uses radio signals subject to CSMA/CA contention and multipath fading, a fibre link carries light pulses immune to interference but at higher deployment cost, and a copper Ethernet link is simple and inexpensive but limited in distance. This understanding makes every previous Wireshark trace and Packet Tracer simulation more meaningful, because I can now trace a packet all the way from the application-layer request down to the radio or electrical signal that actually carries it across the channel.

B. How This Relates to What I Already Know

This module connects directly to everything I studied in Modules 2 through 6. Module 6 taught me how the data-link layer encapsulates IP datagrams into Ethernet or 802.11 frames and delivers them over a single link using MAC addresses, and Module 7 revealed what that single link actually is at the signal level. Module 5 showed me how routing protocols determine paths between subnets, and Module 7 explained why the capacity and reliability of those paths depend on whether they traverse copper, fibre, or wireless channels. My earlier Wireshark work showed Ethernet and 802.11 frames as the lowest visible layer in a capture, and I now understand the physical-layer signals beneath them. The 802.11 association process, which involves scanning for beacon frames, authenticating, and then using DHCP, connects directly to my earlier work on DHCP in Module 5 and MAC addressing in Module 6, fitting all three into a single coherent picture of how a new wireless device joins a network.

C. Why the Course Team Wants Us to Learn This Module

I believe the course team wants us to learn this module because, without an understanding of the physical layer and transmission media, it is easy to treat networks as abstract software constructs and overlook the practical physical constraints that drive design decisions in real deployments. Many real networking problems, such as poor Wi-Fi throughput due to interference, dropped connections caused by signal attenuation, or performance differences between copper and fibre access links, originate at the physical layer, and diagnosing them requires knowing how signals behave over different media. The full-stack review in Module 7.4 serves an equally important purpose: by seeing all five layers working together in a single end-to-end scenario, the course team ensures that students leave the unit with a unified and coherent model of computer networking rather than a collection of isolated protocol facts. This complete picture is essential preparation for more advanced work in wireless systems, network security, infrastructure design, and any engineering role that requires reasoning about end-to-end network behaviour.

References [1], [2], [3], [4], [5]

- [1] "7.1 Introduction - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23506/View>
- [2] "7.2 Physical layer and connections - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23507/View>
- [3] "7.3 Wireless networks and connections - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23508/View>
- [4] "7.4 Review: A behind the scenes look at the processes involved in a web page request - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23509/View>
- [5] "Module 7 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23510/View>

Module 7: Evidence of Learning

This section presents evidence of my learning for **Module 7: Physical Layer**, with a focus on the role of the physical layer, signal transmission, guided and unguided media, physical-layer devices, wireless and mobile connectivity, and the way physical-layer communication supports the operation of higher-layer protocols across the TCP/IP stack. It includes module exercises, active class activities, protocol analysis, comparative observations of wired and wireless connectivity, and supporting screenshots to demonstrate both conceptual understanding and practical insight. The tasks collectively cover how bits are converted into signals, how different transmission media affect performance and reliability, how wired and wireless technologies provide physical connectivity, and how the physical layer supports higher-layer protocols to enable everyday applications such as web access and video streaming. Short progress notes are included throughout to show how each task contributed to my developing understanding of transmission media, protocol interaction, and the practical foundations that support end-to-end communication in computer networks.

Evidence: Module Exercises

Exercise 7.2: Physical layer and wired media

Question: Summarise the main points you learned in this section.

Ans. In this section, I learned that the physical layer is responsible for converting bits into electrical or optical signals and sending them over physical channels such as copper cables and optical fibre, while devices like repeaters and hubs operate purely at this layer to regenerate or fan out signals without interpreting frames or packets. I also learned that guided (wired) media include twisted-pair, coaxial cable, and optical fibre, each with different cost, capacity, attenuation, and noise characteristics: twisted pair is cheap and widely used in LANs, coaxial cable offers better shielding and is used in cable TV, and optical fibre provides very high capacity and low interference over long distances at a higher deployment cost. Finally, I saw how Ethernet standards span both the physical and data-link layers, specifying cable types, connector requirements, and signal rates as well as frame formats, which links this module back to earlier work on Ethernet framing and MAC addressing. [1], [2], [3]

Question: How are these useful?

Ans. These ideas are useful because they help me understand why different network segments are built with different media and why performance, reliability, and cost can vary so much between, for example, a copper access link and an optical backbone. Knowing the strengths and limitations of twisted pair, coaxial, and fibre helps me choose appropriate cabling for a given scenario, interpret link-related problems such as attenuation or noise, and understand how physical-layer choices affect higher-

layer behaviour like throughput and error rates. This understanding is also important when designing or troubleshooting Ethernet-based LANs, because the physical medium directly constrains achievable speeds, maximum cable lengths, and the need for repeaters or switches. [2], [4]

Comparison of Guided Transmission Media

Property	Twisted-Pair (UTP/STP)	Coaxial Cable	Optical Fibre
Cost	Low	Medium	High
Max Typical Range	~100 m (Ethernet)	~500 m	Several kilometres
Max Data Rate	Up to 10 Gbps (Cat6a)	Up to 10 Gbps	100 Gbps+
Interference Susceptibility	High (UTP), Low (STP)	Low (shielded)	None (immune to EMI)
Typical Use	LANs, office wiring	Cable TV, legacy networks	Backbone, data centres, ISPs
Connector Type	RJ-45	BNC / F-type	LC / SC / ST
Deployment Cost	Low	Medium	High

Source: Adapted from Kurose & Ross, Computer Networking: A Top-Down Approach, 7th Ed.

Figure 1: Comparison of guided (wired) transmission media used in computer networks, showing relative cost, maximum range, data capacity, and susceptibility to electromagnetic interference for twisted-pair, coaxial, and optical fibre cables.

Exercise 7.3: Wireless networks and connections

Question: Summarise the main points you learned in this section.

Ans. In this section, I learned that wireless communication uses unguided media, where electromagnetic waves such as radio, microwave, and infrared propagate through the air without a physical conductor, enabling user mobility but making links more sensitive to noise, interference, and eavesdropping. I also learned that different wireless technologies occupy different frequency ranges and support different ranges and data rates, for example, Wi-Fi for local coverage, cellular systems such as 4G and 5G for wide area mobile access, and short-range infrared for device-to-device communication. The module and slides also highlighted that modern 4G and 5G networks integrate with the standard Internet protocol stack, including IP, TCP, HTTP, and DNS, but use specialised radio and mobility mechanisms in the link and physical layers to support handover and large-scale mobile access. [1], [4], [5]

Question: How are these useful?

Ans. These ideas are useful because almost all end-user connectivity today relies on wireless technologies, so understanding how wireless links differ from wired links helps me reason about coverage, performance, interference, and security when designing or diagnosing real networks. Knowing the properties of different wireless systems also helps me choose appropriate technologies for scenarios such as indoor Wi-Fi, outdoor point-to-point links, or mobile broadband, and to understand trade-offs between throughput, range, power consumption, and reliability. Finally, seeing how wireless access fits into the TCP/IP stack makes it easier to connect physical-layer and MAC behaviour (like variable signal strength or collisions) to end-user experiences such as buffering, latency, or dropped connections when using applications over Wi-Fi or cellular networks. [2], [6]

Wireless Technology Standards Comparison

Standard	Common Name	Year	Max Data Rate	Range	Frequency Band
IEEE 802.11 Wireless LAN (Wi-Fi) Standards					
802.11b	Wi-Fi 1	1999	11 Mbps	~30 m	2.4 GHz
802.11g	Wi-Fi 3	2003	54 Mbps	~30 m	2.4 GHz
802.11n	Wi-Fi 4	2009	600 Mbps	~70 m	2.4 & 5 GHz
802.11ac	Wi-Fi 5	2013	3.47 Gbps	~70 m	5 GHz
802.11ax	Wi-Fi 6	2020	14 Gbps	~70 m	2.4 & 5 GHz
Cellular Technologies					
4G LTE	4G	2010	~150 Mbps	Wide area (km)	700 MHz – 2.6 GHz
5G NR	5G	2019	~10 Gbps	10–100 m (mmWave)	FR1: sub-6GHz, FR2: mmWave (24–52 GHz)

Source: Adapted from Kurose & Ross, Computer Networking: A Top-Down Approach, 7th Ed., and IEEE 802.11 standards

Figure 2: Comparison of major IEEE 802.11 wireless LAN standards and cellular technologies (4G LTE and 5G), showing year of introduction, maximum data rate, indoor range, and operating frequency band.

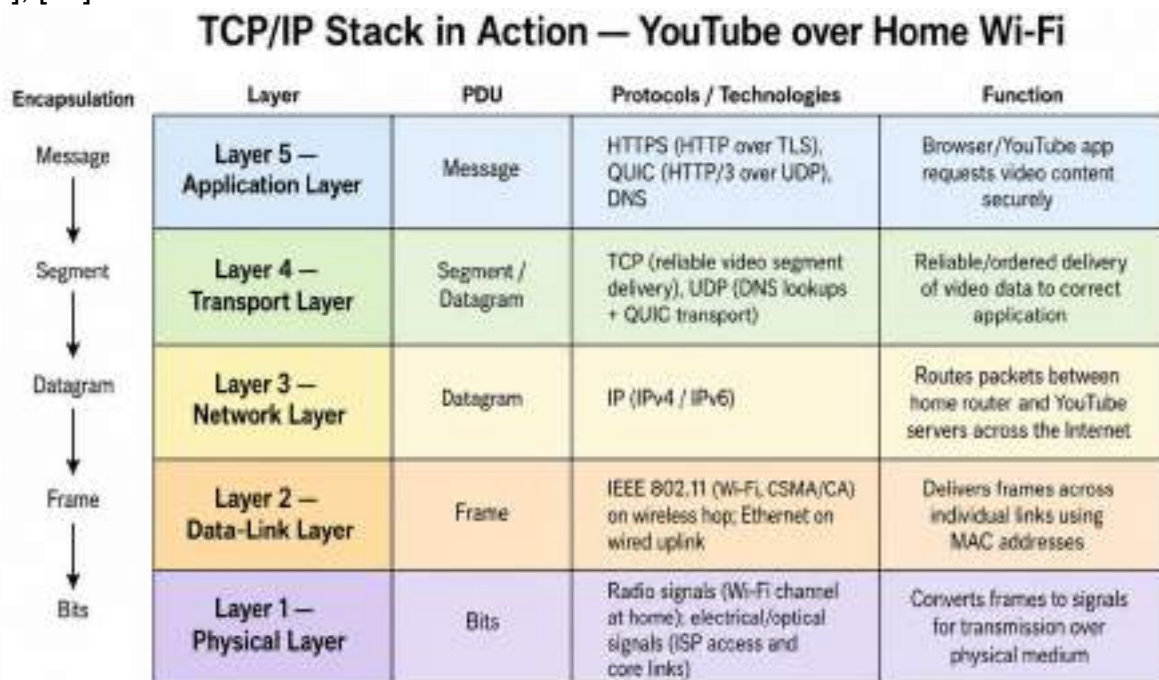
Exercise 7.4: Watching a YouTube video over home Wi-Fi

Question: Explain the behind-the-scenes process and protocols at each layer when watching a YouTube video using home Wi-Fi.

Ans. When I watch a YouTube video over my home Wi-Fi, the process begins at the **application layer**, where my browser or YouTube app uses HTTPS (HTTP over

TLS) to request the video from YouTube's web servers using URL, cookies, and other application metadata. At the **transport layer**, the client and server establish one or more TCP connections (and a TLS session) to provide reliable, ordered delivery of video segments, while UDP is used specifically for DNS lookups (a separate supporting protocol), and QUIC (which runs over UDP) is increasingly used by YouTube for actual video delivery as it offers faster connection establishment and better performance over lossy networks. At the **network layer**, IP (typically IPv4, sometimes IPv6) routes each packet across the Internet between my home router's public interface and YouTube's servers, using source and destination IP addresses and routing tables maintained by routers in my ISP and the wider Internet. [7], [8]

At the **data-link layer**, my laptop's Wi-Fi interface uses 802.11 frames with MAC addresses for my device and the access point, handling local delivery over the wireless link, medium access using CSMA/CA, and link-layer retransmissions when frames are corrupted. The home router or gateway then forwards the traffic on its wired or optical uplink using Ethernet frames, which encapsulate each IP datagram with appropriate source and destination MAC addresses for each hop. At the **physical layer**, all of these frames are converted into radio signals over the Wi-Fi channel in my home and into electrical or optical signals on the ISP's access and core links, where modulation, power levels, and channel properties determine how bits are actually transmitted and received. Supporting protocols like DHCP, DNS, and ARP also play a role: DHCP may have previously given my device a local IP address and default gateway, ARP resolves MAC addresses inside the LAN, and DNS translates the YouTube domain into the server IP address that my client contacted. [9], [10]



Source: Adapted from Kurose & Ross, *Computer Networking: A Top-Down Approach*, 7th Ed.

Figure 3: TCP/IP protocol stack showing the specific protocols and technologies active at each layer during YouTube video streaming over a home Wi-Fi network,

including HTTPS and QUIC at the application layer, TCP/UDP at the transport layer, IP at the network layer, 802.11 at the data-link layer, and radio signal transmission at the physical layer.

Evidence: Class Notes

6

media

Guided (Wired)

Unguided (Wireless)

Signal long distance usually through a twisted pair or fiber optic cable

all different types of cable

Twisted Pair Cables

Coaxial Cable

Fiber Optic Cable

Optics

Advantages

- Low cost
- Easy to install
- High speed
- Long distance transmission

7

attenuation - loss of signal strength

shielded twisted pair (STP)

losses of a shielded jacket (copper braided)

Twisted Pair Cables

Coaxial Cable

Fiber Optic Cable

Advantages

- Low cost
- Easy to install
- High speed
- Long distance transmission

8

Uniqued (WIRELESS)

WIRELESS TRANSMISSION

- Radio waves: 300 MHz - 1 GHz
- Infrared: 300 MHz - 300 GHz
- Microwave: 300 GHz - 400 THz

HY INTERNET

LAN = Local Area Network

INTERNET

→ global network used

1. Ethernet: 802.3 by IEEE standardized

→ works in three main layers used physical layer

2. Gigabit Ethernet:

1. Gigabit Ethernet
2. Wireless Ethernet

9

Internet network can be classified into four types:

- i) Standard Ethernet
- ii) Fast Ethernet
- iii) Gigabit Ethernet
- iv) 10-Gigabit Ethernet

Evidence: Active Classes Activities

Activities for Active Class 10: Physical Layer

Activity 1: Behind-the-scenes process of accessing CloudDeakin

Question 1: Major steps from power-on to page download (no NAT)

Ans. When I power on my laptop and connect to my home Wi-Fi, the first step is **link-layer association and physical connectivity**, where my Wi-Fi interface discovers the access point's SSID, authenticates if needed, and completes the 802.11 association so that it can send and receive frames over the wireless channel. Next, if my laptop does not already have a valid IP configuration, it uses **DHCP** to obtain an IP address, subnet mask, default gateway, and DNS server by exchanging DHCP Discover, Offer, Request, and ACK messages with the home router acting as a DHCP server.

Once the interface is configured, my laptop needs to resolve the CloudDeakin hostname, so it sends a **DNS query** for d2l.deakin.edu.au to the configured DNS server, either directly or via the home router, and receives a DNS response containing the IP address of the CloudDeakin front-end server. To actually send packets toward this server, the laptop checks its routing table, sees that the destination is outside the local subnet, and uses **ARP** to resolve the MAC address of the default gateway, allowing it to encapsulate IP datagrams in 802.11 frames destined for the home router. Then, the laptop's browser initiates one or more **TCP connections** (with TLS) to the CloudDeakin server's IP address and port 443, performing the TCP three-way handshake and TLS negotiation before sending encrypted HTTPS GET requests for the page and embedded resources.

During this process, each request and response is carried as TCP segments inside IP datagrams, which are in turn encapsulated into Wi-Fi frames on the wireless hop to the router and then into Ethernet frames on the ISP's access network and backhaul links, until they reach the CloudDeakin infrastructure. The CloudDeakin server processes the requests, generates dynamic HTML, CSS, JavaScript, and other assets, and sends these back over the same end-to-end TCP/TLS connection, after which my browser reassembles the data stream, parses the HTML, and renders the SIT202 CloudDeakin page on the screen.

Question 2: Protocols used and their functionality

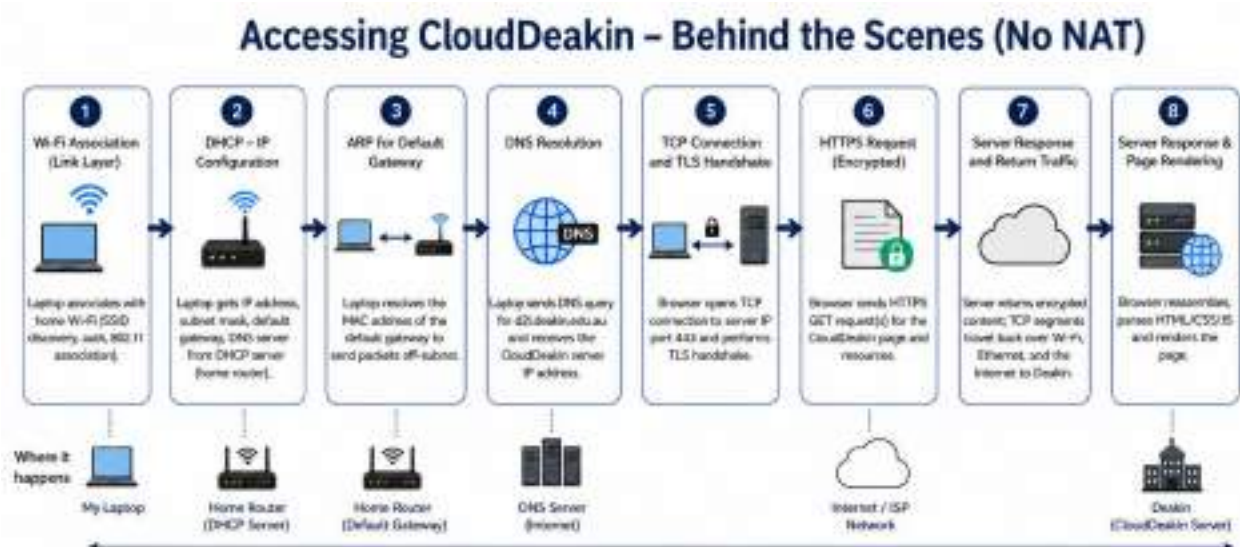
Ans. At the **application layer**, HTTPS (HTTP over TLS) is used by the browser to request the CloudDeakin page and subsequent resources securely, while DNS is used to translate the human-readable domain name into an IP address that the network layer can route to. At the **transport layer**, TCP provides a reliable, connection-oriented service that ensures ordered, loss-free delivery of the HTTP

request and response streams between my laptop and the CloudDeakin server, including congestion control and flow control. At the **network layer**, IP routes datagrams between my home network and Deakin's network using the source and destination IP addresses, with routers in the ISP and Internet core using routing protocols like OSPF and BGP to maintain forwarding tables.

At the **data-link layer**, 802.11 Wi-Fi and Ethernet provide local delivery over individual links: Wi-Fi frames carry the traffic between my laptop and the home router using MAC addresses and CSMA/CA, while Ethernet frames carry the same IP datagrams over the router's wired uplink to the ISP. At the **physical layer**, these frames are converted into radio signals on my home Wi-Fi and electrical or optical signals on copper or fibre in the ISP network, so that bits can travel across the physical infrastructure from my home to the university's servers. Supporting protocols like DHCP assign my laptop a usable IP configuration, ARP resolves MAC addresses for the default gateway and local peers, and ICMP may be used for diagnostic messages, although it is not directly visible to the user during normal page loading.

Question 3: What changes if the home network uses NAT?

Ans. If my home network uses NAT, the **end-to-end logical process remains the same**, but the source addressing seen on the public Internet changes: internally, my laptop still uses a private IP address assigned by DHCP, but the home router translates this to a single public IP address when forwarding outbound packets. For each TCP connection to the CloudDeakin server, the NAT router records a mapping from the internal (private IP, source port) pair to an external (public IP, translated port) pair and rewrites the source IP and port in outgoing packets, as well as the destination IP and port in incoming replies, to maintain consistency. From my laptop's perspective, it still communicates with the same destination IP and port using standard TCP and HTTPS, but any device on the Internet only sees the router's public IP address as the client, so multiple devices in my home can share a single public IP.



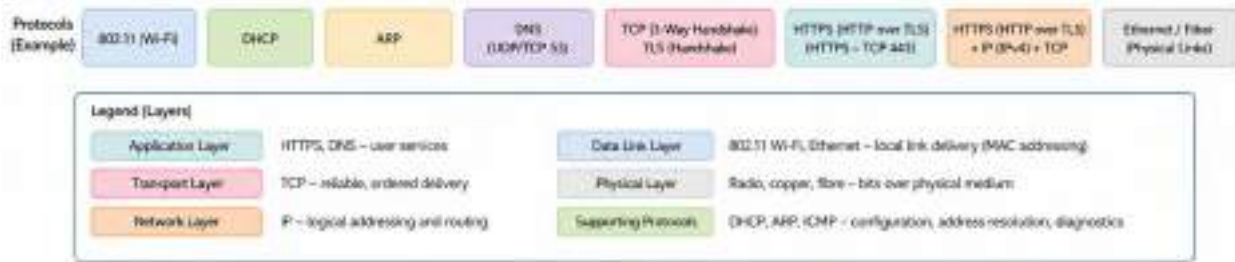


Figure 4: Step-by-step process and protocols involved in accessing CloudDeakin (d2l.deakin.edu.au) over a home Wi-Fi network without NAT, showing the sequence from 802.11 association through DHCP, ARP, DNS, TCP/TLS handshake, and HTTPS request/response across the TCP/IP layers.

Activity 2: Wireshark – explaining the process

Question 7(a) Order of protocols and messages

Ans. In my Wireshark capture for <http://www.discoverourtown.com>, the observed protocol sequence matched the expected conceptual flow: DHCP, ARP, DNS, TCP handshake, and HTTP request/response appeared in that order, confirming the theoretical process described in Activity 1. First, my laptop exchanged **DHCP** messages (Discover, Offer, Request, and ACK) with the home router to confirm its IP configuration, because the DNS cache and network connection had been reset. Next, it used **ARP** to resolve the MAC address of the default gateway so that it could send IP packets toward the Internet, followed by one or more **DNS queries and responses** to translate the web server's hostname into an IP address. Once the IP address was known, the client opened a **TCP connection** to the server's HTTP port by sending a SYN, receiving a SYN-ACK, and replying with an ACK, after which it sent an **HTTP GET** request and received the HTTP response containing the page content and related resources. Each of these higher-layer exchanges was carried in IP datagrams and encapsulated into Ethernet frames at the data link layer, and these frames were also visible in the capture at the link layer.

Question 7(b) Use of each protocol and its layer

Ans. **DHCP** operates at the application layer and uses UDP to automatically assign IP configuration parameters, such as the client's IP address, subnet mask, default gateway, and DNS server, enabling the host to participate in the IP network without manual configuration. **ARP** operates between the network and data-link layers to map IP addresses to MAC addresses on the local subnet, enabling IP datagrams to be encapsulated into frames with correct link-layer destinations. **DNS** is an application-layer protocol, typically running over UDP, that resolves human-readable domain names into IP addresses so that the underlying IP network can route traffic to the correct destination.

TCP is a transport-layer protocol that establishes a reliable, connection-oriented channel between the client and server, ensuring ordered, loss-free delivery of the HTTP request and response messages by using sequence numbers, acknowledgements, and retransmissions. **HTTP** is an application-layer protocol used by the browser to request web resources such as HTML pages, images, and scripts, using methods like GET and responses with status codes indicating success or failure. At lower layers, **IP** provides host-to-host delivery of the TCP segments across multiple networks, while Ethernet provides local frame delivery between the laptop, access point, and router over wired or wireless links.

Question 7(c) Checking ARP, DHCP, DNS, HTTP packet details

Ans. In the **DHCP exchange**, the Discover and Request messages from my laptop used the all-ones broadcast MAC address in the Ethernet header and 0.0.0.0 as the source IP address, because the client did not yet have an assigned address, while the Offer and ACK from the router specified the assigned client IP, default gateway, and lease time. The **ARP requests** had the destination MAC field set to the broadcast address and asked “Who has the router’s IP address? Tell [client IP]”, whereas the ARP replies from the router were unicast and provided the router’s MAC address, which was then cached in the client’s ARP table. The **DNS queries** contained the fully qualified domain name of www.discoverourtown.com, and the corresponding responses carried one or more A records giving the IPv4 address of the web server.

In the **TCP three-way handshake**, the initial SYN from the client included a sequence number and destination port 80, the SYN-ACK from the server echoed the client’s sequence and added its own, and the final ACK completed the handshake, after which the first **HTTP GET** request appeared with headers such as Host, User-Agent, and Accept. The subsequent **HTTP responses** from the server contained status codes (for example, 200 OK) and payloads carrying HTML and other content, which were fragmented into multiple TCP segments when necessary and reassembled by the client. Screenshots of these key packets—DHCP negotiation, ARP resolution, DNS query/response, TCP handshake, and HTTP request/response—are included in my evidence to illustrate how the theoretical protocol flow appears in an actual packet trace.

Question 8. Comparison with Activity 1

Ans. Comparing the Wireshark capture with the conceptual process from Activity 1, I found that the **overall sequence of protocols closely matched the theory**: DHCP configuration, ARP resolution, DNS lookup, TCP connection establishment, and HTTP request/response all appeared in the expected order. The main differences were in additional low-level details that the abstraction in Activity 1 glossed over,

such as retransmissions, multiple DNS queries, and ancillary background traffic like IPv6 neighbour discovery or TLS-related exchanges, which are visible on the wire but not central to the basic story of fetching a single web page. Seeing both views together helped me connect the high-level description of “requesting a webpage” to the actual packets and headers used at each layer, reinforcing my understanding of how theory maps to real network behaviour.

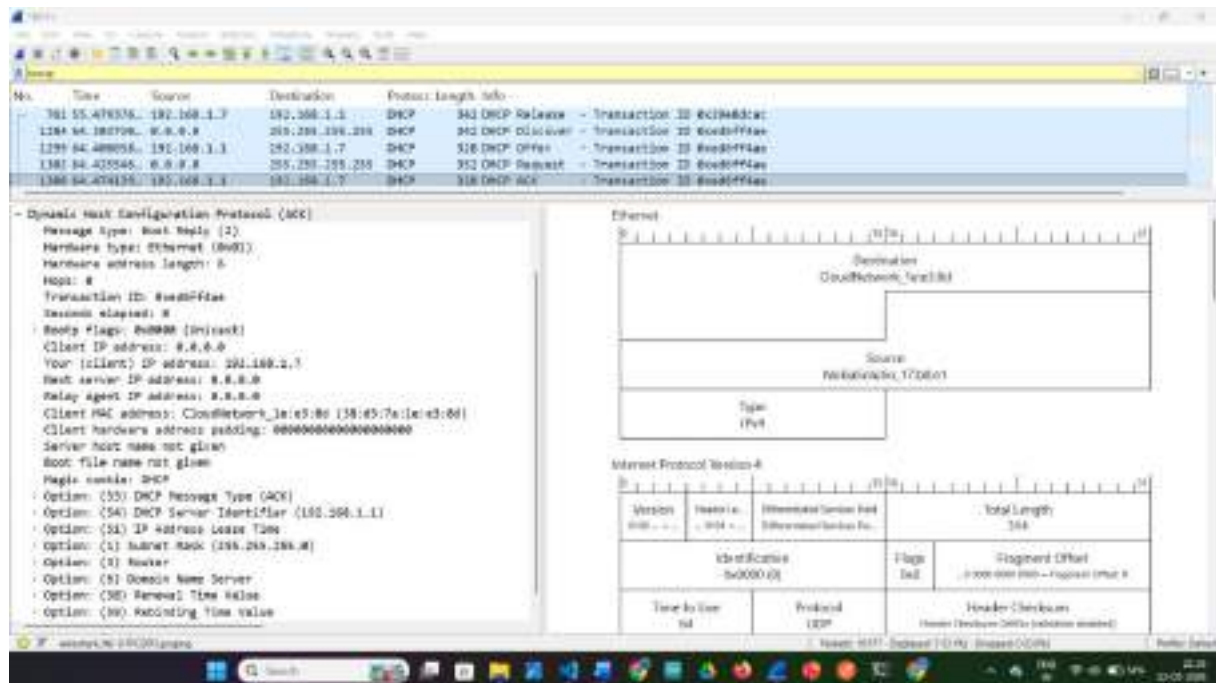


Figure 5(a): Wireshark capture of the DHCP exchange obtained using the bootp filter, showing DHCP Discover, Offer, Request, and ACK messages; the selected DHCP ACK packet confirms the client IP address, subnet mask, default gateway, DNS server, and lease information.

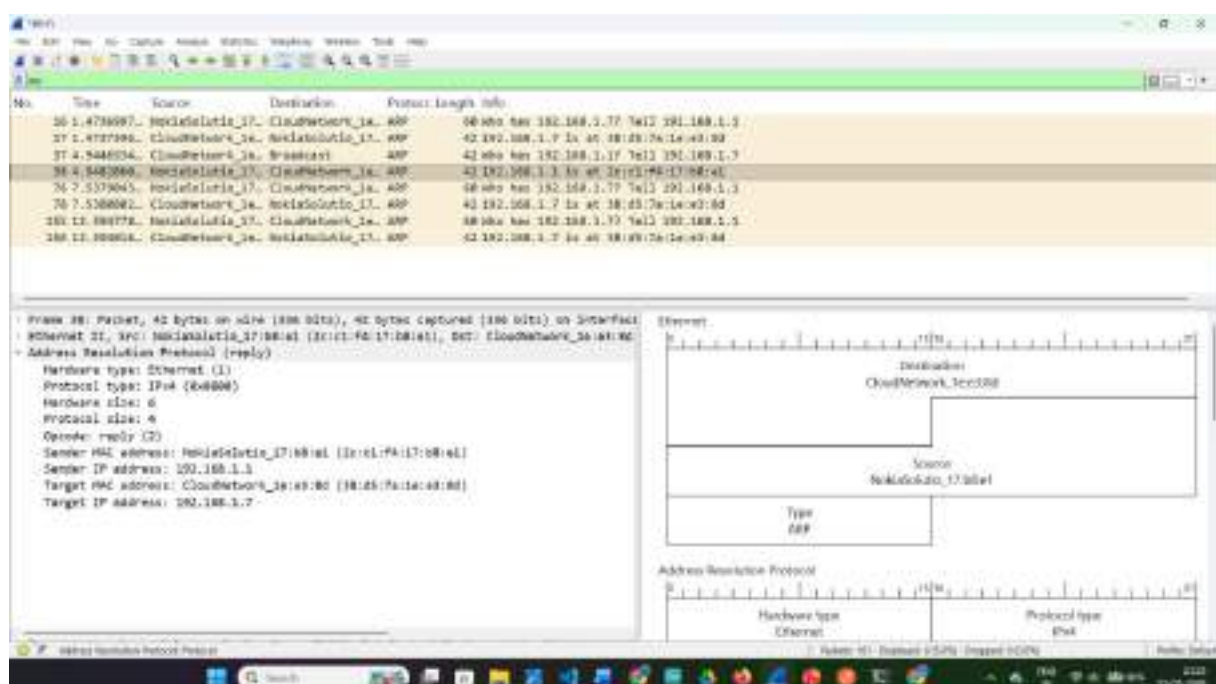


Figure 5(b): Wireshark capture of ARP resolution using the arp filter, showing the ARP Request from the client (192.168.1.7) asking "Who has 192.168.1.1?" and the ARP Reply from the gateway (192.168.1.1) confirming its MAC address (2c:c1:f4:17:b8:e1), enabling the client to encapsulate IP packets into Ethernet frames for forwarding.

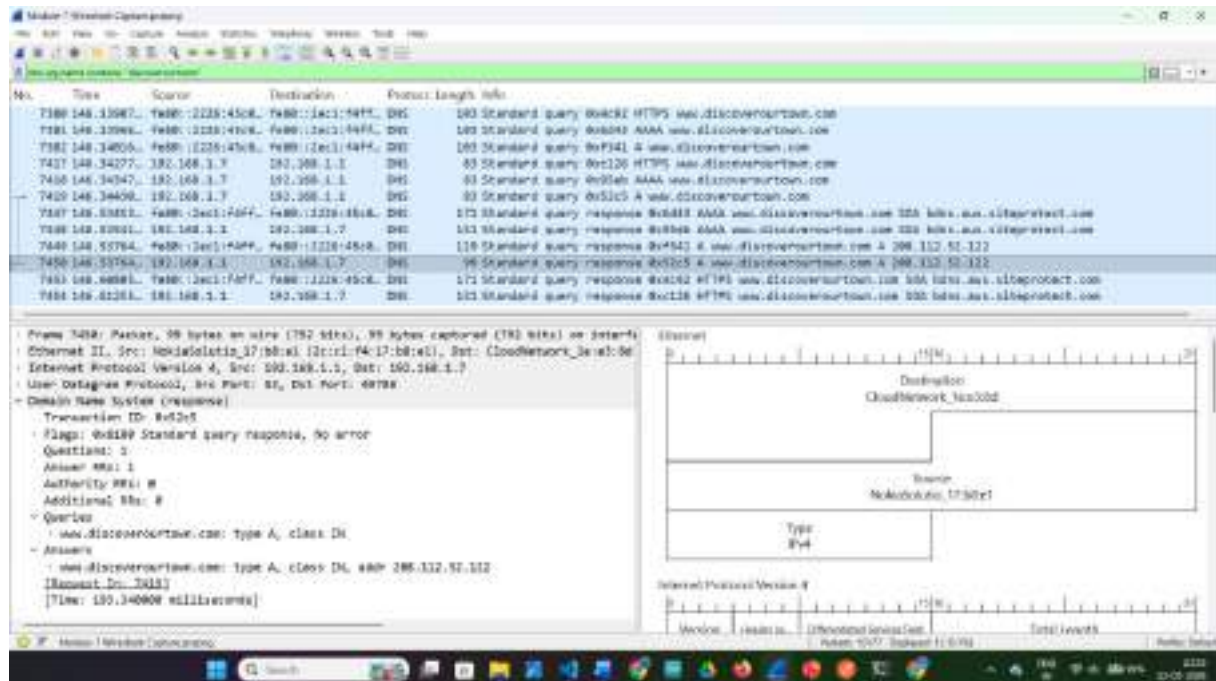


Figure 5(c): Wireshark capture of DNS resolution using the dns filter, showing the DNS A record query for www.discoverourtown.com from the client (192.168.1.7) and the corresponding DNS response resolving the domain to IP address 208.112.52.122.

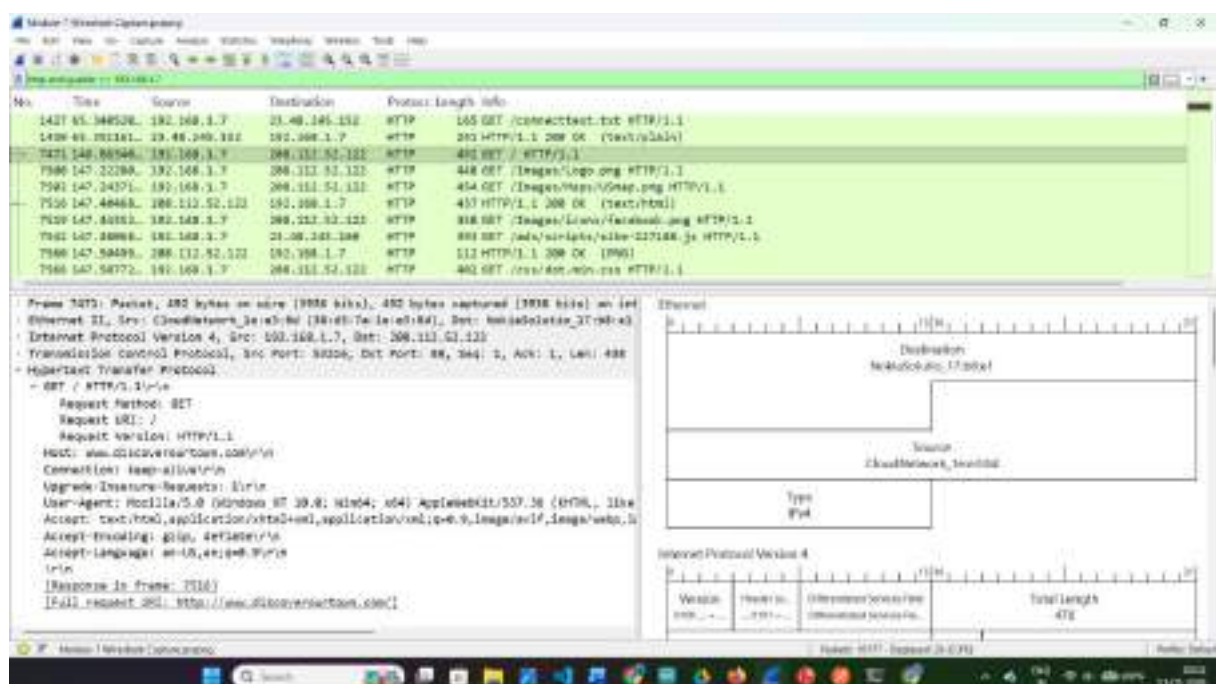


Figure 5(d): Wireshark capture of HTTP communication using the http filter, showing the HTTP GET request from the client (192.168.1.7) to the web server at 208.112.52.122 (www.discoverourtown.com) and the corresponding HTTP/1.1 200 OK response, confirming successful retrieval of the webpage.

Activity 3: Wired vs wireless networks in Packet Tracer

Question 1–2. Similarities and differences between wired and wireless PDUs

Ans. When I sent a test PDU from **PC1 to PC0** over the wired LAN, the simulation showed a simple path where the ICMP Echo Request was encapsulated in an IP datagram and then in Ethernet frames with a single pair of MAC addresses (source = PC1's NIC, destination = PC0's NIC), confirming that the switch operates purely at the data-link layer and does not inspect or modify IP or transport headers. The PDU details confirmed that the link-layer header contained exactly two MAC addresses and that the switch did not change the IP header, but only forwarded frames on the correct port after learning the source address and performing a lookup for the destination.

In contrast, when I sent a test PDU from **Laptop1 to TabletPC0** over the wireless LAN, the PDU passed through the wireless router, and the 802.11 frame headers contained **three MAC addresses** corresponding to the transmitting station, receiving station, and access point (BSSID), as shown in the PDU details. The IP and ICMP headers remained the same between the end hosts, but the link-layer encapsulation reflected the fact that traffic was flowing between wireless clients via an access point rather than directly over a single shared Ethernet segment. Despite these differences, both wired and wireless paths ultimately delivered the same ICMP Echo Request and Echo Reply messages at the network and transport layers, demonstrating that higher-layer protocols can operate consistently over very different physical and link-layer technologies.

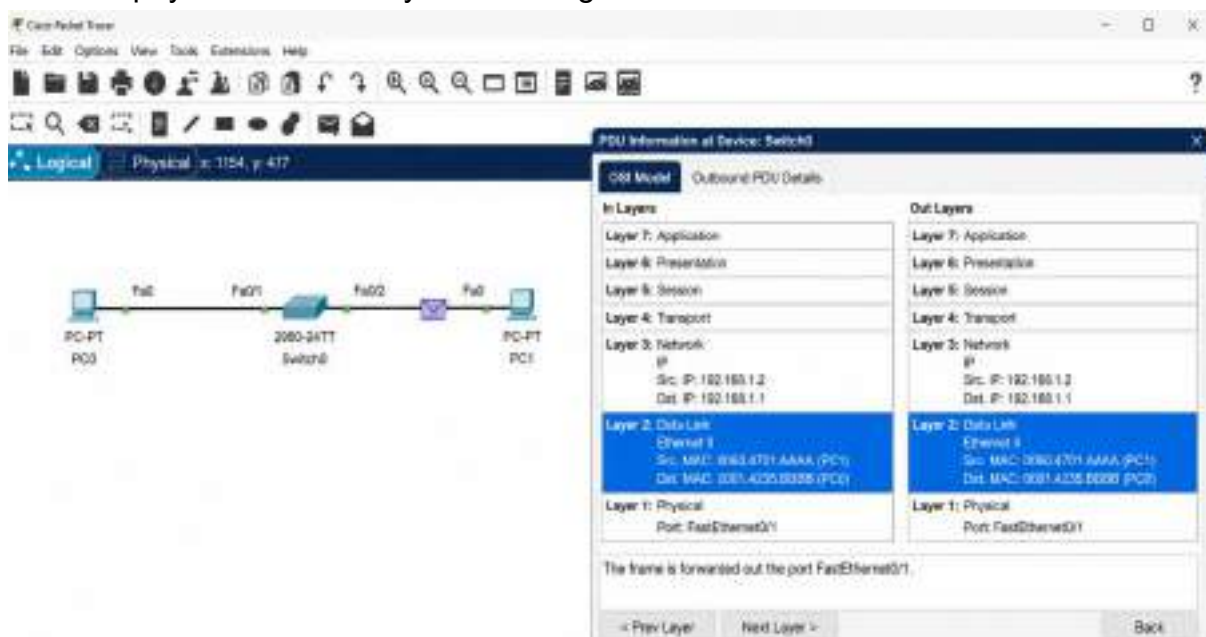


Figure 6(a-i): Cisco Packet Tracer PDU Information window showing the OSI Model for wired transmission from PC1 to PC0 at Switch0, confirming that Layer 2 contains only two MAC addresses and that the switch forwards the frame without modifying the IP header.

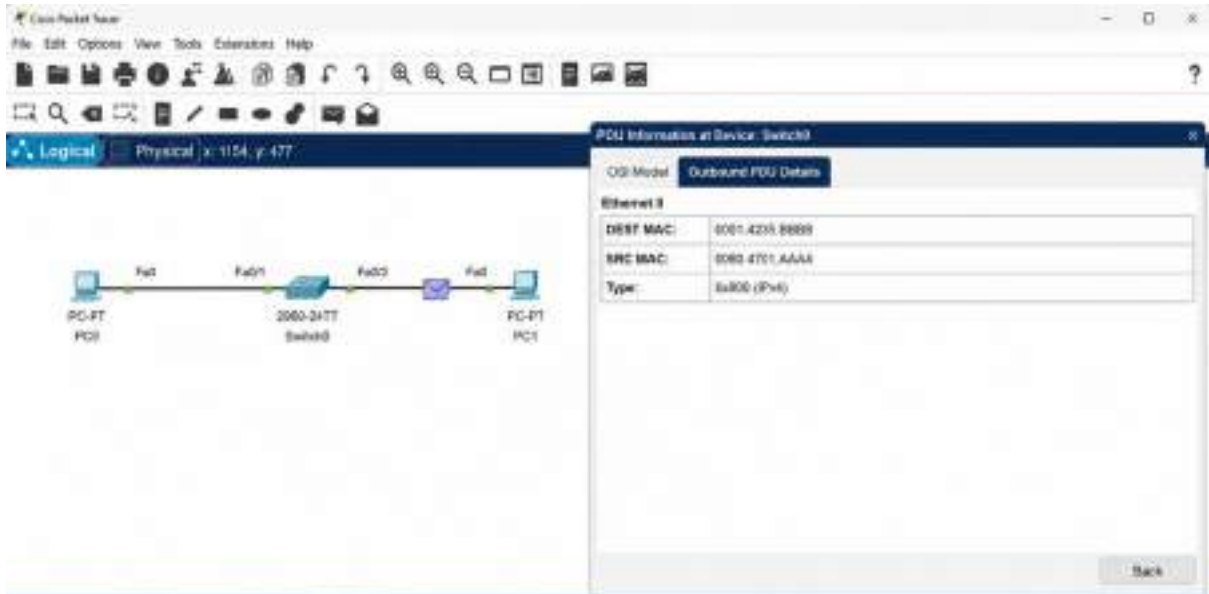


Figure 6(a-ii): Cisco Packet Tracer Outbound PDU Details at Switch0 showing the raw Ethernet II frame with source MAC 0060.4701.AAAA, destination MAC 0001.4235.BBBB, and EtherType 0x800, confirming standard two-address wired encapsulation.

Question 3–4. Why are there three MAC addresses in 802.11 PDUs

Ans. In 802.11 wireless PDUs, there are often **three MAC addresses** because the frame format must represent not only the sender and receiver but also the access point or distribution-system address that relays frames between the wireless and wired domains. In the scenario where a wireless client communicates with another device via an access point, one address identifies the **receiver** of the frame at the current hop, one identifies the **transmitter** that sent the frame onto the wireless medium, and the third identifies the **BSSID** (the access point's MAC address) that anchors the wireless cell and connects it to the wired network.

This three-address structure allows the same frame format to support different forwarding modes, including communication from a wireless client to the access point, from the access point to a client, and, in more advanced cases, distribution between access points, while still keeping track of the original source and ultimate destination. By contrast, standard Ethernet frames only require two MAC addresses because each frame is always between two interfaces on a single link, with intermediate switches operating transparently at the link layer



Figure 6(b-i): Cisco Packet Tracer OSI Model view at AccessPoint0 showing Layer 2 as 802.11 (Dot11) with three MAC addresses (Address 1/BSSID: 00E0.A3B1.CC01, Address 2/Transmitter: 0060.2F45.DD02, Address 3/Destination: 0090.0C78.EE03) and Layer 3 IP header unchanged (SRC: 192.168.0.2, DEST: 192.168.0.3), confirming that wireless relay via the access point requires additional MAC addressing at the data link layer.

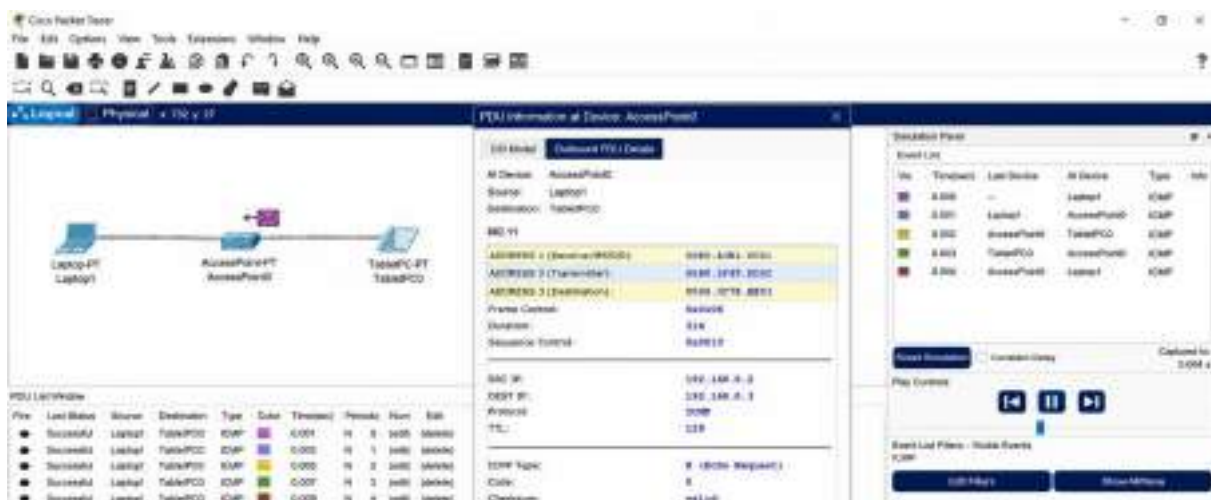


Figure 6(b-ii): Cisco Packet Tracer Outbound PDU Details at AccessPoint0 showing the 802.11 wireless frame with three MAC addresses (Address 1/BSSID: 00E0.A3B1.CC01, Address 2/Transmitter: 0060.2F45.DD02, Address 3/Destination: 0090.0C78.EE03) and IP header unchanged (SRC: 192.168.0.2, DEST: 192.168.0.3), confirming the additional addressing required for wireless relay via the access point.

Above and Beyond: Straight-through vs crossover Ethernet cables

Ans. A **straight-through Ethernet cable** is a twisted-pair cable where both ends are terminated using the same wiring standard (typically T568B on both connectors), so that each pin on one end connects directly to the corresponding pin on the other end; this type of cable is used to connect devices of different types, such as a PC to a switch, a PC to a router, or a switch to a router, in modern Ethernet networks.

A **crossover Ethernet cable** swaps the transmit and receive pairs between the two ends by using T568A on one end and T568B on the other, so that the transmit pins on one device connect to the receive pins on the other; historically, crossover cables were used to connect similar devices directly, such as PC-to-PC or switch-to-switch, before auto MDI-X became common. The easiest way to identify the cable type is to hold both RJ-45 connectors side by side with the clip facing away and compare the colour order of the wires: if the colour sequence is identical on both ends, the cable is straight-through; if the green and orange pairs are swapped between ends, the cable is a crossover.

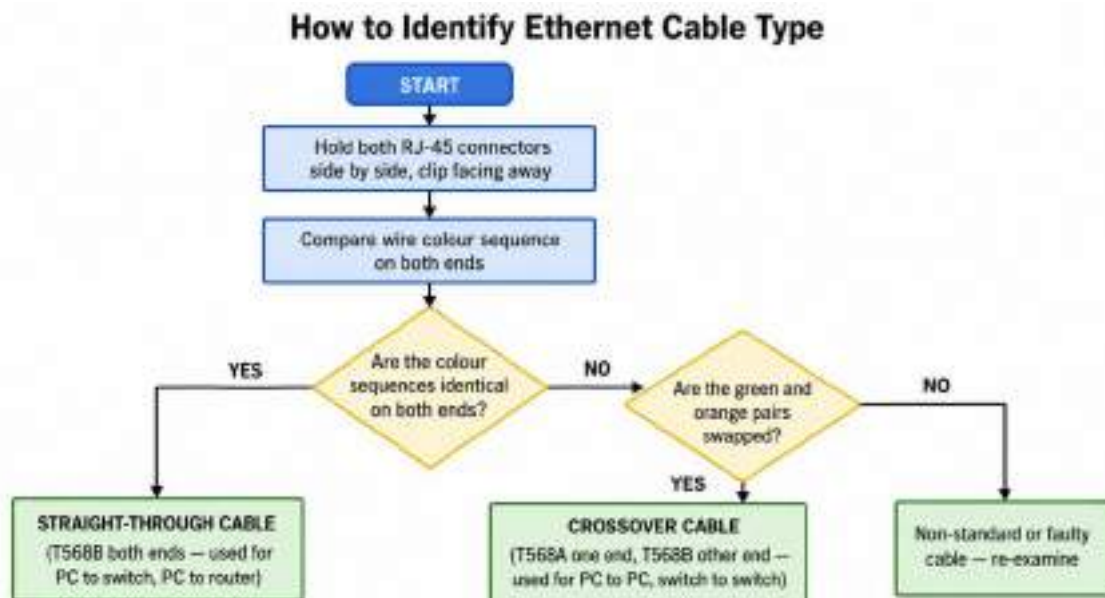


Figure 7: Ethernet Cable Identification Flowchart showing the step-by-step process of comparing RJ-45 connector wire colour sequences to distinguish a straight-through cable (T568B on both ends, used for PC to switch or PC to router) from a crossover cable (T568A on one end, T568B on the other, used for PC to PC or switch to switch), with a third outcome for non-standard or faulty cables.

References

- [1] "Physical Layer in OSI Model - GeeksforGeeks." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/physical-layer-in-osi-model/>
- [2] "Transmission Media in Computer Networks - GeeksforGeeks." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/types-transmission-media/>
- [3] "IEEE SA - IEEE 802.3-2022." Accessed: Apr. 28, 2026. [Online]. Available: <https://standards.ieee.org/ieee/802.3/10422/>
- [4] "Module 7 PPT - Network and Communications_Sem2_2025." Accessed: Apr. 28, 2026. [Online]. Available: <https://culms.chitkara.edu.in/d2l/le/content/7806/viewContent/23510/View>
- [5] "UnGuided Media - GeeksforGeeks." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/unguided-media/>
- [6] "Wired vs Wireless: Which Connectivity Wins? - SynchroNet." Accessed: Apr. 28, 2026. [Online]. Available: <https://synchronet.net/wired-vs-wireless/>
- [7] "Why is YouTube using TCP but not UDP? - GeeksforGeeks." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/why-is-youtube-using-tcp-but-not-udp/>
- [8] "TCP/IP Model Explained – 4 Layers, Protocols, Encapsulation, OSI Comparison & Troubleshooting | CCNA Mastery | NetsTuts." Accessed: Apr. 28, 2026. [Online]. Available: <https://netstuts.com/tcpip-model>
- [9] "IEEE 802.11 Standards Explained: PHY, Frame Format & Measurement | Tektronix." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.tek.com/en/documents/primer/wi-fi-overview-80211-physical-layer-and-transmitter-measurements>
- [10] "Network layer (CS 4410, Summer 2017)." Accessed: Apr. 28, 2026. [Online]. Available: <https://www.cs.cornell.edu/courses/cs4410/2017su/lectures/lec20-netlayer.html>

[Quiz List](#) › [Submissions](#)

Quiz Submissions – Module 7: Physical Layer

Individual Attempts	Grade
Attempt 1	10 / 10 - 100 %
Overall Grade (highest attempt):	10 / 10 - 100 %

Quiz List - Network and Comm

cafebaobhkanee.edu.my/03/firstpage.php?user/quizs_list&id=1006

cafebaobhkanee.edu.my

Network and Communications_Sem2_2025

JAHANGIRG

Course HomeContentAssignmentsDiscussionsQuizzesClasslistGradesClass ProgressCourse Tools

Quiz List

Help

Current Quizzes	Evaluation Status	Attempts
Module 7: Physical Layer Due on 30 April 2026 11:55 PM Available on 10 April 2026 12:01 AM until 30 April 2026 11:59 PM	Feedback: On Attempt	1 / 3
Past Quizzes	Evaluation Status	Attempts
Module 1: Introduction to Computer Networking Due on 19 February 2026 4:00 PM Available on 04 February 2026 9:00 AM until 19 February 2026 4:00 PM	Feedback: On Attempt	1 / 3
Module 2: Application Layer Due on 21 April 2026 4:00 PM Available on 11 February 2026 12:00 PM until 21 April 2026 4:00 PM	Feedback: On Attempt	1 / 4

Search

23:59 27-04-2026

 Springer

Quiz Item	Points	Grade	Comments and Assessments
Module 1: Introduction to Computer Networking	10 / 50	100%	 View Quiz Attempts
Module 2: Application Layer	10 / 50	100%	 View Quiz Attempts
Module 3: Transport Layer	10 / 50	100%	 View Quiz Attempts
Unfiled	10 / 50	100%	 View Quiz Attempts
Module 6: Data Link Layer	10 / 50	100%	 View Quiz Attempts
Module 7: Physical Layer	10 / 50	100%	 View Quiz Attempts
Module 4: Network Layer Data Plane	10 / 50	100%	 View Quiz Attempts